

BIEK LECTURE-WISE STUDY GUIDES

BIEK Computer Science XII

One PDF per lecture — plus a full chapter pack for each unit.

Bilingual support: English + اردو

بی آئی ای کے کمپیوٹر سائنس — بارہویں جماعت

XII

Lecture index

Chapter packs are named **Lecture 01–06**. Individual topics are **ChNN-LNN**. ★ marks Golden (high-yield) exam topics.

Code	File	Title
Lecture 01	CS-XII-Lecture-01-Computer-Systems-HCI.pdf	Computer Systems (HCI)
Ch01-L01	CS-XII-Ch01-L01-1.1-Fundamentals-of-Human-Computer-Interaction-HCI.pdf	1.1 Fundamentals of Human Computer Interaction (HCI)
Ch01-L02	CS-XII-Ch01-L02-1.1.1-1.1.2-Traditional-vs-Natural-Interaction.pdf	1.1.1 – 1.1.2 Traditional vs Natural Interaction
Ch01-L03	CS-XII-Ch01-L03-1.2-Applications-of-HCI-Domains-of-Life.pdf	1.2 Applications of HCI (Domains of Life) ★
Ch01-L04	CS-XII-Ch01-L04-1.3-Components-of-HCI.pdf	1.3 Components of HCI
Ch01-L05	CS-XII-Ch01-L05-1.3-Types-of-User-Interaction.pdf	1.3 Types of User Interaction
Ch01-L06	CS-XII-Ch01-L06-1.3-Interface-Types-Environment-Feedback.pdf	1.3 Interface Types, Environment & Feedback
Ch01-L07	CS-XII-Ch01-L07-1.4-Importance-of-HCI.pdf	1.4 Importance of HCI ★
Ch01-L08	CS-XII-Ch01-L08-1.5-Accessibility-Principles.pdf	1.5 Accessibility Principles
Ch01-L09	CS-XII-Ch01-L09-1.6-Need-Analysis-for-Interface-Design.pdf	1.6 Need Analysis for Interface Design
Ch01-L10	CS-XII-Ch01-L10-1.7-Human-Computer-Interaction-Problems.pdf	1.7 Human Computer Interaction Problems ★
Ch01-L11	CS-XII-Ch01-L11-1.8-Methods-for-Improving-HCI.pdf	1.8 Methods for Improving HCI
Ch01-L12	CS-XII-Ch01-L12-1.9-User-Interface-Design-UI-vs-UX.pdf	1.9 User Interface Design (UI vs UX) ★
Ch01-L13	CS-XII-Ch01-L13-1.9.3-1.9.5-Wireframing-Figma-and-Prototypes.pdf	1.9.3 – 1.9.5 Wireframing, Figma, and Prototypes
Ch01-L14	CS-XII-Ch01-L14-1.11-Test-and-Evaluate-HCI-Evaluation-Methods.pdf	1.11 Test and Evaluate HCI (Evaluation Methods)

Ch01-L15	CS-XII-Ch01-L15-1.11.2-Testing-Methods-Usability-AB-Automated.pdf	1.11.2 Testing Methods (Usability, A/B, Automated) ★
Lecture 02	CS-XII-Lecture-02-Computational-Thinking-Algorithms.pdf	Computational Thinking & Algorithms
Ch02-L01	CS-XII-Ch02-L01-2.1-Analyze-algorithms-for-correctness.pdf	2.1 Analyze algorithms for correctness
Ch02-L02	CS-XII-Ch02-L02-2.1.1-2.1.3-Trace-Tables-vs-Stepwise-Reasoning.pdf	2.1.1 – 2.1.3 Trace Tables vs Stepwise Reasoning ★
Ch02-L03	CS-XII-Ch02-L03-2.2-Evaluating-the-Clarity-of-an-Algorithm.pdf	2.2 Evaluating the Clarity of an Algorithm
Ch02-L04	CS-XII-Ch02-L04-2.2.1-2.2.2-Modularity-and-Readability.pdf	2.2.1 – 2.2.2 Modularity and Readability
Ch02-L05	CS-XII-Ch02-L05-2.3-Assess-Algorithm-Efficiency.pdf	2.3 Assess Algorithm Efficiency ★
Ch02-L06	CS-XII-Ch02-L06-2.3.1-Number-of-Steps-Big-O-Notation.pdf	2.3.1 Number of Steps (Big O Notation) ★
Ch02-L07	CS-XII-Ch02-L07-2.3.2-Number-of-Conditions.pdf	2.3.2 Number of Conditions
Ch02-L08	CS-XII-Ch02-L08-2.3.3-Number-of-Repetitions.pdf	2.3.3 Number of Repetitions
Ch02-L09	CS-XII-Ch02-L09-2.3.4-The-Efficiency-Evaluation-Framework.pdf	2.3.4 The Efficiency Evaluation Framework
Ch02-L10	CS-XII-Ch02-L10-2.4-Refinements-to-improve-clarity-and-efficiency.pdf	2.4 Refinements to improve clarity and efficiency
Ch02-L11	CS-XII-Ch02-L11-2.5-Concept-of-Data-Structure.pdf	2.5 Concept of Data Structure ★
Ch02-L12	CS-XII-Ch02-L12-2.5.1-Linear-Data-Structures-Array-Linked-List-Stack-Queue.pdf	2.5.1 Linear Data Structures (Array, Linked List, Stack, Queue) ★
Ch02-L13	CS-XII-Ch02-L13-2.5.2-Non-Linear-Data-Structures-Tree-Graph.pdf	2.5.2 Non-Linear Data Structures (Tree, Graph)
Ch02-L14	CS-XII-Ch02-L14-2.5.3-Operations-on-Data-Structures.pdf	2.5.3 Operations on Data Structures

Ch02-L15	CS-XII-Ch02-L15-2.6-Identify-data-structures-in-problem-scenarios.pdf	2.6 Identify data structures in problem scenarios
Ch02-L16	CS-XII-Ch02-L16-2.7-Trace-data-retrieval-in-Array-List-and-Queue.pdf	2.7 Trace data retrieval in Array, List and Queue
Lecture 03	CS-XII-Lecture-03-Programming-Fundamentals.pdf	Programming Fundamentals
Ch03-L01	CS-XII-Ch03-L01-3.1-Data-Structures-in-Python.pdf	3.1 Data Structures in Python
Ch03-L02	CS-XII-Ch03-L02-3.1.1-Lists-Ordered-Mutable-Methods.pdf	3.1.1 Lists (Ordered, Mutable, Methods) ★
Ch03-L03	CS-XII-Ch03-L03-3.1.2-Tuples-Ordered-Immutable.pdf	3.1.2 Tuples (Ordered, Immutable) ★
Ch03-L04	CS-XII-Ch03-L04-3.1.3-Sets-Unordered-Unique-Math-Operations.pdf	3.1.3 Sets (Unordered, Unique, Math Operations)
Ch03-L05	CS-XII-Ch03-L05-3.1.4-Dictionaries-Key-Value-pairs.pdf	3.1.4 Dictionaries (Key-Value pairs) ★
Ch03-L06	CS-XII-Ch03-L06-3.1.5-Common-Built-in-Functions-min-max-len-sum.pdf	3.1.5 Common Built-in Functions (min, max, len, sum)
Ch03-L07	CS-XII-Ch03-L07-3.1.6-Activity-Students-Attendance-Program.pdf	3.1.6 Activity: Students Attendance Program
Ch03-L08	CS-XII-Ch03-L08-3.1.7-Activity-Word-Frequency-Counter.pdf	3.1.7 Activity: Word Frequency Counter
Ch03-L09	CS-XII-Ch03-L09-3.2-Functions-in-Python.pdf	3.2 Functions in Python ★
Ch03-L10	CS-XII-Ch03-L10-3.2.1-Types-of-Functions-Built-in-vs-User-defined.pdf	3.2.1 Types of Functions (Built-in vs User-defined)
Ch03-L11	CS-XII-Ch03-L11-3.2.2-Functions-with-Return-Values.pdf	3.2.2 Functions with Return Values
Ch03-L12	CS-XII-Ch03-L12-3.2.3-Activity-Arithmetic-Calculator-Using-Functions.pdf	3.2.3 Activity: Arithmetic Calculator Using Functions
Ch03-L13	CS-XII-Ch03-L13-3.2.4-Scope-of-Local-and-Global-Variables.pdf	3.2.4 Scope of Local and Global Variables

Ch03-L14	CS-XII-Ch03-L14-3.3-3.4-File-Handling-and-Operations-Modes-r-w-a.pdf	3.3 & 3.4 File Handling and Operations (Modes r, w, a) ★
Ch03-L15	CS-XII-Ch03-L15-3.6-with-Statement-Context-Manager.pdf	3.6 with Statement (Context Manager) ★
Ch03-L16	CS-XII-Ch03-L16-3.8-Errors-and-Exceptions-Try-Except.pdf	3.8 Errors and Exceptions (Try-Except)
Ch03-L17	CS-XII-Ch03-L17-3.9-Mini-File-Handling-Project-Grade-Tracker.pdf	3.9 Mini File Handling Project: Grade Tracker
Lecture 04	CS-XII-Lecture-04-Data-and-Analysis.pdf	Data and Analysis
Ch04-L01	CS-XII-Ch04-L01-4.1-Concept-of-Data-Analysis.pdf	4.1 Concept of Data Analysis
Ch04-L02	CS-XII-Ch04-L02-4.1.1-4.1.2-Data-Sources-and-Database-Connection.pdf	4.1.1 – 4.1.2 Data Sources and Database Connection ★
Ch04-L03	CS-XII-Ch04-L03-4.2-Creating-a-SQLite-Database-in-Python.pdf	4.2 Creating a SQLite Database in Python
Ch04-L04	CS-XII-Ch04-L04-4.3-Basic-Operations-on-Data-using-Pandas.pdf	4.3 Basic Operations on Data using Pandas ★
Ch04-L05	CS-XII-Ch04-L05-4.3.3-Handling-Missing-Values-NaN.pdf	4.3.3 Handling Missing Values (NaN) ★
Ch04-L06	CS-XII-Ch04-L06-4.4-Data-Organization-and-Representation.pdf	4.4 Data Organization and Representation
Ch04-L07	CS-XII-Ch04-L07-4.5-Data-Visualization-Types-of-Graphs.pdf	4.5 Data Visualization & Types of Graphs ★
Ch04-L08	CS-XII-Ch04-L08-4.6-Descriptive-Statistics.pdf	4.6 Descriptive Statistics ★
Lecture 05	CS-XII-Lecture-05-Application-and-Impacts-of-Computing.pdf	Application and Impacts of Computing
Ch05-L01	CS-XII-Ch05-L01-5.1-Machine-Learning-Neural-Networks-Deep-Learning.pdf	5.1 Machine Learning, Neural Networks & Deep Learning
Ch05-L02	CS-XII-Ch05-L02-5.1.1-Machine-Learning-ML.pdf	5.1.1 Machine Learning (ML)
Ch05-L03	CS-XII-Ch05-L03-5.1.2-Neural-Networks-NN.pdf	5.1.2 Neural Networks (NN) ★

Ch05-L04	CS-XII-Ch05-L04-5.1.3-Deep-Learning-DL.pdf	5.1.3 Deep Learning (DL)
Ch05-L05	CS-XII-Ch05-L05-5.1.4-Components-of-Neural-Networks.pdf	5.1.4 Components of Neural Networks ★
Ch05-L06	CS-XII-Ch05-L06-5.1.5-Components-of-Deep-Learning.pdf	5.1.5 Components of Deep Learning
Ch05-L07	CS-XII-Ch05-L07-5.2-Applications-of-Neural-Networks-and-Deep-Learning.pdf	5.2 Applications of Neural Networks and Deep Learning
Ch05-L08	CS-XII-Ch05-L08-5.3-Secure-Collaboration-Authentication-Access-Control.pdf	5.3 Secure Collaboration (Authentication, Access Control) ★
Ch05-L09	CS-XII-Ch05-L09-5.3.2-Data-Protection-Encryption-Passwords-Backup-Firewall.pdf	5.3.2 Data Protection (Encryption, Passwords, Backup, Firewall)
Ch05-L10	CS-XII-Ch05-L10-5.4-Security-Threats-and-Mitigation-Techniques.pdf	5.4 Security Threats and Mitigation Techniques ★
Ch05-L11	CS-XII-Ch05-L11-5.4.2-Identify-Security-Threats.pdf	5.4.2 Identify Security Threats
Ch05-L12	CS-XII-Ch05-L12-5.4.3-Apply-Mitigation-Techniques.pdf	5.4.3 Apply Mitigation Techniques
Ch05-L13	CS-XII-Ch05-L13-5.5-Equity-and-Equal-Access-in-Digital-Collaboration.pdf	5.5 Equity and Equal Access in Digital Collaboration
Ch05-L14	CS-XII-Ch05-L14-5.5.1-Functions-of-Equity-and-Equal-Access.pdf	5.5.1 Functions of Equity and Equal Access ★
Ch05-L15	CS-XII-Ch05-L15-5.5.2-Collaboration-Tools.pdf	5.5.2 Collaboration Tools
Lecture 06	CS-XII-Lecture-06-Entrepreneurship-in-the-Digital-Age.pdf	Entrepreneurship in the Digital Age
Ch06-L01	CS-XII-Ch06-L01-6.1-Entrepreneur-and-Entrepreneurship.pdf	6.1 Entrepreneur and Entrepreneurship
Ch06-L02	CS-XII-Ch06-L02-6.2-Entrepreneurship-in-the-Digital-Age.pdf	6.2 Entrepreneurship in the Digital Age ★

Ch06-L03	CS-XII-Ch06-L03-Table-6.1-Local-examples-of-digital-entrepreneurship.pdf	Table 6.1 Local examples of digital entrepreneurship
Ch06-L04	CS-XII-Ch06-L04-6.3-From-Problem-to-Business-Idea.pdf	6.3 From Problem to Business Idea
Ch06-L05	CS-XII-Ch06-L05-6.4-Prototype.pdf	6.4 Prototype ★
Ch06-L06	CS-XII-Ch06-L06-6.5-Types-of-Prototypes-Low-Mid-High-Fidelity.pdf	6.5 Types of Prototypes (Low, Mid, High-Fidelity)
Ch06-L07	CS-XII-Ch06-L07-6.6-Prototype-Development-Cycle-Design-Build-Test-Iterate.pdf	6.6 Prototype Development Cycle (Design, Build, Test, Iterate) ★
Ch06-L08	CS-XII-Ch06-L08-6.7-Class-Activity-Create-a-Prototype.pdf	6.7 Class Activity: Create a Prototype
Ch06-L09	CS-XII-Ch06-L09-6.8-Minimum-Viable-Product-MVP.pdf	6.8 Minimum Viable Product (MVP) ★
Ch06-L10	CS-XII-Ch06-L10-6.9-Difference-between-Prototype-and-MVP.pdf	6.9 Difference between Prototype and MVP
Ch06-L11	CS-XII-Ch06-L11-6.10-Identifying-the-Riskiest-Assumption.pdf	6.10 Identifying the Riskiest Assumption
Ch06-L12	CS-XII-Ch06-L12-6.11-Developing-an-MVP.pdf	6.11 Developing an MVP
Ch06-L13	CS-XII-Ch06-L13-6.12-Case-Study-MVP-for-a-Canteen-Pre-Order-System.pdf	6.12 Case Study: MVP for a Canteen Pre-Order System
Ch06-L14	CS-XII-Ch06-L14-6.13-Testing-an-MVP.pdf	6.13 Testing an MVP
Ch06-L15	CS-XII-Ch06-L15-6.14-Beachhead-Market.pdf	6.14 Beachhead Market ★
Ch06-L16	CS-XII-Ch06-L16-6.15-6.16-Complete-Student-Project-Ethical-Use.pdf	6.15 & 6.16 Complete Student Project & Ethical Use
Lecture 07	CS-XII-Lecture-07-Final-Revision-and-Mock-Paper.pdf	Final Revision and Mock Paper

Lecture 01

Computer Systems (HCI)

Chapter lecture — 15 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

BIEK Computer Science XII · Bilingual support: English + اردو

بی آئی ای کے کمپیوٹر سائنس — بارہویں جماعت

15 book lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XII

How to use this BLEK study-guide lecture

Each PDF follows one textbook lecture from the BLEK / Sindh Computer Science notes (New Sindh Curriculum). Read *Learn it*, study the SVG diagram, try the worked example on paper, then answer *Check yourself* before the answers. **Extra board practice** (MCQs, short and long questions) is added after the book content — it is not in the textbook. Career-only extras that were not in the book lecture list have been removed. Math is rendered with MathJax. ★ Golden topics are high-yield in board exams. The green box repeats the idea in Urdu.

ہر لیکچر کتاب کے درس کے مطابق ہے۔ کتاب کے بعد اضافی MCQs اور سوالات ہیں۔ جو اضافی کیریئر صفحات کتاب کی فہرست میں نہیں تھے وہ ہٹا دیے گئے ہیں۔

Lectures in this pack

Chapter 1: Computer Systems (HCI). Study one lecture at a time, then use the chapter review and answer key at the end of this PDF.

L01. 1.1 Fundamentals of Human Computer Interaction (HCI)

L03. 1.2 Applications of HCI (Domains of Life)

L05. 1.3 Types of User Interaction

L07. 1.4 Importance of HCI

L09. 1.6 Need Analysis for Interface Design

L11. 1.8 Methods for Improving HCI

L13. 1.9.3 – 1.9.5 Wireframing, Figma, and Prototypes

L15. 1.11.2 Testing Methods (Usability, A/B, Automated)

L02. 1.1.1 – 1.1.2 Traditional vs Natural Interaction

L04. 1.3 Components of HCI

L06. 1.3 Interface Types, Environment & Feedback

L08. 1.5 Accessibility Principles

L10. 1.7 Human Computer Interaction Problems

L12. 1.9 User Interface Design (UI vs UX)

L14. 1.11 Test and Evaluate HCI (Evaluation Methods)

Computer Systems (HCI)

This chapter teaches you Human-Computer Interaction (HCI): how people and computers talk to each other, how to design screens that are easy and fair to use, and how to test those designs before they go live. You will apply every idea to things you already use in Pakistan — a bank ATM, JazzCash, and a school portal.

اس باب میں آپ ہیومن کمپیوٹر انٹرایکشن (HCI) سیکھیں گے: انسان اور کمپیوٹر آپس میں کیسے بات کرتے ہیں، اسکرینیں کیسے آسان اور منصفانہ بنائی جاتی ہیں، اور انہیں عوام تک پہنچانے سے پہلے کیسے جانچا جاتا ہے۔

What you will learn

- Define HCI (also called CHI and MMI) and name the five sensory channels of interaction.
- Describe HCI applications in health care, banking, education, and networking.
- Explain environment and the two kinds of feedback.
- Carry out a need analysis before designing an interface.
- Differentiate UI from UX, and compare wireframes, Figma work and prototypes.
- Differentiate traditional interaction from natural interaction with local examples.
- Name the components of HCI and classify types of user interaction and interface.
- State why HCI is important and apply five accessibility principles.
- Identify common HCI problems and methods for improving HCI.
- Choose evaluation and testing methods (SUS, NPS, usability, A/B, automated audits).

1.1 Fundamentals of Human Computer Interaction (HCI)

LEARN IT

Human-Computer Interaction (HCI) is the study and design of technology so that it **serves people**. An HCI system is easy to use, efficient, and responsive to human needs. HCI is also known as **CHI** (Computer-Human Interface) or **MMI** (Man-Machine Interaction). The three names mean the same idea: the meeting point of a person and a machine.

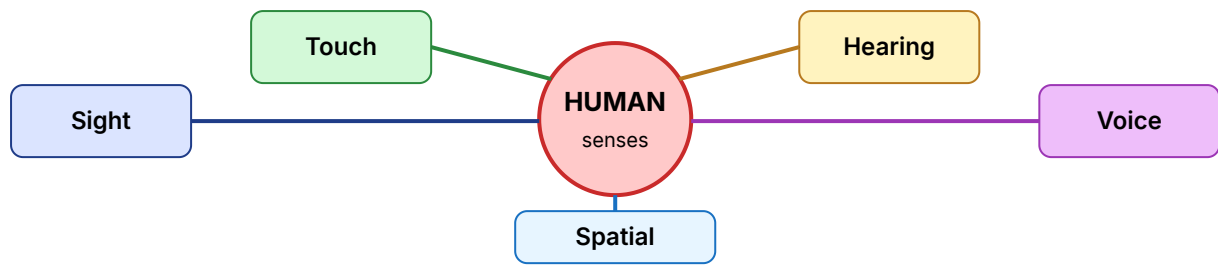
HCI is not only "making a pretty screen". It asks: Can a student in Hyderabad find results on the school portal in 30 seconds? Can a grandmother send JazzCash without help? Can a person standing at an outdoor ATM in bright sunlight read the buttons?

Sensory channels of interaction

Humans and computers communicate through **sensory channels**. A sensory channel is a path that carries information to (or from) one of the human senses. Computers must send and receive signals on the same channels, or the conversation breaks.

Channel	How humans use it	How computers respond
Sight (بصارت)	Look at screens, graphics or videos.	Displays visuals on monitors and screens.
Touch (چھونا)	Tap, type or swipe.	Responds with clicks or vibrations (haptics).
Hearing (سننا)	Listen to sounds or voices.	Provides audio feedback via speakers.
Voice (آواز)	Speak commands (for example, "Alexa").	Processes speech and replies.
Spatial (مکانی)	Move in physical space.	Tracks position or body movement (for example, VR).

Haptics means physical feedback you can feel, such as a phone vibrating when a JazzCash payment succeeds. **Spatial** interaction means the system notices where your body is, as in a VR headset or a Kinect camera.



Five sensory channels meet at the human. HCI design must support the channels the user actually needs.

WORKED EXAMPLE

Worked example: map a JazzCash payment to channels. Sara in Sukkur sends Rs 500 to her brother.

1. **Sight:** she looks at the JazzCash home screen and the amount field.
2. **Touch:** she taps "Send Money" and types the mobile number.
3. **Voice (optional):** she could say "JazzCash, send five hundred rupees" if voice commands are on.
4. **Hearing + haptics:** a beep and a short vibration confirm "Payment successful".

If the app showed no tick, no sound and no vibration, Sara would not know whether the money left. That is an HCI failure, not a banking failure.

REMEMBER IT

Remember the three names: HCI = CHI = MMI. Think "Human meets Machine." Five channels: **S-T-H-V-S** — Sight, Touch, Hearing, Voice, Spatial. "See The Handy Voice in Space."

COMMON MISTAKE

Mistake: Writing "HCI is only about computer hardware" or mixing up Voice with Hearing.

Fix: HCI is about the *design of the conversation* between people and machines. Hearing is what you *listen to* (speakers). Voice is what you *speak* (microphone + speech software).

اردو میں سمجھیں

بیومن کمپیوٹر انٹریکشن (HCI) کا مطلب ہے ٹیکنالوجی کو اس طرح ڈیزائن کرنا کہ وہ انسانوں کے لیے فائدہ مند، آسان اور مؤثر ہو۔ اسے CHI یا MMI بھی کہتے ہیں۔ انسان اور کمپیوٹر پانچ حسّی راستوں سے بات کرتے ہیں: بصارت، چھونا، سنا، آواز اور مکانی حرکت۔ ہیپٹکس (Haptics) وہ ارتعاش ہے جو آپ کو چھو کر محسوس ہوتا ہے۔

CHECK YOURSELF

1. Give two other names for HCI.
2. Which channel tracks body movement in VR?
3. Is a phone vibration sight, touch (haptics), or voice?

Answers: 1. CHI (Computer-Human Interface) and MMI (Man-Machine Interaction). 2. Spatial.
3. Touch / haptics.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. HCI studies:

- a. How people and computers work together
- b. Only CPU clocks
- c. Only NAND
- d. Only SQL keys

MCQ 2. Another name mentioned is:

- a. CHI / MMI
- b. MVP
- c. SDLC only
- d. K-map

MCQ 3. Sensory channels include:

- a. Sight, touch, hearing, voice, spatial
- b. Only smell in this course
- c. Only taste
- d. Only OSI layer 7

MCQ 4. Good HCI makes systems:

- a. Usable, efficient, fitting human needs
- b. Harder on purpose
- c. Mouse-only always
- d. Text in 4pt always

MCQ 5. Haptics are:

- a. Touch feedback (vibration)
- b. Pie charts
- c. Foreign keys
- d. Sprints

MCQ 6. VR tracking is a:

- a. Spatial channel
- b. Primary key
- c. Bubble sort
- d. Histogram

Extra short questions

1. Define HCI.
2. Five sensory channels from the notes.
3. Give a computer response for touch and for voice.
4. Why HCI is not "just drawing a pretty screen".

Extra long question

1. Tabulate the five channels with a human action and a computer response. Then explain one device that uses three channels at once.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Human-Computer Interaction: design of technology around people.
2. Sight, touch, hearing, voice, spatial.
3. Touch: click/haptic. Voice: speech recognition and spoken reply.
4. It includes users, tasks, environment, feedback, accessibility.

Long

1. Smartphone: sight (GUI), touch (tap/haptic), hearing (ringtone), optional voice assistant. Closing: if one channel fails, another should still work (accessibility).

1.1.1 – 1.1.2 Traditional vs Natural Interaction

LEARN IT

1.1.1 Traditional interaction is the basic way people use everyday devices. It relies on standard input and output: a mouse, a keyboard, a keypad, a monitor, speakers, or a simple touchscreen tap and swipe.

- **Computer:** moving a mouse and typing on a keyboard; output via monitor and speakers.
- **Smartphone (traditional use):** tapping and swiping on a touchscreen; output via screen lighting and vibrations.

1.1.2 Natural interaction is advanced, smart technology that lets people interact **without a mouse or keyboard**. It feels more human — you speak, move, or show your face, and the system understands.

- **Voice:** speaking commands into a device (for example, "Alexa, play a song").
- **Body and gesture:** facial recognition to unlock a phone, or playing VR games by waving hands.

Feature	Traditional	Natural
Main input	Mouse, keyboard, keypad, simple tap/swipe	Voice, face, hand/body gesture, movement
Feels like	Operating a machine	Talking or moving as you do with people
Examples	TV remote, ATM keypad, school-portal login typed on a PC	FaceID, Siri, Kinect games, "Hey Google"
Typical output	Screen, speakers, vibration	Spoken reply, unlock, tracking of the body in VR

WORKED EXAMPLE

Worked example: sort today's tools. List three traditional and three natural interfaces you might use in one day in Sindh.

Traditional: (1) HBL ATM keypad to withdraw cash. (2) College computer lab: mouse + keyboard to open the school portal. (3) Television remote at home.

Natural: (1) Unlocking a phone with FaceID. (2) Asking Google Assistant "Call Aslam". (3) A VR science demo in which you pick a molecule by waving your hand.

Notice: the *same smartphone* can be traditional (you type a PIN) or natural (the camera reads your face). The classification depends on **how you interact**, not on the brand of the device.

REMEMBER IT

Test sentence: If you must press keys or tap labelled buttons, it is traditional. If the machine understands speech, face or body movement, it is natural.

COMMON MISTAKE

Mistake: "Touchscreens are always natural interaction."

Fix: Ordinary tapping and swiping on a phone is still listed as traditional in this chapter. Natural interaction starts when the system understands voice, face, or full-body gesture without a mouse or keyboard.

اردو میں سمجھیں

روایتی تعامل (Traditional Interaction) میں ماؤس، کی بورڈ، کی پیڈ یا عام ٹیپ اور سوائپ استعمال ہوتے ہیں، جیسے اے ٹی ایم کی پیڈ۔ قدرتی تعامل (Natural Interaction) میں آواز، چہرہ یا جسم کی حرکت سے کام ہوتا ہے، جیسے فیس آئی ڈی (FaceID) یا "الیکسا، گانا چلاؤ"۔ درجہ بندی آلے کی نہیں، تعامل کے طریقے کی ہوتی ہے۔

CHECK YOURSELF

1. Is an ATM keypad traditional or natural?
2. Give one voice example of natural interaction from the source.
3. Why can one smartphone be both traditional and natural?

Answers: 1. Traditional. 2. "Alexa, play a song" (or any spoken command). 3. Because typing a PIN is traditional, while FaceID or a voice assistant on the same phone is natural — it depends on the method, not the device.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Traditional interaction uses:

- a. Standard controls — keys, mouse, menus
- b. Gaze and gesture as the default 1980 PC
- c. Smell
- d. Telepathy

MCQ 2. Natural interaction tries to:

- a. Use human skills (speech, gesture, touch) with less training
- b. Force people to learn command codes only
- c. Remove all feedback
- d. Use only punch cards

MCQ 3. A command-line is more:

- a. Traditional / learned
- b. Natural baby-talk
- c. IoT soil
- d. A pie

MCQ 4. Pinch-to-zoom on a phone is more:

- a. Natural (gesture)
- b. A foreign key
- c. A K-map
- d. Waterfall

MCQ 5. Both still need:

- a. Feedback so the user knows the system heard them
- b. No UI
- c. No errors ever
- d. No accessibility

MCQ 6. Alexa-style speech is:

- a. Natural-ish voice interaction
- b. A datasheet
- c. A histogram
- d. A stack pop

Extra short questions

1. Define traditional vs natural interaction.
2. One example each in a college.
3. A drawback of “natural”.
4. Can a product mix both?

Extra long question

1. Design a library search: one traditional UI and one natural UI. Compare learnability, errors, and accessibility.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Traditional: trained widgets (keyboard/mouse). Natural: speech, gesture, touch that feel like the real world.
2. Traditional: SIS forms. Natural: fingerprint attendance, touch kiosk.
3. False gesture recognition; privacy of voice; not natural for every disability.
4. Yes — a phone has icons (traditional GUI) and gestures/voice.

Long

1. Traditional: typed query + filters. Natural: speak the title / point at a shelf QR.
Learnability vs noisy halls. Accessibility: keyboard vs voice. Recommend both.

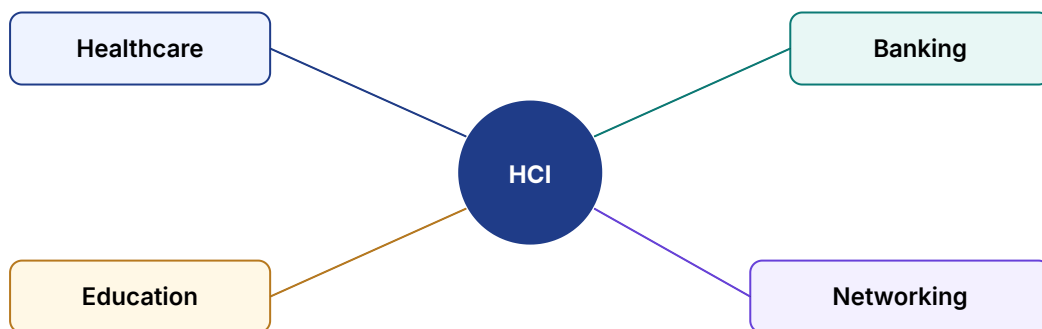
1.2 Applications of HCI (Domains of Life) ★ Golden

LEARN IT

HCI is used to design and develop interactive systems such as **desktop applications, websites and mobile apps**. These are **human-centred solutions** built to meet user needs across many domains of life. A domain is an area of daily activity — health, money, school, talking to people.

When you design for a domain, you design for that domain's **users, tasks and risks**. A mistake on a games app is annoying. A mistake on a medical or banking screen can harm a person or lose money. That is why examiners treat this topic as golden.

Domain	Application examples	User benefit
Health care	Patient portals, telemedicine apps, assistive technologies	Improves access to medical services remotely
Banking	Mobile banking apps, secure ATM interfaces	Enables quick transactions and secure financial management
Education	E-learning platforms (LMS), online exam portals	Enhances teaching, learning and student engagement
Networking and communication	Social media, messaging apps (WhatsApp), Zoom	Connects people globally through collaboration tools



HCI sits at the centre of four everyday domains: healthcare, banking, education and networking.

WORKED EXAMPLE

Worked example: the same HCI idea in three Pakistani systems.

Step 1 — ATM (banking). You insert a card, the screen shows large buttons (Fast Cash, Balance, Other), you enter a PIN on the keypad, and you take cash. HCI benefit: a quick, secure transaction without a human teller.

Step 2 — JazzCash (banking + communication). You log in with a PIN or fingerprint, choose Send Money, type a number and amount, confirm, and see "Successful" with a JazzCash SMS. HCI benefit: money moves in seconds from a shop in Larkana to a student in Karachi.

Step 3 — School portal (education). A Grade XII student opens the college LMS, taps "Download date sheet", and submits a leave form. HCI benefit: learning and admin tasks happen without standing in a queue at the office.

Step 4 — Compare the user benefit. Banking HCI saves time and keeps money safe. Education HCI raises engagement. Health HCI (a Sehat/telemedicine app) brings a doctor to a village phone. Networking HCI (WhatsApp, Zoom) keeps a family in Dubai in touch with parents in Sukkur.

REMEMBER IT

Exam mnemonic H-BEN: Health, Banking, Education, Networking. For each, write: one application + one user benefit. That structure scores full marks on "applications of HCI".

COMMON MISTAKE

Mistake: Listing brand names only ("WhatsApp, JazzCash, Zoom") without saying the *domain* and the *user benefit*.

Fix: Always write three parts: domain → example → what the user gains. "Banking: JazzCash mobile app — enables quick, secure transfers."

اردو میں سمجھیں

ایچ سی آئی کی ایپلیکیشنز صحت، بینکنگ، تعلیم اور نیٹ ورکنگ کے شعبوں میں انسانی ضروریات پوری کرتی ہیں۔ صحت میں مریض پورٹل اور ٹیلی میڈیسن دور سے علاج دیتے ہیں، بینکنگ میں موبائل ایپ اور اے ٹی ایم تیز اور محفوظ لین دین دیتے ہیں، تعلیم میں ای لرننگ اور آن لائن امتحان مشغولیت بڑھاتے ہیں، اور واٹس ایپ یا زوم لوگوں کو جوڑتے ہیں۔

EXAM TIP

Very common 5–8 mark question: "Explain applications of HCI in different domains of life with examples." Draw a four-row table (Health care, Banking, Education, Networking & communication). Each row: one or two application names + the user benefit from the source. Add a local line (ATM / JazzCash / school portal) to show you can apply the idea. Do not write a story about only one app.

CHECK YOURSELF

1. Name the four domains of HCI given in the source.
2. What user benefit does HCI bring in banking?
3. Give one education application and one networking application.

Answers: 1. Health care, banking, education, networking and communication. 2. Quick transactions and secure financial management. 3. Education: LMS or online exam portal. Networking: social media, WhatsApp or Zoom.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. HCI applies to:

- a. Desktops, web, mobile, kiosks, embedded, VR
- b. Only mainframes in 1970
- c. Only NAND trainers
- d. Only paper

MCQ 2. A hospital infusion pump UI is HCI in:

- a. Health / safety-critical life
- b. Sports memes only
- c. Fashion only
- d. OSI cabling only

MCQ 3. Education HCI example:

- a. LMS that students can actually submit homework on
- b. A random 4pt PDF
- c. An unlabelled form
- d. A hidden button

MCQ 4. Bad HCI in a bank app can:

- a. Make people send money to the wrong account
- b. Speed bubble sort
- c. Fix NAND
- d. Change Big O of Python

MCQ 5. Domains of life in the book include:

- a. Work, health, education, home, public services
- b. Only gaming
- c. Only compilers
- d. Only ER

MCQ 6. This is ★ because you must:

- a. Name domains with a concrete interface each
- b. Draw K-maps
- c. Write SQL only
- d. List sprints

Extra short questions

1. Five domains of HCI application.
2. Why HCI in medical devices is life-critical.
3. A public-service example in Pakistan.
4. What "application of HCI" means in a 3-mark.

Extra long question

1. Pick four domains. For each: one system, one user, one HCI requirement. ★ Golden 10-mark.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Work software, health devices, education, home/consumer, government/public kiosks, transport...
2. A confusing unit (mg vs ml) can dose wrongly.
3. NADRA/passport kiosks, metro ticket machines.
4. Not the definition — a place + the interface + why users matter.

Long

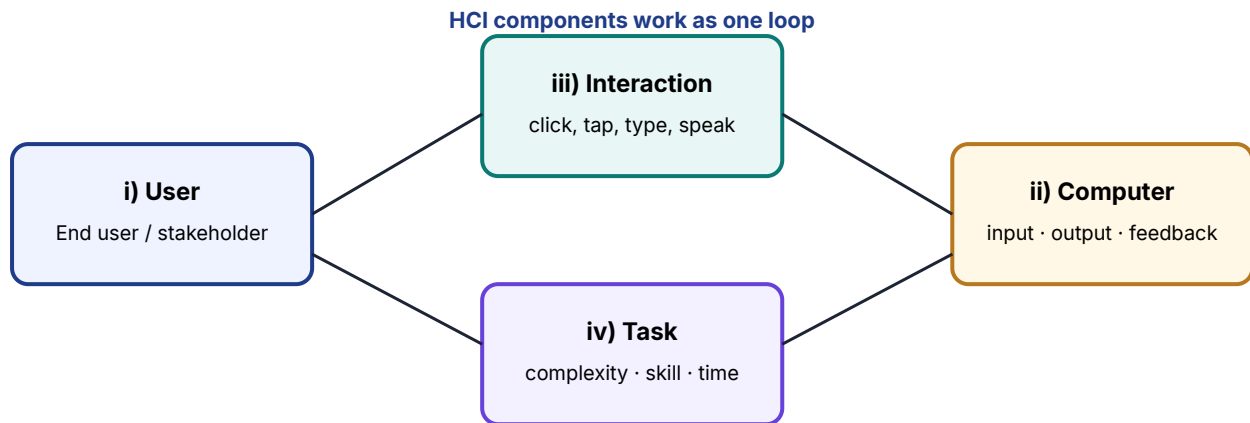
1. Clinic: nurse, glanceable alarms. College portal: student, language + mobile. ATM: elderly, privacy hood + large text. Car dash: driver, eyes-up. Table.

1.3 Components of HCI

LEARN IT

HCI relies on several primary components that work together. If one component is weak, the whole interaction feels broken. The first four are the people, the machine, the conversation, and the job to be done.

	Component	Meaning
i	User (صارف)	Any individual or group that interacts with a computer system to achieve defined goals. Two categories: end users (people who directly operate the system, from beginners to experts) and stakeholders (people with an interest in the outcome, such as managers and clients).
ii	Computer system (کمپیوٹر سسٹم)	The platform where humans and computers communicate through input, output and feedback using an interface. It connects human actions with machine responses.
iii	User interaction (صارف کا تعامل)	The process through which a user communicates with a system by performing actions (clicking, tapping, typing, speaking). It transforms user intent into system responses.
iv	Task (کام)	The specific action users perform (for example, booking a ticket). It has three characteristics: task complexity (filling a form vs designing a 3D model), user knowledge (basic typing vs advanced programming), and time requirement (ATM withdrawal vs online passport application).



The user performs a task through interaction with a computer system. All four parts must fit together.

WORKED EXAMPLE

Worked example: label the four components of an HBL ATM withdrawal.

1. **User:** end user = the account holder standing at the machine; stakeholder = the bank that owns the ATM and must keep it safe and working.
2. **Computer system:** the ATM hardware plus the bank's software; it takes PIN input, shows a screen, and dispenses notes (output and feedback).
3. **User interaction:** insert card → type PIN → choose "Cash" → type amount → confirm. Intent ("I want Rs 5,000") becomes a machine response (cash + receipt).
4. **Task:** withdraw cash. Complexity: low (a few buttons). User knowledge: basic (PIN and amounts). Time: about one minute — much shorter than an online passport application.

REMEMBER IT

U-C-I-T: User, Computer system, Interaction, Task. "U Can Interact with Tasks." Then add three task questions: How hard? What skill? How long?

COMMON MISTAKE

Mistake: Calling the bank manager an "end user" of the ATM, or saying a task has only one property (difficulty).

Fix: The manager is usually a **stakeholder**. The person pressing the keys is the **end user**. A task always has complexity, knowledge and time — mention all three in long answers.

ایچ سی آئی کے چار بنیادی اجزاء ہیں: صارف (اینڈ یوزر جو خود سسٹم چلاتا ہے اور اسٹیک ہولڈر جس کا نتیجے میں مفاد ہے)، کمپیوٹر سسٹم (ان پٹ، آؤٹ پٹ اور فیڈبیک کا پلیٹ فارم)، صارف کا تعامل (کلک، ٹیپ، ٹائپ، بولنا) اور کام (ٹاسک) جس کی پیچیدگی، مطلوبہ مہارت اور وقت مختلف ہوتا ہے۔ اے ٹی ایم سے رقم نکالنا آسان اور مختصر کام ہے؛ پاسپورٹ کی آن لائن درخواست لمبا کام ہے۔

CHECK YOURSELF

1. What is the difference between an end user and a stakeholder?
2. Name the three characteristics of a task.
3. In a school portal, who is a stakeholder?

Answers: 1. End users directly operate the system; stakeholders have an interest in the outcome (managers, clients) but may not click the buttons. 2. Task complexity, user knowledge, time requirement. 3. The principal, IT admin or board office — they care about results even if students are the end users.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. The user in HCI is:

- a. The human with goals, limits, skills
- b. The CPU
- c. The database
- d. The sprint log

MCQ 2. The task is:

- a. What the user is trying to get done
- b. The Wi-Fi password
- c. The NAND count
- d. The pie flavour

MCQ 3. The context/environment is:

- a. Where/when/noise/light/stress
- b. Only RGB of the logo
- c. Only Big O
- d. Only SQL

MCQ 4. Missing any component in design causes:

- a. HCI problems
- b. Faster CPUs
- c. Better keys
- d. Shorter OSI

MCQ 5. A tired nurse at night is part of:

- a. User + environment
- b. Only the algorithm
- c. Only the DBMS
- d. Only Figma auto-layout

MCQ 6. Components work:

- a. Together
- b. In isolation always
- c. Only in waterfall phase 6
- d. Only after MVPs ship

Extra short questions

1. List the primary HCI components from the notes.
2. Give a college-lab example of each.
3. What if you design only for an expert user?
4. Why environment?

Extra long question

1. Analyse JazzCash (or a similar app) using HCI components. Where might it fail for a first-time Urdu user in bright sunlight?

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. User, task, interface, environment/context, feedback (and system).
2. User: XI student. Task: submit practical. UI: LMS upload. Env: noisy lab, short period. Feedback: "uploaded" tick.
3. Beginners fail — a typical HCI problem.
4. A UI that needs silence fails in a crowded canteen kiosk.

Long

1. User literacy/language; task send-money; UI contrast; outdoor glare; feedback of "success". Recommend larger type, Urdu toggle, confirmation screen.

1.3 Types of User Interaction

LEARN IT

User interaction can happen in many different ways depending on the technology used. Learn every type with one clean example — board questions often ask you to "list and explain" five or more.

Interaction type	Description	Examples
Direct manipulation	Handling digital objects on a screen exactly like physical objects.	Dragging files, zooming images with fingers
Command-based	Giving precise instructions through typed text commands.	MS-DOS, Linux terminal (<code>dir</code> , <code>cd ..</code>)
Menu-driven	Selecting from predefined lists or options instead of typing.	Clicking "Start Test" or "Download" on a portal
Form fill	Entering structured data into blank text fields.	Sign-up forms, ticket booking screens
Conversational	Communicating with systems using natural human language (text or voice).	Chatbots, ChatGPT, Google Assistant
Gesture-based	Using body or hand movements to control systems.	Swiping touchscreens, VR hand gestures
Voice interaction	Speaking commands directly to the system.	"Call Aslam" on a smartphone
Context-aware	Systems sense the environment and act automatically without a prompt.	Smart lighting turning on when you enter a room
Biometric	Using biological features for interaction and authentication.	Fingerprint unlock, FaceID
Haptic and tactile	Providing physical feedback to confirm actions.	Phone vibration when typing
Multimodal	Combining two or more interaction types together.	Voice + touch (speak a search, then tap)

WORKED EXAMPLE

Worked example: a full JazzCash "Send Money" session, labelled by type.

1. Open the app with a fingerprint → **biometric**.
2. Home screen shows Send Money, Bill Payment, Load → you tap Send Money → **menu-driven**.
3. Type the receiver's mobile number and the amount → **form fill**.
4. The keypad vibrates on each tap → **haptic**.
5. Optional: say "Send five hundred to Ali" then tap Confirm → **multimodal** (voice + touch).

Same idea on a **school portal**: menu-driven ("Date Sheet", "Fee Challan"), form fill (leave application), conversational (a chatbot "When is the chemistry practical?").

REMEMBER IT

Group them: hands on screen (direct, gesture, haptic) · words (command, conversational, voice) · lists and blanks (menu, form) · automatic/body (context-aware, biometric) · mix (multimodal).

COMMON MISTAKE

Mistake: Treating "voice interaction" and "conversational" as identical, or calling a dropdown menu "command-based".

Fix: Voice is spoken commands only. Conversational includes typed chat (ChatGPT) as well as voice. Command-based means you type precise codes such as `dir`, not clicking a menu.

اردو میں سمجھیں

تعامل کی اقسام میں براہ راست ہیرا پھیری، کمانڈ، مینو، فارم، گفتگو (چیٹ بوٹ)، اشارہ، آواز، سیاق و سباق سے آگاہ نظام، بائیومیٹرک، بیپیٹک اور ملٹی موڈل شامل ہیں۔ جاز کیش میں فنگر پرنٹ بائیومیٹرک ہے، "رقم بھیجیں" مینو ہے، نمبر لکھنا فارم ہے، اور آواز کے ساتھ ٹیپ ملٹی موڈل ہے۔

CHECK YOURSELF

1. Which type is a Linux `cd ..` command?
2. Smart lighting that turns on when you enter a room is which type?
3. Why is speaking a search and then tapping a result called multimodal?

Answers: 1. Command-based. 2. Context-aware. 3. Because it combines two or more types (voice + touch) in one task.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. GUI means:

- a. Graphical User Interface
- b. Great Universal Internet
- c. Gate Unary Inverter
- d. Grouped Unique Index

MCQ 2. CLI is:

- a. Command-line (typed commands)
- b. Only touch
- c. Only VR
- d. Only pie menus of 1990 Mac always

MCQ 3. Voice UI example:

- a. Siri / Alexa / Google Assistant
- b. A datasheet
- c. A K-map
- d. A foreign key

MCQ 4. Gesture UI example:

- a. Swipe, pinch, Wii/Kinect-style move
- b. SQL SELECT
- c. Bubble swap
- d. HTTP GET only

MCQ 5. The best interaction type depends on:

- a. User, task, environment
- b. Whatever is newest
- c. Always voice
- d. Always CLI

MCQ 6. A surgeon in a sterile field may prefer:

- a. Voice/gesture over touch
- b. A dusty mouse
- c. Tiny 4pt CLI
- d. Captcha on the pump

Extra short questions

1. Name four interaction types with examples.
2. One strength of CLI.
3. One weakness of voice.
4. Can one app offer two types?

Extra long question

1. Recommend an interaction type for: (a) programmer (b) ATM (c) car while driving (d) museum kiosk. Justify with user+environment.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. GUI desktop; CLI terminal; touch; voice; gesture; AR/VR — any four.
2. Fast and scriptable for experts.
3. Noise, privacy, accents, no visual scan of options.
4. Yes — VS Code GUI + terminal; phones GUI + voice.

Long

1. (a) GUI+CLI (b) GUI+keys, privacy (c) voice/steering-wheel, eyes up (d) large touch, language choice. Table.

1.3 Interface Types, Environment & Feedback

LEARN IT

Components v–vii complete the HCI picture: the look of the controls, the place where you use them, and the signals that tell you "it worked" or "try again".

v) User interface (UI)

The **user interface** is the medium through which humans interact with computer systems.

- **Graphical User Interface (GUI):** visual elements such as windows, icons and menus (Windows OS, Android).
- **Command-Line Interface (CLI):** text-based typing (MS-DOS, Linux terminal).
- **Touch interface:** tapping, swiping, pinching (smartphones, tablets).
- **Voice User Interface (VUI):** spoken commands (smart speakers).
- **Natural User Interface (NUI):** gestures, body movements or immersive environments (VR headsets, Microsoft Kinect).

vi) Environment

The **environment** is the real-world setting where a person uses a computer system. It is shaped by:

- **Physical conditions** — an ATM outdoors in sunlight, heat, noise, rain.
- **Social conditions** — a quiet classroom, a crowded bazaar, privacy at a PIN pad.
- **Technical hardware** — slow college Wi-Fi, a cracked phone screen, an old browser.

vii) Feedback

Feedback is the signal or response given to confirm or guide actions.

System feedback

Response from the computer: a "Loading..." bar, a "Message Sent!" tick, or an error sound.

User feedback

Input from the user to the system: tapping a "Submit" button, giving a 5-star rating.

WORKED EXAMPLE

Worked example: environment and feedback at a roadside ATM in Hyderabad.

Physical: noon sunlight washes out a dim screen — the GUI must use high contrast.

Social: people stand in a queue; the user needs a privacy shield around the PIN pad.

Technical: the printer may be out of paper, so on-screen "Take your cash" plus a beep still has to confirm the transaction.

System feedback: "Please wait", cash coming out, a printed or SMS receipt.

User feedback: the user presses Confirm, or later rates the bank's app 4 stars.

On a **school portal** during board-exam week, hundreds of students on weak Wi-Fi need a visible "Loading results..." bar. Silence after clicking "View marks" is poor system feedback.

REMEMBER IT

GUI vs CLI vs NUI: GUI = pictures you click. CLI = words you type. NUI = body you move. VUI = voice you speak. Touch = fingers on glass.

Feedback rule: **system talks back; user talks in.**

COMMON MISTAKE

Mistake: Mixing up system feedback with user feedback, or saying "environment" means only the software theme (dark mode).

Fix: A tick mark is system feedback. A 5-star rating is user feedback. Environment is the *real world* around the user — sun, noise, hardware — not the colour of the app (though a good UI will adapt to that world).

اردو میں سمجھیں

یوزر انٹرفیس وہ واسطہ ہے جس سے انسان سسٹم سے بات کرتا ہے: GUI تصاویر اور مینو، CLI لکھی کمانڈ، ٹچ، آواز (VUI) اور قدرتی اشارے (NUI)۔ ماحول وہ حقیقی جگہ ہے جہاں سسٹم استعمال ہوتا ہے — دھوپ میں اے ٹی ایم، خاموش کلاس روم، کمزور وائی فائی۔ سسٹم فیڈبیک مشین کا جواب ہے ("بھیج دیا گیا")؛ صارف کا فیڈبیک صارف کا ان پٹ ہے (سبمٹ بٹن یا پانچ ستارے)۔

CHECK YOURSELF

1. Name five types of user interface given in the source.
2. Give one physical and one social example of environment.
3. Is a "Message Sent" tick system feedback or user feedback?

Answers: 1. GUI, CLI, Touch, VUI, NUI. 2. Physical: ATM in sunlight. Social: a quiet classroom.
3. System feedback.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Feedback is:

- a. The system telling the user what happened
- b. A student evaluation form only
- c. A foreign key
- d. A sprint retrospective only

MCQ 2. A spinner/progress bar is:

- a. Feedback that work is ongoing
- b. A primary key
- c. A sort
- d. A NAND

MCQ 3. No feedback after Save causes:

- a. Double clicks / lost trust
- b. Faster CPUs
- c. Better Big O
- d. Shorter OSI

MCQ 4. The environment includes:

- a. Noise, light, device, social setting
- b. Only CSS colours
- c. Only RAM
- d. Only Agile

MCQ 5. A GUI button should look:

- a. Clickable (affordance) and then respond
- b. Like body text
- c. Invisible
- d. Like a pie

MCQ 6. Error messages should:

- a. Say what went wrong and what to do
- b. Say ERROR 0xUnknown only
- c. Be in 4pt grey
- d. Vanish in 0.1s

Extra short questions

1. Define interface and feedback.
2. Three feedback channels.
3. Example of poor environment–UI fit.
4. What is an affordance?

Extra long question

1. Audit the college portal login: environment, UI type, three feedback events (success, fail, loading). Rewrite one bad message.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. UI: the medium of interaction. Feedback: system response the user can sense.
2. Visual (message), audio (beep), haptic (vibrate).
3. Voice-only help in a noisy canteen; tiny text on a sunny ticket machine.
4. The look that suggests how to use a control (a button looks pressable).

Long

1. Lab noise/shared PCs. GUI form. Success: "Welcome Ali". Fail: "Wrong password — try again or reset". Loading: spinner. Bad: "Error 500" → "Server busy, wait 1 min or see the lab attendant".

1.4 Importance of HCI ★ Golden

LEARN IT

HCI ensures that machines are approachable and usable, turning complex technology into everyday tools. **Good design** makes technology simple and enjoyable (touchscreens). **Poor design** causes confusion and risks (unclear buttons on medical devices).

This is why HCI is not optional decoration. A confusing JazzCash confirm button can send money to the wrong number. A confusing school-portal "Submit" can lock a student out of an online exam. Importance questions in the board exam expect the five implications below, each with a sentence of explanation.

Implication	Description
1. Enhance productivity	Well-designed interfaces help users work faster, make fewer mistakes, and achieve goals efficiently.
2. Promotes inclusivity	Accessibility features (screen readers, voice commands) empower marginalized groups and ensure equal use of technology for everyone.
3. Drives innovation	Emerging tech such as AR/VR, AI and IoT rely strictly on solid HCI principles for people to actually adopt and use them.
4. Economic impact	User-friendly products gain higher adoption rates, customer loyalty and market success, driving profits.
5. Ethical responsibility	Responsible design ensures technology respects user privacy, reduces bias and supports human well-being.

WORKED EXAMPLE

Worked example: good vs poor HCI, same task.

Task: send Rs 2,000 through JazzCash to a parent.

Good HCI: large "Send Money" button, amount shown in words and numbers, last 4 digits of the number repeated, fingerprint confirm, green "Successful" plus SMS. Result: fast, few mistakes, the user trusts the app (productivity + economic loyalty + ethics of clear consent).

Poor HCI: tiny grey buttons, no amount-in-words, no confirm screen, no tick. Result: wrong number, no idea if it worked, user deletes the app and goes back to a bank queue.

Medical contrast from the source: unclear buttons on a medical device are not just ugly — they are dangerous. That is why ethical responsibility sits in this table.

REMEMBER IT

P-I-I-E-E: Productivity, Inclusivity, Innovation, Economic impact, Ethics. Sentence: "Please Include Innovative, Ethical Earnings."

COMMON MISTAKE

Mistake: Writing only "HCI is important because it makes apps look nice."

Fix: Looks are a tiny part of UI. Importance answers must name productivity, inclusivity, innovation, money and ethics. Quote the medical-device warning if you have time.

اردو میں سمجھیں

اچھا ایچ سی آئی پیچیدہ ٹیکنالوجی کو روزمرہ کا آسان اوزار بناتا ہے؛ برا ڈیزائن الجھن اور خطرہ پیدا کرتا ہے، جیسے طبی آلے پر غیر واضح بٹن۔ پانچ مضمرات: پیداواری بڑھتی ہے، شمولیت آتی ہے، جدت اپنائی جاتی ہے، معیشت مضبوط ہوتی ہے، اور رازداری و انصاف کی اخلاقی ذمہ داری پوری ہوتی ہے۔

EXAM TIP

Typical wording: "Why is HCI important? Discuss its implications." (8 marks) Write a short opening (good design vs poor design) then five numbered points with one example each. Local examples that earn marks: ATM PIN privacy (ethics), screen reader on a school portal (inclusivity), JazzCash one-tap send (productivity), a bank's app beating a rival because it is easier (economic), AR classroom app that students actually use (innovation). Do not skip ethics — examiners look for privacy and well-being.

CHECK YOURSELF

1. Give one example of poor design mentioned in the source.
2. Which implication is about screen readers and equal use?
3. How does HCI create economic impact?

Answers: 1. Unclear buttons on medical devices. 2. Promotes inclusivity. 3. User-friendly products get higher adoption, loyalty and profits.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. HCI matters because:

- a. It turns complex machines into usable tools
- b. It makes CPUs clock faster by magic
- c. It replaces maths
- d. It deletes ethics

MCQ 2. Usability includes:

- a. Effectiveness, efficiency, satisfaction
- b. Only colour
- c. Only NAND count
- d. Only price

MCQ 3. Poor HCI costs:

- a. Errors, time, accidents, abandoned products
- b. Nothing
- c. Only printer ink
- d. Only font licences

MCQ 4. Good HCI can:

- a. Widen who can use the system (accessibility)
- b. Reduce RAM physically
- c. Change OSI to 2 layers
- d. Remove the need for passwords always

MCQ 5. This is ★: expect a question:

- a. Why HCI is important — 3–5 points with examples
- b. Trace a K-map
- c. Write bubble sort
- d. Normalise to 5NF

MCQ 6. A usable result portal:

- a. Students find the mark without a tutorial
- b. Hides the mark in 12 menus
- c. Uses 4pt text
- d. Needs a CS degree

Extra short questions

1. Three reasons HCI is important.
2. Link to a Pakistani e-service.
3. HCI vs pretty graphics.
4. One safety example.

Extra long question

1. Write a 10-mark essay: importance of HCI with five points, each with a local example.
★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Fewer errors; faster tasks; more users included; safer devices; product success.
2. If the tax/passport site is confusing, people pay agents — HCI failure with social cost.
3. A pretty unusable app still fails; a plain clear form can succeed.
4. Car dashboards / medical pumps — confusion harms people.

Long

1. Learnability (first-year portal); efficiency (JazzCash); safety (clinic software); accessibility (screen reader); business (app store ratings). Short intro/conclusion.

1.5 Accessibility Principles

LEARN IT

Accessibility principles are HCI guidelines that ensure digital systems are usable by **all** individuals, including people with diverse physical, sensory and cognitive abilities. They remove barriers to create equal access.

Accessibility is not a "special extra". It is how you make sure a student with low vision, a shopkeeper who cannot use a mouse, or a colour-blind driver can still complete the task.

Principle	What it means	Example
1. High contrast (واضح رنگ)	Make text and pictures stand out clearly against the background.	Black text on a white background in a railway app, readable in bright sunlight.
2. Readable fonts (پڑھنے میں آسان تحریر)	Clear, simple fonts with proper size and spacing.	A school website using large Calibri so homework instructions are easy to read.
3. Colour-blind friendly design (رنگوں کی پہچان)	Do not depend <i>only</i> on colours; also use symbols or text labels.	A traffic app showing a red circle with a "STOP" label, not just a red colour.
4. Inclusive navigation (آسان نیویگیشن)	Give users different ways to move around a system.	A website that lets you use the Tab key to jump between buttons if you cannot use a mouse.
5. Multimodal detection (متعدد حسی ذرائع)	Present information in different formats (sight, sound, touch).	A video with text captions, or a phone vibrating to alert a user.

WORKED EXAMPLE

Worked example: audit a school portal against the five principles.

1. **Contrast:** are result PDFs pale grey on white? Fail — use dark text.
2. **Fonts:** is the fee-challan note in 9-point decorative script? Fail — large Calibri (or a similar simple font), as the source recommends.
3. **Colour:** is "fail" only a red row with no word "Fail"? Fail — add the word, for colour-blind students.
4. **Navigation:** can a student who cannot use a mouse reach "Download date sheet" with Tab + Enter? If not, fail.
5. **Multimodal:** does the principal's video assembly have captions, and does a fee-due alert vibrate as well as flash? If not, deaf or busy students miss it.

The same five checks apply to an outdoor ATM (sunlight contrast) and to JazzCash (large numerals, Urdu labels, vibration on success).

REMEMBER IT

C-F-C-N-M: Contrast, Fonts, Colour-blind, Navigation, Multimodal. "Can Friends Catch New Messages?"

COMMON MISTAKE

Mistake: "Accessibility is only for people with disabilities, so our class portal does not need it."

Fix: The source says usable by **all** individuals. High contrast also helps in sunlight; captions help in a noisy bus; large fonts help tired eyes during board exams.

اردو میں سمجھیں

رسائی پذیری (Accessibility) کے اصول یقینی بناتے ہیں کہ ہر انسان سسٹم استعمال کر سکے، بشمول مختلف جسمانی، حسی اور ذہنی صلاحیتوں والے افراد۔ پانچ اصول: اونچا تضاد (کنٹراسٹ)، پڑھنے میں آسان فونٹ، صرف رنگ پر انحصار نہ کرنا، کی بورڈ سمیت متبادل نیویگیشن، اور معلومات کو نظر، آواز اور لمس کئی شکلوں میں دینا۔

CHECK YOURSELF

1. Why is a red-only "STOP" sign a problem?
2. Which principle does the Tab key support?
3. Give the source's school-website font example.

Answers: 1. Colour-blind users may not see the meaning if there is no symbol or "STOP" text.
2. Inclusive navigation. 3. Large Calibri on a school website for homework instructions.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Accessibility means:

- a. People with diverse abilities can use the system
- b. Only experts can use it
- c. Colourful GIFs always
- d. Mouse-only

MCQ 2. A principle is providing:

- a. Multiple ways to see/operate (text, caption, keyboard)
- b. Captcha of 12 steps
- c. 4pt grey on grey
- d. Auto-playing sound only

MCQ 3. Keyboard access helps:

- a. Motor impairment and power users
- b. Nobody
- c. Only AI models
- d. Only IoT soil

MCQ 4. Contrast helps:

- a. Low vision
- b. Faster bubble sort
- c. TCP window
- d. Primary keys

MCQ 5. Captions help:

- a. Deaf users and noisy labs
- b. Routers
- c. NAND
- d. K-maps

MCQ 6. Accessibility is part of:

- a. HCI quality, not an optional sticker
- b. Only marketing
- c. Only XII entrepreneurship
- d. Only physical cabling

Extra short questions

1. Define accessibility in HCI.
2. Four practical techniques.
3. Who else benefits besides disabled users?
4. Link to assistive tech in XI.

Extra long question

1. Audit a video lecture page against accessibility: list 6 fixes. Include one for vision, hearing, motor, language.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Design so people with disabilities (and many others) can perceive, operate and understand the UI.
2. Alt text, captions, keyboard, contrast, resizable text, Urdu/English toggle.
3. Broken arm, noisy room, bright sun, second-language users.
4. Screen readers need labelled buttons — that is an HCI job.

Long

1. Transcript+captions; contrast; pause; keyboard; alt on diagrams; Urdu summary. Do not autoplay. Test with Tab and a screen reader.

1.6 Need Analysis for Interface Design

LEARN IT

Need analysis is the process of identifying the gap between the **existing situation** and the **desired outcome**. In HCI it is done **before** designing a UI, so that you understand users' goals, tasks, abilities and expectations.

If you skip need analysis, you guess. Guessing is how a school portal ends up with 40 menus and no "Download date sheet" button on the home screen.

Steps in need analysis:



1. **Identify the users:** who exactly will use the system? (elderly people, teenagers, ...)
2. **Information collection:** gather data using interviews, questionnaires or observations.
3. **Analyse user needs:** identify the core problems, goals and requirements.
4. **Prioritise needs:** decide which features are the most critical to build first.
5. **Develop solutions:** design a system that meets those verified needs.

WORKED EXAMPLE

Case study from the source: mobile banking app. Before building the app, the team interviews customers and discovers:

- Users want secure login via Fingerprint / FaceID.
- Elderly users need large buttons and simple navigation.
- Users want support for both English and local languages.

Result: the UI is designed around these proven needs, which makes success more likely.

Apply the five steps to a college portal in Sindh.

1. Users: Grade XI–XII students, parents, clerks, teachers. Not "everyone in Pakistan".
2. Collect: interview 20 students; questionnaire to parents; watch a queue at the office on fee day.
3. Analyse: students cannot find date sheets; parents cannot read English-only fee notices; clerks re-type the same form.
4. Prioritise: (1) date sheet on home screen, (2) Urdu/Sindhi fee notice, (3) online leave form. Fancy animations wait.
5. Solution: a simple portal with those three features, large buttons, bilingual labels.

REMEMBER IT

Need analysis = gap finding. Write: Existing = what hurts now. Desired = what "done" looks like. The UI must close that gap, not decorate it.

COMMON MISTAKE

Mistake: Starting in Figma on day one, then "asking users later".

Fix: Need analysis comes **before** UI design. Features you did not hear from users are guesses. Guessed features waste time and fail in exams when the question is "why do we analyse needs?"

اردو میں سمجھیں

نیڈ اینالیسس موجودہ صورتحال اور مطلوبہ نتیجے کے فرق کو یو آئی بنانے سے پہلے تلاش کرتا ہے۔ پانچ قدم: صارفین کی پہچان، انٹرویو یا مشاہدے سے معلومات، ضروریات کا تجزیہ، اہم خصوصیات کو ترجیح، پھر تصدیق شدہ ضرورتوں پر حل۔ موبائل بینکنگ کی مثال: فنگر پرنٹ لاگ ان، بڑے بٹن، اور انگریزی کے ساتھ مقامی زبان۔

CHECK YOURSELF

1. When in the design process is need analysis done?
2. List the five steps in order.
3. Name three needs found in the mobile-banking case study.

Answers: 1. Before designing the UI. 2. Identify users; collect information; analyse needs; prioritise; develop solutions. 3. Fingerprint/FaceID login; large buttons and simple navigation for elderly users; English plus local languages.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Need analysis finds:

- a. The gap between now and the desired UI outcome
- b. The CPU benchmark
- c. The NAND delay
- d. The sprint velocity only

MCQ 2. You study:

- a. Users, tasks, pain points, constraints
- b. Only colours you like
- c. Only competitors' logos
- d. Only Big O

MCQ 3. A persona is:

- a. A realistic portrait of a user type
- b. A primary key
- c. A pie chart
- d. A gate

MCQ 4. Skipping need analysis causes:

- a. Pretty UIs that solve the wrong problem
- b. Correct Big O automatically
- c. Free RAM
- d. Shorter OSI

MCQ 5. Constraints include:

- a. Time, devices, literacy, connectivity
- b. Only Figma seats
- c. Only NAND trainers
- d. Only exam memes

MCQ 6. Output of need analysis is:

- a. Requirements for the interface
- b. A compiled kernel
- c. A K-map
- d. A histogram of NAND

Extra short questions

1. Define need analysis in HCI.
2. Three techniques.
3. Give a college example.
4. What is a constraint here?

Extra long question

1. Do a mini need analysis for a hostel complaint app: users, top 3 tasks, environment, constraints, 5 requirements.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Identify the gap between the current situation and what users need the interface to do.
2. Interview, observe, survey, task analysis, look at old system logs.
3. Students miss fee deadlines → need SMS reminders + a 3-step mobile payment UI.
4. Many students on low-end Androids and 2G — no huge animations.

Long

1. Users: residents, warden. Tasks: report, track, urgent leak. Env: phone on the stairs.
Constraints: Urdu, photos, weak wifi. Req: offline draft, photo, status, privacy, large buttons.

1.7 Human Computer Interaction Problems ★ Golden

LEARN IT

HCI problems are common difficulties that stop users from using technology smoothly. They happen when systems fail to meet human needs or cognitive limits. **Cognitive limits** means the mind can only hold and process so much at once — if an app asks you to remember three SMS codes at the same time, it has gone past that limit.

Board answers score well when you name the problem, explain it in one line, and give a concrete example (the source examples are safe; a matching local example is even better).

Problem	Explanation and example
Interface complexity	Too many options or a confusing layout. Example: a shopping app with so many menus you cannot find the checkout button.
Poor feedback	The system does not show if an action worked or failed. Example: clicking Submit and the screen freezes with no confirmation.
Inconsistency	Different parts of the app look or act differently. Example: a magnifying-glass icon for Search on one page, a binocular icon on another.
Accessibility gaps	Design ignores people with disabilities. Example: a video tutorial platform without captions for deaf users.
Compatibility issues	Interface fails on different devices or browsers. Example: a website that works on a laptop but crashes on a mobile browser.
Contextual neglect	The system does not adjust to the user's situation. Example: a map app with no dark mode, blinding the driver at night.
High cognitive load	Users must memorize too much or follow overly complex steps. Example: a registration system forcing you to remember three different SMS codes at once.
Trust deficiency	The system feels unsafe or hides security cues. Example: an online store asking for credit cards but lacking an SSL/HTTPS lock icon.

WORKED EXAMPLE

Worked example: diagnose three local systems.

ATM: six tiny unmarked buttons beside the screen and no "Cash is coming" message → interface complexity + poor feedback. A user walks away while cash is still coming out.

JazzCash: "Send" is green on the home screen but orange on the confirm screen, and there is no padlock on the payment page → inconsistency + trust deficiency.

School portal: works in Chrome on a lab PC, blank on a student's Android browser; exam instructions are a caption-free video; login needs a portal password, an email OTP and an SMS OTP on the same screen → compatibility, accessibility gap, high cognitive load.

Once you can name the problem, Section 1.8 tells you how to fix it.

REMEMBER IT

Eight problems in two families: (1) Screen problems — complexity, inconsistency, accessibility, compatibility, context. (2) Mind-and-trust problems — poor feedback, cognitive load, trust. If the user is lost, look at the screen family. If the user is scared or exhausted, look at the mind family.

COMMON MISTAKE

Mistake: Writing "the app is bad" as an HCI problem, or mixing up cognitive load with compatibility.

Fix: Use the exact names from the table. Cognitive load is "too much to remember". Compatibility is "breaks on another device or browser".

اردو میں سمجھیں

ایچ سی آئی مسائل وہ رکاوٹیں ہیں جو سسٹم کو انسانی ضرورت یا ذہنی حد کے مطابق نہ ہونے پر پیدا ہوتی ہیں۔ آٹھ مسائل: انٹرفیس کی پیچیدگی، کمزور فیڈبیک، عدم مطابقت (ایک جیسا نہ ہونا)، رسائی کی کمی، آلہ/براؤزر سے ناسازگاری، سیاق کو نظرانداز کرنا، زیادہ ذہنی بوجھ، اور اعتماد کی کمی (جیسے HTTPS تالے کے بغیر کارڈ مانگنا)۔

EXAM TIP

Common 8-mark question: "Describe problems of HCI with examples." Aim for at least six of the eight, each in the pattern Name → one-line meaning → example. Memorise the source examples (checkout buried in menus, frozen Submit, binocular vs magnifying glass, no captions, laptop-only site, no dark mode, three SMS codes, missing HTTPS lock). Adding ATM / JazzCash / portal examples shows application and often gains the last marks.

CHECK YOURSELF

1. Which problem is a missing HTTPS lock on a payment page?
2. Why are three SMS codes at once an HCI problem?
3. A site works on a laptop but crashes on a phone. Name the problem.

Answers: 1. Trust deficiency. 2. High cognitive load — users must memorize too much. 3. Compatibility issues.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. HCI studies:

- a. How people and computers work together
- b. Only CPU clocks
- c. Only NAND
- d. Only SQL keys

MCQ 2. Another name mentioned is:

- a. CHI / MMI
- b. MVP
- c. SDLC only
- d. K-map

MCQ 3. Sensory channels include:

- a. Sight, touch, hearing, voice, spatial
- b. Only smell in this course
- c. Only taste
- d. Only OSI layer 7

MCQ 4. Good HCI makes systems:

- a. Usable, efficient, fitting human needs
- b. Harder on purpose
- c. Mouse-only always
- d. Text in 4pt always

MCQ 5. Haptics are:

- a. Touch feedback (vibration)
- b. Pie charts
- c. Foreign keys
- d. Sprints

MCQ 6. VR tracking is a:

- a. Spatial channel
- b. Primary key
- c. Bubble sort
- d. Histogram

Extra short questions

1. Define HCI.
2. Five sensory channels from the notes.
3. Give a computer response for touch and for voice.
4. Why HCI is not "just drawing a pretty screen".

Extra long question

1. Tabulate the five channels with a human action and a computer response. Then explain one device that uses three channels at once.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Human-Computer Interaction: design of technology around people.
2. Sight, touch, hearing, voice, spatial.
3. Touch: click/haptic. Voice: speech recognition and spoken reply.
4. It includes users, tasks, environment, feedback, accessibility.

Long

1. Smartphone: sight (GUI), touch (tap/haptic), hearing (ringtone), optional voice assistant. Closing: if one channel fails, another should still work (accessibility).

1.8 Methods for Improving HCI

LEARN IT

Once you can name an HCI problem, you fix it with a matching method. The source lists eight methods. Learn each with its tiny example — they pair almost one-to-one with Section 1.7.

- **Enhance usability:** simplify navigation (a clear "Buy Now" button).
- **Strengthen accessibility:** support disabilities (captions on videos).
- **Ensure consistency:** keep layouts uniform (the same "Save" button everywhere).
- **Provide clear feedback:** show action results ("Payment Successful" popup).
- **Improve adaptability:** responsive design (the website scales to mobile screens).
- **Apply user-centred design (UCD):** focus on real user goals and pilot-test.
- **Use multimodal interaction:** allow touch, voice or gesture inputs combined.
- **Continuous testing:** regularly update based on user feedback.

User-centred design (UCD) means the real user's goals drive every decision, and you trial (pilot-test) the design with those users instead of guessing in an office.

Responsive design means the layout rearranges itself for phone, tablet and PC, instead of staying a laptop-only page.

WORKED EXAMPLE

Worked example: match each JazzCash / portal problem to a fix.

Problem (1.7)	Method (1.8)
Cannot find "Send Money"	Enhance usability — one clear button
No captions on help video	Strengthen accessibility
Search icon changes from page to page	Ensure consistency
Submit freezes with no message	Clear feedback — "Payment Successful"
Portal crashes on mobile	Improve adaptability — responsive layout
Built features nobody asked for	UCD — interview, then pilot-test
Elderly user cannot type a long number	Multimodal — voice + touch
Complaints ignored after launch	Continuous testing and updates

REMEMBER IT

Think "problem → method" as a pair. Complexity ↔ usability. No captions ↔ accessibility. Different icons ↔ consistency. Freeze ↔ feedback. Phone crash ↔ adaptability. Guessing ↔ UCD. One input only ↔ multimodal. Launch and forget ↔ continuous testing.

COMMON MISTAKE

Mistake: Naming methods without saying *what they change* ("we will improve HCI" is not an answer).

Fix: Always attach the source example: "Provide clear feedback, for example a Payment Successful popup."

اردو میں سمجھیں

ایچ سی آئی بہتر بنانے کے طریقے: نیویگیشن آسان کرنا، رسائی بڑھانا (کیپشن)، ایک جیسا لے آؤٹ، واضح فیڈبیک ("ادائیگی کامیاب")، ریسپانسیو ڈیزائن، صارف مرکز ڈیزائن (UCD) اور پائلٹ ٹیسٹ، ملٹی موڈل ان پٹ، اور مسلسل جانچ۔ ہر مسئلے کا ایک جوڑی دار حل یاد رکھیں۔

CHECK YOURSELF

1. What does UCD stand for, and what extra step does it include?
2. Which method stops a website from failing on a phone?
3. Give the source's example of clear feedback.

Answers: 1. User-Centred Design; focus on real user goals and pilot-test. 2. Improve adaptability (responsive design). 3. A "Payment Successful" popup.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Improving HCI often starts with:

- a. Watching real users (usability test)
- b. Adding more clipart
- c. Reducing contrast
- d. Hiding labels

MCQ 2. Consistency means:

- a. Same words/icons/positions for the same action
- b. A new metaphor every page
- c. Random colours
- d. Surprise menus

MCQ 3. Simple IA (information architecture) means:

- a. Fewer, clearer paths to tasks
- b. Deeper 12-level menus
- c. All links in a PDF dump
- d. No search

MCQ 4. Prototypes help you:

- a. Fix HCI cheaply before code
- b. Skip users
- c. Skip accessibility
- d. Skip feedback

MCQ 5. Training is a fix when:

- a. The task is rare/complex — but do not use it to excuse a bad UI
- b. The button is invisible
- c. Contrast is 1:1
- d. Errors are in hex only

MCQ 6. Iterative improvement is:

- a. Test → change → test again
- b. Ship once never look
- c. Waterfall with no users
- d. Only career slides

Extra short questions

1. Five methods to improve HCI.
2. Why test with real students not only the developer?
3. A quick win for a form.
4. What is a heuristic evaluation?

Extra long question

1. Take a 7-step fee-payment UI. Cut it to 3 steps, list the HCI problems you removed, and how you would re-test.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Simplify, consistent patterns, better feedback, accessibility, user tests, prototypes, reduce steps.
2. Developers know the map; users get lost.
3. Show required fields, inline errors, remember the class dropdown.
4. An expert checks the UI against rules (consistency, feedback...) before/besides user tests.

Long

1. Steps: amount → confirm → OTP. Removed: extra account-type screens, jargon, no progress. Re-test: 5 students timed, errors counted, one interview.

1.9 User Interface Design (UI vs UX) ★ Golden

LEARN IT

UI design is the process of creating and organizing the **visual elements** of a software application so users can interact with it easily and efficiently.

1.9.1 Components of a user interface

Component	Role
Buttons	Clickable elements that perform actions (Submit, Cancel).
Menus	Lists of options for navigating the system.
Icons	Small symbols representing tools or actions.
Text fields	Boxes where users type information (passwords, search).
Checkboxes / radio buttons	Selecting options in a form. Checkboxes allow several choices; radio buttons allow only one in a group.
Sliders	Adjusting values along a bar (volume, brightness).

1.9.2 User experience (UX)

UX is the overall experience a user has while interacting with a system — how smooth, comfortable and satisfying it feels.

UI is what you see; UX is how it feels.

WORKED EXAMPLE

Source example: food-delivery app. The UI is the nice pictures of burgers and the red buttons. The UX is that the app loads fast, tracking works, and the food arrives without the user getting frustrated.

Local walk-through: JazzCash.

- **UI:** green Send button, JazzCash logo, amount text field, fingerprint icon, menu of Bill / Load / Send.
- **UX:** login in two seconds, no crash in a weak-signal bazaar, money arrives, SMS comes, the user feels safe and finished — or, if it hangs with no tick, the UX is bad even if the buttons look pretty.

School portal: UI = logo, "Login" button, date-sheet menu icon. UX = a student finds the chemistry practical date in under a minute on a phone, without calling the clerk.

Pretty UI + slow, confusing flow = poor UX. Plain UI + fast, clear flow = good UX.

REMEMBER IT

Clothes vs day out. UI is the outfit (colours, buttons, icons). UX is the whole day (Did you arrive on time? Did you get wet? Were you stressed?). Examiners love the line: *UI is what you see; UX is how it feels.*

COMMON MISTAKE

Mistake: Writing "UI and UX are the same" or "UX is just colours".

Fix: UI = visual components (buttons, menus, icons, fields, checkboxes, sliders). UX = overall smoothness, comfort and satisfaction, including speed and whether the task succeeds.

اردو میں سمجھیں

یو آئی ڈیزائن بصری عناصر — بٹن، مینو، آئیکن، ٹیکسٹ فیلڈ، چیک باکس/ریڈیو، سلائیڈر — کو ترتیب دینا ہے۔ یو ایکس پورا تجربہ ہے: کتنا ہموار، آرام دہ اور اطمینان بخش لگا۔ یو آئی وہ ہے جو آپ دیکھتے ہیں؛ یو ایکس وہ ہے جو آپ محسوس کرتے ہیں۔ کھانے کی ڈیلیوری ایپ میں برگر کی خوبصورت تصویر یو آئی ہے، تیز لوڈنگ اور وقت پر کھانا یو ایکس ہے۔

EXAM TIP

Highest-yield short question in the chapter: "Differentiate between UI and UX with an example." Write: (1) the two definitions, (2) the sentence "UI is what you see; UX is how it feels", (3) the food-delivery example from the source, (4) optionally JazzCash or the school portal. For 5 marks also list four UI components. Do not spend the answer describing Figma — that is the next topic.

CHECK YOURSELF

1. List six UI components from the source.
2. In the food-delivery example, what is UX?
3. Complete: "UI is what you see; UX is ____."

Answers: 1. Buttons, menus, icons, text fields, checkboxes/radio buttons, sliders. 2. The app loads fast, tracking works, food arrives without frustration. 3. how it feels.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. UI is:

- a. The look and interactive controls
- b. The whole research journey only
- c. The database schema
- d. The sprint velocity

MCQ 2. UX is:

- a. The overall experience (useful, usable, satisfying)
- b. Only the colour palette
- c. Only the logo
- d. Only the NAND count

MCQ 3. You can have pretty UI and bad UX when:

- a. The task still takes 20 confusing steps
- b. Buttons are clear and short
- c. Feedback is excellent
- d. Accessibility is strong

MCQ 4. UX research includes:

- a. Interviews, tests, personas
- b. Only picking hex colours
- c. Only SQL
- d. Only OSI

MCQ 5. This is ★: define both, then:

- a. Give a product example of each going wrong
- b. Draw a K-map
- c. Write a sort
- d. List Access objects

MCQ 6. A progress tracker in a 3-step form is mainly:

- a. UX (and a bit of UI)
- b. A foreign key
- c. A histogram
- d. A gate

Extra short questions

1. Define UI vs UX.
2. Who typically does each job.
3. Example: ATM.
4. Why both matter in exams.

Extra long question

1. A food-delivery app is gorgeous but checkout fails twice. Analyse UI vs UX, then list 4 UX fixes that may not need new colours. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. UI: visual/interactive layer. UX: how the whole journey feels and whether the goal is achieved easily.
2. UI designer: screens/components. UX: research, flows, prototypes, testing. They overlap.
3. UI: button layout. UX: whether you felt safe and finished withdrawal quickly.
4. Papers ask the difference plus one example — do not mix the letters.

Long

1. UI may be fine. UX: errors, extra steps, unclear fees. Fixes: guest checkout, saved address, fee shown early, retry without losing the cart, better messages.

1.9.3 – 1.9.5 Wireframing, Figma, and Prototypes

LEARN IT

1.9.3 Wireframing is creating a basic visual outline or blueprint of a user interface **before** the final design. It plans the structure without focusing on colours or detailed graphics.

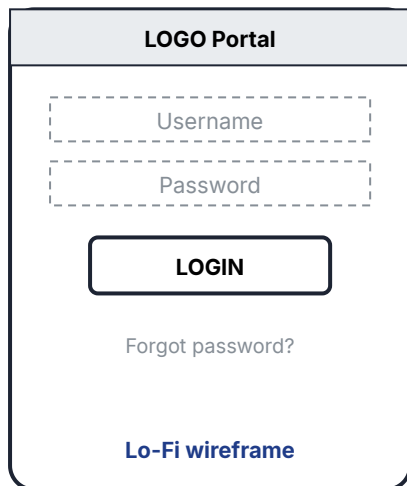
Low-fidelity wireframes (Lo-Fi)

A simple black-and-white sketch showing basic layout (boxes and lines). Used early to test and share ideas quickly. Focuses purely on structure and flow.

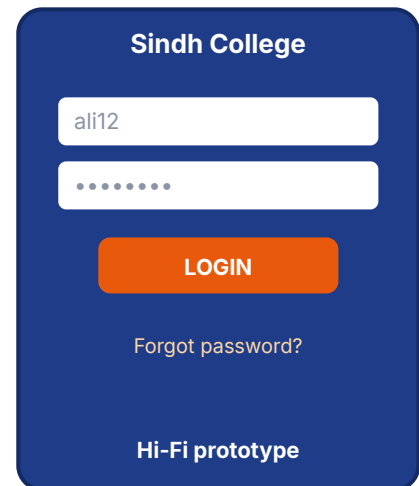
High-fidelity prototypes (Hi-Fi)

A detailed version that looks almost exactly like the final product. Includes real colours, fonts, images and interactive clickable buttons. Used for final user testing.

Lo-Fi sketch → Add details → Hi-Fi prototype



→
add colour



A school-portal login: Lo-Fi boxes and labels on the left; a coloured, realistic Hi-Fi prototype on the right.

1.10 What is Figma?

Figma is a popular, free, web-based tool used to design and collaborate on UI and prototypes. It works directly in a web browser — you do not have to install a heavy program on the college lab PCs.

Creating a low-fidelity design in Figma

1. **Sign up:** access the Figma workspace online.
2. **Create frames:** select the Frame tool (for example an iPhone-sized frame) to act as your screen canvas.
3. **Add shapes:** draw basic rectangles for buttons and circles for avatars.
4. **Add text:** use the text tool to label buttons (for example, "Sign Up").
5. **Duplicate and edit:** copy the frame to create the next screen (for example, the Home Screen).

1.9.5 Prototype design

A **prototype** is an early interactive model used to test design ideas **before writing actual code**. In Figma you use the Prototype tab to link screens together.

- Set a **Trigger** (for example, "On Tap").
- Set an **Action** (for example, "Navigate to Home Screen").

Real users can then click through the fake app to see if the navigation makes sense.

WORKED EXAMPLE

Worked example: JazzCash-style "Send Money" in Figma, without writing code.

1. Lo-Fi on paper or in Figma: three frames — Home (boxes: Send, Bills, Load), Amount (text field + Next), Success (tick + Done).
2. Show the sketch to two classmates. If they cannot find Send, move that box to the top. This is the point of Lo-Fi: cheap changes.
3. In Figma, add a phone frame, rectangles, and labels. Duplicate the frame for Home, Amount and Success.
4. Prototype tab: Trigger = On Tap of the Send rectangle; Action = Navigate to Amount. On Tap of Next → Success.
5. Hand the laptop to a student. If they tap Bills by mistake, the navigation is still wrong — fix it before any programmer writes Java or Kotlin.

REMEMBER IT

Lo-Fi = boxes. Hi-Fi = almost the real app. Prototype = clickable. Figma can hold all three stages. Trigger = what the user does. Action = where the fake app goes.

COMMON MISTAKE

Mistake: Colouring a Lo-Fi wireframe, or thinking a Figma prototype is the finished coded app.

Fix: Lo-Fi stays black-and-white on purpose so people discuss structure, not taste in colours. A prototype tests ideas before code; developers still have to build the real system afterwards.

اردو میں سمجھیں

وائر فریم یو آئی کا بنیادی خاکہ ہے، رنگوں سے پہلے۔ لو فائی سیاہ و سفید ڈبے ہیں؛ ہائی فائی تقریباً حتمی شکل ہے، رنگ، فونٹ اور کلک ہونے والے بٹن کے ساتھ۔ فگما (Figma) مفت ویب ٹول ہے: فریم بناؤ، شکل اور متن ڈالو، فریم کاپی کرو۔ پروٹوٹائپ کوڈ سے پہلے کا چلنے والا نمونہ ہے جس میں ٹرگر (On Tap) اور ایکشن (اگلی اسکرین) جوڑے جاتے ہیں۔

CHECK YOURSELF

1. What does a Lo-Fi wireframe deliberately leave out?
2. Where does Figma run?
3. In Figma prototyping, what is a trigger? Give the source example.

Answers: 1. Colours and detailed graphics (it is structure only). 2. Directly in a web browser.
3. The user event that starts a move between screens, e.g. "On Tap".

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A wireframe is:

- a. A low-detail blueprint of screens
- b. The final branded mock
- c. A running MVP
- d. A database dump

MCQ 2. Figma is:

- a. A collaborative UI/prototype tool in the browser
- b. A DBMS
- c. An OSI sniffer
- d. A sort

MCQ 3. A prototype here is:

- a. A clickable model of the UI
- b. The production server
- c. A primary key
- d. A pie

MCQ 4. You wireframe:

- a. Before polishing pixels
- b. After launch only
- c. Instead of need analysis
- d. Instead of testing ever

MCQ 5. Low fidelity means:

- a. Boxes and labels, not final art
- b. Photoreal 3D
- c. Compiled ARM
- d. Encrypted SQL

MCQ 6. Figma is taught because the book uses it to:

- a. Design and share UI without installing heavy tools
- b. Replace Python
- c. Replace ER
- d. Replace SDLC

Extra short questions

1. Define wireframe and prototype.
2. Why Figma (from the notes)?
3. Order: need analysis → wireframe → prototype → test.
4. What belongs on a wireframe?

Extra long question

1. Describe three Figma wireframe frames for college attendance: login, mark, confirm. List what you would test with five students.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Wireframe: structure/layout. Prototype: interactive simulation of the UI.
2. Free, web-based, collaborative, good for class.
3. Do not skip to colour before structure.
4. Boxes, headings, buttons, notes — not lorem-pretty photos.

Long

1. Frame contents; tap targets; Urdu. Test: time to mark, errors, could they undo. Change the prototype, not the database first.

1.11 Test and Evaluate HCI (Evaluation Methods)

LEARN IT

Testing checks if the product works properly (finding bugs). **Evaluation** determines whether it meets users' needs (usability). Together they improve the product before release.

A school portal can be "bug-free" (testing passed) and still unusable if students cannot find the date sheet (evaluation failed). You need both.

1.11.1 Evaluation methods

Method	Description
SUS (System Usability Scale)	A standard 10-item questionnaire used to calculate a score for how usable a system is.
NPS (Net Promoter Score)	Asks users: "How likely are you to recommend this app to a friend?" Measures user loyalty.
Formative evaluation	Done during development to collect early feedback and improve the design continuously.
Summative evaluation	Done at the end of development to measure overall success against the original goals.
Expert reviews	Usability specialists review the app against established design standards (heuristics).

Heuristics are agreed rules of good design (for example "show system status", "be consistent"). An expert review checks the product against those rules without waiting for a huge user survey.

WORKED EXAMPLE

Worked example: when to use each method on a school portal.

1. Week 2 of building: show the Lo-Fi to five students, change the menu. That is **formative** evaluation.
2. A teacher who has studied HCI checks the screens against standard rules (Is feedback visible? Are labels consistent?). That is an **expert review**.
3. After the portal is "finished", 30 students fill the 10 SUS questions. A low score means it is still hard to use, even if there are no crashes.
4. Ask: "On a scale of 0–10, would you recommend this portal to a friend in another college?" That is **NPS** (loyalty).
5. Compare the finished portal with the original goals (date sheet in one tap, bilingual fee notice). That is **summative** evaluation.

REMEMBER IT

Formative = forming it (during). Summative = summing it up (end). SUS = 10 questions on usability. NPS = "Would you recommend it?" Expert = specialist vs heuristics.

COMMON MISTAKE

Mistake: Saying testing and evaluation are the same, or mixing formative with summative.

Fix: Testing = bugs. Evaluation = does it meet human needs. Formative is during; summative is at the end against original goals.

اردو میں سمجھیں

ٹیسٹنگ یہ دیکھتی ہے کہ پروڈکٹ ٹھیک چلتا ہے (بگز)؛ ایویلیویشن یہ دیکھتی ہے کہ صارف کی ضرورت پوری ہوئی۔ SUS دس سوالوں والا استعمال کا اسکور ہے، NPS پوچھتا ہے کہ آپ یہ ایپ دوست کو کتنی تجویز کریں گے۔ فارمیٹو جانچ ترقی کے دوران ہوتی ہے، سمیٹو آخر میں اہداف کے خلاف۔ ماہر جائزہ ہیورسٹکس (Heuristics) پر ہوتا ہے۔

CHECK YOURSELF

1. What is the difference between testing and evaluation?
2. How many items are on the SUS questionnaire?
3. When is summative evaluation done?

Answers: 1. Testing finds bugs / checks the product works; evaluation checks it meets users' needs (usability). 2. 10. 3. At the end of development, against the original goals.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Testing asks:

- a. Does it work / any bugs?
- b. Is the logo pretty only
- c. What is Big O
- d. What is a FK

MCQ 2. Evaluation asks:

- a. Does it meet user needs / usability goals?
- b. Only CPU temp
- c. Only compile errors
- d. Only SQL syntax

MCQ 3. You need both because:

- a. A bug-free UI can still be unusable
- b. Bugs never matter
- c. Users never matter
- d. HCI is only art

MCQ 4. A usability goal example:

- a. First-time student pays fees in < 3 minutes
- b. More colours
- c. More menus
- d. More jargon

MCQ 5. Evaluation can be:

- a. With users or with experts/heuristics
- b. Only with the CEO's nephew
- c. Only after 5 years
- d. Only in assembly

MCQ 6. Do this:

- a. After a prototype, not only after full code
- b. Never
- c. Only on paper NAND
- d. Only on OSI diagrams

Extra short questions

1. Differentiate testing and evaluation in HCI.
2. Give a metric for each.
3. When is expert review useful?
4. Why include real students?

Extra long question

1. Plan an evaluation of the LMS assignment-upload: 2 test cases (bugs) and 2 evaluation questions (usability) with measures.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Testing: system behaviour/bugs. Evaluation: human fit/usability/need.
2. Test: crash rate. Eval: task success, time, SUS satisfaction, error types.
3. Early, cheap, catches obvious heuristic breaks before booking users.
4. They hold the real tasks and language.

Long

1. Test: 20MB PDF; no network. Eval: can a new student succeed unaided; time; satisfaction 1–5. n=8. What you would change if time > 5 min.

1.11.2 Testing Methods (Usability, A/B, Automated) ★ Golden

LEARN IT

Testing involves observing how the system behaves under specific conditions. Learn all seven methods. The three named in the heading — usability, A/B and automated — are the ones examiners ask most, but a complete answer also names the others.

1. **Usability testing:** observing real users while they perform realistic tasks, to measure efficiency and satisfaction. If the user gets stuck, designers know where to fix the UI.
2. **Task-based testing:** assigning a specific task (for example, "Find a product and add it to cart") and strictly measuring **completion time**, **success rate** and **number of errors**.
3. **Automated audits:** software bots scan websites to detect technical issues such as broken links, slow loading speeds or low colour contrast.
4. **A/B testing:** comparing two versions of an interface (A and B). Half the users see A, half see B. The one with more clicks or sign-ups wins.
5. **Compatibility testing:** verifying the system on different browsers (Chrome, Safari), devices (mobiles, tablets, PCs) and operating systems.
6. **Performance and load testing:** simulating high traffic (for example 10,000 users logging in at once) to see if speed drops or the server crashes.
7. **Keyboard and screen-reader walkthroughs:** navigating using *only* the keyboard (Tab/Enter) or assistive technologies, to check accessibility for users with disabilities.

Task-based

"Add to cart"

Time: 12 s

Success: Yes

A/B test

Version A | Version B

half / half users

more clicks wins

Automated audit

Bot scanning...

3 broken links

Low contrast

Three golden testing pictures: timed tasks, A versus B, and a bot that finds broken links and weak contrast.

WORKED EXAMPLE

Worked example: test JazzCash "Send Money" and a school portal with numbers.

Usability testing. Sit beside five students. Task: "Send Rs 100 to this test number." Do not help. Note where they pause. If three people look for Confirm at the bottom and it is at the top, that is your UI fix.

Task-based testing (numbers). Same task for 10 students.

- Success: 8 of 10 finish = success rate $(8 \div 10) \times 100 = 80\%$.
- Times: 12, 14, 11, 40, 13, 12, 15, 50, 14, 12 seconds. The two slow times (40, 50) are the students who got stuck — they also made errors.
- Errors: extra taps, wrong field. Count them. High errors + high time = poor HCI, even if most people eventually succeed.

A/B testing. Version A: green "Send" at the top. Version B: large orange "Send" under the amount. 200 users each. If B gets 140 completed payments and A gets 90, B wins. You keep B.

Automated audit. A bot reports 3 broken "Fee challan" links and low contrast on the portal's grey-on-grey marks table. Fix those before the board-exam rush.

Load testing. On result day, 10,000 students log in at 9:00 a.m. If the server crashes, performance testing failed — the UI never even appears.

Keyboard walkthrough. Unplug the mouse. Can a student Tab to "Download date sheet" and press Enter? If focus jumps randomly, accessibility testing failed.

REMEMBER IT

Task-based three numbers: Time, Success, Errors. **A/B:** two versions, split users, the higher clicks/sign-ups win. **Bot:** links, speed, contrast. **Load:** many users at once.

Keyboard: Tab and Enter only.

COMMON MISTAKE

Mistake: Calling A/B testing "showing both designs to the same user and asking which is prettier".

Fix: Half the users see only A, half see only B. You measure behaviour (clicks, sign-ups), not a taste vote. Also: an automated audit does not replace watching real users — bots cannot see that a student is confused.

سات ٹیسٹنگ طریقے: یوزیبلٹی (حقیقی صارف کو کام کرتے دیکھنا)، ٹاسک بیسڈ (وقت، کامیابی، غلطیاں ناپنا)، خودکار آڈٹ (بوت ٹوٹے لنک، سست رفتار، کم کنٹراسٹ ڈھونڈے)، اے/بی ٹیسٹ (آدھا صارف اے، آدھا بی؛ زیادہ کلک والا جیتے)، کمپیٹیبلٹی (براؤزر اور آلہ)، لوڈ ٹیسٹ (مثلاً دس ہزار لاگ ان)، اور صرف کی بورڈ یا اسکرین ریڈر سے چل کر رسائی کی جانچ۔

EXAM TIP

Golden 8-mark question: "Explain testing methods used in HCI." Write seven short paragraphs or a seven-row table. For full marks, define usability testing (observe real users on realistic tasks), give the three task-based measures, describe automated audits with the three bot findings (broken links, slow load, low contrast), explain A/B with the half-and-half rule, then add compatibility, load (10,000 users), and keyboard/screen-reader walkthroughs. If the paper asks only "usability, A/B and automated", go deep on those three and still mention the rest in one closing line.

CHECK YOURSELF

1. Name the three measurements in task-based testing.
2. In A/B testing, who sees version A?
3. Give two issues an automated audit can detect.

Answers: 1. Completion time, success rate, number of errors. 2. Half of the users (the other half see B). 3. Any two of: broken links, slow loading, low colour contrast.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Usability testing is:

- a. Watching people do real tasks
- b. Only pinging servers
- c. Only unit tests of Python
- d. Only load tests of CPUs

MCQ 2. A/B testing compares:

- a. Two versions on similar users
- b. NAND vs NOR only
- c. Mean vs mode only
- d. TCP vs UDP only

MCQ 3. Automated UI tests are good for:

- a. Regressions of known flows
- b. Discovering new user feelings
- c. Replacing all interviews
- d. Measuring joy perfectly

MCQ 4. Think-aloud means:

- a. Users speak while they try the task
- b. The computer reads NAND
- c. SQL dumps
- d. Sprinters shout

MCQ 5. A/B needs:

- a. A clear metric (clicks, success)
- b. Random colours with no goal
- c. $n=1$
- d. No ethics

MCQ 6. This is ★: name the method, when to use it, one limit.

- a. Yes — that is the 5-mark recipe
- b. No — only definitions
- c. No — write code
- d. No — draw OSI

Extra short questions

1. Define usability testing, A/B, automated testing.
2. One limit of each.
3. When A/B is overkill in XI/XII class.
4. What do you record in a usability test?

Extra long question

1. Design a usability test + a tiny A/B for a “download slip” button (blue text vs big labelled button). Tasks, n, metric, ethics. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Usability: observe tasks. A/B: two live variants. Automated: scripts click through.
2. Usability: small n, lab effect. A/B: needs traffic, can harm half the users. Auto: misses confusion that does not throw errors.
3. Five classmates on a Figma prototype is enough — no live traffic.
4. Task, success, time, errors, quotes, where they hesitated.

Long

1. Task: get the slip. $n=10$ (5+5). Metric: success and time. Ethics: consent, no marks attached. Predict big button wins. Also one automated check that the link still works.

Computer Systems (HCI)

15 single-page lessons · 5 Golden · read, close, say aloud, then sit the exam

Chapter in one look

- **HCI** (also CHI, MMI) designs technology to be easy, efficient and responsive to people, using five sensory channels: sight, touch (haptics), hearing, voice and spatial.
- **Traditional interaction** uses mouse, keyboard, keypad or simple tap/swipe; **natural interaction** uses voice, face or body gesture without a mouse or keyboard.
- HCI applications cover **health care, banking, education, and networking/communication**, each with a clear user benefit (remote care, quick secure money, better learning, global connection).
- Four core components: **user** (end user vs stakeholder), **computer system**, **user interaction**, and **task** (complexity, knowledge, time).
- Interaction types include direct manipulation, command-based, menu-driven, form fill, conversational, gesture, voice, context-aware, biometric, haptic and multimodal.
- Interfaces: GUI, CLI, touch, VUI, NUI. **Environment** is physical, social and technical. **System feedback** is the machine's reply; **user feedback** is what the person sends in.
- HCI matters for productivity, inclusivity, innovation, economic impact and ethical responsibility. Poor design (unclear medical buttons) creates risk.
- Five accessibility principles: high contrast, readable fonts, colour-blind friendly labels, inclusive navigation, multimodal presentation.
- **Need analysis** finds the gap before UI design: identify users, collect data, analyse, prioritise, develop solutions (banking case: FaceID, large buttons, local language).
- Eight HCI problems: complexity, poor feedback, inconsistency, accessibility gaps, compatibility, contextual neglect, high cognitive load, trust deficiency.
- Improve HCI with usability, accessibility, consistency, clear feedback, adaptability (responsive design), UCD and pilot tests, multimodal input, and continuous testing.
- **UI** is the visual components (buttons, menus, icons, fields, checkboxes, sliders); **UX** is how the whole experience feels. UI is what you see; UX is how it feels.
- Lo-Fi wireframes plan structure; Hi-Fi prototypes look real; **Figma** is a free browser tool; prototypes use Trigger (On Tap) and Action (navigate) before code.
- Evaluation (SUS, NPS, formative, summative, expert/heuristic reviews) checks needs; testing (usability, task-based, automated, A/B, compatibility, load, keyboard/screen-reader) checks behaviour under conditions.

Key terms

Term	Meaning	اردو
HCI	Human-Computer Interaction; designing technology that is easy, efficient and responsive to people. Also called CHI or MMI.	انسان اور کمپیوٹر کا باہمی تعامل
Haptics	Physical feedback you can feel, such as a vibration.	لمسی / ارتعاشی جواب
Traditional interaction	Using standard input/output such as mouse, keyboard, keypad or simple tap and swipe.	روایتی تعامل

Natural interaction	Interacting without mouse or keyboard, using voice, face or body gesture.	قدرتی تعامل
End user	A person who directly operates the system.	حتمی صارف
Stakeholder	A person or group with an interest in the system's outcome (manager, client).	فریق مفاد
GUI	Graphical User Interface using windows, icons and menus.	تصویری صارف انٹرفیس
CLI	Command-Line Interface using typed text commands.	کمانڈ لائن انٹرفیس
NUI	Natural User Interface using gestures, body movement or immersive environments (VR, Kinect).	قدرتی صارف انٹرفیس
System feedback	The computer's response that confirms or guides an action.	نظام کا جواب
User feedback	Input the user sends to the system (Submit click, star rating).	صارف کی رائے / ان پٹ
Accessibility	Making digital systems usable by all, including people with diverse abilities.	رسائی پذیری

CHAPTER 1 EXAM · CLOSED BOOK

Computer Systems (HCI)

75 marks · 1 hour 15 minutes · no notes · no phones

CHAPTER EXAM RULES

Instructions. Sit this exam after Lesson 15 and the recap. **Section A** — all 15 MCQs (1 mark). **Section B** — all 10 short questions (3 marks). **Section C** — attempt any **three** long questions (10 marks). At least one Section C answer must be a ★ Golden topic. Write in your notebook. Do not open the key until time is up.

Marks. 15 + 30 + 30 = 75. A 5-mark / 10-mark answer needs Define · Explain · Example · Diagram · Working.

سبق اور خلاصے کے بعد یہ امتحان بغیر نوٹس کے حل کریں۔ وقت پورا ہونے پر کلید کھولیں۔

Section A — Multiple choice (15 × 1 = 15)

1. HCI is also known as:

- a. CPU and ALU
- b. CHI or MMI
- c. HTTP or HTML
- d. GUI only

2. Which sensory channel tracks body movement in VR?
- a. Sight
 - b. Hearing
 - c. Spatial
 - d. Voice
3. Unlocking a phone with FaceID is an example of:
- a. natural interaction
 - b. command-based interaction only
 - c. traditional interaction
 - d. CLI
4. Which domain of HCI includes patient portals and telemedicine?
- a. Banking
 - b. Education
 - c. Networking
 - d. Health care
5. A bank manager who never uses the ATM but cares that it works is a:
- a. stakeholder
 - b. end user
 - c. haptic device
 - d. wireframe
6. Typing `dir` in MS-DOS is:
- a. menu-driven
 - b. command-based
 - c. context-aware
 - d. biometric
7. A "Message Sent!" tick mark is:
- a. user feedback
 - b. a stakeholder
 - c. need analysis
 - d. system feedback
8. Unclear buttons on a medical device mainly show that poor HCI:
- a. only affects game apps
 - b. is a colour preference
 - c. causes confusion and risk
 - d. improves load testing
9. Not depending only on colour, but adding a "STOP" label, is:
- a. colour-blind friendly design
 - b. load testing
 - c. CLI design
 - d. A/B testing
10. Need analysis for a UI should be done:
- a. after the product is sold
 - b. before designing the UI
 - c. only by programmers
 - d. instead of testing
11. An online store that asks for a credit card but shows no HTTPS lock has:
- a. high contrast
 - b. natural interaction
 - c. summative evaluation
 - d. trust deficiency

2. "UI is what you see; UX is how it feels" means:

- a. UI and UX are identical
- b. UX is only the logo colour
- c. UI is code and UX is hardware
- d. UI is visual elements; UX is the overall experience

3. A Lo-Fi wireframe should focus on:

- a. final photographs and brand colours
- b. structure and flow using boxes and lines
- c. server load of 10,000 users
- d. writing the finished Python code

4. The System Usability Scale (SUS) is:

- a. a 10-item questionnaire that scores usability
- b. a bot that finds broken links
- c. a type of ATM keypad
- d. a Figma trigger

5. In A/B testing:

- a. every user sees both versions at once and votes for the prettier one
- b. only experts review heuristics
- c. half the users see A, half see B, and the version with more clicks or sign-ups wins
- d. the server is crashed on purpose

Section B — Short questions (10 × 3 = 30)

1. Define HCI. Give two other names for it and list the five sensory channels.
2. Differentiate traditional interaction from natural interaction with one example of each from daily Pakistani use.
3. State four domains of HCI. For each, give one application and one user benefit.
4. Name the four primary HCI components. Distinguish end user from stakeholder, and list the three characteristics of a task.
5. Define GUI, CLI, VUI and NUI, with one example of each.
6. Write five implications of the importance of HCI.
7. List the five accessibility principles with one example each.
8. What is need analysis? Write its five steps and three findings from the mobile-banking case study.
9. Differentiate UI from UX using the food-delivery example. Name six UI components.
10. What is A/B testing? How is it different from an automated audit?

Section C — Long questions (attempt any 3 × 10 = 30)

1. Explain the components of HCI (user, computer system, user interaction, task, interface types, environment, feedback). Illustrate every part with a labelled walk-through of a person withdrawing cash from an outdoor ATM in Hyderabad.

2. Describe eight HCI problems with examples. Then match each problem to a method of improving HCI. Use JazzCash or a school portal wherever it makes the example clearer.
3. "UI is what you see; UX is how it feels." Explain UI design, its six components, and UX. Then describe Lo-Fi wireframes, Hi-Fi prototypes, the steps of making a low-fidelity design in Figma, and how a Figma prototype uses a trigger and an action. Sketch a Lo-Fi school-portal login.
4. Differentiate testing from evaluation. Explain SUS, NPS, formative evaluation, summative evaluation and expert reviews. Then explain usability testing, task-based testing (including the three measurements), automated audits, A/B testing, compatibility testing, performance/load testing, and keyboard/screen-reader walkthroughs. Apply three of these methods to a college portal on result day.
5. A team is designing a JazzCash-style "Send Money" screen for elderly users in Sukkur who speak Sindhi, stand in bright sunlight, and have never used a CLI. Carry out a need analysis (five steps), list accessibility principles you will apply, choose interaction types, and say which evaluation and testing methods you will use before launch and why.

Answer key. Finish the Chapter Exam first, then mark. Do not peek while the clock is running.

پہلے امتحان مکمل کریں، پھر اس کلید سے نمبر لگائیں۔

Check yourself — answers

1. **1.1 Fundamentals of Human Computer Interaction (HCI).** Answers: 1. CHI (Computer-Human Interface) and MMI (Man-Machine Interaction). 2. Spatial. 3. Touch / haptics.
2. **1.1.1 – 1.1.2 Traditional vs Natural Interaction.** Answers: 1. Traditional. 2. "Alexa, play a song" (or any spoken command). 3. Because typing a PIN is traditional, while FaceID or a voice assistant on the same phone is natural — it depends on the method, not the device.
3. **1.2 Applications of HCI (Domains of Life).** Answers: 1. Health care, banking, education, networking and communication. 2. Quick transactions and secure financial management. 3. Education: LMS or online exam portal. Networking: social media, WhatsApp or Zoom.
4. **1.3 Components of HCI.** Answers: 1. End users directly operate the system; stakeholders have an interest in the outcome (managers, clients) but may not click the buttons. 2. Task complexity, user knowledge, time requirement. 3. The principal, IT admin or board office — they care about results even if students are the end users.
5. **1.3 Types of User Interaction.** Answers: 1. Command-based. 2. Context-aware. 3. Because it combines two or more types (voice + touch) in one task.
6. **1.3 Interface Types, Environment & Feedback.** Answers: 1. GUI, CLI, Touch, VUI, NUI. 2. Physical: ATM in sunlight. Social: a quiet classroom. 3. System feedback.
7. **1.4 Importance of HCI.** Answers: 1. Unclear buttons on medical devices. 2. Promotes inclusivity. 3. User-friendly products get higher adoption, loyalty and profits.
8. **1.5 Accessibility Principles.** Answers: 1. Colour-blind users may not see the meaning if there is no symbol or "STOP" text. 2. Inclusive navigation. 3. Large Calibri on a school website for homework instructions.
9. **1.6 Need Analysis for Interface Design.** Answers: 1. Before designing the UI. 2. Identify users; collect information; analyse needs; prioritise; develop solutions. 3. Fingerprint/FaceID login; large buttons and simple navigation for elderly users; English plus local languages.
10. **1.7 Human Computer Interaction Problems.** Answers: 1. Trust deficiency. 2. High cognitive load — users must memorize too much. 3. Compatibility issues.
11. **1.8 Methods for Improving HCI.** Answers: 1. User-Centred Design; focus on real user goals and pilot-test. 2. Improve adaptability (responsive design). 3. A "Payment Successful" popup.
12. **1.9 User Interface Design (UI vs UX).** Answers: 1. Buttons, menus, icons, text fields, checkboxes/radio buttons, sliders. 2. The app loads fast, tracking works, food arrives without frustration. 3. how it feels.
13. **1.9.3 – 1.9.5 Wireframing, Figma, and Prototypes.** Answers: 1. Colours and detailed graphics (it is structure only). 2. Directly in a web browser. 3. The user event that starts a move between screens, e.g. "On Tap".
14. **1.11 Test and Evaluate HCI (Evaluation Methods).** Answers: 1. Testing finds bugs / checks the product works; evaluation checks it meets users' needs (usability). 2. 10. 3. At the end of development, against the original goals.
15. **1.11.2 Testing Methods (Usability, A/B, Automated).** Answers: 1. Completion time, success rate, number of errors. 2. Half of the users (the other half see B). 3. Any two of: broken links, slow loading, low colour contrast.

Answer key

Multiple choice

- | | | | | |
|------|------|------|-------|-------|
| 1. b | 4. d | 7. d | 10. b | 13. b |
| 2. c | 5. a | 8. c | 11. d | 14. a |
| 3. a | 6. b | 9. a | 12. d | 15. c |

Short questions — model answers

1. HCI is the design of technology to serve people; systems that are easy to use, efficient and responsive. Also called CHI (Computer-Human Interface) and MMI (Man-Machine Interaction). Channels: sight, touch, hearing, voice, spatial.
2. Traditional uses standard I/O (mouse, keyboard, keypad, simple tap/swipe), e.g. an ATM keypad or typing on a school portal. Natural lets people interact without mouse or keyboard, using voice, face or gesture, e.g. FaceID or "Alexa, play a song".
3. Health care: patient portals / telemedicine — remote access to medical services. Banking: mobile banking and ATMs — quick, secure transactions. Education: LMS and exam portals — better teaching, learning and engagement. Networking: WhatsApp, Zoom, social media — global connection and collaboration.
4. User, computer system, user interaction, task. End users directly operate the system; stakeholders (managers, clients) have an interest in the outcome. Task characteristics: complexity, user knowledge, time requirement.
5. GUI: windows, icons, menus (Windows, Android). CLI: typed commands (MS-DOS, Linux). VUI: spoken commands (smart speakers). NUI: gestures, body, immersive VR/Kinect.
6. Enhance productivity; promote inclusivity; drive innovation (AR/VR, AI, IoT need solid HCI); economic impact (adoption, loyalty, profit); ethical responsibility (privacy, less bias, well-being).
7. High contrast (black text on white in a railway app); readable fonts (large Calibri on a school site); colour-blind friendly (STOP label, not colour alone); inclusive navigation (Tab key); multimodal detection (captions or vibration).
8. Need analysis finds the gap between the existing situation and the desired outcome, before UI design. Steps: identify users; collect information (interviews, questionnaires, observation); analyse needs; prioritise; develop solutions. Banking case: Fingerprint/FaceID login; large buttons and simple navigation for elderly users; English and local languages.
9. UI is the visual elements so people can interact easily; UX is the overall smoothness, comfort and satisfaction. Food app: UI = burger pictures and red buttons; UX = fast load, working tracking, food arrives without frustration. Components: buttons, menus, icons, text fields, checkboxes/radio buttons, sliders.
10. A/B testing shows version A to half the users and version B to the other half; the version with more clicks or sign-ups wins. An automated audit uses bots to scan for technical issues such as broken links, slow loading or low contrast; it does not split users between two designs.

Long questions — model answers

- 1. Components through an outdoor ATM.** User: end user = account holder; stakeholder = bank. Computer system: ATM hardware and bank software linking PIN input to cash output. Interaction: insert card, type PIN, choose Cash, confirm — intent becomes a response. Task: withdraw cash; low complexity, basic knowledge, about one minute (contrast with a long passport application). Interface: GUI on screen + keypad (traditional); possibly fingerprint (biometric). Environment: physical sunlight so contrast must be high; social queue so a privacy shield is needed; technical printer or network may fail. System feedback: "Please wait", cash, receipt/SMS. User feedback: pressing Confirm. If any part is weak (glare, no "cash coming" message), HCI fails even if the bank account is correct.
- 2. Problems and methods.** Interface complexity (buried checkout / hidden Send Money) → enhance usability (clear Buy Now / Send). Poor feedback (frozen Submit) → Payment Successful popup. Inconsistency (magnifying glass vs binoculars, or Send changing colour) → same Save/Send control everywhere. Accessibility gaps (no captions) → strengthen accessibility. Compatibility (laptop-only portal) → responsive/adaptability. Contextual neglect (map with no night mode; ATM unreadable in sun) → design for environment. High cognitive load (three SMS codes) → simplify steps. Trust deficiency (no HTTPS lock on JazzCash-style payment) → show security cues. Also apply UCD with a pilot test, multimodal input (voice + touch for elderly users), and continuous testing after launch.
- 3. UI, UX, wireframe, Figma, prototype.** UI design organises visual elements. Six components: buttons (Submit/Cancel), menus, icons, text fields, checkboxes/radio, sliders. UX = overall experience; UI is what you see, UX is how it feels; food-delivery pictures vs fast delivery. Lo-Fi: black-and-white boxes for structure; Hi-Fi: real colour, type, images, clickable controls for final tests. Figma: sign up in the browser; Frame tool for a phone canvas; rectangles and circles; text labels such as Sign Up; duplicate frames for the next screen. Prototype: before code, Prototype tab, Trigger = On Tap, Action = Navigate to Home. Sketch: header "LOGO / Portal", dashed Username and Password boxes, a LOGIN rectangle, "Forgot password?" — the Lo-Fi login in this chapter.
- 4. Test vs evaluate, then methods.** Testing finds bugs / whether it works; evaluation checks whether it meets user needs. SUS: 10-item usability score. NPS: "recommend to a friend?" (loyalty). Formative: during development. Summative: at the end vs original goals. Expert reviews: specialists vs heuristics. Usability testing: watch real users on realistic tasks. Task-based: time, success rate, errors (e.g. Add to cart in 12 s, success yes). Automated audit: bots find broken links, slow load, low contrast. A/B: half/half, more clicks win. Compatibility: Chrome/Safari, phone/tablet/PC, OS. Load: 10,000 logins at once. Keyboard/screen-reader: Tab/Enter and assistive tech. Result-day portal: load-test the 9:00 a.m. rush; usability-test "find chemistry marks"; automated audit for broken result links and grey-on-grey contrast; optional A/B of two home-screen layouts.
- 5. Send Money for elderly users in Sukkur.** Need analysis: (1) users = elderly Sindhi speakers, not teenagers; (2) collect: interviews and watch them at a JazzCash shop agent; (3) analyse: fear of wrong number, cannot read small English, glare on the phone; (4) prioritise: huge Sindhi "رقم بھیجیں", amount in words, confirm screen, high contrast; (5)

solution: that UI, not a CLI. Accessibility: high contrast for sunlight, large simple fonts, not colour-only status, Tab/large targets, vibration + spoken confirm (multimodal). Interaction: menu-driven + form fill + biometric + haptic; optional voice. Evaluate formatively on paper then in Figma; expert review against heuristics; SUS and NPS before launch. Test: usability and task-based ("send Rs 100") measuring time/success/errors; A/B two button sizes; compatibility on cheap Androids; keyboard if a kiosk is used; do not skip a confirm screen — trust and feedback are the main risks.

Self-assessment: I can...

- ☐ define HCI, CHI and MMI and name the five sensory channels.
- ☐ describe HCI in health, banking, education and networking.
- ☐ name the types of user interaction and give an example of each.
- ☐ apply the five accessibility principles and carry out a need analysis.
- ☐ differentiate UI from UX and list six UI components.
- ☐ choose evaluation methods (SUS, NPS, formative, summative, expert).
- ☐ tell traditional interaction from natural interaction with local examples.
- ☐ label user, computer system, interaction, task, UI type, environment and feedback on an ATM, JazzCash or portal.
- ☐ explain why HCI is important using the five implications.
- ☐ name eight HCI problems and the methods that fix them.
- ☐ compare Lo-Fi, Hi-Fi, Figma steps and prototype triggers/actions.
- ☐ explain usability, task-based, automated, A/B, compatibility, load and keyboard tests.

Lecture 02

Computational Thinking & Algorithms

Chapter lecture — 16 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

BIEK Computer Science XII · Bilingual support: English + اردو

بی آئی ای کے کمپیوٹر سائنس — بارہویں جماعت

16 book lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XII

How to use this BIEK study-guide lecture

Each PDF follows one textbook lecture from the BIEK / Sindh Computer Science notes (New Sindh Curriculum). Read *Learn it*, study the SVG diagram, try the worked example on paper, then answer *Check yourself* before the answers. **Extra board practice** (MCQs, short and long questions) is added after the book content — it is not in the textbook. Career-only extras that were not in the book lecture list have been removed. Math is rendered with MathJax. ★ Golden topics are high-yield in board exams. The green box repeats the idea in Urdu.

ہر لیکچر کتاب کے درس کے مطابق ہے۔ کتاب کے بعد اضافی MCQs اور سوالات ہیں۔ جو اضافی کیریئر صفحات کتاب کی فہرست میں نہیں تھے وہ ہٹا دیے گئے ہیں۔

Lectures in this pack

Chapter 2: Computational Thinking & Algorithms. Study one lecture at a time, then use the chapter review and answer key at the end of this PDF.

L01. 2.1 Analyze algorithms for correctness

L02. 2.1.1 – 2.1.3 Trace Tables vs Stepwise Reasoning

L03. 2.2 Evaluating the Clarity of an Algorithm

L04. 2.2.1 – 2.2.2 Modularity and Readability

L05. 2.3 Assess Algorithm Efficiency

L06. 2.3.1 Number of Steps (Big O Notation)

L07. 2.3.2 Number of Conditions

L08. 2.3.3 Number of Repetitions

L09. 2.3.4 The Efficiency Evaluation Framework

L10. 2.4 Refinements to improve clarity and efficiency

L11. 2.5 Concept of Data Structure

L12. 2.5.1 Linear Data Structures (Array, Linked List, Stack, Queue)

L13. 2.5.2 Non-Linear Data Structures (Tree, Graph)

L14. 2.5.3 Operations on Data Structures

L15. 2.6 Identify data structures in problem scenarios

L16. 2.7 Trace data retrieval in Array, List and Queue

Computational Thinking & Algorithms

Class XI taught you how to write algorithms. This chapter teaches you how to **judge** them: is the algorithm correct, clear and efficient? You will then learn the main **data structures** — array, linked list, stack, queue, tree and graph — so you can store data the smart way and trace how it is retrieved.

گیارہویں جماعت میں آپ نے الگورتھم لکھنا سیکھا۔ اس باب میں آپ سیکھیں گے کہ الگورتھم درست، واضح اور تیز ہے یا نہیں، اور ایرے، لنکڈ لسٹ، سٹیک، کیو، ٹری اور گراف جیسے ڈیٹا سٹرکچرز میں ڈیٹا کیسے رکھا اور نکالا جاتا ہے۔

What you will learn

- Define **correctness** and prove it with a complete **trace table**.
- Evaluate **clarity** using modularity (SRP, coupling, abstraction) and readability (names, flow, consistency).
- Explain how the number of **conditions** and **repetitions** changes efficiency, and apply the three-question efficiency framework.
- Define a **data structure** and classify it as linear or non-linear.
- Draw a tree and a graph, and name the five operations on data structures.
- Apply **stepwise reasoning** and compare it with a trace table on at least four points.
- Define **efficiency** in time and space, and state Big O for $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$ and $O(n^2)$ with examples.
- Refine an algorithm for clearer names, modules, comments and fewer wasted steps.
- Draw and trace an array, linked list, stack (PUSH/POP) and queue (ENQUEUE/DEQUEUE).
- Choose a data structure for a real scenario and trace retrieval in an array, a linked list and a queue.

2.1 Analyze algorithms for correctness

LEARN IT

An algorithm is **correct** if its **output matches the expected results** for a given input. Analysing correctness means showing that the algorithm produces the intended output for **all valid inputs**, not just one lucky example.

Two methods are used in this course:

Trace tables → paper execution, line by line → Stepwise reasoning → logic blocks, overall state

You already met the idea of a dry run in Class XI. Here you learn to do it formally, and to choose the method that fits the size of the algorithm.

WORKED EXAMPLE

Worked example: what “correct” means. An algorithm is supposed to count how many of the heights 1.91, 2.01, 1.87, 1.57, 1.48 lie between 1.57 m and 1.91 m inclusive.

Expected output: **3** (1.91, 1.87 and 1.57 qualify; 2.01 is too tall; 1.48 is too short).

If a program prints 4, it **runs** but it is **not correct**. Correctness is about the right answer, not about “no crash”.

REMEMBER IT

Remember: **runs ≠ correct**. A logical error still produces output — just the wrong output. That is why we analyse correctness on paper before we trust the code.

COMMON MISTAKE

Mistake: testing only one “normal” input and calling the algorithm correct.

Fix: also try edge cases (empty list, $N = 0$, the exact boundary values 1.57 and 1.91). The definition says **all valid inputs**.

اردو میں سمجھیں

الگورتھم تب درست (Correct) سمجھا جاتا ہے جب دیے گئے ان پٹ پر اس کا آؤٹ پٹ وہی ہو جو ہونا چاہیے۔ صرف چل جانا کافی نہیں؛ منطقی غلطی سے غلط جواب بھی نکل سکتا ہے۔ درستگی جانچنے کے دو طریقے ہیں: ٹریس ٹیبل (Trace Table) اور مرحلہ وار استدلال (Stepwise Reasoning)۔

CHECK YOURSELF

1. When is an algorithm considered correct?
2. Name the two methods used to analyse correctness in this chapter.
3. Why is “the program ran without an error” not enough?

Answers: 1. When its output matches the expected results for a given input (and does so for all valid inputs). 2. Trace tables and stepwise reasoning. 3. Because a logical error can still produce output — the wrong output.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An algorithm is correct if:

- a. Outputs match the spec for the intended inputs (incl. edges)
- b. It is short
- c. It uses XOR
- d. It prints Hello

MCQ 2. A failed edge case means:

- a. Not fully correct
- b. Still correct enough always
- c. Faster Big O
- d. Better UX

MCQ 3. You demonstrate correctness in this chapter with:

- a. Trace tables / stepwise reasoning / tests
- b. Only careers
- c. Only Figma
- d. Only pie charts

MCQ 4. A spec is:

- a. The expected behaviour
- b. The CPU model
- c. The font
- d. The sprint name

MCQ 5. Partial correctness often means:

- a. IF it halts, the answer is right — you still need termination
- b. It never needs to halt
- c. It is Agile
- d. It is a GUI

MCQ 6. Off-by-one in a loop is a:

- a. Correctness bug
- b. OSI issue
- c. NAND issue
- d. Career issue

Extra short questions

1. Define correctness.
2. Two ways it can fail.
3. Why test edges.
4. Link to later trace tables.

Extra long question

1. Give a “find maximum” algorithm, two traces (normal and $n=1$), and an incorrect version that fails on negatives. Explain.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. For every specified input, the algorithm produces the specified output and finishes.
2. Wrong answer; infinite loop (does not finish).
3. Empty, one element, already sorted, not found, max int.
4. A dry run is a manual proof for a small case.

Long

1. Init max to first element not 0. Trace [3,-1,5] and [-4]. Bug: init 0 fails all-negative lists. That is correctness, not Big O.

2.1.1 – 2.1.3 Trace Tables vs Stepwise Reasoning ★ Golden

LEARN IT

A **trace table** is a technique used to monitor how the values of variables change during the execution of an algorithm. It is a **paper execution** of the code — a manual walkthrough, also called a **dry run**. You act as the computer and fill in one row after another.

Purpose of trace tables

- **Dry running:** execute the algorithm line by line by hand.
- **Debugging:** find logical errors where the algorithm runs but the output is wrong.
- **Validation:** check that the algorithm does what it is intended to do.

Components of a trace table

- **Line / step number** — which line is being executed.
- **Variables** — one column for every variable.
- **Input / output** — user inputs and printed results.
- **Condition check** — True/False for IF, WHILE, FOR, and similar tests.

WORKED EXAMPLE

Worked example: soldiers who meet the height rule. To qualify for training, a soldier's height must be between **1.57 and 1.91 metres, inclusive**.

Inputs: all soldiers' heights. Output: the count of eligible soldiers.

Step 1: Start

Step 2: SET Total = 0

Step 3: Set AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]

Step 4: FOR EACH height FROM AllHeights DO

Step 5: IF height $\geq$ 1.57 AND height $\leq$ 1.91 THEN

Step 6: SET Total = Total + 1

Step 7: END IF

Step 8: END FOR EACH

Step 9: Print Total

Step 10: End

Inclusive means the two boundary values **count**: 1.57 and 1.91 are eligible.

Step	Total	Height	AllHeights (input)	Height between 1.57 and 1.91?	Output
2	0				
3	0		1.91, 2.01, 1.87, 1.57, 1.48		
4	0	1.91			
5	0	1.91		True	
6	1	1.91			
4	1	2.01			
5	1	2.01		False (skip Step 6)	
4	1	1.87			
5	1	1.87		True	
6	2	1.87			
4	2	1.57			
5	2	1.57		True (boundary)	

6	3	1.57			
4	3	1.48			
5	3	1.48		False (skip Step 6)	
9	3				3

Eligible: 1.91, 1.87, 1.57. Not eligible: 2.01 (above 1.91) and 1.48 (below 1.57). Printed Total = 3.

```
AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]
Total = 0
for height in AllHeights:
    if height ≥ 1.57 and height ≤ 1.91:
        Total = Total + 1
print(Total)
```

OUTPUT
3

REMEMBER IT

How to fill a trace table without getting lost. Write a new row every time a variable **changes** or a condition is **tested**. Copy unchanged values into the new row so you never guess. Circle the final output.

Mnemonic: **V-I-C-O** — **V**ariables, **I**ntermediate, **C**ondition, **O**utput. If a column is missing, the table is incomplete.

COMMON MISTAKE

Mistake 1: treating 1.57 or 1.91 as out of range. The words “inclusive” and the operators $\geq$ and $\leq$ mean the boundaries **are allowed**.

Mistake 2: updating Total on a False condition. If the IF is False, skip the SET line and leave Total unchanged.

Mistake 3: writing only the last Total. Examiners award marks for the **changing values in the table**, not only for the printed 3.

ٹریس ٹیبل الگورتھم کا کاغذی اجرا (Dry Run) ہے۔ ہر قطار میں لکیر نمبر، متغیرات کی نئی قیمتیں، ان پٹ / آؤٹ پٹ اور شرط کا نتیجہ (True/False) لکھا جاتا ہے۔ فوجیوں کی مثال میں قد 1.57 سے 1.91 میٹر تک ہونا چاہیے، حدود سمیت؛ پانچ قدوں میں سے تین اہل ہیں اس لیے آؤٹ پٹ 3 ہے۔

EXAM TIP

Very common long question: "Dry-run the given algorithm using a trace table and state the output." Draw columns for **every** variable plus a condition column. Show the loop once per item. Boundary values ($\geq$, $\leq$, empty list) are favourite traps. Short questions ask: purpose of a trace table (dry run / debug / validate) and its components (line, variables, I/O, condition).

CHECK YOURSELF

1. List the four usual columns / components of a trace table.
2. In the soldier example, why is height 1.57 counted but 1.48 not counted?
3. What is printed, and how many times is Step 6 executed?

Answers: 1. Line/step number, variables, input/output, condition check. 2. 1.57 is the lower boundary (inclusive, so True); 1.48 is below 1.57 (False). 3. 3 is printed; Step 6 runs three times (for 1.91, 1.87 and 1.57).

LEARN IT

Stepwise reasoning (also called **logic verification**) breaks an algorithm into logical sections and checks the **state** of the algorithm at each stage. A **state** is a snapshot of what is true about the variables at that moment (for example, "Sum currently holds the total of the numbers seen so far").

You do **not** fill a row for every line. You think through the logic:

1. Analyse each step of the algorithm.
2. Check: does it **start in a valid state**?
3. Does it **handle all possible cases**?
4. Are the **conditions correct**?
5. If the loop is correct at the **start** of an iteration, does it remain correct at the **end**?

WORKED EXAMPLE

Worked example: sum of the numbers from 1 to N.

Step 1: Start

Step 2: Input N

Step 3: SET Sum = 0

Step 4: For i = 1 to N

Step 5: Sum = Sum + i

Step 6: END For

Step 7: Print Sum

Step 8: End

Stepwise reasoning breakdown

Check	What you verify
Initial state	Sum = 0 before the loop. Valid: nothing has been added yet.
Loop boundary	The loop runs exactly from i = 1 to i = N. No missing number, no extra number.
Iteration	Each i is added once: after k iterations, Sum = 1+2+...+k.
Final state	When the loop ends, Sum holds 1+2+...+N, which is the required total.
Edge case N = 0	The loop body never runs, Sum stays 0. Correct (the sum of no numbers is 0).
Check case N = 5	Sum becomes 15, which matches $5 \times 6 / 2 = 15$.

```
N = 5
Sum = 0
for i in range(1, N + 1):
    Sum = Sum + i
print(Sum)
```

OUTPUT
15

```
N = 0
Sum = 0
for i in range(1, N + 1):
    Sum = Sum + i
print(Sum)
```

REMEMBER IT

Think of stepwise reasoning as a **teacher checking a proof**, not as a cashier counting coins one by one. You ask “is this block always true?” instead of “what is Sum when $i = 17$?”.

COMMON MISTAKE

Mistake: rewriting the whole trace table and calling it stepwise reasoning.

Fix: write short comments about **state** (start, loop, finish, edges). If you listed every i from 1 to 1000, you used the wrong method.

اردو میں سمجھیں

مرحلہ وار استدلال میں الگورتھم کو منطقی حصوں میں توڑ کر ہر مرحلے پر اس کی حالت (State) دیکھی جاتی ہے: کیا شروع درست ہے، کیا تمام صورتیں سنبھلی ہیں، کیا شرط صحیح ہے، اور کیا لوپ کے ہر چکر کے بعد بھی نتیجہ درست رہتا ہے۔ ایک سے N تک جمع کی مثال میں Sum شروع میں 0 ہے، لوپ 1 سے N تک چلتا ہے، اور $N=0$ پر جواب 0 ہی رہتا ہے۔

CHECK YOURSELF

1. What other name is given to stepwise reasoning?
2. For the sum algorithm, why is $\text{Sum} = 0$ a valid start?
3. What happens when $N = 0$, and is that correct?

Answers: 1. Logic verification. 2. Because no numbers have been added yet, so the running total should be zero. 3. The loop does not run and Sum remains 0, which is the correct sum of no numbers.

LEARN IT

Both methods verify correctness, but they fit different situations depending on the **complexity and length** of the algorithm.

Feature	Trace table	Stepwise reasoning
Best for	Small algorithm blocks, complex loops, and beginners learning to track variables.	Large systems, architectural logic, and abstract mathematical proofs.

Effort required	High. Very tedious and impractical if a loop runs 1,000 times.	Moderate. Needs strong logical deduction and abstract thinking.
Primary goal	Catching syntax misinterpretations or specific arithmetic errors line by line.	Catching conceptual or structural flaws in the overall design.
Level	Micro-level (line by line). Checks exact values.	Macro-level (logic blocks). Checks overall state.

Trace table

Micro-level
Line by line
Exact values

Stepwise reasoning

Macro-level
Logic blocks
Overall state

Trace tables zoom in on values; stepwise reasoning zooms out on design.

WORKED EXAMPLE

Worked example: which method would you pick?

1. **A 12-line loop that adds five heights.** Use a **trace table**. The loop is short, you are a beginner, and you need the exact Total.
2. **A billing system with 40 functions.** Use **stepwise reasoning**. You cannot fill 10,000 rows. You check each module's start state, end state and edge cases.
3. **A loop from $i = 1$ to 1,000 that should produce $\text{Sum} = 500500$.** Do **not** write 1,000 rows. Argue: "Sum starts at 0; each i is added once; final $\text{Sum} = n(n+1)/2 = 1000 \times 1001/2 = 500500$." That is stepwise reasoning. Then dry-run a tiny N (say 5) with a trace table if you want extra confidence.

REMEMBER IT

Micro vs macro. Trace table = microscope (every cell). Stepwise reasoning = map (whole city). Beginners start with the microscope; designers need the map.

COMMON MISTAKE

Mistake: saying one method is "better" in every question.

Fix: they have different jobs. For a short dry-run in the exam, draw the table. For "explain why the algorithm is correct for any N ", write stepwise reasoning.

ٹریس ٹیبل خرد بینی (Micro) سطح پر کام کرتا ہے: ہر لائن اور ہر قیمت۔ مرحلہ وار استدلال بڑی (Macro) سطح پر ڈیزائن اور حالت چیک کرتا ہے۔ چھوٹے لوپ اور مبتدیوں کے لیے ٹیبل بہتر ہے؛ ہزار بار چلنے والے لوپ یا بڑے سسٹم کے لیے استدلال بہتر ہے۔

EXAM TIP

Differentiate questions are almost guaranteed: "Compare a trace table with stepwise reasoning." Write a **four-row table** (best for, effort, primary goal, micro/macro). Add one sentence of advice: table for small loops and exact values; reasoning for large systems and proofs. Do not write "trace table is always better".

CHECK YOURSELF

1. Which method is micro-level and checks exact values?
2. Why is a trace table a poor choice if a loop runs 1,000 times?
3. Which method is better for catching a structural flaw in a large design?

Answers: 1. Trace table. 2. It is very tedious and impractical (too many rows). 3. Stepwise reasoning (macro-level, conceptual / structural flaws).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A trace table is:

- a. A dry-run grid of variables vs steps
- b. A database table
- c. A pie
- d. A Figma frame

MCQ 2. Stepwise reasoning checks:

- a. The state at logical sections, not every micro-step
- b. Only Big O
- c. Only colours
- d. Only SQL

MCQ 3. Use a trace table when:

- a. Few variables, you must not miss an assignment
- b. The algorithm is a huge system diagram only
- c. You need UX quotes
- d. You need a histogram

MCQ 4. Use stepwise when:

- a. The logic is chunky (phases) rather than many tiny vars
- b. $n=2$ and 8 columns of i
- c. You sort visually
- d. You draw OSI

MCQ 5. Both methods aim at:

- a. Correctness
- b. Careers
- c. Fonts
- d. TCP windows

MCQ 6. A soldier-eligibility trace would track:

- a. Each input and the count so far
- b. CSS padding
- c. NAND groups
- d. Sprint points

Extra short questions

1. Define trace table vs stepwise reasoning.
2. When each is better (book comparison).
3. What must every trace column include?
4. A common student error.

Extra long question

1. Build a full trace table for: count how many of 3 heights are ≥ 5.5 . Then write three stepwise sentences. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Trace: column per variable, row per step. Stepwise: argue each phase's post-condition.
2. Trace: loops with a few vars. Stepwise: long algorithms with clear sections.
3. Changing variables and the output/decision.
4. Updating *i* in the table but forgetting the accumulator.

Long

1. Columns: *i*, height, count. Rows for each soldier. Stepwise: init 0; for each, if tall count++; after loop count is the answer. Compare the two explanations.

2.2 Evaluating the Clarity of an Algorithm

LEARN IT

Clarity means an algorithm is easy to **read, understand and modify**. In many cases, evaluating clarity is even more important than making the algorithm slightly faster. A clear algorithm is easier to **maintain, debug and extend** over time.

The two key aspects of clarity are:

Clarity → Modularity (parts) → Readability (text and flow)

WORKED EXAMPLE

Worked example: speed versus clarity. Algorithm A finds the class average in 8 confusing lines with variables a, b, c . Algorithm B uses two named functions and takes the same number of additions. Both are $O(n)$. Algorithm B is the better school solution because the next person (or you, in three months) can change it without breaking it.

REMEMBER IT

Ask two questions: **"Can I split this into parts?"** (modularity) and **"Can another student read it aloud and understand it?"** (readability).

COMMON MISTAKE

Mistake: treating a clever one-line trick as "better" than a clear five-line solution.

Fix: the source says clarity is often more important than speed. Board answers should praise meaningful names and modules, not compacted code.

اردو میں سمجھیں

وضاحت (Clarity) کا مطلب ہے الگورتھم پڑھنا، سمجھنا اور بدلنا آسان ہو۔ بہت سی صورتوں میں وضاحت رفتار سے بھی اہم ہے۔ اس کے دو حصے ہیں: ماڈیولریٹی (Modularity) یعنی کام کو حصوں میں بانٹنا، اور پڑھائی پن (Readability) یعنی متن اور بہاؤ صاف ہونا۔

CHECK YOURSELF

1. What does clarity mean?
2. Name the two key aspects of clarity.
3. Why can clarity matter more than a small speed gain?

Answers: 1. The algorithm is easy to read, understand and modify. 2. Modularity and readability. 3. Because a clear algorithm is easier to maintain, debug and extend over time.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Clarity means the algorithm is:

- a. Easy to read, understand, change
- b. As short as possible even if cryptic
- c. Written in one letter variables only
- d. Uncommented hex

MCQ 2. Modularity helps clarity because:

- a. Named pieces have one job
- b. Everything is in main
- c. There are no names
- d. Gotos everywhere

MCQ 3. Readability includes:

- a. Names, structure, comments, consistent style
- b. Removing all spaces
- c. 1-letter names always
- d. No line breaks

MCQ 4. A 200-line monoblock is:

- a. Low modularity
- b. High modularity
- c. A primary key
- d. A pie

MCQ 5. Clarity can matter more than a tiny speed gain when:

- a. Humans must maintain the code
- b. $n=10^{12}$ always
- c. It is a one-off NAND
- d. It is OSI layer 1

MCQ 6. Poor names are a:

- a. Readability problem
- b. TCP problem
- c. Power problem
- d. RAM manufacturing problem

Extra short questions

1. Define clarity of an algorithm.
2. Two ways to improve it.
3. Can a correct algorithm be unclear?
4. Link to functions in Ch 3.

Extra long question

1. Rewrite a messy "average of array" monoblock as three modules (input, compute, output). Explain the clarity gain.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. A person can see the purpose and logic without running it.
2. Decompose into modules; meaningful names; comments on why; consistent structure.
3. Yes — and then it will be modified wrongly.
4. Functions are modularity in Python.

Long

1. Show before/after pseudocode. Each module tested. Names: `sum_list`, `mean`.
Comments why `n=0` is rejected.

2.2.1 – 2.2.2 Modularity and Readability

LEARN IT

Modularity means breaking an algorithm into smaller, independent parts such as **functions, classes or modules**. This makes the algorithm more organised and easier to work with.

How to evaluate modularity

1. **Single Responsibility Principle (SRP)**. Each module should focus on **one** specific task. If a function calculates a sum, prints a report and saves a file, it violates SRP.
2. **Coupling**. Modules should be reusable in different contexts. Good modularity means **loose coupling**: a change in one part does not smash the others. Tight coupling is when two functions share so many hidden details that you cannot reuse one without the other.
3. **Abstraction**. A well-designed algorithm hides complex implementation details. You can call a sorting function without knowing whether it uses Bubble Sort or Selection Sort inside.

WORKED EXAMPLE

Worked example: average of an array.

Poor modularity (monoblock)

All logic sits in one block, so you cannot reuse the sum.

Step 1: Start
Step 2: Set A = [1, 2, 3, 4, 5]
Step 3: Set n = 5
Step 4: Set total = 0
Step 5: Set avg = 0
Step 6: For I = 1 to n
Step 7: total = total + A[I]
Step 8: End For
Step 9: avg = total / n
Step 10: Print avg
Step 11: End

Good modularity (functions)

Two clear functions: `CalculateSum()` and `FindAverage()`.

Step 1: Start
Step 2: Set A = [1, 2, 3, 4, 5]
Step 3: Set n = 5
Step 4: Print FindAverage(A, n)
Step 5: End

```
Function FindAverage(A, n)
    Sum = CalculateSum(A, n)
    Avg = Sum / n
    Return Avg
End Function
```

```
Function CalculateSum(A, n)
    Set Total = 0
    For I = 1 to n
        Total = Total + A[I]
    End For
    Return Total
End Function
```

SRP check: `CalculateSum` only adds; `FindAverage` only divides. Loose coupling: you can reuse `CalculateSum` to find a total without computing an average. Abstraction: the main algorithm just calls `FindAverage` and does not see the loop.

The source pseudocode numbers array positions from 1. Python numbers them from 0, so the loop is `for i in range(n)`.

```
def calculate_sum(A, n):
    total = 0
    for i in range(n):
        total = total + A[i]
    return total

def find_average(A, n):
    return calculate_sum(A, n) / n

A = [1, 2, 3, 4, 5]
n = 5
print(find_average(A, n))
```


OUTPUT
3.0

REMEMBER IT

SRP in one line: **one function, one job**. If you cannot name the function with a short verb-phrase (`CalculateSum`, `FindAverage`), it is doing too much.

COMMON MISTAKE

Mistake: copying the 1-based loop `for I = 1 to n: total = total + A[I]` into Python. Then `A[1]` skips the first item and `A[5]` crashes.

Fix: in Python use `for i in range(n)` so the indices are 0, 1, 2, 3, 4.

اردو میں سمجھیں

ماڈیولریٹی کا مطلب الگورتھم کو آزاد حصوں (فنکشن، کلاس، ماڈیول) میں توڑنا ہے۔ جانچ کے تین اصول: واحد ذمہ داری (SRP) یعنی ایک فنکشن ایک کام؛ ڈھیلی جڑت (Loose Coupling) تاکہ ایک حصہ بدلنے سے دوسرے نہ ٹوٹیں؛ اور تجرید (Abstraction) یعنی اندرونی تفصیل چھپا کر صرف کام منگوانا، جیسے سورٹ فنکشن بلائے بغیر یہ جانے کہ ببل ہے یا سلیکشن۔

CHECK YOURSELF

1. What does SRP say each module should do?
2. In the good average algorithm, which function hides the loop?
3. Why is the monoblock harder to reuse?

Answers: 1. One specific task. 2. `CalculateSum` (`FindAverage` only divides; the main program only prints). 3. All logic is in one block, so you cannot call "just the sum" from another algorithm.

LEARN IT

Readability is how easily another person can understand the purpose and logic of an algorithm **without running it**. It is about how clear the text and the flow are.

How to evaluate readability

1. **Naming conventions** — consistent rules for names. Good names make an algorithm easier to search and understand.
Bad: $v = d / t$
Good: `velocity = distance / time`
2. **Self-documentation** — the algorithm explains itself, so you need fewer comments. Meaningful names and clean formatting make the purpose obvious.
3. **Control-flow simplicity** — keep the logic straightforward. Avoid deep nesting (IF inside IF inside IF), limit extra conditions, and keep each function on one task.
4. **Consistency** — one style throughout: indentation, spacing, and casing (`camelCase` or `snake_case`). Then the logic stands out instead of the mess.

Names → Self-doc → Simple flow → Consistent

WORKED EXAMPLE

Worked example: the same formula, two styles.

```
# Hard to read
d = 100
t = 4
s = 20
v = d / t
if v > s:
    if t > 0:
        print(v)
```

OUTPUT
25.0

```
# Readable
distance = 100
time = 4
velocity = distance / time
speed_limit = 20
if time > 0 and velocity > speed_limit:
    print(velocity)
```

OUTPUT
25.0

The second version uses names, avoids an IF nested inside an IF, and keeps one consistent snake_case style. You can understand it without running it. Both print 25.0, but only the second is clear.

REMEMBER IT

Remember readability as **N-S-S-C**: **N**ames, **S**elf-documentation, **S**imple flow, **C**onsistency.

COMMON MISTAKE

Mistake: dumping a comment on every line (`total = total + 1 # add one to total`).

Fix: if the name already says it, skip the comment. Comments should explain **why**, not repeat **what**.

پڑھائی پن یہ ہے کہ دوسرا شخص الگورتھم چلائے بغیر اس کا مقصد سمجھ لے۔ اچھے نام (`velocity = distance / time`)، خود بیان (کم تبصرے)، سادہ کنٹرول فلو (گہری IF کے بغیر) اور ایک جیسا انداز (casing اور indentation) اسے آسان بناتے ہیں۔

CHECK YOURSELF

1. Rewrite `v = d / t` with readable names.
2. What is deep nesting, and why is it a readability problem?
3. Name two style habits that support consistency.

Answers: 1. `velocity = distance / time`. 2. IF inside IF inside IF (or similar); it makes the flow hard to follow. 3. Any two of: consistent indentation, spacing, and casing (camelCase or snake_case).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Modularity is:

- a. Splitting into independent pieces with clear interfaces
- b. One file of 2000 mixed lines
- c. Random copy-paste
- d. A chart type

MCQ 2. A module should have:

- a. One job
- b. Every job
- c. No name
- d. No inputs

MCQ 3. Readability is:

- a. How easily another person understands the algorithm
- b. Font size of the PDF only
- c. RAM size
- d. Cable length

MCQ 4. Reusing a module is easier if:

- a. It does not depend on hidden globals
- b. It prints inside and uses global n
- c. It is unnamed
- d. It mixes I/O and maths

MCQ 5. Comments should explain:

- a. Why, not every obvious $i=i+1$
- b. Nothing
- c. Only your WhatsApp
- d. Only Big O symbols

MCQ 6. Poor readability increases:

- a. Bugs when changing code
- b. CPU GHz
- c. OSI layers
- d. Pie slices

Extra short questions

1. Define modularity and readability.
2. One benefit of each.
3. A readable name vs a bad one.
4. How they interact.

Extra long question

1. Take a messy attendance script (globals, 1-letter names). Produce a modular readable version and a 5-point rubric a teacher could mark.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Modularity: parts. Readability: understandable text/structure.
2. Modularity: test/reuse. Readability: fewer maintenance bugs.
3. total_marks vs x1.
4. Small named modules are easier to read.

Long

1. Modules: load, mark, save, report. Rubric: names, functions, no magic numbers, comments why, handles empty file.

2.3 Assess Algorithm Efficiency ★ Golden

LEARN IT

Algorithm efficiency is measured in terms of **time** and **space** — how fast it runs and how much memory it uses.

The key considerations for assessing efficiency are:

1. **Number of steps** (described with **Big O notation**).
2. **Number of conditions** (IF decisions).
3. **Number of repetitions** (loops).

Time (speed) + Space (memory) → Efficiency

WORKED EXAMPLE

Worked example: two ways to find a name in 1,000,000 sorted records.

Linear search checks about 1,000,000 names in the worst case (time grows with n).

Binary search checks about 20 names (it halves the list each time). Both can be correct and clear. They are not equally efficient.

REMEMBER IT

Efficiency is always “as the input grows”. An algorithm that feels instant on 10 items can freeze on 10 million items if it is $O(n^2)$.

COMMON MISTAKE

Mistake: judging efficiency by how short the code looks.

Fix: a five-line nested loop can do n^2 work. Count steps, conditions and repetitions — not lines of text.

اردو میں سمجھیں

کارکردگی (Efficiency) وقت (رفتار) اور جگہ (میموری) سے ناپی جاتی ہے۔ تین چیزیں دیکھی جاتی ہیں: قدموں کی تعداد (بگ او)، شرطوں کی تعداد، اور دہرائی (لوپ) کی تعداد۔ چھوٹے ان پٹ پر تیز لگنے والا الگورتھم بڑے ان پٹ پر سست پڑ سکتا ہے۔

CHECK YOURSELF

1. Efficiency is measured in terms of which two resources?
2. List the three key considerations named in the source.

Answers: 1. Time (speed) and space (memory). 2. Number of steps (Big O), number of conditions, number of repetitions.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Efficiency is mainly:

- a. Time and extra memory vs n
- b. How pretty the UI is
- c. How many comments
- d. Font choice

MCQ 2. As n grows, we care about:

- a. Growth class (Big O), not a 2 ms blip
- b. Only the constant 2 ms
- c. Only variable names
- d. Only colours

MCQ 3. Space efficiency is:

- a. Extra RAM/disk beyond the input
- b. Screen size
- c. PDF margins
- d. Cable thickness

MCQ 4. An algorithm can be:

- a. Correct but inefficient
- b. Efficient but never correct and still useful always
- c. A pie
- d. A sprint

MCQ 5. This chapter's framework later uses:

- a. Steps, conditions, repetitions
- b. Only UX
- c. Only ER
- d. Only Figma

MCQ 6. Why efficiency is ★:

- a. Board likes compare-two-algorithms questions
- b. It replaces correctness
- c. It replaces HCI
- d. It replaces Python

Extra short questions

1. Define time and space efficiency.
2. Why not measure only one run on your PC?
3. Example of wasting space.
4. Example of wasting time.

Extra long question

1. Compare linear vs binary search on $n=2$, $n=1\,000\,000$ for time and space. State preconditions. 10-mark table.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Time: how running time grows with input. Space: extra memory.
2. Machines differ; we need growth vs n .
3. Copying a huge list on every recursive call.
4. Linear search 1 million times on a static sorted list instead of binary/index.

Long

1. Time $\sim n$ vs $\sim \log n$. Space $O(1)$ both (iterative). Binary needs sorted. For $n=1e6$, $\sim 1e6$ vs ~ 20 comparisons. Correctness not sacrificed.

2.3.1 Number of Steps (Big O Notation) ★ Golden

LEARN IT

When we discuss efficiency we ask how performance changes as the **input size n** increases. **Big O notation** is a mathematical method used to describe how the running time (or memory) **grows** as n grows. We ignore small constants and keep the most important term: if an algorithm does $3n + 5$ steps we write $O(n)$.

In computer science, **log n** almost always means log base 2 (how many times you can halve n until you reach 1).

Five common classes

Class	Name	What happens when n grows	Typical example	Grade
$O(1)$	Constant	Time stays the same for 10 items or 10 million.	Access <code>arr[0]</code> (or any known index).	Excellent
$O(\log n)$	Logarithmic	Time grows very slowly; the problem is halved each step.	Binary search on a sorted list.	Great
$O(n)$	Linear	If n doubles, time roughly doubles.	Linear search of an unsorted list.	Fair / Good
$O(n \log n)$	Linearithmic	A little worse than linear; much better than n^2 . Common in good sorting.	Merge Sort (split in half, then merge).	Good
$O(n^2)$	Quadratic	Time grows with the square of n . Nested loops. Painful for large n .	Bubble Sort; compare every item with every other item.	Poor

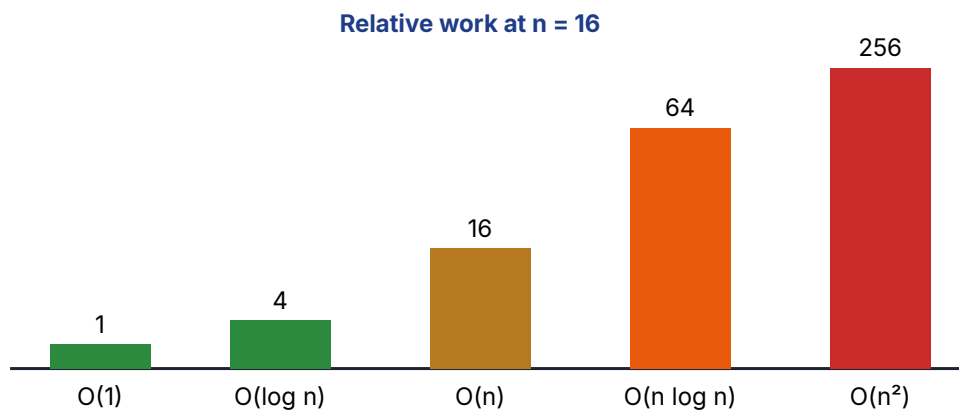
The source stresses $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$. $O(n \log n)$ is the class of efficient general-purpose sorts, so you should recognise it when a question says “better than Bubble Sort but more than linear”.

Growth table (log is $\log_2$)

Each cell is the approximate number of operations for that n . Read down a column to see how fast that class explodes.

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
2	1	1	2	2	4
4	1	2	4	8	16
8	1	3	8	24	64
16	1	4	16	64	256
32	1	5	32	160	1,024
1,024	1	10	1,024	10,240	1,048,576
1,000,000	1	~20	1,000,000	~20,000,000	1,000,000,000,000

From $n = 8$ to $n = 1,000,000$, $O(1)$ does not move, $O(\log n)$ only goes from 3 to about 20, $O(n)$ becomes a million, and $O(n^2)$ becomes a million million — which is why nested loops “fail for large datasets”.



Same $n = 16$. Quadratic work is already four times $n \log n$ and sixteen times linear.

WORKED EXAMPLE

Worked example A — $O(1)$: first element of an array. Whether the array holds 10 or 10 million values, `arr[0]` is one jump to a known address.

```
arr = [23, 76, 12, 90, 34, 68]
print(arr[0])
```

OUTPUT
23

Worked example B — $O(n)$: linear search. Each extra item adds one extra check.

```
def linear_search(items, key):
    for i, value in enumerate(items):
        if value == key:
            return i
    return -1

print(linear_search([10, 50, 20, 70, 90, 30], 20))
print(linear_search([10, 50, 20, 70, 90, 30], 99))
```

OUTPUT
2
-1

Worked example C — $O(\log n)$: binary search on a sorted list. Each step throws away half of the remaining items. For 8 sorted numbers the worst case is about 4 comparisons ($\text{floor}(\log_2 8) + 1$).

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    steps = 0
    while low <= high:
        mid = (low + high) // 2
        steps = steps + 1
        if arr[mid] == key:
            return mid, steps
        if arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, steps

data = [3, 8, 12, 17, 23, 31, 40, 52]
print(binary_search(data, 40))
print(binary_search(data, 3))
```

OUTPUT

(6, 3)
(0, 3)

Key 40 is found at index 6 after 3 comparisons. Key 3 is found at index 0 after 3 comparisons. Never more than a handful of steps, even if the list later grows to a million (about 20 steps).

Worked example D — $O(n \log n)$: Merge Sort idea. To sort 8 items, Merge Sort makes $\log_2 8 = 3$ split-levels, and each level walks all 8 items once, so work $\approx 8 \times 3 = 24$, which matches the growth table. Bubble Sort on 8 items is on the order of $8^2 = 64$ comparisons. That gap becomes enormous at $n = 1,000,000$.

Worked example E — $O(n^2)$: nested loops.

```
n = 4
count = 0
for i in range(n):
    for j in range(n):
        count = count + 1
print(count)
```

OUTPUT

16

Inner body runs $n \times n = 16$ times. That is the pattern of Bubble Sort and of “compare every item with every other item”.

REMEMBER IT

Order from fastest to slowest (for large n): $O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2)$.

Mnemonic: **“Constant Logs Never Need Squares”** — Constant, Logarithmic, N, N-log-N, Squares.

COMMON MISTAKE

Mistake 1: writing $O(n)$ for binary search because “there is a loop”. The loop runs about $\log n$ times, not n times.

Mistake 2: writing $O(2n)$ or $O(n+5)$. Big O drops constants: both are $O(n)$.

Mistake 3: using binary search on an **unsorted** list. Binary search is $O(\log n)$ only on sorted data.

بگ او (Big O) بتاتا ہے کہ ان پٹ n بڑھنے پر وقت یا میموری کیسے بڑھتی ہے۔ $O(1)$ مستقل (ایرے) کا $O(\log n)$ بر قدم پر آدھا (بائنری سرچ)، $O(n)$ لکیری (لینیئر سرچ)، $O(n \log n)$ اچھے سارٹ جیسے مرچ سارٹ، اور $O(n^2)$ مربع (بیل سارٹ)، اندرونی لوپ۔ بڑے n پر $O(n^2)$ ناقابل استعمال ہو جاتا ہے۔

EXAM TIP

Memorise the five rows of the class table (name, growth, one example, grade). Typical MCQ: "Accessing arr[0] is ...?" $\rightarrow O(1)$. "Binary search is ...?" $\rightarrow O(\log n)$. "Nested loops / Bubble Sort ...?" $\rightarrow O(n^2)$. Long questions ask you to fill or interpret a growth table, or to say what happens if n becomes 10 times larger (linear $\rightarrow 10\times$ time; quadratic $\rightarrow 100\times$ time). Always mention that $\log n$ is "halving" and n^2 is "nested loops".

CHECK YOURSELF

1. Give one example each of $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$.
2. From the growth table, how many operations does $O(n^2)$ need when $n = 32$?
3. Why is Merge Sort described as $O(n \log n)$?

Answers: 1. $O(1)$ arr[0]; $O(\log n)$ binary search; $O(n)$ linear search; $O(n^2)$ Bubble Sort / nested loops. 2. 1,024. 3. It makes about $\log n$ split-levels and does about n work on each level, so $n \times \log n$.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. $O(1)$ means:

- a. Constant time, independent of n
- b. Linear in n
- c. Quadratic
- d. Factorial

MCQ 2. $O(n)$ example:

- a. Linear search
- b. Binary search
- c. Nested $n \times n$ always
- d. Hash $O(1)$ best always guaranteed

MCQ 3. $O(\log n)$ example:

- a. Binary search
- b. Bubble sort
- c. Reading all n items once
- d. Nested loops n^2

MCQ 4. $O(n^2)$ example:

- a. Bubble / selection sort
- b. Binary search
- c. Closed formula sum
- d. Array index

MCQ 5. We drop constants and small terms because:

- a. Growth class dominates for large n
- b. They are always 0
- c. Big O is UX
- d. Boards forbid numbers

MCQ 6. $\log n$ grows:

- a. Slowly
- b. Faster than n^2
- c. Faster than $n!$
- d. As a straight 45° line always

Extra short questions

1. Define Big O in one student sentence.
2. Order these from fast to slow: n^2 , 1, $\log n$, n .
3. Why is $3n+5$ still $O(n)$?
4. What does $O(\log n)$ look like on a phone contacts list?

Extra long question

1. For each: array index, binary search, linear search, bubble sort — state Big O and a 1-line why. Then pick one for $n=50$ vs $n=5$ million. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. An upper-class of how time (or space) grows as n gets large.
2. $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$.
3. Constants ignored; linear term dominates.
4. Each step halves the remaining names.

Long

1. $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$. $n=50$ anything fine; $n=5e6$ avoid n^2 , prefer log/linear as appropriate. Mention sort cost if you need binary.

2.3.2 Number of Conditions

LEARN IT

Conditions (IF statements) are decision points. Every condition forces the CPU to evaluate True or False, so excessive branching reduces performance.

- **Computational cost.** Each IF is extra work, even if the rest of the algorithm is simple.
- **Nested conditions.** IF inside IF often means a brute-force approach. It hurts both readability and speed.
- **Short-circuiting** (the optimisation goal). Stop evaluating as soon as the result is known. In a logical AND, if the first part is False the second part is skipped. In a logical OR, if the first part is True the second part is skipped.

WORKED EXAMPLE

Worked example: cheap test first. You only divide if the denominator is not zero. Place the cheap, safe test on the left so Python never evaluates the dangerous part.

```
x = 0
y = 10
if x != 0 and (y / x) > 2:
    print("ratio is large")
else:
    print("skipped safely")
```

OUTPUT

```
skipped safely
```

$x \neq 0$ is False, so AND short-circuits and y / x is never attempted. If you wrote the division first, the program would crash. Same idea in the soldier algorithm: a False height test skips Step 6.

REMEMBER IT

Put the **cheapest / most likely False** test first in an AND, and the **most likely True** test first in an OR. That is short-circuiting used on purpose.

COMMON MISTAKE

Mistake: writing a chain of nested IFs when one compound condition would do (as in the velocity example in 2.2.2).

Fix: flatten with AND/OR, and rely on short-circuiting instead of extra layers.

IF پروسیسر سے True/False پوچھتی ہے اس لیے زیادہ شاخیں کارکردگی گھٹاتی ہیں۔ اندرونی IF اکثر brute-force ہوتی ہے۔ شارٹ سرکٹ (Short-circuit) کا مطلب ہے جیسے ہی جواب معلوم ہو جانچ روک دی جائے، مثلاً AND میں پہلی شرط False ہو تو دوسری نہ چلے۔

CHECK YOURSELF

1. Why do extra IF statements reduce performance?
2. What does nested IF often indicate?
3. In $x \neq 0$ and $(y / x) > 2$ with $x = 0$, why is there no division-by-zero?

Answers: 1. Each condition must be evaluated True/False by the CPU. 2. A brute-force approach (and it reduces readability and speed). 3. AND short-circuits: the first part is False, so the division is never evaluated.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Number of conditions affects:

- a. How many decisions (and often branches) you pay for
- b. Only the font
- c. Only RAM manufacturing
- d. Only UX colour

MCQ 2. A nested if inside a loop can make work:

- a. Depend on data (sometimes skip inner work)
- b. Always n^2
- c. Always $O(1)$
- d. Always crash

MCQ 3. Short-circuit and/or can:

- a. Skip later tests
- b. Sort the array
- c. Open a file
- d. Draw a pie

MCQ 4. Too many rare special cases:

- a. Hurt clarity and can hurt time if mis-ordered
- b. Always reduce Big O class
- c. Always increase RAM
- d. Always fix UX

MCQ 5. Binary search's conditions:

- a. Few, but they cut n in half — very efficient
- b. n^2
- c. Are UX heuristics
- d. Are SQL joins

MCQ 6. Counting conditions is part of the:

- a. Efficiency evaluation framework
- b. HCI Nielsen list only
- c. ER cardinality only
- d. MVP canvas only

Extra short questions

1. What is a "condition" in this chapter's sense?
2. How can conditions save time?
3. How can they waste time?
4. Link to readability.

Extra long question

1. Two versions of search: (A) no break (B) break when found. Compare conditions, time, traces on a hit at position 2 of 10.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. A true/false test the algorithm performs (if, loop test, while).
2. Early exit: if found, break; if $x < 0$, skip expensive work.
3. Re-testing the same expensive predicate; poorly ordered or/and.
4. A tower of nested ifs is slow to understand even if $O(n)$.

Long

1. A always 10 tests. B 2 tests. Same Big O class worst-case still $O(n)$, but average better. Show trace tables.

2.3.3 Number of Repetitions

LEARN IT

Repetitions (loops) are the most significant factor in algorithm efficiency. The **structure of the loop** decides how time grows with n .

Type of repetition	Conceptual impact	Efficiency grade
Linear loop	Processes each element exactly once. Time grows in proportion to size.	Good — $O(n)$
Logarithmic repetition	Reduces the problem size by half each iteration (binary search). Huge time savings.	Excellent — $O(\log n)$
Nested loops	For each element, process all other elements. Operations explode.	Poor — $O(n^2)$

WORKED EXAMPLE

Worked example: count the inner-body runs.

```
n = 5
linear = 0
for i in range(n):
    linear = linear + 1

nested = 0
for i in range(n):
    for j in range(n):
        nested = nested + 1

print(linear, nested)
```

OUTPUT

```
5 25
```

One loop: 5. Nested loops: $25 = 5^2$. For $n = 1,000$ that would be 1,000 versus 1,000,000.

REMEMBER IT

Find the **deepest loop**. One loop $\approx O(n)$. Loop inside a loop $\approx O(n^2)$. A loop that halves $n \approx O(\log n)$.

COMMON MISTAKE

Mistake: counting the word `for` and ignoring how the index moves.

Fix: a loop `while n > 1: n = n // 2` is logarithmic, even though it is still “a loop”.

اردو میں سمجھیں

دہرائی (لوپ) کارکردگی کا سب سے بڑا عامل ہے۔ ایک بار ہر عنصر پر چلنے والا لوپ $O(n)$ (اچھا)،
ہر بار مسئلہ آدھا کرنے والا $O(\log n)$ (بہترین)، اور اندرونی لوپ $O(n^2)$
(خراب) ہے۔ اندرونی جسم کے چلنے کی گنتی سے یہ فرق صاف نظر آتا ہے۔

CHECK YOURSELF

1. What grade does the source give nested loops?
2. Why is binary search's loop logarithmic rather than linear?
3. If $n = 5$, how many times does a fully nested n -by- n pair of loops run?

Answers: 1. Poor — $O(n^2)$. 2. Each iteration halves the remaining problem, so the number of iterations is about $\log n$. 3. 25.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. for i in range(3) runs:

- a. i=0,1,2
- b. i=1,2,3
- c. i=3,2,1
- d. forever

MCQ 2. while repeats while the condition is:

- a. True
- b. False
- c. A file
- d. None always

MCQ 3. break:

- a. Exits the loop immediately
- b. Skips one line only
- c. Closes Python
- d. Commits SQL

MCQ 4. continue:

- a. Skips the rest of this iteration
- b. Ends the program
- c. Defines a function
- d. Opens a file

MCQ 5. A nested loop: inner runs:

- a. Fully for each outer step
- b. Once in total
- c. Never
- d. Only if NAND

MCQ 6. range(2, 6) yields:

- a. 2,3,4,5
- b. 2,3,4,5,6
- c. 1,2,3,4,5,6
- d. 6,5,4

Extra short questions

1. When for vs while?
2. What is an infinite loop?
3. Trace s=0; for i in range(1,4): s+=i.
4. Shopping bill uses which loop typically?

Extra long question

1. Write nested loops to print a 3×3 table of i,j. Then rewrite the shopping-bill outline with a while, a running total, and an if on quantity.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. for: known count / iterate a sequence. while: unknown count, stop on a condition.
2. while True without break, or a condition that never becomes False.
3. i=1 s=1; i=2 s=3; i=3 s=6.
4. while more items (unknown count) or for a known list of items.

Long

1. for i in range(3): for j in range(3): print(i,j). Bill: total=0; while True: read item/qty; if qty<=0: continue; if item=="end": break; total+=...; print total.

2.3.4 The Efficiency Evaluation Framework

LEARN IT

To evaluate an algorithm at a conceptual level, ask three questions:

1. **Does it scale?** If n grows 10 times, does time jump 10 times (linear) or 100 times (quadratic)? Quadratic scaling fails for large datasets.
2. **What is the bottleneck?** Focus on the most time-consuming section (usually the deepest loop) and try to improve that part.
3. **Is the cost worth it?** A theoretically faster $O(\log n)$ algorithm can be too complex to code and maintain. For a list of just 50 items, a simple $O(n)$ linear search is practically better because it is simple.

Does it scale?



Where is the bottleneck?



Is the extra complexity worth it?

WORKED EXAMPLE

Worked example: 50 names versus 1 million names.

Situation	Scale	Bottleneck	Worth the cost?	Choice
50 unsorted class names	Tiny n	A single linear scan	Binary search needs a sort first, which is extra work and extra code	Linear search $O(n)$
1,000,000 sorted account numbers, searched all day	$n \times 10$ compared with 100,000 is already huge; quadratic would be impossible	The search loop	Yes — binary search is standard and well understood	Binary search $O(\log n)$

REMEMBER IT

Framework mnemonic: **S-B-W** — **S**cale, **B**ottleneck, **W**orth it.

COMMON MISTAKE

Mistake: always choosing the theoretically fastest Big O.

Fix: question 3 exists for a reason. For $n = 50$, simple $O(n)$ beats a fragile $O(\log n)$ implementation.

کارکردگی کے تین سوال: کیا الگورتھم بڑے n پر چل سکتا ہے (Scale)؟ سب سے سست جگہ کون سی ہے (Bottleneck، عموماً گہرا لوپ)؟ کیا تیز تر طریقے کی پیچیدگی اس چھوٹے فائدے کے لائق ہے؟ پچاس

ناموں کے لیے سادہ $O(n)$ تلاش عملی طور پر بہتر ہو سکتی ہے۔

CHECK YOURSELF

1. If n grows $10\times$, how much slower is a quadratic algorithm?
2. Where do you usually look for the bottleneck?
3. Why might linear search beat binary search for 50 items?

Answers: 1. About $100\times$ slower (10^2). 2. The most time-consuming section, usually the deepest loop. 3. Binary search is more complex to code and maintain, and for $n = 50$ the simplicity of $O(n)$ is practically better.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. The framework looks at:

- a. Steps, conditions, repetitions (then Big O)
- b. Only logos
- c. Only careers
- d. Only Figma

MCQ 2. "Number of steps" is:

- a. Basic operations in one pass / closed form
- b. The page count of the PDF
- c. OSI layers
- d. Sprint days

MCQ 3. You combine the three to:

- a. Justify a Big O class
- b. Pick a colour
- c. Pick a font
- d. Pick a career

MCQ 4. A 5-mark compare question wants:

- a. The framework on both algorithms + a verdict
- b. Only names
- c. Only Python libraries
- d. Only UX

MCQ 5. Constants matter at tiny n but the framework emphasises:

- a. Growth
- b. Hex colours
- c. Teacher names
- d. PDF margins

MCQ 6. If repetitions are nested $n \times n$, Big O is likely:

- a. n^2 regardless of a few extra ifs
- b. $\log n$
- c. 1
- d. $n!$

Extra short questions

1. State the efficiency evaluation framework.
2. Apply in one line to binary search.
3. Apply to bubble sort.
4. When would you still pick the slower class?

Extra long question

1. Using the framework, compare selection sort vs binary search as if a student mixed them up. Then say why they are not alternatives.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Count significant steps, conditions, repetitions; combine into time/space class; compare to requirements.
2. Few steps per shot, one/two conditions, log n repetitions → $O(\log n)$ time, $O(1)$ space.
3. Compare+swap steps, a condition, nested repetitions → $O(n^2)$.
4. Tiny n, need stability/simplicity, or the fast one needs a costly precondition (sort first).

Long

1. Different problems (order vs find). Framework still: both have loops; sort n^2 vs search log n. Verdict: sort then many binary queries if the list is reused.

2.4 Refinements to improve clarity and efficiency

LEARN IT

Improving an algorithm means reviewing it step by step: simplifying logic, removing redundant work, and picking smart data structures. A clear algorithm is easier to follow; an efficient one wastes fewer steps.

2.4.1 Refinements for clarity

1. **Use meaningful names.** Replace vague variables with descriptive ones. Bad: `a = b / c`. Good: `average = totalSum / totalItems`.
2. **Break into modules.** Divide a massive block into functions, for example separating `calculateSum()` from the printing logic.
3. **Improve structure and indentation.** Packed loops and IF statements confuse the reader. Spacing reduces mental effort.
4. **Add comments.** Keep them short. Explain **why** something is done, not just what is happening, especially for tricky logic.

2.4.2 Refinements for efficiency

1. **Reduce unnecessary steps.** In Bubble Sort, after the first pass the largest value has settled at the end. Ignore that sorted tail on the next pass by shrinking the inner loop, and you save comparisons.
2. **Use better algorithms.** Searching 1 million **sorted** records with Binary Search instead of Linear Search changes the time from seconds to milliseconds.
3. **Avoid re-computing values** (memoization / caching). If a value never changes, compute it once outside the loop, store it, and reuse it.
4. **Use appropriate data structures.** Direct access → array. Hierarchical data → tree. Do not force one structure everywhere.

WORKED EXAMPLE

Worked example 1 — shrinking Bubble Sort's inner loop ($n = 5$). A naive inner loop always runs $n-1 = 4$ times, for 4 passes: 16 comparisons. The refined inner loop runs 4, then 3, then 2, then 1: 10 comparisons.

```
n = 5
naive = 0
for i in range(n - 1):
    for j in range(n - 1):
        naive = naive + 1

refined = 0
for i in range(n - 1):
    for j in range(n - 1 - i):
        refined = refined + 1

print(naive, refined)
```

OUTPUT
16 10

Worked example 2 — do not re-compute a constant inside the loop.

```
A = 3
B = 4
N = 5

# BAD: C is recalculated every time
for i in range(1, N + 1):
    C = A + B
    print(C * i)
```

OUTPUT
7
14
21
28
35

```
A = 3
B = 4
N = 5
# GOOD: compute C once
C = A + B
for i in range(1, N + 1):
    print(C * i)
```

OUTPUT

7
14
21
28
35

Same output, fewer additions. That is caching: compute once, reuse N times.

REMEMBER IT

Clarity: **N-M-I-C** (Names, Modules, Indentation, Comments). Efficiency: **S-A-C-D** (Shrink steps, Algorithm choice, Cache, Data structure).

COMMON MISTAKE

Mistake: “optimising” by renaming variables and thinking that is an efficiency refinement.

Fix: meaningful names improve **clarity**. Efficiency refinements change the **amount of work** (fewer loop iterations, a better algorithm, no re-compute, a better structure).

اردو میں سمجھیں

وضاحت بہتر کرنے کے لیے معنی خیز نام، ماڈیول، درست indentation اور مختصر تبصرے استعمال کریں۔ کارکردگی کے لیے فضول قدم ہٹائیں (بیل سارٹ میں ترتیب شدہ دم نظر انداز کریں)، بہتر الگورتھم چنیں (لینیئر کی جگہ بائنری سرچ)، لوپ کے اندر وہی حساب بار بار نہ کریں (Caching/Memoization)، اور مناسب ڈیٹا سٹرکچر رکھیں۔

CHECK YOURSELF

1. Rewrite $a = b / c$ with meaningful names for an average.
2. After Bubble Sort's first pass, what should the next pass ignore?
3. In the A+B example, where should $c = A + B$ be written?

Answers: 1. $average = totalSum / totalItems$. 2. The already-sorted largest value at the end (shrink the inner loop). 3. Once, outside the loop, then reuse C inside.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A clarity refinement is:

- a. Rename, split, comment, simplify logic
- b. Add more nested loops for fun
- c. Shorten names to a,b,c
- d. Remove all spaces

MCQ 2. An efficiency refinement is:

- a. Cut extra work (e.g. already-sorted inner of bubble)
- b. Add random sleeps
- c. Copy the list 12 times
- d. Print inside the inner loop 1e6 times

MCQ 3. Flagged bubble sort's early exit is:

- a. Efficiency refinement
- b. A new UX colour
- c. A foreign key
- d. A pie

MCQ 4. Extracting a function is:

- a. Clarity (and sometimes reuse) refinement
- b. Always worse Big O
- c. OSI layering
- d. A histogram

MCQ 5. Do not "optimise" first if:

- a. The algorithm is still incorrect or unreadable
- b. $n=10^{12}$ already proven
- c. Tests all pass and profiler points here
- d. It is $O(n!)$ on huge n and you must ship

MCQ 6. Redundant work means:

- a. Computing the same thing twice
- b. A needed loop
- c. A primary key
- d. A caption

Extra short questions

1. Two clarity refinements.
2. Two efficiency refinements from the book.
3. A refinement that hurts clarity for 1% speed is:
4. How do you know a refinement worked?

Extra long question

1. Show bubble sort before/after: (1) meaningful names (2) last-i unsorted bound (3) swapped flag. State what each changed (clarity vs efficiency).

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Better names; extract module; remove duplicate logic; add pre/post comments.
2. Stop inner bubble as the suffix grows sorted; avoid extra copies; fewer conditions in the inner loop.
3. Usually a bad deal in XI/XII and in teams.
4. Same tests (correctness) + fewer operations or clearer reading.

Long

1. Code/pseudocode both versions. (1) clarity (2) fewer comparisons $n^2/2$ still $O(n^2)$ (3) best case $O(n)$. Tests unchanged.

2.5 Concept of Data Structure ★ Golden

LEARN IT

A **data structure** is a systematic way of **organising, storing and managing** data so that an algorithm can access, modify and process it efficiently.

Real-life analogy. Books on a shelf are arranged in a set order, so you can find a title quickly. Files in folders are grouped on purpose. Data structures do the same job inside computer memory.

1. Linear data structures

Elements stand in a strict sequence, one after another. Each element (except the first and last) has exactly **one predecessor** and **one successor**.

Examples: Array, Linked List, Stack, Queue.

Linear (sequence)

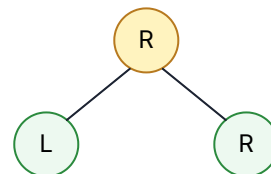


2. Non-linear data structures

Elements are connected in a hierarchy or a network. One element can connect to **several** others at the same time.

Examples: Tree, Graph.

Non-linear (branch / net)



Linear: a line of neighbours. Non-linear: a node with more than one connection.

WORKED EXAMPLE

Worked example: classify each everyday picture.

1. Students standing in a single assembly line → **linear** (each has one in front and one behind).
2. A family tree of grandparents, parents and children → **non-linear** (one parent, several children).
3. Roads linking Karachi, Hyderabad, Sukkur and Larkana in many ways → **non-linear** (a network).

REMEMBER IT

Ask: "Does each item have at most one next item?" Yes → linear. "Can one item link to many, or can links form a net?" → non-linear.

COMMON MISTAKE

Mistake: calling a tree “linear” because you can walk it from top to bottom.

Fix: walking is traversal. The **shape** is hierarchical: one parent, many children. That is non-linear.

اردو میں سمجھیں

ڈیٹا سٹرکچر ڈیٹا کو منظم طریقے سے رکھنے کا نظام ہے تاکہ الگورتھم اسے جلدی استعمال کر سکے۔ لکیری ڈھانچوں میں عناصر قطار میں ہوتے ہیں (ایرے، لنکڈ لسٹ، سٹیک، کیو)۔ غیر لکیری میں ایک عنصر کئی سے جڑ سکتا ہے (ٹری، گراف)۔ الماری کی کتابیں اور فولڈرز کی مثال یاد رکھیں۔

EXAM TIP

Define data structure in full (organise, store, manage; efficient access and processing). Then split linear vs non-linear with the “one predecessor and one successor” test, and list the four linear and two non-linear examples. The shelf / folder analogy earns a mark. MCQs: “Which is non-linear?” → Tree or Graph.

CHECK YOURSELF

1. Define data structure.
2. Name four linear and two non-linear examples.
3. What is true of each middle element in a linear structure?

Answers: 1. A systematic way of organising, storing and managing data for efficient access, modification and processing. 2. Linear: array, linked list, stack, queue. Non-linear: tree, graph. 3. It has exactly one predecessor and one successor.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A data structure is:

- a. An organised way to store and access data
- b. A random pile
- c. A font
- d. A sprint

MCQ 2. We pick a structure for:

- a. The operations we need (fast insert? fast search?)
- b. The logo
- c. The teacher's birthday
- d. The PDF theme

MCQ 3. Linear vs non-linear is about:

- a. Whether elements form a sequence vs branches/links
- b. Colour
- c. File format .py vs .txt
- d. Agile vs waterfall

MCQ 4. An array gives fast:

- a. Index access $O(1)$
- b. Insert at front $O(1)$ always
- c. Arbitrary graph neighbours
- d. Sorted insert $O(1)$ always

MCQ 5. This is ★ because papers ask:

- a. Define + why we need DS + one example
- b. Only careers
- c. Only Figma
- d. Only OSI

MCQ 6. A queue of print jobs is a DS chosen because:

- a. Order of arrival matters (FIFO)
- b. LIFO undo
- c. Hierarchy of boss-staff
- d. Random friends links

Extra short questions

1. Define data structure.
2. Why not always use one big list?
3. Give one linear and one non-linear example.
4. What is an operation on a DS?

Extra long question

1. "If you pick the wrong structure, the algorithm looks harder." Explain with college examples of array, stack, queue, tree. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. A systematic way to organise data for efficient access and modification.
2. Some problems need LIFO, FIFO, hierarchy, or many-to-many links.
3. Linear: array/stack/queue. Non-linear: tree/graph.
4. Insert, delete, search, traverse, peek...

Long

1. Roll list: array. Undo typing: stack. Printer/canteen line: queue. Org chart/folders: tree.
Wrong pick: using a stack for a fair queue angers people.

2.5.1 Linear Data Structures (Array, Linked List, Stack, Queue)

★ Golden

LEARN IT

An **array** holds a collection of **similar items** (the same data type) placed next to each other in **contiguous memory** (one continuous block).

Each item has a position number called an **index**, usually starting from **0**. Because the address of any cell can be calculated from the index, you can jump straight to it. That is **direct access** (also called random access), and it is $O(1)$.

Array name: A

23	76	12	90	34	68
0	1	2	3	4	5

A[3] = 90 (index starts at 0)

Six integers in contiguous cells. Direct access: A[3] jumps to 90 without walking 23, 76 and 12.

WORKED EXAMPLE

Worked example: read by index.

```
A = [23, 76, 12, 90, 34, 68]
print(A[3])
print(A[0])
```

OUTPUT

```
90
23
```

REMEMBER IT

Index 0 is the first shelf on the row. The third item that humans call “number 4” is index 3. Formula in your head: **position from the left, minus one**.

COMMON MISTAKE

Limitation. Arrays are fast to read by index but **inflexible** to grow in the middle. Inserting a new value between 12 and 90 means shifting 90, 34 and 68 one cell to the right. That shift is $O(n)$, so arrays are a poor fit when you insert and delete in the middle all the time.

ایرے ایک جیسے عناصر کو میموری کے لگاتار بلاک میں رکھتا ہے۔ ہر عنصر کا انڈیکس (عموماً 0 سے) ہوتا ہے جس سے براہ راست رسائی $O(1)$ میں مل جاتی ہے، جیسے $A[3] = 90$ ۔ درمیان میں نیا عنصر ڈالنے کے لیے دائیں طرف شفٹ کرنا پڑتا ہے جو سست ہے۔

CHECK YOURSELF

1. What does "contiguous memory" mean?
2. In the diagram, what is $A[2]$?
3. Why is inserting in the middle of an array inefficient?

Answers: 1. The items sit in one continuous block, next to each other. 2. 12. 3. Later elements must all be shifted right to make room.

LEARN IT

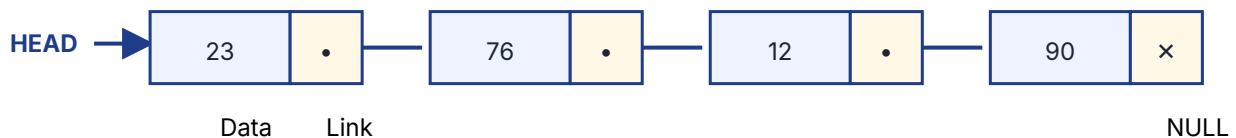
A **linked list** stores data as a chain. Unlike an array, the items are **not** one continuous block. Each separate piece is a **node** sitting somewhere in memory.

Each node has two parts:

1. the actual **data**;
2. a **link** (pointer) holding the memory address of the **next** node.

The first node is pointed to by **HEAD**. The last node's link is **NULL** (nowhere). To find an item you start at HEAD and follow links one by one — **sequential access**, $O(n)$.

Real-life example: browser history. Clicking Back follows the pointer to the previous page-node.



Follow HEAD → 23 → 76 → 12 → 90 → NULL

Four nodes. The last link is NULL, which ends the chain.

WORKED EXAMPLE

Worked example: walk from HEAD to print every data field.

```
nodes = [23, 76, 12, 90]
for data in nodes:
    print(data, end=" → ")
print("NULL")
```

OUTPUT

23 → 76 → 12 → 90 → NULL

There is no `nodes[2]` jump in a real linked list. The loop above models following each link once. Inserting a new node between 76 and 12 only means changing two pointers — you do not shift the rest of a big block, which is why lists beat arrays for frequent insert/delete.

REMEMBER IT

Array = row of lockers with numbers on the doors. Linked list = treasure hunt: each clue (link) tells you where the next box is.

COMMON MISTAKE

Mistake: writing `list[3]` and calling it linked-list access.

Fix: a true linked list has no index. Python's `list` type is an array-like structure. In this chapter, "linked list" means HEAD-and-pointers.

اردو میں سمجھیں

لنکڈ لسٹ زنجیر کی طرح نوڈز پر مشتمل ہے جو میموری میں بکھرے ہو سکتے ہیں۔ ہر نوڈ میں ڈیٹا اور اگلے نوڈ کا پتہ (Link) ہوتا ہے۔ پہلا نوڈ HEAD سے ملتا ہے، آخری کا لنک NULL ہوتا ہے۔ براؤزر کا Back بٹن پچھلے نوڈ کے پوائنٹر پر چلتا ہے۔ رسائی ترتیب وار ہے، انڈیکس سے چھلانگ نہیں۔

CHECK YOURSELF

1. Name the two parts of a node.
2. What does the last node's link store?
3. Why is browser history a fair analogy?

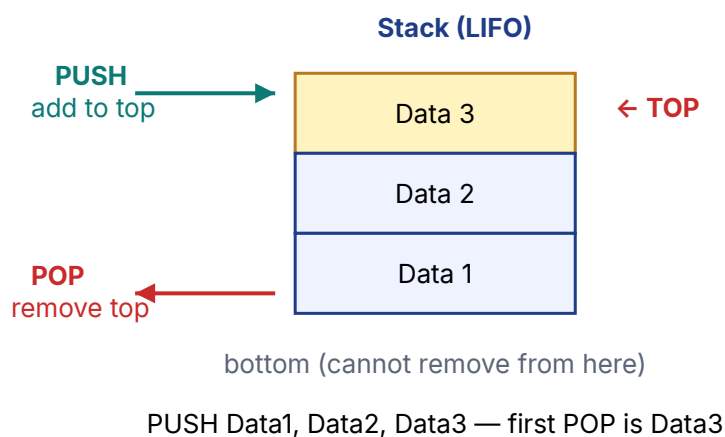
Answers: 1. Data, and a link/pointer to the next node. 2. NULL (no next node). 3. Back follows the pointer to the previous page-node instead of jumping by index.

LEARN IT

A **stack** is a linear data structure that works on the **LIFO** principle: **Last-In, First-Out**. The last element added is the first one removed. You cannot pull an item from the bottom or the middle.

- **PUSH** — add an item to the **top**.
- **POP** — remove the item from the **top**.

A **TOP** pointer always names the current top item. **Real-life example:** Undo in a text editor. Every action is piled on the stack; Undo POPs the most recent action first.



Three PUSH operations. The TOP is Data 3, so the first POP returns Data 3.

WORKED EXAMPLE

Worked example: PUSH / POP trace. Start with an empty stack. Then PUSH Data1, PUSH Data2, PUSH Data3, POP, POP.

Step	Operation	Stack (bottom → top)	TOP	Popped value
0	(empty)	[]	none	
1	PUSH Data1	[Data1]	Data1	
2	PUSH Data2	[Data1, Data2]	Data2	
3	PUSH Data3	[Data1, Data2, Data3]	Data3	
4	POP	[Data1, Data2]	Data2	Data3
5	POP	[Data1]	Data1	Data2

```
stack = []
stack.append("Data1")
stack.append("Data2")
stack.append("Data3")
print("after 3 PUSHes:", stack)
print("POP:", stack.pop())
print("POP:", stack.pop())
print("left:", stack)
```

OUTPUT

```
after 3 PUSHes: ['Data1', 'Data2', 'Data3']
POP: Data3
POP: Data2
left: ['Data1']
```

Python's `list.append` is PUSH and `list.pop()` (no index) is POP from the top.

REMEMBER IT

LIFO = Last In First Out = "last plate washed is first plate used" on a canteen pile. Undo, Redo, Back button and function calls are all stacks.

COMMON MISTAKE

Mistake: POPing from the bottom, or mixing LIFO with FIFO.

Fix: stack = plates (top only). Queue = ticket line (front leaves, rear joins). If the first POP is Data1, you drew a queue, not a stack.

سٹیک LIFO پر چلتا ہے: جو سب سے بعد میں آیا وہ سب سے پہلے نکلے گا۔ اوپر رکھنا PUSH، اوپر سے ہٹانا POP، اور TOP پوائنٹر موجودہ سب سے اوپر عنصر بتاتا ہے۔ انڈو (Undo) کی مثال: ہر عمل ڈھیر پر پڑتا ہے، انڈو سب سے تازہ عمل ہٹاتا ہے۔

EXAM TIP

Define stack + LIFO, define PUSH and POP, mention TOP, give the Undo example, and draw the vertical boxes with TOP labelled. Trace questions: "Show the stack after PUSH A, PUSH B, POP, PUSH C." Work left to right; the first POP must be B. Never remove from the middle.

CHECK YOURSELF

1. Expand LIFO.
2. After PUSH Data1, Data2, Data3, what does the first POP return?
3. Why is Undo modelled as a stack?

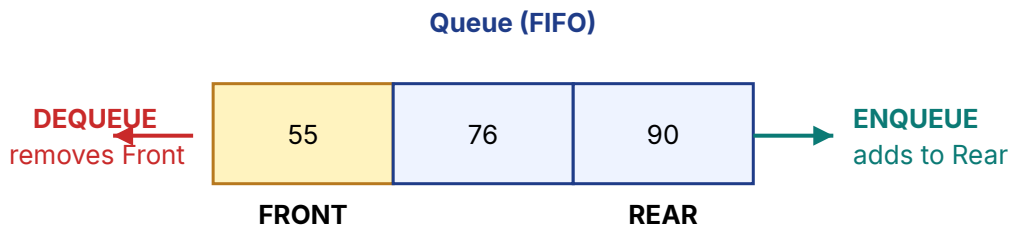
Answers: 1. Last-In, First-Out. 2. Data3. 3. Each action is PUSHed; Undo POPs the most recent action first.

LEARN IT

A **queue** organises data on the **FIFO** principle: **First-In, First-Out**. Whoever arrives first is served first. Order truly matters.

- **ENQUEUE** — add an element at the **Rear** (back).
- **DEQUEUE** — remove an element from the **Front**.

Real-life example: several documents sent to a printer. The printer prints the first document received, then the next.



Next ENQUEUE joins after 90. Next DEQUEUE removes 55.

Three jobs waiting. New jobs join at the rear; service always takes the front.

WORKED EXAMPLE

Worked example: ENQUEUE / DEQUEUE trace. Start empty. ENQUEUE 55, 76, 90, then ENQUEUE D, then DEQUEUE.

Step	Operation	Queue (Front → Rear)	Front	Rear	Removed
0	(empty)	[]	—	—	
1	ENQUEUE 55	[55]	55	55	
2	ENQUEUE 76	[55, 76]	55	76	
3	ENQUEUE 90	[55, 76, 90]	55	90	
4	ENQUEUE D	[55, 76, 90, D]	55	D	
5	DEQUEUE	[76, 90, D]	76	D	55

```
from collections import deque
q = deque()
q.append(55)
q.append(76)
q.append(90)
q.append("D")
print("after ENQUEUE D:", list(q))
print("DEQUEUE:", q.popleft())
print("left:", list(q))
```

OUTPUT

```
after ENQUEUE D: [55, 76, 90, 'D']
DEQUEUE: 55
left: [76, 90, 'D']
```

`append` is ENQUEUE at the rear; `popleft` is DEQUEUE from the front. The first DEQUEUE is 55 (the oldest), never D.

REMEMBER IT

FIFO = First In First Out = a canteen line. Front = the person being served. Rear = the person who just joined.

COMMON MISTAKE

Mistake: DEQUEUEing from the rear, or ENQUEUEing at the front.

Fix: new items only at Rear; removal only at Front. If your first removal is D, you treated the queue as a stack.

کیو FIFO پر چلتی ہے: جو پہلے آیا وہ پہلے نکلے۔ پیچھے (Rear) شامل کرنا ENQUEUE، آگے (Front) سے ہٹانا DEQUEUE ہے۔ پرنٹر کی قطار: جو فائل پہلے بھیجی گئی وہ پہلے چھپے گی۔ پہلی DEQUEUE ہمیشہ سب سے پرانا عنصر نکالتی ہے۔

EXAM TIP

Define queue + FIFO, define ENQUEUE (rear) and DEQUEUE (front), give the printer example, draw the labelled Front/Rear diagram, and trace a short sequence. Examiners mix stack and queue on the same paper: if they say "first come, first served" write Queue; if they say "Undo" write Stack.

CHECK YOURSELF

1. Expand FIFO. Where is an item added, and where is it removed?
2. After ENQUEUE 55, 76, 90, what does DEQUEUE return?
3. Why is a printer a queue, not a stack?

Answers: 1. First-In, First-Out; added at Rear, removed from Front. 2. 55. 3. Documents must print in the order they arrived (first received, first printed), not last-received first.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An array stores items:

- a. In contiguous indexed slots of one type
- b. With arbitrary node links only
- c. As a hierarchy always
- d. As a pie

MCQ 2. A linked list stores items:

- a. In nodes with pointers to the next
- b. In one block of equal indexes only
- c. As a tree of bosses
- d. As a hash of pies

MCQ 3. A stack is:

- a. LIFO
- b. FIFO
- c. A graph
- d. A GUI

MCQ 4. A queue is:

- a. FIFO
- b. LIFO
- c. A binary tree
- d. A K-map

MCQ 5. Undo in an editor uses a:

- a. Stack
- b. Queue
- c. Pie
- d. OSI layer

MCQ 6. A printer line is a:

- a. Queue
- b. Stack
- c. AVL tree always
- d. NAND

Extra short questions

1. Define array, linked list, stack, queue in one line each.
2. One op each: push/pop, enqueue/dequeue.
3. Array vs list for insert at front.
4. Peek means:

Extra long question

1. Draw stack and queue after the sequence: insert A,B,C then remove one. Show remaining order and a real example of each. ★ Golden for stack/queue.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Array: indexed block. List: nodes+links. Stack: LIFO. Queue: FIFO.

2. Stack push/pop top. Queue enqueue rear, dequeue front.
3. Array shifts $O(n)$; list pointer change $O(1)$ if you have the head.
4. Read the top/front without removing.

Long

1. Stack remaining B,A (C popped). Queue remaining B,C (A served). Examples: back button vs ticket counter. Diagrams with top/front labelled.

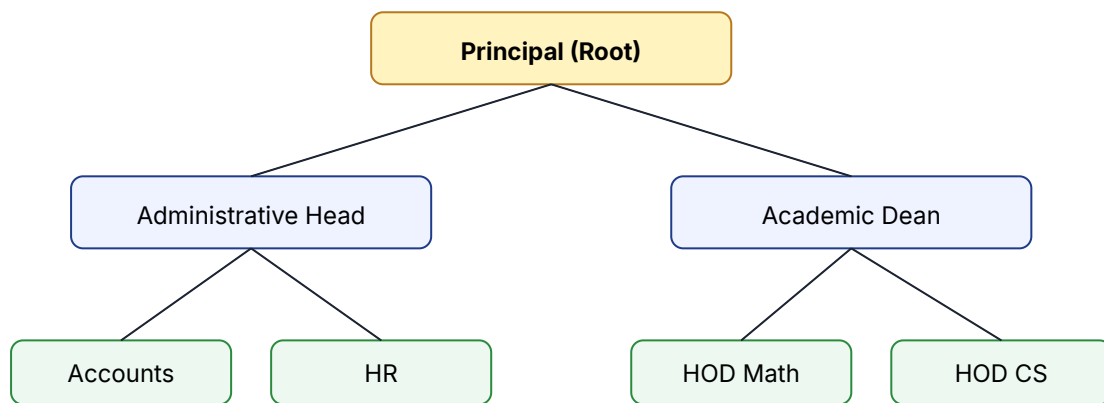
2.5.2 Non-Linear Data Structures (Tree, Graph)

LEARN IT

A **tree** organises data in a **hierarchy**. Everything branches out from a single starting point called the **root**.

It creates a strict **one-to-many (parent-child)** relationship. That is an excellent way to show how roles flow in an organisation.

Real-life examples: a family tree (grandparents → parents → children) and folders inside folders on a drive.



College tree: one root (Principal), then parent-child branches. No child has two parents in this model.

WORKED EXAMPLE

Worked example: name the relationships.

- Root: Principal.
- Children of the root: Administrative Head, Academic Dean.
- Children of Academic Dean: HOD Math, HOD CS.
- HOD CS has one parent (Academic Dean), not two — that is the tree rule. If HOD CS also reported sideways to Accounts as an equal link, the picture would become a **graph**.

REMEMBER IT

Tree = family / folders / organogram. One root. Branches down. Parent-child, not a web of equals.

COMMON MISTAKE

Mistake: drawing a tree with two roots, or with an arrow going back up to the Principal from HOD CS (a cycle).

Fix: one root, links only parent → child, no loops. Cycles belong to graphs.

اردو میں سمجھیں

ٹری ڈیٹا کو درجہ بندی (Hierarchy) میں رکھتی ہے جو ایک جڑ (Root) سے پھوٹتی ہے۔ تعلق والدین-بچہ (ایک سے کئی) ہوتا ہے، جیسے پرنسپل → ڈین → ایچ او ڈی، خاندانی شجرہ، یا فولڈر کے اندر فولڈر۔ دو جڑیں یا چکر ٹری کو گراف بنا دیتے ہیں۔

CHECK YOURSELF

1. What is the starting node of a tree called?
2. What kind of relationship does a tree model?
3. Give two real-life tree examples from the source.

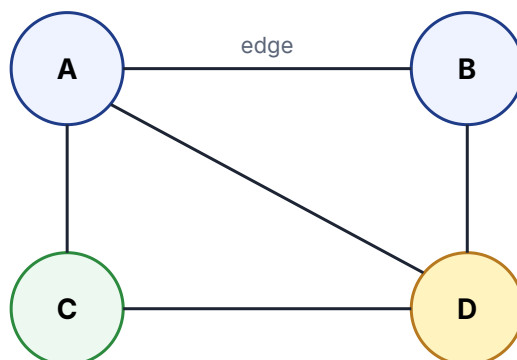
Answers: 1. The root. 2. One-to-many parent-child. 3. A family tree, and folders inside folders (or a college organogram).

LEARN IT

A **graph** organises data when things connect in a flexible, complex way. Unlike a tree, there is **no strict hierarchy**: any node can connect to any other node.

- **Nodes (vertices)** — the items (cities, people).
- **Edges** — the connections (roads, friendships).

Examples: Facebook friends; GPS mapping (the fastest route between two cities).



Four vertices A, B, C, D. Edges can form a cycle (A-B-D-C-A). A tree cannot do that.

WORKED EXAMPLE

Worked example: Karachi – Hyderabad – Sukkur – Larkana. Cities are vertices. Roads are edges. You can travel Karachi → Hyderabad → Sukkur or Karachi → Larkana → Sukkur. There is no single “root city”. That is a graph, and GPS uses it to pick a fast route.

REMEMBER IT

Tree $\subset$ Graph in shape-talk: a tree is a special graph with one root and no cycles. If you need cycles or many-to-many links, say **graph**.

COMMON MISTAKE

Mistake: using a tree for GPS because “Hyderabad comes under Sindh”.

Fix: administration can be a tree; **roads** are a graph because you can loop and take alternative paths.

اردو میں سمجھیں

گراف لچکدار روابط کے لیے ہے: کوئی سخت درجہ بندی نہیں، کوئی بھی نوڈ کسی سے جڑ سکتا ہے۔ نوڈز (Vertices) چیزیں ہیں، کنارے (Edges) تعلق۔ فیس بک دوست اور GPS نقشہ کلاسیکی مثالیں ہیں۔ چکر (Cycle) اور کئی راستے گراف کو ٹری سے الگ کرتے ہیں۔

CHECK YOURSELF

1. What are the two parts of a graph?
2. Give two source examples of graphs.
3. State one difference between a tree and a graph.

Answers: 1. Nodes/vertices and edges. 2. Social networks (Facebook friends) and GPS mapping. 3. A tree has a hierarchy and one root; a graph has no strict hierarchy and any node may connect to any other (cycles allowed).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An array stores items:

- a. In contiguous indexed slots of one type
- b. With arbitrary node links only
- c. As a hierarchy always
- d. As a pie

MCQ 2. A linked list stores items:

- a. In nodes with pointers to the next
- b. In one block of equal indexes only
- c. As a tree of bosses
- d. As a hash of pies

MCQ 3. A stack is:

- a. LIFO
- b. FIFO
- c. A graph
- d. A GUI

MCQ 4. A queue is:

- a. FIFO
- b. LIFO
- c. A binary tree
- d. A K-map

MCQ 5. Undo in an editor uses a:

- a. Stack
- b. Queue
- c. Pie
- d. OSI layer

MCQ 6. A printer line is a:

- a. Queue
- b. Stack
- c. AVL tree always
- d. NAND

Extra short questions

1. Define array, linked list, stack, queue in one line each.
2. One op each: push/pop, enqueue/dequeue.
3. Array vs list for insert at front.
4. Peek means:

Extra long question

1. Draw stack and queue after the sequence: insert A,B,C then remove one. Show remaining order and a real example of each. ★ Golden for stack/queue.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Array: indexed block. List: nodes+links. Stack: LIFO. Queue: FIFO.

2. Stack push/pop top. Queue enqueue rear, dequeue front.
3. Array shifts $O(n)$; list pointer change $O(1)$ if you have the head.
4. Read the top/front without removing.

Long

1. Stack remaining B,A (C popped). Queue remaining B,C (A served). Examples: back button vs ticket counter. Diagrams with top/front labelled.

2.5.3 Operations on Data Structures

LEARN IT

Whatever the shape, you keep meeting the same five operations:

Operation	Meaning	Quick picture
Insertion	Adding a new item into the structure.	PUSH on a stack; ENQUEUE on a queue; putting 40 into an array.
Deletion	Removing an item that is no longer needed.	POP; DEQUEUE; deleting a node from a list.
Traversal	Going through each item one by one.	Walking HEAD → ... → NULL; visiting every tree node.
Searching	Finding a specific item.	Linear search in an array; following links until 87 is found.
Sorting	Arranging items in a proper order (ascending or descending).	Bubble Sort on an array of marks.

WORKED EXAMPLE

Worked example: name the operation.

1. "Print every song from the start of the playlist." → **Traversal**.
2. "Add a print job at the rear." → **Insertion** (ENQUEUE).
3. "Undo the last typing action." → **Deletion** (POP).
4. "Is roll number 87 in the list?" → **Searching**.
5. "Arrange heights from shortest to tallest." → **Sorting**.

REMEMBER IT

Memorise **I-D-T-S-S**: Insert, Delete, Traverse, Search, Sort.

COMMON MISTAKE

Mistake: calling a PUSH "sorting" because the stack looks tidy.

Fix: PUSH is insertion. Sorting means putting values in ascending or descending order.

ہر ڈیٹا سٹرکچر پر پانچ عمل ہوتے ہیں: داخل کرنا (Insertion)، حذف کرنا (Deletion)، ایک ایک کر کے گزرنا (Traversal)، تلاش (Searching) اور ترتیب (Sorting، صعودی یا نزولی)۔

CHECK YOURSELF

1. List the five operations.
2. Which operation is ENQUEUE?
3. Traversal vs searching — one difference?

Answers: 1. Insertion, deletion, traversal, searching, sorting. 2. Insertion (at the rear of a queue). 3. Traversal visits every item; searching stops when the target is found (or the structure ends).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Common DS operations include:

- a. Insert, delete, search, traverse
- b. Only compile
- c. Only ping
- d. Only print the logo

MCQ 2. Traverse means:

- a. Visit elements in some order
- b. Delete all
- c. Encrypt all
- d. Sort the OS

MCQ 3. Search cost depends on:

- a. The structure (array vs sorted vs hash vs tree)
- b. The PDF theme
- c. The teacher name
- d. The sprint

MCQ 4. Insert in a full array may:

- a. Fail or require resizing
- b. Always $O(1)$ no matter what
- c. Create a graph edge automatically
- d. Pop a stack

MCQ 5. Delete from a queue happens at the:

- a. Front
- b. Rear only always
- c. Random index
- d. Root of a tree

MCQ 6. You choose a DS by the:

- a. Operations you will do most
- b. First letter of the course
- c. Colour
- d. NAND count

Extra short questions

1. List five operations.
2. Give costs qualitatively: array index vs list search.
3. What does traverse a tree mean?
4. Why "update" might be search+insert.

Extra long question

1. For array, stack, queue, tree: name the natural insert/delete ends and one costly operation. Table.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Create, insert, delete, search, traverse, update, sort (as applicable).
2. Array index $O(1)$; unsorted list search $O(n)$.
3. Visit every node (pre/in/post or level order).
4. You must find the item first.

Long

1. Array: insert middle costly. Stack: top only cheap. Queue: rear in front out. Tree: search/insert depend on balance. 8-row table.

2.6 Identify data structures in problem scenarios

LEARN IT

To pick a structure, first see how the data is organised. If each item has a clear “before and after”, it is **linear**. If it branches or connects to many items, it is **non-linear**.

2.6.1 Linear scenarios

1. **Storing student grades for a class → Array.** You usually know how many students there are, and all grades are the same type. Indices give rapid lookup.
2. **Music playlist / browser history → Linked list.** Every song knows what comes next. When one finishes, follow the pointer. Frequent insert/delete is cheap.
3. **Undo or Redo → Stack (LIFO).** Each typing or formatting action is PUSHed. Undo POPs the most recent action.
4. **Printer task scheduling → Queue (FIFO).** Documents print in the exact order they were received. First-come, first-served.

2.6.2 Non-linear scenarios

1. **Organisational chart of a college → Tree.** Principal is the root, then deans, then HODs, then teachers. Clear parent-child links.
2. **GPS navigation and maps → Graph.** Cities are nodes, roads are edges. No strict top-down hierarchy. Several routes can reach the same destination.

If the data...	Then it is...	Recommended structure
Follows “next” or “previous” logic	Linear	Array, Linked list
Follows “parent” and “child” logic	Non-linear	Tree
Follows “first-come, first-served” logic	Linear	Queue
Follows “connections / relationships” logic	Non-linear	Graph
Needs to be accessed in the reverse order of entry	Linear	Stack

WORKED EXAMPLE

Worked example: match five school problems.

Scenario	Structure	Why (one line)
Store 40 practical marks you already counted	Array	Fixed size, same type, need A[i] lookup.
A playlist where the teacher inserts a nasheed between two songs	Linked list	Insert by moving two pointers, no big shift.
Ctrl+Z after formatting a heading	Stack	Last action must come off first.
Students lining up for the photocopier	Queue	First in line is served first.
Friend links in a class WhatsApp "everyone knows everyone"	Graph	Any student can connect to any other.

REMEMBER IT

Decision order: reverse-of-entry? → stack. First-come? → queue. Parent-child? → tree. Many-to-many routes? → graph. Known list with indexes? → array. Lots of insert/delete in a sequence? → linked list.

COMMON MISTAKE

Mistake: choosing an array for Undo because "actions are stored in a list".

Fix: the **access rule** matters more than "it is a list". Undo is reverse order of entry → stack.

اردو میں سمجھیں

اگر ہر عنصر کا واضح اگلا/پچھلا ہے تو ڈھانچہ لکیری ہے؛ اگر شاخیں یا کئی روابط ہیں تو غیر لکیری۔ نمبر: ایرے؛ پلے لسٹ/ہسٹری: لنکڈ لسٹ؛ انڈو: سٹیک؛ پرنٹر: کیو؛ کالج کا چارٹ: ٹری؛ GPS: گراف۔ فیصلے کی جدول یاد کر لیں۔

CHECK YOURSELF

1. Which structure stores a class's grades, and why?
2. Printer jobs: stack or queue? Why?
3. "Needs reverse order of entry" maps to which structure?

Answers: 1. Array — known size, same type, fast index lookup. 2. Queue, because first-come, first-served. 3. Stack.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An IDE is:

- a. One app to write, run and debug code
- b. A printer
- c. A sort
- d. A cable

MCQ 2. VS Code is:

- a. A popular editor/IDE
- b. A DBMS
- c. An OSI layer
- d. A gate

MCQ 3. The terminal/run button is for:

- a. Executing the program
- b. Drawing pie charts only
- c. Sending TCP SYN only
- d. K-maps

MCQ 4. Syntax highlighting helps you:

- a. See keywords and mistakes
- b. Sort faster
- c. Encrypt disks
- d. Route packets

MCQ 5. A file explorer in VS Code shows:

- a. Project files
- b. OSI layer 1
- c. NAND groups
- d. Beachhead users

MCQ 6. Debugging in an IDE uses:

- a. Breakpoints and variable watch
- b. Only bubble sort
- c. Only pie charts
- d. Only Agile stickers

Extra short questions

1. Define IDE.
2. Name four VS Code areas.
3. Why not use Notepad only?
4. What is an extension?

Extra long question

1. Describe how you would create, run and debug a first Python file in VS Code. Include two interface elements from the notes.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Integrated Development Environment: editor + run + debug + files in one place.

2. Activity bar/sidebar (files), editor, terminal, status bar (and extensions).
3. No easy run, no highlighting, no debugger.
4. An add-on (Python plugin) that teaches VS Code a language.

Long

1. New file .py, type print, Run Python, read terminal. Mention top menu, explorer, terminal panel. Breakpoint on a wrong variable.

2.7 Trace data retrieval in Array, List and Queue

LEARN IT

Tracing data retrieval means following the step-by-step path of how an algorithm finds or accesses data. Array, linked list and queue are all linear, but the **effort** to find an item is very different because of how they are organised inside.

2.7.1 Retrieval in an array

Every element has a fixed index from 0 to $n-1$ pointing at a spot in memory. If the algorithm already knows the index, it does not search: it **jumps** (direct access, $O(1)$).

Array = [10, 50, 20, 70, 90, 30]. Direct access: Array[2] = 20.

Loop iteration	Index	Array access	Value
1	0	Array[0]	10
2	1	Array[1]	50
3	2	Array[2]	20

```
Array = [10, 50, 20, 70, 90, 30]
print("Direct Array[2] =", Array[2])
for i in range(3):
    print(i + 1, i, f"Array[{i}]", Array[i])
```

OUTPUT

```
Direct Array[2] = 20
1 0 Array[0] 10
2 1 Array[1] 50
3 2 Array[2] 20
```

2.7.2 Retrieval in a linked list

Linked lists have **no index**. The only way is to start at HEAD and follow links one node at a time (sequential access, $O(n)$).

List: [12] → [98] → [23] → [87] → [56] → NULL. Search key: 87.

Step	Node data	Compare with 87	Action
1	12	12 != 87	Move next
2	98	98 != 87	Move next
3	23	23 != 87	Move next
4	87	87 == 87	FOUND — output True

```

nodes = [12, 98, 23, 87, 56]
key = 87
found = False
for i, value in enumerate(nodes, start=1):
    print(f"Step {i}: Check Node {i} ({value}).")
    if value == key:
        found = True
        print("FOUND! Output: True")
        break
    print("Not equal. Move Next.")
print("Result:", found)

```

OUTPUT

```

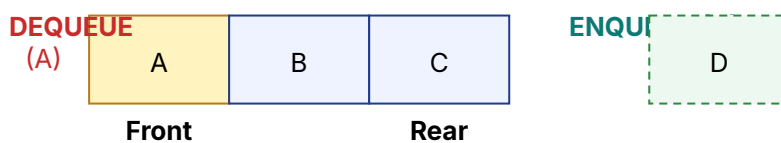
Step 1: Check Node 1 (12).
Not equal. Move Next.
Step 2: Check Node 2 (98).
Not equal. Move Next.
Step 3: Check Node 3 (23).
Not equal. Move Next.
Step 4: Check Node 4 (87).
FOUND! Output: True
Result: True

```

Key 87 sits at the fourth node, so you paid four checks. An array that already knew index 3 would have paid one jump.

2.7.3 Retrieval in a queue

A queue is FIFO. You **cannot** retrieve from the middle or the rear. The oldest item at the Front is always DEQUEUEED first.



You cannot pull B until A has left.

Queue A, B, C. ENQUEUE D joins the rear. DEQUEUE must remove A, not C or B.

```

from collections import deque
q = deque(["A", "B", "C"])
q.append("D")
print("after ENQUEUE D:", list(q))
print("DEQUEUE:", q.popleft())
print("now:", list(q))

```


OUTPUT

after ENQUEUE D: ['A', 'B', 'C', 'D']
 DEQUEUE: A
 now: ['B', 'C', 'D']

WORKED EXAMPLE

Worked example: three retrievals compared for "get 20 / get 87 / get A".

Structure	Target	Path	Can you jump?
Array [10, 50, 20, ...]	20 at index 2	Array[2] in one step, or scan 10, 50, 20	Yes, if the index is known
Linked list 12→98→23→87→...	87	Must visit 12, 98, 23, then 87	No
Queue A-B-C	A (the only legal retrieval)	DEQUEUE Front	No — never the middle

REMEMBER IT

Array jumps, list walks, queue waits. If the exam asks "why is array retrieval different?", write **direct access by index** versus **sequential links** versus **Front only**.

COMMON MISTAKE

Mistake: DEQUEUEing C because "C is the interesting letter", or reading linked-list node 4 as `list[3]` without walking.

Fix: show every hop in the table. Marks are for the path, not only for FOUND.

اردو میں سمجھیں

ایرے میں اگر انڈیکس معلوم ہو تو سیدھی چھلانگ لگتی ہے، جیسے $\text{Array}[2] = 20$ ۔ لنکڈ لسٹ میں HEAD سے ایک ایک نوڈ دیکھنا پڑتا ہے؛ 87 چوتھے نوڈ پر ملا۔ کیوں صرف Front نکلتا ہے، درمیان سے نہیں؛ A, B, C میں DEQUEUE ہمیشہ A ہے۔

CHECK YOURSELF

1. What is `Array[2]` in `[10, 50, 20, 70, 90, 30]`?
2. How many nodes are checked before 87 is found in `12→98→23→87→56`?
3. In queue A, B, C, can you retrieve B without removing A?

Answers: 1. 20. 2. Four (12, 98, 23, 87). 3. No — a queue only DEQUEUEs the Front, so A must leave first.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Tracing retrieval means:

- a. Following how an algorithm finds/returns an item
- b. Deleting the DS
- c. Drawing Figma
- d. Running pip

MCQ 2. Array retrieval by index is:

- a. Direct: $a[i]$
- b. Walk from head always
- c. Pop until empty always
- d. BFS always

MCQ 3. Linked-list retrieval of the k-th is:

- a. Walk k links
- b. $O(1)$ index
- c. A binary cut
- d. A hash

MCQ 4. Queue retrieval of the next client is:

- a. Dequeue front (FIFO)
- b. Pop top (LIFO)
- c. Random node
- d. Tree root

MCQ 5. You cannot take the middle of a queue without:

- a. Breaking FIFO (or using another DS)
- b. It is the definition of queue
- c. A pie
- d. A sprint

MCQ 6. A trace should show:

- a. Pointers/indexes before and after
- b. Only the career names
- c. Only colours
- d. Only OSI

Extra short questions

1. Trace getting $a[2]$ from array $[10,20,30,40]$.
2. Trace finding 30 in a list $10 \rightarrow 20 \rightarrow 30$.
3. Trace queue A-B-C serve one.
4. Why tracing retrieval is in the syllabus.

Extra long question

1. Same three names stored in array, list, queue. Trace "get the second person" in each. Who is not allowed in a pure queue?

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Index 2 $\rightarrow$ 30 in one step.
2. Start 10 $\neq$, next 20 $\neq$, next 30 $=$. Three visits.
3. Dequeue A; front is B. Remaining B-C.
4. So you can justify the DS with actual steps, not slogans.

Long

1. Array `a[1]`; list two hops; queue: you may only legally take the front — getting the second requires dequeue then the new front, which also removes the first. That is the point.

Computational Thinking & Algorithms

16 single-page lessons · 5 Golden · read, close, say aloud, then sit the exam

Chapter in one look

- **Correctness:** output matches expected results for valid inputs. Test with a trace table (manual dry run) or stepwise reasoning (logic verification).
- A **trace table** is micro-level: line, variables, I/O, conditions. Best for small loops. The soldier heights [1.91, 2.01, 1.87, 1.57, 1.48] print Total = 3.
- **Stepwise reasoning** is macro-level: valid start, all cases, correct conditions, loop stays correct. Sum 1..N starts at 0; N = 0 stays 0; N = 5 becomes 15.
- **Clarity** (read, understand, modify) often beats a tiny speed gain. It has two parts: modularity and readability.
- **Modularity:** SRP (one job), loose coupling, abstraction (call Sort without knowing Bubble vs Selection). Split CalculateSum and FindAverage.
- **Readability:** names (`velocity = distance / time`), self-documentation, simple flow, consistent style.
- **Efficiency** = time + space. Drivers: number of steps (Big O), conditions, repetitions.
- **Big O:** $O(1)$ constant (arr[0]), $O(\log n)$ binary search, $O(n)$ linear search, $O(n \log n)$ Merge Sort, $O(n^2)$ nested loops / Bubble Sort. Quadratic fails as n grows.
- **Framework:** Does it scale? What is the bottleneck (deepest loop)? Is the extra complexity worth it? For n = 50, simple $O(n)$ can beat fragile $O(\log n)$.
- **Refinements:** meaningful names, modules, indentation, why-comments; shrink Bubble Sort's inner loop; better algorithms; cache constants; pick the right structure.
- **Data structure:** systematic organising of data. Linear (one after another): array, linked list, stack, queue. Non-linear: tree, graph.
- **Array:** contiguous, index from 0, direct access $O(1)$, shifting on mid-insert. **Linked list:** nodes of data+link, HEAD to NULL, sequential access.
- **Stack LIFO:** PUSH/POP at TOP (Undo). **Queue FIFO:** ENQUEUE rear, DEQUEUE front (printer).
- **Tree:** one root, parent-child (college chart). **Graph:** vertices + edges, no strict hierarchy (GPS, friends).
- Five operations: insert, delete, traverse, search, sort. Choose by next/previous, parent/child, FIFO, relationships, or reverse-of-entry.
- **Retrieval:** array can jump to an index; a list walks from HEAD; a queue only removes the Front.

Key terms

Term	Meaning	اردو
Correctness	The algorithm's output matches the expected results for valid inputs.	درستگی
Trace table	A paper dry-run table of steps, variables, I/O and conditions.	ٹریس ٹیبل
Dry run	Executing an algorithm by hand, line by line.	دستی اجرا
Stepwise reasoning	Checking each logic block's state; also called logic verification.	مرحلہ وار استدلال

Clarity	How easy an algorithm is to read, understand and modify.	وضاحت
Modularity	Breaking work into independent functions, classes or modules.	ماڈیولریتی
Single Responsibility Principle	Each module should do one specific task.	واحد ذمہ داری کا اصول
Coupling	How tightly modules depend on each other; loose coupling is the goal.	ماڈیولز کی جڑت
Abstraction	Hiding implementation details so a caller need not know the inner method.	تجربہ
Readability	How easily another person understands the logic without running it.	پڑھائی پن
Efficiency	How well an algorithm uses time (speed) and space (memory).	کارکردگی
Big O notation	A description of how running time or memory grows as n grows.	بگ او نوٹیشن

CHAPTER 2 EXAM • CLOSED BOOK

Computational Thinking & Algorithms

75 marks • 1 hour 15 minutes • no notes • no phones

CHAPTER EXAM RULES

Instructions. Sit this exam after Lesson 16 and the recap. **Section A** — all 15 MCQs (1 mark). **Section B** — all 10 short questions (3 marks). **Section C** — attempt any **three** long questions (10 marks). At least one Section C answer must be a ★ Golden topic. Write in your notebook. Do not open the key until time is up.

Marks. 15 + 30 + 30 = 75. A 5-mark / 10-mark answer needs Define • Explain • Example • Diagram • Working.

سبق اور خلاصے کے بعد یہ امتحان بغیر نوٹس کے حل کریں۔ وقت پورا ہونے پر کلید کھولیں۔

Section A — Multiple choice (15 × 1 = 15)

1. An algorithm is correct when:

- a. it compiles with no syntax error
- b. its output matches the expected results for the given input
- c. it uses Big O notation
- d. it has no comments

2. A trace table is best described as:

- a. a paper execution that tracks variables line by line
- b. a diagram of a tree
- c. a type of graph edge
- d. an $O(n^2)$ nested loop

3. Stepwise reasoning mainly checks:

- a. printer ink levels
- b. the colour of variable names
- c. only the last printed line
- d. the overall state of logic blocks

4. Which method is impractical if a loop runs 1,000 times?

- a. Abstraction
- b. A full trace table
- c. Stepwise reasoning
- d. Using a function

5. The Single Responsibility Principle says each module should:

- a. focus on one specific task
- b. print and save and calculate everything
- c. avoid meaningful names
- d. use nested IF only

6. Which assignment is more readable?

- a. `v = d / t`
- b. `x1 = a2 / b2`
- c. `velocity = distance / time`
- d. `a = b / c`

7. Accessing `arr[0]` is:

- a. $O(n^2)$
- b. $O(n \log n)$
- c. $O(n)$
- d. $O(1)$

8. Binary search on a sorted list is:

- a. $O(\log n)$
- b. $O(n^2)$
- c. $O(1)$ always for any n
- d. $O(n)$ because a loop exists

9. Bubble Sort and nested "compare every pair" loops are:

- a. $O(1)$
- b. $O(\log n)$
- c. $O(n)$
- d. $O(n^2)$

10. If n grows 10 times, a quadratic algorithm's time grows about:

- a. 2 times
- b. 10 times
- c. 100 times
- d. log 10 times

11. LIFO is the rule of a:

- a. queue
- b. graph of cities
- c. tree root only
- d. stack

- .2. ENQUEUE adds an item at the:
- a. Front of a queue
 - b. TOP of a stack only
 - c. Rear of a queue
 - d. root of a tree
- .3. A college organogram with a Principal at the top is modelled as a:
- a. tree
 - b. queue
 - c. stack
 - d. contiguous array of roads
- .4. GPS maps (cities and roads, many possible routes) need a:
- a. stack of undo actions
 - b. graph
 - c. strict one-parent tree only
 - d. single array index
- .5. To retrieve 87 from HEAD→12→98→23→87→56 you must:
- a. jump with Array[87]
 - b. DEQUEUE the rear
 - c. follow four nodes from HEAD until 87 matches
 - d. POP from the bottom of a stack

Section B — Short questions (10 × 3 = 30)

1. Define correctness of an algorithm. Name the two analysis methods used in this chapter.
2. List the components of a trace table. Why is a dry run useful for debugging?
3. Give four differences between a trace table and stepwise reasoning.
4. What is modularity? Explain SRP, coupling and abstraction in one sentence each.
5. State four ways to evaluate readability, with one example of a bad name and a good name.
6. Define Big O notation. Give one example each of $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$ and $O(n^2)$.
7. Write the three questions of the efficiency evaluation framework and answer them for searching 50 unsorted names.
8. List four refinements for clarity and four for efficiency.
9. Define data structure. Distinguish linear from non-linear with examples.
- .0. Define stack and queue. Include LIFO/FIFO, PUSH/POP, ENQUEUE/DEQUEUE, and one real-life example of each.

Section C — Long questions (attempt any 3 × 10 = 30)

1. The soldier-eligibility algorithm uses AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48] and counts heights from 1.57 to 1.91 inclusive. (a) Draw a complete trace table. (b) State the printed Total. (c) Explain in stepwise-reasoning style why the boundaries 1.57 and 1.91 are counted. (d) Write equivalent Python and its output.
2. Copy the monoblock average algorithm for A = [1, 2, 3, 4, 5] and rewrite it with CalculateSum and FindAverage. (a) Show that SRP, loose coupling and abstraction are

- improved. (b) Dry-run both versions to the same average. (c) Give two readability improvements you would still make to the functions.
3. Using the growth table idea, (a) fill $n = 8$ and $n = 32$ for $O(1)$, $O(\log_2 n)$, $O(n)$, $O(n \log_2 n)$ and $O(n^2)$. (b) Explain what happens if n becomes 10 times larger for $O(n)$ and for $O(n^2)$. (c) A library has 1,000,000 sorted IDs. Choose linear or binary search using the Scale / Bottleneck / Worth-it framework. (d) Show a 4-by-4 nested loop counts 16 operations in Python.
4. Draw a labelled stack and a labelled queue. Then trace: (a) PUSH Data1, PUSH Data2, PUSH Data3, POP, POP. (b) ENQUEUE 55, 76, 90, D then DEQUEUE. Include Python that matches both traces. (c) Why is Undo a stack and a printer a queue?
5. A college needs structures for: (i) 200 integer marks, (ii) a playlist with frequent insertions, (iii) Undo in the fee-office editor, (iv) photocopier jobs, (v) the Principal–Dean–HOD chart, (vi) roads between Sindh cities. Identify each structure using the decision guide, draw the tree and the graph, and then trace retrieval of 20 from Array = [10, 50, 20, 70, 90, 30] and of 87 from the linked list $12 \rightarrow 98 \rightarrow 23 \rightarrow 87 \rightarrow 56 \rightarrow \text{NULL}$.

Answer key. Finish the Chapter Exam first, then mark. Do not peek while the clock is running.

پہلے امتحان مکمل کریں، پھر اس کلید سے نمبر لگائیں۔

Check yourself — answers

1. **2.1 Analyze algorithms for correctness.** Answers: 1. When its output matches the expected results for a given input (and does so for all valid inputs). 2. Trace tables and stepwise reasoning. 3. Because a logical error can still produce output — the wrong output.
2. **2.1.1 – 2.1.3 Trace Tables vs Stepwise Reasoning.** Answers: 1. Line/step number, variables, input/output, condition check. 2. 1.57 is the lower boundary (inclusive, so True); 1.48 is below 1.57 (False). 3. 3 is printed; Step 6 runs three times (for 1.91, 1.87 and 1.57).
3. **2.2 Evaluating the Clarity of an Algorithm.** Answers: 1. The algorithm is easy to read, understand and modify. 2. Modularity and readability. 3. Because a clear algorithm is easier to maintain, debug and extend over time.
4. **2.2.1 – 2.2.2 Modularity and Readability.** Answers: 1. One specific task. 2. CalculateSum (FindAverage only divides; the main program only prints). 3. All logic is in one block, so you cannot call “just the sum” from another algorithm.
5. **2.3 Assess Algorithm Efficiency.** Answers: 1. Time (speed) and space (memory). 2. Number of steps (Big O), number of conditions, number of repetitions.
6. **2.3.1 Number of Steps (Big O Notation).** Answers: 1. $O(1)$ arr[0]; $O(\log n)$ binary search; $O(n)$ linear search; $O(n^2)$ Bubble Sort / nested loops. 2. 1,024. 3. It makes about $\log n$ split-levels and does about n work on each level, so $n \times \log n$.
7. **2.3.2 Number of Conditions.** Answers: 1. Each condition must be evaluated True/False by the CPU. 2. A brute-force approach (and it reduces readability and speed). 3. AND short-circuits: the first part is False, so the division is never evaluated.
8. **2.3.3 Number of Repetitions.** Answers: 1. Poor — $O(n^2)$. 2. Each iteration halves the remaining problem, so the number of iterations is about $\log n$. 3. 25.
9. **2.3.4 The Efficiency Evaluation Framework.** Answers: 1. About $100\times$ slower (10^2). 2. The most time-consuming section, usually the deepest loop. 3. Binary search is more complex to code and maintain, and for $n = 50$ the simplicity of $O(n)$ is practically better.
10. **2.4 Refinements to improve clarity and efficiency.** Answers: 1. average = totalSum / totalItems. 2. The already-sorted largest value at the end (shrink the inner loop). 3. Once, outside the loop, then reuse C inside.
11. **2.5 Concept of Data Structure.** Answers: 1. A systematic way of organising, storing and managing data for efficient access, modification and processing. 2. Linear: array, linked list, stack, queue. Non-linear: tree, graph. 3. It has exactly one predecessor and one successor.
12. **2.5.1 Linear Data Structures (Array, Linked List, Stack, Queue).** Answers: 1. The items sit in one continuous block, next to each other. 2. 12. 3. Later elements must all be shifted right to make room.
13. **2.5.2 Non-Linear Data Structures (Tree, Graph).** Answers: 1. The root. 2. One-to-many parent-child. 3. A family tree, and folders inside folders (or a college organogram).
14. **2.5.3 Operations on Data Structures.** Answers: 1. Insertion, deletion, traversal, searching, sorting. 2. Insertion (at the rear of a queue). 3. Traversal visits every item; searching stops when the target is found (or the structure ends).
15. **2.6 Identify data structures in problem scenarios.** Answers: 1. Array — known size, same type, fast index lookup. 2. Queue, because first-come, first-served. 3. Stack.

16. **2.7 Trace data retrieval in Array, List and Queue.** Answers: 1. 20. 2. Four (12, 98, 23, 87). 3. No — a queue only DEQUEUES the Front, so A must leave first.

Answer key

Multiple choice

- | | | | | |
|------|------|------|-------|-------|
| 1. b | 4. b | 7. d | 10. c | 13. a |
| 2. a | 5. a | 8. a | 11. d | 14. b |
| 3. d | 6. c | 9. d | 12. c | 15. c |

Short questions — model answers

1. An algorithm is correct if its output matches the expected results for a given input (and does so for all valid inputs). The two methods are trace tables and stepwise reasoning (logic verification).
2. Components: line/step number, a column per variable, input/output, and a condition-check column. A dry run acts as the computer, so you see logical errors where the program runs but the values go wrong, and you can validate that the intention is met.
3. Trace table: small blocks/complex loops/beginners; high effort (bad for 1,000 iterations); catches arithmetic/line mistakes; micro-level exact values. Stepwise reasoning: large systems and proofs; moderate effort; catches conceptual/structural flaws; macro-level overall state.
4. Modularity is breaking an algorithm into smaller independent parts (functions, classes, modules). SRP: each module does one task. Coupling: modules should be loosely coupled so a change in one does not break others. Abstraction: hide inner details (call Sort without knowing Bubble vs Selection).
5. Naming conventions, self-documentation, simple control flow, consistency of indentation/spacing/casing. Bad: $v = d / t$. Good: $velocity = distance / time$.
6. Big O describes how time or memory grows as n grows. $O(1)$: `arr[0]`; $O(\log n)$: binary search; $O(n)$: linear search; $O(n \log n)$: Merge Sort; $O(n^2)$: Bubble Sort / nested loops.
7. Does it scale? $n = 50$ is tiny, even $O(n)$ is fine. Bottleneck? A single scan of 50 names. Worth it? Binary search needs sorted data and more complex code, so linear search is practically better.
8. Clarity: meaningful names; modules; structure/indentation; short why-comments. Efficiency: reduce extra steps (shrink Bubble Sort's inner loop); better algorithms (binary vs linear on sorted millions); avoid re-computing (cache $C = A+B$ outside the loop); appropriate data structures (array for direct access, tree for hierarchy).
9. A data structure is a systematic way of organising, storing and managing data for efficient access, modification and processing. Linear: sequence with one predecessor and one successor (array, linked list, stack, queue). Non-linear: hierarchical or network links (tree, graph).
10. Stack: LIFO; PUSH adds at TOP; POP removes from TOP; example Undo. Queue: FIFO; ENQUEUE adds at Rear; DEQUEUE removes from Front; example printer jobs.

Long questions — model answers

1. (a) Columns: Step, Total, Height, AllHeights, condition, output. Step 2 sets Total = 0. Step 3 stores the five heights. Then five loop cycles:

- 1.91 → True → Total 1
- 2.01 → False → Total stays 1
- 1.87 → True → Total 2
- 1.57 → True (boundary) → Total 3
- 1.48 → False → Total stays 3

(b) Printed Total = **3**.

(c) The condition is height ≥ 1.57 AND height ≤ 1.91 , so the closed interval includes both ends. 1.57 and 1.91 satisfy the inequalities; 2.01 and 1.48 do not. Initial Total = 0 is valid; each True branch adds one; False skips the add; after five items Total is the count of eligible soldiers.

(d) Python:

```
AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]
Total = 0
for height in AllHeights:
    if height ≥ 1.57 and height ≤ 1.91:
        Total = Total + 1
print(Total)
```

OUTPUT
3

2. (a) Monoblock: one sequence does the loop, the division and the print, so you cannot reuse the sum. Modular version: CalculateSum only adds; FindAverage only divides; main only prints. SRP holds. CalculateSum can be called from another program (loose coupling). Main does not see the loop (abstraction).

(b) Both compute total = 15 and avg = $15/5 = 3.0$.

```
def calculate_sum(A, n):
    total = 0
    for i in range(n):
        total = total + A[i]
    return total

def find_average(A, n):
    return calculate_sum(A, n) / n

print(find_average([1, 2, 3, 4, 5], 5))
```

OUTPUT
3.0

(c) Keep snake_case names; avoid 1-based I in Python; a one-line comment on why you divide by n (empty-list guard could be added later) is enough — do not comment every addition.

3. (a) Growth ($\log_2$):

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
8	1	3	8	24	64
32	1	5	32	160	1,024

(b) $O(n)$: 10× input → about 10× time. $O(n^2)$: 10× input → about 100× time, which fails for large sets.

(c) Scale: a million IDs; linear worst case is a million checks, binary about 20. Bottleneck: the search. Worth it: the IDs are already sorted and binary search is standard, so choose **binary search** $O(\log n)$.

(d)

```
n = 4
count = 0
for i in range(n):
    for j in range(n):
        count = count + 1
print(count)
```

OUTPUT
16

4. Draw the vertical stack with TOP on Data3 after three PUSHes, and the horizontal queue with FRONT on 55 and REAR on 90 (then D).

(a) Stack: [] → [Data1] → [Data1, Data2] → [Data1, Data2, Data3] → POP Data3 → POP Data2 → [Data1].

```
stack = []
stack.append("Data1")
stack.append("Data2")
stack.append("Data3")
print(stack.pop())
print(stack.pop())
print(stack)
```

OUTPUT
Data3
Data2
['Data1']

(b) Queue: [55] → [55, 76] → [55, 76, 90] → [55, 76, 90, D] → DEQUEUE 55 → [76, 90, D].

```
from collections import deque
q = deque()
q.append(55)
q.append(76)
q.append(90)
q.append("D")
print(q.popleft())
print(list(q))
```

OUTPUT

```
55
[76, 90, 'D']
```

(c) Undo must cancel the most recent action (LIFO → stack). A printer must respect arrival order (FIFO → queue).

5. (i) Array — known count, same type, index lookup. (ii) Linked list — next-pointer, cheap insert. (iii) Stack — reverse order of entry. (iv) Queue — first-come, first-served. (v) Tree — parent-child from the Principal root (Administrative Head / Academic Dean, then Accounts, HR, HOD Math, HOD CS). (vi) Graph — cities as vertices, roads as edges, alternative routes.

Array retrieval: `Array[2] = 20` in one direct access (or after visiting 10 then 50 then 20 if you scan).

List retrieval: Step1 12 ≠ 87, Step2 98 ≠ 87, Step3 23 ≠ 87, Step4 87 = 87 FOUND True.

Self-assessment: I can...

- ☐ prove correctness with a full trace table, including boundary values.
- ☐ judge modularity (SRP, coupling, abstraction) and readability (N-S-S-C).
- ☐ apply the Scale / Bottleneck / Worth-it framework, including the $n = 50$ warning.
- ☐ define a data structure and classify linear vs non-linear.
- ☐ draw a tree and a graph and pick a structure from a scenario using the decision guide.
- ☐ write stepwise reasoning (start, loop, finish, edges) and compare it with a trace table.
- ☐ state $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$ and $O(n^2)$ with examples and a growth table.
- ☐ refine an algorithm for names, modules, fewer loop steps and cached values.
- ☐ draw and trace an array, linked list, stack (PUSH/POP) and queue (ENQUEUE/DEQUEUE).
- ☐ trace retrieval: array jump, list walk from HEAD, queue Front only.

Lecture 03

Programming Fundamentals

Chapter lecture — 17 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

BIEK Computer Science XII · Bilingual support: English + اردو

بی آئی ای کے کمپیوٹر سائنس — بارہویں جماعت

17 book lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XII

How to use this BIEK study-guide lecture

Each PDF follows one textbook lecture from the BIEK / Sindh Computer Science notes (New Sindh Curriculum). Read *Learn it*, study the SVG diagram, try the worked example on paper, then answer *Check yourself* before the answers. **Extra board practice** (MCQs, short and long questions) is added after the book content — it is not in the textbook. Career-only extras that were not in the book lecture list have been removed. Math is rendered with MathJax. ★ Golden topics are high-yield in board exams. The green box repeats the idea in Urdu.

ہر لیکچر کتاب کے درس کے مطابق ہے۔ کتاب کے بعد اضافی MCQs اور سوالات ہیں۔ جو اضافی کیریئر صفحات کتاب کی فہرست میں نہیں تھے وہ ہٹا دیے گئے ہیں۔

Lectures in this pack

Chapter 3: Programming Fundamentals. Study one lecture at a time, then use the chapter review and answer key at the end of this PDF.

L01. 3.1 Data Structures in Python

L03. 3.1.2 Tuples (Ordered, Immutable)

L05. 3.1.4 Dictionaries (Key-Value pairs)

L07. 3.1.6 Activity: Students Attendance Program

L09. 3.2 Functions in Python

L11. 3.2.2 Functions with Return Values

L13. 3.2.4 Scope of Local and Global Variables

L15. 3.6 with Statement (Context Manager)

L17. 3.9 Mini File Handling Project: Grade Tracker

L02. 3.1.1 Lists (Ordered, Mutable, Methods)

L04. 3.1.3 Sets (Unordered, Unique, Math Operations)

L06. 3.1.5 Common Built-in Functions (min, max, len, sum)

L08. 3.1.7 Activity: Word Frequency Counter

L10. 3.2.1 Types of Functions (Built-in vs User-defined)

L12. 3.2.3 Activity: Arithmetic Calculator Using Functions

L14. 3.3 & 3.4 File Handling and Operations (Modes r, w, a)

L16. 3.8 Errors and Exceptions (Try-Except)

Programming Fundamentals

This chapter teaches you how Python stores collections of data, how you write reusable functions, and how you save information in files so it is still there after the program ends. You will build small programs — attendance, a word counter, a calculator, and a grade tracker — that you can type and run yourself.

اس باب میں آپ سیکھیں گے کہ پائتھن (Python) ڈیٹا کو فہرست، ٹیپل، سیٹ اور ڈکشنری میں کیسے رکھتا ہے، فنکشن کیسے لکھے جاتے ہیں، اور فائلوں میں ڈیٹا کیسے محفوظ کیا جاتا ہے تاکہ پروگرام بند ہونے کے بعد بھی وہ موجود رہے۔

What you will learn

- Distinguish primitive types from abstract data structures (list, tuple, set, dictionary).
- Use tuples as ordered, immutable collections, including indexing and the two tuple methods.
- Store and retrieve key–value data with dictionaries, including safe access with `.get()`.
- Write user-defined functions with parameters and `return` values, and contrast them with built-in functions.
- Open text files in modes `r`, `w` and `a`, and close them safely with `with`.
- Create, index, slice, traverse and update lists using the main list methods.
- Build sets of unique items and compute union, intersection, difference and symmetric difference.
- Apply `min()`, `max()`, `len()`, `sum()` and `sorted()` across data structures.
- Explain local and global variable scope with short programs.
- Catch file errors with `try/except` and complete a grade-tracker file project.

3.1 Data Structures in Python

LEARN IT

A **data structure** is a way of organising values in a program so you can store them, find them and change them. Every useful program is built on data structures. Python gives you two layers:

- **Primitive data structures** — the basic types that hold **one** value. Examples: integers (`int`), floating-point numbers (`float`), and booleans (`bool`).
- **Abstract data structures** — higher-level containers built from primitive types so you can hold **many** values and run richer operations. Examples: **lists, tuples, sets, dictionaries**.

Python has these four abstract structures built in. You do not install anything extra. That is why collecting marks, names or settings is simpler in Python than in many other languages.

Category	Description	Examples
Primitive	Basic types that represent a single value.	<code>int</code> , <code>float</code> , <code>bool</code>
Abstract	Higher-level structures for collections and complex operations.	Lists, tuples, sets, dictionaries

WORKED EXAMPLE

One value versus a collection. A student's age is one integer. That student's three test marks need a list.

```
age = 17
height = 5.8
passed = True
marks = [80, 75, 90]
print(type(age))
print(type(height))
print(type(passed))
print(type(marks))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'list'>
```

`type()` tells you which structure a value has. `age` is a primitive integer. `marks` is an abstract list that holds three integers together.

REMEMBER IT

Primitive = one piece. Abstract = a box of pieces. Think of a single rupee coin (primitive) versus a wallet that can hold many coins (list, tuple, set or dictionary).

COMMON MISTAKE

Do not call a list a “variable type” and stop there. A list is a **structure**: it has an order, methods, and rules. The four abstract structures look similar at first (they all hold several values) but they follow different rules. Learn those rules one by one in 3.1.1–3.1.4.

اردو میں سمجھیں

ڈیٹا سٹرکچر (Data Structure) پروگرام میں ڈیٹا کو ترتیب دینے کا طریقہ ہے۔ پرائمیٹو اقسام جیسے عدد (int) صرف ایک قیمت رکھتی ہیں، جبکہ فہرست، ٹیپل، سیٹ اور ڈکشنری کئی قیمتیں ایک ساتھ سنبھالتی ہیں۔ پائتھن میں یہ چاروں سٹرکچرز پہلے سے موجود ہیں، اس لیے ڈیٹا جمع کرنا دوسری زبانوں کے مقابلے میں آسان ہے۔

CHECK YOURSELF

1. Name two primitive types and two abstract data structures in Python.
2. Why is a list called an abstract data structure rather than a primitive type?

Answers: 1. Primitive: any two of int, float, bool. Abstract: any two of list, tuple, set, dictionary.
2. Because it is a higher-level container built from primitive values, holding many items and supporting operations such as append and slice.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Python built-in structures in this chapter include:

- a. list, tuple, set, dict
- b. only arrays of C
- c. only trees
- d. only graphs

MCQ 2. You choose among them by:

- a. Order? Duplicates? Mutability? Key lookup?
- b. The logo
- c. The font
- d. The sprint

MCQ 3. Mutable means:

- a. You can change it in place
- b. Frozen forever
- c. Always sorted
- d. Always unique

MCQ 4. A dict maps:

- a. Keys to values
- b. Only numbers to NAND
- c. Only OSI layers
- d. Only files

MCQ 5. A set is best for:

- a. Unique items / membership tests
- b. Ordered ranking with duplicates
- c. LIFO
- d. FIFO

MCQ 6. This lecture is the map; later lectures:

- a. Zoom into each type
- b. Delete Python
- c. Draw only ER
- d. Draw only Figma

Extra short questions

1. One-line each: list, tuple, set, dict.
2. Which for student names that may repeat seats?
3. Which for a record of roll→marks?
4. Which as a function return of (x,y) that must not change?

Extra long question

1. Four college needs → choose the Python DS and justify with one operation each.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. List: ordered mutable sequence. Tuple: ordered immutable. Set: unique unordered.
Dict: key→value.
2. List (order). If unique roll numbers: set or dict.
3. Dict.
4. Tuple.

Long

1. Attendance ordered: list. RGB colour constant: tuple. Club unique IDs: set. Phonebook: dict. Show one line of code each.

3.1.1 Lists (Ordered, Mutable, Methods) ★ Golden

LEARN IT

A **list** is Python's most versatile built-in data structure. It stores an **ordered** collection of items in a single variable. Lists are **mutable** (you can change them after you create them) and they are written with **square brackets** `[]`.

Why use a list? One variable can hold thousands of values. You can mix types if you need to. You can add, remove and sort items as the program runs.

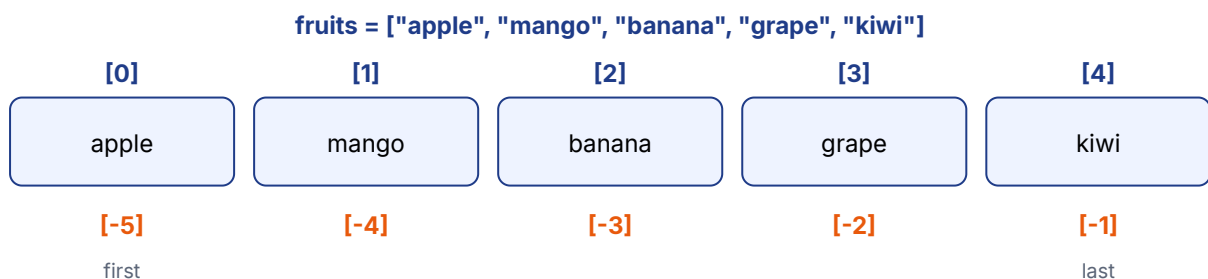
```
numbers = [10, 2, 15, 14, 3, 33]      # list of integers
names = ["Hassan", "Qasim", "Malika"]  # list of strings
mixed = [1, "Python", 3.14, True]     # mixed data types
print(numbers)
print(names)
print(mixed)
```

OUTPUT

```
[10, 2, 15, 14, 3, 33]
['Hassan', 'Qasim', 'Malika']
[1, 'Python', 3.14, True]
```

List indexing (positive and negative)

Each item sits in a numbered slot called an **index**. Numbering starts at **0** from the left. Python also supports **negative indexing** from the right: `-1` is the last item, `-2` is the second last, and so on.



Positive indices count left to right from 0. Negative indices count right to left from `-1`.

```
fruits = ["apple", "mango", "banana", "grape", "kiwi"]
print(fruits[0])    # first item
print(fruits[4])    # last item (when you know the length)
print(fruits[-1])   # last item (works even if the list grows)
print(fruits[-5])   # first item, counted from the right
```

OUTPUT

```
apple
kiwi
kiwi
apple
```

REMEMBER IT

Always remember `[-1]`. If a list has 10,000 items and you do not want to compute `len(list) - 1`, write `list[-1]`. It grabs the last item in one step.

Slicing a list

Slicing copies a portion of a list into a **new** list. The original is not changed.

Syntax: `list[start:stop:step]`

- `start` — first index to include (default 0).
- `stop` — first index to **exclude**. The slice goes up to, but does **not** include, this index.
- `step` — jump size (default 1). A step of `-1` walks backwards.

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(nums[2:6])
print(nums[:4])
print(nums[6:])
print(nums[::2])
print(nums[::-1])
```

OUTPUT

```
[2, 3, 4, 5]
[0, 1, 2, 3]
[6, 7, 8, 9]
[0, 2, 4, 6, 8]
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Trace of `nums[2:6]`: start at index 2 (value 2), take 3, 4, 5, and stop **before** index 6. Result `[2, 3, 4, 5]`. `nums[::-1]` is the standard way to reverse a list without changing the original (it returns a new reversed list).

Traversal

Traversal means visiting each element, one by one, in order. Use a `for` loop (the usual Python style) or a `while` loop with an index.

```
numsList = [10, 20, 30]
for num in numsList:
    print(num)
```

```
numsList = [10, 20, 30]
index = 0
while index < len(numsList):
    print(numsList[index])
    index += 1
```

OUTPUT

```
10
20
30
```

OUTPUT

```
10
20
30
```

The `for` loop gives you each value directly. The `while` loop needs a counter, a test `index < len(numsList)`, and `index += 1` so the loop ends.

Built-in list methods

Because a list is mutable, Python gives you methods that change it in place. `len()` is listed with them in many notes, but it is a **built-in function**: you write `len(fruits)`, not `fruits.len()`.

Operation	What it does
<code>len(list)</code>	Returns the number of items.
<code>.append(value)</code>	Adds an item at the end .
<code>.insert(index, value)</code>	Inserts an item at a chosen position (later items shift right).
<code>.remove(value)</code>	Removes the first matching value. Raises <code>ValueError</code> if it is missing.
<code>.clear()</code>	Removes every item. The list becomes <code>[]</code> .
<code>.count(value)</code>	How many times a value appears.
<code>.index(value)</code>	Index of the first occurrence.
<code>.sort()</code>	Sorts the list in place, ascending. Use <code>reverse=True</code> for descending.
<code>.reverse()</code>	Reverses the list in place (does not sort).

WORKED EXAMPLE

Worked example: methods change the same list step by step.

```
fruits = ['apple', 'banana', 'apple']
print(fruits)
fruits.append('grape')
print(fruits)
fruits.remove('apple')
print(fruits)
fruits.sort()
print(fruits)
```

OUTPUT

```
['apple', 'banana', 'apple']
['apple', 'banana', 'apple', 'grape']
['banana', 'apple', 'grape']
['apple', 'banana', 'grape']
```

After this line	List in memory
create	['apple', 'banana', 'apple']
.append('grape')	['apple', 'banana', 'apple', 'grape']
.remove('apple')	['banana', 'apple', 'grape'] — only the first apple goes
.sort()	['apple', 'banana', 'grape']

More methods on numbers:

```
nums = [10, 20, 30]
print(len(nums))
nums.insert(1, 15)
print(nums)
print(nums.count(20))
print(nums.index(30))
nums.reverse()
print(nums)
nums.clear()
print(nums)
```

OUTPUT

```
3
[10, 15, 20, 30]
1
3
[30, 20, 15, 10]
[]
```

`insert(1, 15)` places 15 at index 1, so 20 and 30 shift right. After that, 30 sits at index 3. `reverse()` flips the current order; it does not sort. `clear()` leaves an empty list `[]`, not the value `None`.

You can also replace an item by index, because lists are mutable:

```
numbers = [10, 2, 15]
numbers[0] = 99
print(numbers)
```

OUTPUT

```
[99, 2, 15]
```

Advantages of lists

- **Mutable** — add, remove or replace items as the program runs.
- **Organised** — thousands of values live in one variable instead of `n1`, `n2`, `n3`...
- **Flexible** — integers, strings, booleans and even other lists can sit in the same list.

Activity: searching an element in a list

This program combines a list, user input, a `for` loop, an `if` test and a **flag** (a Boolean that remembers whether the search succeeded).

```
numbers = [5, 10, 15, 20, 25]
search = int(input("Enter number to search: "))
found = False
for num in numbers:
    if num == search:
        found = True
        break
if found:
    print("Number found in the list.")
else:
    print("Number NOT found in the list.")
```

Execution 1 — the user types 15, which is at index 2. The loop sets `found = True` and `break` stops the scan at once.

OUTPUT

```
Enter number to search: 15
Number found in the list.
```

Execution 2 — the user types 99. The loop finishes without a match, so the flag stays `False`.

OUTPUT

```
Enter number to search: 99
Number NOT found in the list.
```

Why `break`? If the value is found at index 1 in a list of 10,000 items, there is no need to check the remaining 9,998. `break` makes linear search cheaper as soon as it succeeds.

REMEMBER IT

LIST — **L**ots of items, **I**ndexed from 0, **S**quare brackets, **T**weakable (mutable). Slice memory trick: "start is in, stop is out."

COMMON MISTAKE

Mistake 1: `.remove('apple')` deletes every apple. **Fix:** it deletes only the first match. Call it again if you need the next one, or use a loop.

Mistake 2: `.sort()` and `sorted()` are the same. **Fix:** `.sort()` is a list method that changes the original and returns `None`. `sorted()` is a function that works on many structures and returns a **new** list (see 3.1.5).

Mistake 3: `fruits[5]` on a five-item list. Valid positive indices are 0 to 4. Use `[-1]` for the last item.

اردو میں سمجھیں

فہرست (List) مربع بریکٹ `[]` میں لکھی جاتی ہے، ترتیب برقرار رکھتی ہے اور تبدیل کی جا سکتی ہے (Mutable)۔ انڈیکس صفر سے شروع ہوتا ہے، اور `[1-]` آخری عنصر دیتا ہے۔ `.append()` آخر میں جوڑتا ہے، `.remove()` پہلی مشابہ قیمت ہٹاتا ہے، اور `.sort()` اصل فہرست کو ترتیب دیتا ہے۔ تلاش کرتے وقت `break` مل جانے کے بعد لوپ روک دیتا ہے۔

EXAM TIP

Lists appear every year: "What is a list?", "List four methods with one line each", "Differentiate list and tuple", "Write a program to search an element", "Predict the output of a slice." Write: ordered, mutable, `[]`, index from 0, `[-1]` for last. For a slice, always say the stop index is excluded. For `.remove()`, say "first occurrence only." Keep a short program ready that uses `append`, `remove` and `sort` with the printed list after each line.

CHECK YOURSELF

1. What does `nums[2:6]` return if `nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`?
2. After `fruits = ['apple', 'banana', 'apple']` then `fruits.remove('apple')`, what is `fruits`?
3. Why does the search program use `break`?

Answers: 1. `[2, 3, 4, 5]`. 2. `['banana', 'apple']` (only the first apple is removed). 3. To stop scanning once the value is found, so the rest of a long list is not checked.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A list is:

- a. Ordered and mutable
- b. Immutable
- c. Unordered unique only
- d. A dict of keys

MCQ 2. Negative index -1 is:

- a. The last item
- b. The first
- c. An error always
- d. A key

MCQ 3. append adds:

- a. At the end
- b. At the front always
- c. A new dict key
- d. A file

MCQ 4. slicing a[1:3] on [10,20,30,40] is:

- a. [20,30]
- b. [10,20,30]
- c. [30]
- d. [20,30,40]

MCQ 5. Lists can hold:

- a. Mixed types (but usually don't for sanity)
- b. Only ints
- c. Only strings
- d. Only dicts

MCQ 6. remove(x) raises if:

- a. x is not in the list
- b. x is last
- c. The list is long
- d. You used print

Extra short questions

1. Three list methods with effect.
2. How do you walk a list?
3. What does mutable mean here?
4. Show a search snippet.

Extra long question

1. Write a small program: read 5 marks into a list, print max, min, and those above average. Show a sample run.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. append, insert, pop, remove, sort, reverse — any three with one line each.
2. for x in lst: or for i in range(len(lst)).
3. lst[0]=5 is legal; the object changes.
4. if target in lst / for loop with break.

Long

1. Use list, loop, sum/len, if. Output exact for a chosen sample. Mention IndexError if you go past 4.

3.1.2 Tuples (Ordered, Immutable) ★ Golden

LEARN IT

A **tuple** is an ordered collection that is **immutable** — once created, you cannot add, remove or replace items. Think of it as a **read-only list**. Tuples use **parentheses** `()`. Like lists, they allow duplicates, mixed types, and indexing from 0 (including negative indices).

List []

Can add / remove / replace

Tuple ()

LOCKED — read-only

A list is an open box you can refill. A tuple is a sealed box you may only look into.

```
t = (10, 20, 30, 40, 50)
print(t[0])
print(t[-1])
print(t[1:4])
print(t[::-1])
```

OUTPUT

```
10
50
(20, 30, 40)
(50, 40, 30, 20, 10)
```

Slicing a tuple returns a **new tuple**, not a list. Negative indexing and `[::-1]` work exactly as they do for lists.

Immutability

You cannot assign to an index of a tuple. Python raises `TypeError`.

```
t = (1, 2, 3)
t[0] = 99
```

OUTPUT

```
Traceback (most recent call last):
  File "tuple.py", line 2, in <module>
    t[0] = 99
TypeError: 'tuple' object does not support item assignment
```

Exception: if a tuple contains a **mutable object** (a list inside the tuple), that inner object's contents can still change. The tuple still holds the same two list objects; you changed a list, not the tuple's structure.

```
t = ([1, 2], [3, 4])
t[0].append(99)
print(t)
```

```
OUTPUT
([1, 2, 99], [3, 4])
```

`t[0] = [9, 9]` would still be illegal, because that would replace the tuple's first item.
`t[0].append(99)` is legal because it mutates the list that already lives at index 0.

Traversal and methods

Traverse a tuple with `for`, just as you do a list. You are only reading, which is the usual reason to choose a tuple.

```
student = ("Ali", 20, "Computer Science")
for item in student:
    print(item)
```

```
OUTPUT
Ali
20
Computer Science
```

Because a tuple cannot grow or shrink, it has only **two** methods:

Method	Meaning
<code>.count(value)</code>	How many times the value appears.
<code>.index(value)</code>	Index of the first occurrence.

```
t = (10, 20, 10, 30, 10)
print(t.count(10))
print(t.index(20))
```

```
OUTPUT
3
1
```

Advantages of tuples

- **Protection** — coordinates, identity-card numbers or settings cannot be changed by accident.
- **Speed** — a fixed-size tuple is processed slightly faster than a list.
- **Hashable keys** — a tuple of immutables can be used as a dictionary key; a list cannot. (You will use dictionaries in 3.1.4.)

WORKED EXAMPLE

When to choose a tuple. A student's roll number, name and class form a record that should not be edited in place:

```
record = ("XII-CS-07", "Ayesha", "Computer Science")
print(record[0])
print(len(record))
```

OUTPUT

```
XII-CS-07
3
```

If later you need to change the class, you build a **new** tuple. You do not assign to `record[2]`.

REMEMBER IT

Tuple = locked lunchbox. You can see what is inside; you cannot swap the sandwich. Memory: "Tuple is Tight (immutable), written with round brackets that look like a hug holding the data still."

COMMON MISTAKE

One-item trap: `(5)` is just the integer 5 with extra parentheses. A one-item tuple needs a trailing comma: `(5,)`.

```
t1 = (5)
t2 = (5,)
print(type(t1), t1)
print(type(t2), t2)
```

OUTPUT

```
<class 'int'> 5
<class 'tuple'> (5,)
```

An empty tuple is `()`.

اردو میں سمجھیں

ٹیپل (Tuple) گول بریکٹ () میں لکھا جاتا ہے، ترتیب رکھتا ہے مگر تبدیل نہیں ہو سکتا (Immutable)۔ انڈیکس اور سلائس فہرست جیسی ہیں، لیکن `t[0] = 99` سے `TypeError` آتا ہے۔ صرف دو میتھڈز ہیں: `count()` اور `index()`۔ حساس ڈیٹا کے لیے ٹیپل محفوظ ہے۔ اگر ٹیپل کے اندر فہرست ہو تو اس فہرست کے اندر کی اشیاء بدل سکتی ہیں۔

EXAM TIP

Expect “Differentiate list and tuple” (at least four points: brackets, mutability, methods, speed/protection), “Why use a tuple?”, and “What happens if you write `t[0] = 1`?” Quote the `TypeError` wording. Mention the inner-list exception if the question is a 6-mark “explain with example.” A one-item tuple `(5,)` is a favourite trick question.

CHECK YOURSELF

1. Name the two tuple methods.
2. What error does `t[0] = 99` raise for a tuple?
3. After `t = ([1, 2], [3, 4]); t[0].append(99)`, what is `t`?

Answers: 1. `count` and `index`. 2. `TypeError: 'tuple' object does not support item assignment`. 3. `([1, 2, 99], [3, 4])`.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A tuple is:

- a. Ordered and immutable
- b. Mutable like a list
- c. A set of unique keys
- d. A file mode

MCQ 2. `t[0]=5` on a tuple:

- a. `TypeError`
- b. Works
- c. Deletes `t`
- d. Converts to list silently always

MCQ 3. Why immutability?

- a. Safe as dict keys; safer returns; can be hashed
- b. It makes tuples faster than CPUs
- c. It allows `append()`
- d. It stores files

MCQ 4. A 1-item tuple is written:

- a. `(5,)`
- b. `(5)`
- c. `[5]`
- d. `{5}`

MCQ 5. Unpacking `a,b = (1,2)` sets:

- a. `a=1, b=2`
- b. `a=(1,2)`
- c. Syntax error
- d. `a=2`

MCQ 6. tuple methods include:

- a. `count`, `index`
- b. `append`, `sort`
- c. `add`, `discard`
- d. `keys`, `items`

Extra short questions

1. Define tuple and one use.
2. How to "change" a tuple?
3. Why `(5,)` not `(5)`?
4. Can a tuple contain a list?

Extra long question

1. Write code that stores student (name, roll) tuples in a list, prints each, and explain two errors: mutate the tuple; forget the comma in a 1-tuple.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Immutable ordered sequence. RGB colour; returning two values.
2. Build a new one, or convert to list, change, convert back.
3. (5) is just int 5; comma makes a tuple.
4. Yes — the tuple's slots cannot be rebound, but the inner list can still mutate (a subtle exam point).

Long

1. `records.append(("Ali", 12)). for n,r in records. t[0]=... TypeError. (5) vs (5,).` 10-mark with output.

3.1.3 Sets (Unordered, Unique, Math Operations)

LEARN IT

A **set** stores an **unordered, unindexed** collection of **unique** elements. It is inspired by mathematical sets. You write a set with **curly braces** `{}`. Duplicates are removed as soon as the set is created. Because there is no index, `myset[0]` is illegal.

The set itself is **mutable** (you can add and remove items). Each item inside must be **immutable** (numbers, strings, tuples of immutables — not lists).

```
nums = {1, 4, 3, 3, 2, 2}
print(nums)
```

OUTPUT

```
{1, 2, 3, 4}
```

The two 3s and two 2s collapse to one each. Do not depend on the printed order: a set is unordered. CPython often prints small integers in a tidy way, but that is not a language promise.

Adding and removing

Method	Description
<code>.add(value)</code>	Adds one element.
<code>.update(values)</code>	Adds many elements from a list, tuple or set.
<code>.remove(value)</code>	Removes the value. Raises <code>KeyError</code> if it is missing.
<code>.discard(value)</code>	Removes the value. Does not crash if it is missing.
<code>.clear()</code>	Removes every element. Result is <code>set()</code> .

```
s = {1, 2, 3}
s.add(4)
print(s)
s.update([5, 6])
print(s)
s.discard(99)
print(s)
s.remove(1)
print(s)
s.clear()
print(s)
```

OUTPUT

```
{1, 2, 3, 4}
{1, 2, 3, 4, 5, 6}
{1, 2, 3, 4, 5, 6}
{2, 3, 4, 5, 6}
set()
```

`discard(99)` quietly does nothing. After `clear()` Python prints `set()`, not `{}`, because `{}` already means an empty **dictionary**.

Compare with `.remove(99)` on a set that does not contain 99:

```
s = {1, 2, 3}
s.remove(99)
```

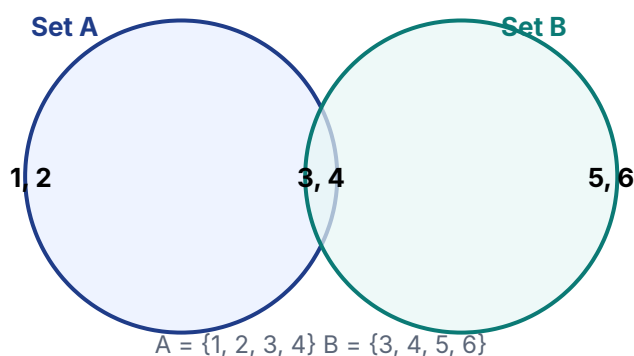
OUTPUT

```
Traceback (most recent call last):
  File "set_remove.py", line 2, in <module>
    s.remove(99)
KeyError: 99
```

Prefer `.discard()` when the item might not be there, so a missing value does not crash the program.

Mathematical operations

Sets are built for union, intersection, difference and symmetric difference.



Overlap {3, 4} is the intersection. Left-only {1, 2} is $A - B$. Right-only {5, 6} is $B - A$.

Operation	Meaning	Syntax	Result for A, B above
Union	All unique items from both	$A \mid B$ or <code>A.union(B)</code>	{1, 2, 3, 4, 5, 6}
Intersection	Items in both	$A \& B$ or <code>A.intersection(B)</code>	{3, 4}
Difference	In A, not in B	$A - B$ or <code>A.difference(B)</code>	{1, 2}

Symmetric difference	In either, but not both	<code>A ^ B</code> or <code>A.symmetric_difference(B)</code>	<code>{1, 2, 5, 6}</code>
----------------------	-------------------------	---	---------------------------

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B)
print(A & B)
print(A - B)
print(A ^ B)
```

OUTPUT

```
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
{1, 2, 5, 6}
```

`B - A` would be `{5, 6}` — difference is not symmetric. Symmetric difference `A ^ B` is `(A - B) | (B - A)`.

Set traversal

There is no index, so you loop with `for`. Print order is not guaranteed. Use `sorted()` when you need a stable alphabetical or numerical order (it returns a list).

```
colors = {"red", "green", "blue"}
for color in sorted(colors):
    print(color)
```

OUTPUT

```
blue
green
red
```

WORKED EXAMPLE

Deduplication. A teacher types roll numbers and some are repeated. A set keeps each roll once:

```
rolls = [12, 5, 12, 8, 5, 19]
unique = set(rolls)
print(unique)
print(sorted(unique))
```

OUTPUT

```
{8, 19, 12, 5}
[5, 8, 12, 19]
```

`set(rolls)` drops duplicates. `sorted(unique)` gives a list in numeric order for a register.

REMEMBER IT

SET = Same-item Eaten Twice-never. Empty set is `set()`, never `{}`. Safe delete = `discard`; angry delete = `remove`.

COMMON MISTAKE

`myset[0]` raises `TypeError` because sets are unindexed. Convert with `list(myset)` only if you accept that the order is arbitrary. Do not put a list inside a set — lists are mutable, so they cannot be set elements.

اردو میں سمجھیں

سیٹ (Set) منفرد عناصر کا بغیر ترتیب اور بغیر انڈیکس مجموعہ ہے، جو `{}` میں لکھا جاتا ہے۔ دہرائی ہوئی قیمتیں خود بخود ختم ہو جاتی ہیں۔ `add()` اور `update()` عناصر جوڑتے ہیں، `discard()` محفوظ طریقے سے ہٹاتا ہے جبکہ `remove()` غیر موجود عنصر پر `KeyError` دیتا ہے۔ اجتماع (Union) `|`، اشتراک (Intersection) `&`، فرق `-` اور متوازی فرق `^` ریاضی کی کارروائیاں ہیں۔

CHECK YOURSELF

1. What is printed by `print({1, 4, 3, 3, 2, 2})`?
2. Why prefer `.discard()` over `.remove()`?
3. If $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$, what is $A \wedge B$?

Answers: 1. `{1, 2, 3, 4}` (duplicates removed; order may vary). 2. `discard` does not raise `KeyError` if the value is missing. 3. `{1, 2, 5, 6}`.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A set holds:

- a. Unique unordered items
- b. Ordered duplicates
- c. Key-value pairs
- d. File handles only

MCQ 2. {1,2,2,3} becomes:

- a. {1,2,3}
- b. {1,2,2,3}
- c. [1,2,2,3]
- d. (1,2,2,3)

MCQ 3. s.add is for:

- a. Insert (no-op if present)
- b. Append at index 0
- c. Sort
- d. Open a file

MCQ 4. Union of {1,2} and {2,3} is:

- a. {1,2,3}
- b. {2}
- c. {1}
- d. {}

MCQ 5. You cannot:

- a. Index s[0] on a set
- b. Iterate a set
- c. Test x in s
- d. Compute intersection

MCQ 6. Sets are great for:

- a. Membership and unique collections
- b. FIFO queues
- c. LIFO undo
- d. Stable sorted ranks with duplicates

Extra short questions

1. Define set and two operations.
2. Why no index?
3. How to make unique names from a list?
4. Empty set syntax?

Extra long question

1. Two clubs A and B as sets. Write Python for members in both, only A, either. Show a sample.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Unordered unique. union | intersection & difference - .

2. No order — use a loop, not `s[0]`.
3. `set(names)` — order lost.
4. `set()` not `{}` (that is a dict).

Long

1. `A&B`, `A-B`, `A|B`. Sample Ali/Sara. Mention add/discard. One exam warning: `{}` is dict.

3.1.4 Dictionaries (Key-Value pairs) ★ Golden

LEARN IT

A **dictionary** stores data as **key–value pairs**. It works like a real dictionary: you look up a word (the **key**) to find its meaning (the **value**). Write a dictionary with curly braces, and put a **colon** between each key and value.

- **Ordered** (Python 3.7+): insertion order is kept.
- **Mutable**: you can add, change and delete pairs.
- **Keys are unique**: if you repeat a key, the last value silently overwrites the earlier one. Values may repeat.



You never search a dictionary by position. You search by key.

A school analogue: a **roll number** (key) can point to only one student (value). If you assign that roll number to someone else, the old link is gone.

Adding, changing and deleting

```
d = {"name": "Alice", "age": 30}
print(d)
d["age"] = 31
print(d)
d["city"] = "NYC"
print(d)
del d["city"]
print(d)
```

OUTPUT

```
{'name': 'Alice', 'age': 30}
{'name': 'Alice', 'age': 31}
{'name': 'Alice', 'age': 31, 'city': 'NYC'}
{'name': 'Alice', 'age': 31}
```

Assigning to an existing key updates the value. Assigning to a new key adds a pair. `del d["city"]` removes that pair.

The duplicate-keys rule. Python does not crash. The last assignment wins.

```
d = {"a": 1, "a": 2, "a": 3}
print(d)
```

OUTPUT

```
{'a': 3}
```

Accessing values safely

1. Direct brackets (risky)

`dict[key]` works if the key exists. If it is missing, the program crashes with `KeyError`.

2. `.get()` (safe)

`dict.get(key)` returns `None` if the key is missing. You may pass a default:

```
dict.get(key, 0).
```

```
person = {"name": "Alice", "age": 30}
print(person["name"])
print(person.get("salary"))
print(person.get("salary", 0))
print("name" in person)
print("salary" in person)
```

OUTPUT

```
Alice
None
0
True
False
```

The operator `in` tests **keys**, not values. `"Alice" in person` would be `False`.

```
person = {"name": "Alice", "age": 30}
print(person["salary"])
```

OUTPUT

```
Traceback (most recent call last):
  File "keyerror.py", line 2, in <module>
    print(person["salary"])
KeyError: 'salary'
```

Dictionary views and traversal

Three view methods return live objects. If the dictionary changes, the view reflects the change immediately.

Method	Returns
<code>.keys()</code>	All keys.
<code>.values()</code>	All values.
<code>.items()</code>	Key-value pairs as tuples.

```
d = {"a": 1}
keys = d.keys()
print(keys)
d["b"] = 2
print(keys)
```

OUTPUT

```
dict_keys(['a'])
dict_keys(['a', 'b'])
```

The variable `keys` was not rebuilt. The view updated itself.

```
inventory = {"apple": 50, "banana": 30, "cherry": 20}
print(list(inventory.keys()))
print(list(inventory.values()))
print(list(inventory.items()))
for key in inventory:
    print(key)
for val in inventory.values():
    print(val)
for key, val in inventory.items():
    print(f"{key} costs {val}")
```

OUTPUT

```
['apple', 'banana', 'cherry']
[50, 30, 20]
[('apple', 50), ('banana', 30), ('cherry', 20)]
apple
banana
cherry
50
30
20
apple costs 50
banana costs 30
cherry costs 20
```

A plain `for key in inventory` loops over keys (the default). `.items()` is the most useful form because you get both sides at once.

Comparison: list, tuple, set, dictionary

Feature	List	Tuple	Set	Dictionary
Written with	[]	()	{ }	{key: value}
Ordered	Yes	Yes	No	Yes (Python 3.7+)
Mutable	Yes	No	Yes (items must be immutable)	Yes

Duplicates	Allowed	Allowed	Removed	Duplicate keys not allowed (last value wins)
Access by	Index 0, 1, 2...	Index 0, 1, 2...	No index	Key
Empty value	<code>[]</code>	<code>()</code>	<code>set()</code>	<code>{}</code>
Typical use	Changing sequences	Fixed records	Unique items, set maths	Look-up tables

WORKED EXAMPLE

Same data, four structures. Three subject marks:

- List `[80, 70, 90]` — easy to append a fourth test.
- Tuple `(80, 70, 90)` — a frozen result slip.
- Set `{80, 70, 90}` — unique scores only; two papers of 80 would become one 80.
- Dictionary `{"math": 80, "eng": 70, "cs": 90}` — you ask "What is CS?" and get 90 without remembering an index.

REMEMBER IT

Keys are unique, like roll numbers. Values are the student records and may repeat (two students can both score 80). Memory: "**get** is gentle, brackets can **break**."

COMMON MISTAKE

Older notes (and the chapter summary in some slides) still call dictionaries "unordered." From Python 3.7 onwards insertion order is preserved, so you may write "ordered since 3.7." Keys must be immutable: a list cannot be a key; a tuple of numbers can. Do not write `{}` when you want an empty set.

اردو میں سمجھیں

ڈکشنری (Dictionary) کلید اور قدر (Key-Value) کے جوڑوں میں ڈیٹا رکھتی ہے۔ کلید منفرد ہوتی ہے؛ دہرائی ہوئی کلید پر آخری قدر رہ جاتی ہے۔ `d[key]` غیر موجود کلید پر `KeyError` دیتا ہے، جبکہ `d.get(key, default)` محفوظ ہے۔ `.values`، `keys()` اور `items()` سے گھما جا سکتا ہے۔ فہرست انڈیکس سے، ڈکشنری کلید سے پہچانی جاتی ہے۔

EXAM TIP

High-yield: definition of a dictionary, difference between `d[key]` and `d.get(key)`, “what if a key is repeated?”, traversal with `.items()`, and a full comparison table of the four structures. Memorise the comparison table in this section — it is a complete 8-mark answer if you add one example row. Always mention that `in` checks keys.

CHECK YOURSELF

1. What does `{"a": 1, "a": 2, "a": 3}` print?
2. What does `person.get("salary", 0)` return if `salary` is missing?
3. Which structure has no index and no duplicate items?

Answers: 1. `{'a': 3}`. 2. `0`. 3. A set.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A dict stores:

- a. key: value pairs
- b. Only a list of values
- c. Only unique numbers without keys
- d. Only files

MCQ 2. Keys in a dict must be:

- a. Hashable (immutable) and unique
- b. Lists
- c. Sets
- d. Other dicts as keys commonly

MCQ 3. `d.get("x")` if missing:

- a. Returns None (or default) instead of crashing
- b. Always `KeyError`
- c. Deletes `d`
- d. Sorts `d`

MCQ 4. `d["x"]` if missing:

- a. `KeyError`
- b. None always
- c. 0 always
- d. Inserts `NAND`

MCQ 5. `d.keys()` is:

- a. A view of keys
- b. A tuple of values
- c. A file
- d. A stack

MCQ 6. Updating `d["a"]=2` if `a` exists:

- a. Overwrites
- b. Creates a second `a`
- c. Errors always
- d. Appends a list

Extra short questions

1. Define dictionary with a 2-pair example.
2. Safe read vs direct read.
3. How to loop keys and values.
4. Why lists cannot be keys.

Extra long question

1. Write a marks dict program: add a student, update, print using `items()`, handle a missing name with `get`. Sample I/O.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. {"Ali": 85, "Sara": 90}
2. get vs [].
3. for k,v in d.items():
4. Mutable, unhashable.

Long

1. CRUD-ish four operations. KeyError demo vs get. 10-mark complete code.

3.1.5 Common Built-in Functions (min, max, len, sum)

LEARN IT

Several built-in functions work across lists, tuples, sets and dictionaries. You do not call them with a dot on the object (except when a type also has its own method, such as list `.sort()`).

Function	Purpose	Works on
<code>min()</code>	Smallest element (numeric or alphabetical)	List, tuple, set, dictionary keys
<code>max()</code>	Largest element	List, tuple, set, dictionary keys
<code>len()</code>	Number of elements (for a dict: number of keys)	List, tuple, set, dictionary
<code>sum()</code>	Adds numeric elements	List, tuple, set, dictionary values via <code>sum(d.values())</code>
<code>sorted()</code>	Returns a new sorted list ; original unchanged	List, tuple, set, dictionary keys

```
s = {5, 1, 9, 3, 7}
print(len(s))
print(min(s))
print(max(s))
print(sum(s))
print(sorted(s))
```

OUTPUT

```
5
1
9
25
[1, 3, 5, 7, 9]
```

`5 + 1 + 9 + 3 + 7 = 25`. `sorted(s)` always returns a **list**, even when the input is a set.

WORKED EXAMPLE

The same five functions on four structures.

```
nums = [8, 3, 5, 3]
t = (8, 3, 5, 3)
s = {8, 3, 5}
d = {"math": 80, "eng": 70, "cs": 90}
print(len(nums), min(nums), max(nums), sum(nums), sorted(nums))
print(len(t), min(t), max(t), sum(t), sorted(t))
print(len(s), min(s), max(s), sum(s), sorted(s))
print(len(d), min(d), max(d), sum(d.values()), sorted(d))
```

OUTPUT

```
4 3 8 19 [3, 3, 5, 8]
4 3 8 19 [3, 3, 5, 8]
3 3 8 16 [3, 5, 8]
3 cs math 240 ['cs', 'eng', 'math']
```

On the dictionary, `min(d)` and `max(d)` compare **keys as strings**: "cs" comes first alphabetically, "math" last. `sum(d)` would try to add the keys "math" + "eng" + ... and fail. Use `sum(d.values())` — $80 + 70 + 90 = 240$. The set dropped the duplicate 3, so `len(s)` is 3 and `sum(s)` is 16.

`sorted()` **versus** `.sort()`

```
original = [8, 3, 5]
print("before sort:", original)
print("sorted():", sorted(original))
print("after sorted():", original)
original.sort()
print("after .sort():", original)
```

OUTPUT

```
before sort: [8, 3, 5]
sorted(): [3, 5, 8]
after sorted(): [8, 3, 5]
after .sort(): [3, 5, 8]
```

`.sort()` exists only on lists and rewrites the original. `sorted()` works on any iterable and leaves the original alone.

REMEMBER IT

LEN-MIN-MAX-SUM — count, smallest, largest, total. For a dictionary:

`len / min / max / sorted` use **keys**; `sum` needs `.values()`.

COMMON MISTAKE

`sum()` needs numbers. `sum(["a", "b"])` raises `TypeError`. Empty structures: `len([])` is 0, but `min([])` and `max([])` raise `ValueError` because there is no smallest of nothing.

اردو میں سمجھیں

`len()` عناصر گنتا ہے، `min()` سب سے چھوٹا، `max()` سب سے بڑا، `sum()` اعداد کا مجموعہ، اور `sorted()` نئی ترتیب شدہ فہرست دیتا ہے بغیر اصل کو بدلے ہیں۔ ڈکشنری پر `min` اور `max` کلیدوں پر چلتے ہیں؛ قدروں کا مجموعہ `sum(d.values())` سے نکالیں۔ `sort()` صرف فہرست کا میتھڈ ہے جو اصل فہرست بدل دیتا ہے۔

CHECK YOURSELF

1. For `s = {5, 1, 9, 3, 7}`, what are `len(s)`, `min(s)`, `max(s)`, `sum(s)`?
2. Does `sorted(s)` return a set or a list?
3. How do you total the values of `{"math": 80, "eng": 70, "cs": 90}`?

Answers: 1. 5, 1, 9, 25. 2. A list: [1, 3, 5, 7, 9]. 3. `sum(d.values())` → 240.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. `len(x)` works on:

- a. Sequences and many collections
- b. Only ints
- c. Only files
- d. Only NAND

MCQ 2. `min([3,1,2])` is:

- a. 1
- b. 3
- c. 2
- d. 6

MCQ 3. `sum((1,2,3))` is:

- a. 6
- b. 123
- c. Error always
- d. (1,2,3)

MCQ 4. `max` on a dict without a key function uses:

- a. Keys (careful!)
- b. Values always
- c. Lengths always
- d. Random

MCQ 5. These functions expect:

- a. Non-empty for min/max
- b. Always empty
- c. Only sets
- d. Only files

MCQ 6. `len("CS")` is:

- a. 2
- b. 1
- c. 0
- d. 3

Extra short questions

1. Four built-ins and a type they work on.
2. What happens `min([])`
3. sum of a list of strings?
4. len vs count in a list.

Extra long question

1. Given marks = [70, 80, 90] write expressions for n, total, mean, best, worst using built-ins. Show values. Warn about empty lists.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. len list; min tuple; max set; sum list of numbers.
2. ValueError.
3. TypeError — sum starts at 0.
4. len whole length; count(x) occurrences of x.

Long

1. n=len=3; sum=240; mean=80; max=90; min=70. Empty: don't divide; min/max error.
Link to Ch 4 statistics.

3.1.6 Activity: Students Attendance Program

LEARN IT

Teachers need a quick way to mark who is in class. This program stores names in a **list**, asks about each student, counts how many said yes, and prints a summary with `len()`.

Store names in a list



Ask present? for each



Count yes



Print totals

```
students = ["Ali", "Sara", "Ayesha"]
present_count = 0
print("--- Attendance System ---\n")
for student in students:
    status = input(f"Is {student} present? (yes/no): ").lower()
    if status == "yes":
        present_count += 1
print("\n--- Attendance Summary ---")
print("Total Students:", len(students))
print("Students Present:", present_count)
print("Students Absent:", len(students) - present_count)
```

WORKED EXAMPLE

Sample run. Ali yes, Sara no, Ayesha yes. `.lower()` means YES or Yes still counts as present.

OUTPUT

```
--- Attendance System ---

Is Ali present? (yes/no): yes
Is Sara present? (yes/no): no
Is Ayesha present? (yes/no): yes

--- Attendance Summary ---
Total Students: 3
Students Present: 2
Students Absent: 1
```

Student	Typed answer	After <code>.lower()</code>	<code>present_count</code>
Ali	yes	yes → add 1	1
Sara	no	not yes	1
Ayesha	yes	yes → add 1	2

Absent = `len(students) - present_count` = 3 - 2 = 1. The counter must start at 0 **before** the loop. If you put `present_count = 0` inside the loop, the count would reset for every student.

REMEMBER IT

Counter pattern: start at 0, loop, add 1 when a condition is true. You will reuse this in the word-frequency program (add 1 per word) and the grade tracker (add each mark).

COMMON MISTAKE

Only the exact string "yes" after `.lower()` counts as present. "y" or "present" is treated as absent. In a longer program you could also accept "y"; this activity keeps the rule simple.

اردو میں سمجھیں

حاضری کا پروگرام طلبہ کے نام ایک فہرست میں رکھتا ہے، ہر نام کے لیے پوچھتا ہے، اور yes پر شمار کار (Counter) ایک بڑھاتا ہے۔ کل طلبہ `len(students)` سے، غیر حاضر کل - حاضر سے نکلتے ہیں۔ شمار کار لوپ سے پہلے صفر ہونا چاہیے۔

CHECK YOURSELF

1. If all three students type `no`, what is printed for present and absent?
2. Why is `.lower()` used on the answer?

Answers: 1. Present 0, Absent 3. 2. So that YES, Yes and yes all match the string "yes".

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A sensible structure for present/absent flags by name is:

- a. dict name→status or two lists/sets
- b. A pie without data
- c. A NAND net
- d. An OSI capture

MCQ 2. You should still:

- a. Validate input (yes/no)
- b. Trust any string
- c. Use eval
- d. Open the OS

MCQ 3. Printing a report uses:

- a. A loop over the collection
- b. A single print of the object id
- c. TCP
- d. Figma

MCQ 4. This activity combines:

- a. DS + input + selection + repetition
- b. Only HCI colours
- c. Only ER
- d. Only Big O proofs

MCQ 5. A set of absentees is useful to:

- a. Unique names, fast membership
- b. Keep duplicate absences as a stack
- c. FIFO print jobs
- d. Store key-value marks

MCQ 6. Never store:

- a. Passwords in the same toy script unprotected
- b. Names
- c. P/A flags
- d. Dates

Extra short questions

1. Outline the attendance program's data and loop.
2. Two Python DS options.
3. How do you count presents?
4. One improvement for XII files.

Extra long question

1. Write a complete small attendance program for 3 names using a dict, print a tidy table, and a sample run.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. For each student: ask status; store; then print counts.
2. dict[str,str] or present_set/absent_set.
3. Counter in the loop or list.count / sum(1 for ...).
4. Save to a text file with with-open.

Long

1. Working code, output, and one validation (only P/A). Mention a logic error if you forget to store.

3.1.7 Activity: Word Frequency Counter

LEARN IT

Count how many times each word appears in a sentence. This is a classic use of a **dictionary** plus `.get()`: the word is the key, the count is the value.

```
sentence = "python is easy python is fun python is powerful"
words = sentence.split()
word_freq = {}
for word in words:
    word_freq[word] = word_freq.get(word, 0) + 1
print(f"Sentence: '{sentence}'\n")
print("Word Frequency Table:")
for word, count in word_freq.items():
    print(f"{word:<10} : {count}")
top_word = max(word_freq, key=word_freq.get)
print(f"\nMost Frequent: '{top_word}' ({word_freq[top_word]} times)")
```

WORKED EXAMPLE

Logic, step by step.

1. `sentence.split()` cuts on spaces and gives
`['python', 'is', 'easy', 'python', 'is', 'fun', 'python', 'is', 'powerful']`.
2. For a new word, `word_freq.get(word, 0)` is 0, then you store $0 + 1 = 1$.
3. For a word already in the dictionary, `.get(word, 0)` returns the current count, then you add 1.
4. `{word:<10}` pads the word to 10 characters so the colons line up.
5. `max(word_freq, key=word_freq.get)` picks the key whose value is largest. If two words share the top count, `max` returns the first of those keys in insertion order.

OUTPUT

Sentence: 'python is easy python is fun python is powerful'

Word Frequency Table:

```
python      : 3
is          : 3
easy        : 1
fun         : 1
powerful    : 1
```

Most Frequent: 'python' (3 times)

After seeing	<code>word_freq</code>
python	<code>{'python': 1}</code>
is	<code>{'python': 1, 'is': 1}</code>
easy	<code>{'python': 1, 'is': 1, 'easy': 1}</code>
python (again)	<code>{'python': 2, 'is': 1, 'easy': 1}</code>
... end of sentence	<code>{'python': 3, 'is': 3, 'easy': 1, 'fun': 1, 'powerful': 1}</code>

Here `python` and `is` both occur 3 times. `max` returns `'python'` because it appears first in the dictionary.

REMEMBER IT

The heart of every counter is one line: `d[item] = d.get(item, 0) + 1`. You can count letters, roll numbers or votes with the same pattern.

COMMON MISTAKE

This program is case-sensitive and space-based. "Python" and "python" would be two keys. Punctuation such as `easy.` would not match `easy`. For a class exercise, keep the sentence lowercase with spaces only, as here. If you wrote `word_freq[word] += 1` without `.get` on a new word, you would get `KeyError`.

اردو میں سمجھیں

جملے کو `.split()` سے الفاظ کی فہرست بناتے ہیں، پھر ہر لفظ ڈکشنری میں کلید ہوتا ہے اور اس کی گنتی قدر۔ نئے لفظ کے لیے `get(word, 0) + 1` سے گنتی 1 سے شروع ہوتی ہے۔ `max(... , key=word_freq.get)` سب سے زیادہ بار آنے والا لفظ دیتا ہے۔

CHECK YOURSELF

1. How many times does "is" occur in the sample sentence?
2. Why use `.get(word, 0)` instead of `word_freq[word]` on the right-hand side?

Answers: 1. Three times. 2. Because a brand-new word is not yet a key; `.get` returns 0 instead of raising `KeyError`.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Word frequency uses a dict as:

- a. word → count
- b. count → word only
- c. A stack of letters
- d. A queue of files

MCQ 2. A standard pattern is:

- a. $d[w] = d.get(w,0)+1$
- b. sort first always $O(n^2)$
- c. binary search the sentence
- d. draw OSI

MCQ 3. You usually:

- a. `split()` and maybe `lower()`
- b. Keep punctuation attached always
- c. Use a set so counts die
- d. Use a tuple of counts you cannot update

MCQ 4. `split("a a b")` has length:

- a. 3
- b. 2
- c. 1
- d. 0

MCQ 5. This activity shows:

- a. Dicts + loops + strings
- b. Only Access reports
- c. Only NAND
- d. Only HCI

MCQ 6. Most frequent word is:

- a. The key with max count
- b. Always the first word
- c. Always the last
- d. A foreign key

Extra short questions

1. Write the counting pattern.
2. How to print nicely.
3. A limitation of naive split.
4. Why a dict not a list of pairs?

Extra long question

1. Count words in "to be or not to be". Show the dict, then the mode word(s). Full code.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. `for w in text.lower().split(): d[w]=d.get(w,0)+1`

2. for k,v in d.items(): print(k,v) or sorted by count.
3. Punctuation ("hello,") is a different word.
4. $O(1)$ average update by word.

Long

1. to:2 be:2 or:1 not:1. Bimodal to/be. Code with get. Mention lower().

3.2 Functions in Python ★ Golden

LEARN IT

When the same instructions are written again and again, that is **duplicate code**. Duplicate code makes a program longer, harder to read, and painful to update (you must fix every copy). A **function** lets you write the instructions **once** and run them whenever you call the function's name.

Benefit	Meaning
Code reusability	Write once, use anywhere.
Modularity	Break a big problem into small named tasks.
Readability	Shorter, cleaner files with meaningful names.
Maintainability	Fix a bug in one place and every call is fixed.

```
def greet():  
    print("Hello, World!")  
  
greet()  
greet()
```

OUTPUT

```
Hello, World!  
Hello, World!
```

The body is written once. Two calls run it twice. To change the message you edit one line inside `greet`.

Key components of a function



- **Keyword** — Python uses `def` to start a definition.
- **Function name** — the identifier you call, e.g. `greet`.
- **Parameters** — input names inside the parentheses. When you call the function, the values you pass are called **arguments**.
- **Body** — the indented block that runs on each call.
- **Return statement** — sends a result back to the caller (3.2.2).


```
def add(a, b):  
    print(f"Adding {a} + {b}")  
    return a + b  
  
result = add(4, 6)  
print(result)
```

OUTPUT

```
Adding 4 + 6  
10
```

Here `a` and `b` are parameters. `4` and `6` are arguments. `return a + b` sends 10 back, and that 10 is stored in `result`.

WORKED EXAMPLE

Without a function you would copy the print-and-add lines every time you needed a sum. With a function, a new sum is one call: `add(10, 20)`, `add(1.5, 2.5)`.

REMEMBER IT

DRY — Don't Repeat Yourself. If you are about to copy-paste a block, wrap it in `def`. Name functions with verbs: `greet`, `add`, `save_grades`.

COMMON MISTAKE

Defining is not calling. After `def greet()`: nothing prints until you write `greet()`. Forgetting the parentheses — writing `greet` alone — does not run the body. The body must be indented; a line at column 0 is outside the function.

اردو میں سمجھیں

فنکشن (Function) ہدایات کا وہ بلاک ہے جو ایک بار لکھا جاتا ہے اور بار بار بلایا جا سکتا ہے۔ اس سے کوڈ مختصر، ماڈیولر اور آسان اصلاح والا بنتا ہے۔ تعریف `def` سے شروع ہوتی ہے، نام اور پیرامیٹرز قوسین میں ہوتے ہیں، جسم انڈینٹ ہوتا ہے، اور `return` نتیجہ واپس بھیج سکتا ہے۔ تعریف کرنے سے فنکشن چلتا نہیں جب تک آپ اسے کال نہ کریں۔

EXAM TIP

Write the four benefits (reusability, modularity, readability, maintainability) as separate bullets — that is a standard short question. For long questions, define a function, label `def`, name, parameters, body and `return`, then show one call. "What is duplicate code and how do functions solve it?" is also common.

CHECK YOURSELF

1. What keyword starts a function definition?
2. How many times does `Hello, World!` print in the `greet` program above, and why?

Answers: 1. `def`. 2. Twice, because `greet()` is called twice.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A function is:

- a. A named reusable block
- b. A random line in main only
- c. A pie
- d. A cable

MCQ 2. You define with:

- a. `def name(params):`
- b. `function name {}`
- c. `fun name`
- d. `lambda` only always

MCQ 3. A call is:

- a. `name(arguments)`
- b. `def` again
- c. `import name` always
- d. `print` the source only

MCQ 4. Duplicate code is bad because:

- a. Fixes must be repeated; more bugs
- b. Python forbids functions
- c. It is faster always
- d. It is required by BLEK

MCQ 5. This is ★: expect:

- a. Write a small function + call + output
- b. Only define HCI
- c. Only OSI
- d. Only ER

MCQ 6. Parameters are:

- a. Placeholders in the definition
- b. The values at the call (arguments)
- c. Always globals
- d. Always files

Extra short questions

1. Define function, parameter, argument.
2. Why functions (three reasons).
3. Show a 3-line add function.
4. What happens if you `def` but never call?

Extra long question

1. Write functions to compute area of rectangle and circle, call both from main, show output. Explain `def` vs `call`. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Reusable block; names in `def`; values you pass.

2. Reuse, clarity, testing, less duplication.
3. `def add(a,b): return a+b`
4. Nothing runs — no output.

Long

1. Two defs, two calls, printed numbers. Mention return vs print inside. $\pi=3.14$ or `math.pi`.

3.2.1 Types of Functions (Built-in vs User-defined)

LEARN IT

There are two main types of functions in Python.

a) Built-in functions are already defined by Python. You just call them. Examples you have already used: `len()`, `range()`, `sum()`, `print()`, `input()`, `min()`, `max()`, `sorted()`, `int()`, `float()`, `type()`.

b) User-defined functions are functions **you** create with `def` for tasks that no built-in covers in one step — greeting a user, dividing with a zero check, writing a grade file.

	Built-in	User-defined
Who writes it?	The Python language	The programmer
How you start	Just call the name	First write <code>def name(...):</code>
Examples	<code>len("Ali")</code> → 3	<code>def add(a, b): return a + b</code>
Purpose	Everyday operations needed by everyone	Your program's specific jobs

WORKED EXAMPLE

Side by side. Counting letters is built-in. Combining a greeting with a name is user-defined.

```
print(len("Sindh"))

def greet_student(name):
    print("Hello,", name)

greet_student("Hassan")
greet_student("Malika")
```

OUTPUT

```
5
Hello, Hassan
Hello, Malika
```

You did not write the code inside `len`. You did write the code inside `greet_student`, then reused it for two names.

REMEMBER IT

If Python already has it (`len`, `sum`, `max`), do not rewrite it. Write a user-defined function when you are packaging **your** steps.

COMMON MISTAKE

Do not name your function `list`, `print`, `sum` or `input`. You would hide the built-in name for the rest of that file. Choose names such as `print_bill` or `sum_marks`.

اردو میں سمجھیں

بلٹ ان فنکشن پائتھن میں پہلے سے موجود ہوتے ہیں، جیسے `len()` اور `print()`۔ یوزر ڈیفائنڈ فنکشن وہ ہیں جو پروگرامر `def` سے خود بناتا ہے تاکہ مخصوص کام بار بار کیا جا سکے۔ جو کام بلٹ ان پہلے کر دیتے ہیں انہیں دوبارہ نہ لکھیں۔

CHECK YOURSELF

1. Is `range()` built-in or user-defined?
2. Which keyword creates a user-defined function?

Answers: 1. Built-in. 2. `def`.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Built-in functions include:

- a. len, print, min (provided by Python)
- b. Only ones you def
- c. Only pip packages
- d. Only SQL

MCQ 2. User-defined functions are:

- a. Ones you write with def
- b. print and len
- c. CPU instructions
- d. OSI layers

MCQ 3. You still:

- a. Call both with name()
- b. Cannot call built-ins
- c. Must recompile C
- d. Must draw Figma

MCQ 4. A module function like math.sqrt is:

- a. Library/built-in-standard, not your def
- b. A user def in your file
- c. A dict
- d. A stack

MCQ 5. Why write your own if print exists?

- a. Your problem's chunks (bill tax, grade band) are not built in
- b. Python forbids print
- c. Built-ins cannot be called
- d. def is illegal

MCQ 6. Help(len) documents a:

- a. Built-in
- b. Your secret function always
- c. A table
- d. A query

Extra short questions

1. Differentiate built-in vs user-defined with examples.
2. Can user-defined call built-in?
3. Where do library functions sit?
4. One exam trap.

Extra long question

1. In one program use a built-in, a standard-library function, and a user-defined function. Label each and show output.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Built-in: print, len. User: def grade(m): ...
2. Yes — that is normal.
3. Neither in your file nor mysterious — imported modules.
4. Calling a function you never defined and is not built-in → NameError.

Long

1. print/len; math.sqrt; def mean. Three labels in comments. Exact numeric output.

3.2.2 Functions with Return Values

LEARN IT

A function can either **print** a result (show it on the screen) or **return** a result (send it back to the caller). A returned value can be stored in a variable, printed later, or used in a further calculation. The call is replaced by the returned value, as if you had written that value in its place.

```
def multiply(a, b):  
    return a * b  
  
result = multiply(5, 3)  
print("The answer is:", result)
```

OUTPUT

The answer is: 15

`multiply(5, 3)` evaluates to 15. That 15 is stored in `result`.

WORKED EXAMPLE

Print versus return. This is the difference examiners want.

```
def show_sum(a, b):  
    print(a + b)  
  
def give_sum(a, b):  
    return a + b  
  
x = show_sum(2, 3)  
print("stored from show_sum:", x)  
y = give_sum(2, 3)  
print("stored from give_sum:", y)  
print("double it:", y * 2)
```

OUTPUT

5
stored from show_sum: None
stored from give_sum: 5
double it: 10

`show_sum` prints 5 inside the function, then returns `None` (the default when there is no `return`). You cannot double that answer later. `give_sum` hands 5 back, so `y * 2` is 10. Use `print` for messages; use `return` for results the rest of the program still needs.

REMEMBER IT

return = send back; print = show now. `return` also **exits** the function immediately. Lines after `return` in the same function do not run.

COMMON MISTAKE

Writing `print(a * b)` inside `multiply` and then `result = multiply(5, 3)` stores `None` in `result`, not 15. If you later write `result + 1` you get `TypeError`. Return the number; let the caller decide whether to print it.

اردو میں سمجھیں

`return` نتیجہ کال کرنے والے پروگرام کو واپس بھیجتا ہے تاکہ وہ متغیر میں محفوظ ہو سکے یا آگے حساب میں لگے۔ اگر `return` نہ ہو تو فنکشن `None` دیتا ہے۔ `print` صرف اسکرین پر دکھاتا ہے؛ اس سے حاصل شدہ قیمت دوبارہ استعمال نہیں ہوتی۔

CHECK YOURSELF

1. What is stored in `result` after `result = multiply(5, 3)` with `return a * b`?
2. What does a function return if it has no `return` statement?

Answers: 1. 15. 2. `None`.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. return sends a value:

- a. Back to the caller
- b. To the printer always
- c. To a file always
- d. To OSI layer 1

MCQ 2. A function with only print:

- a. Shows something but gives None to the caller
- b. Returns the text automatically
- c. Returns 0
- d. Cannot run

MCQ 3. `x = f()` if `f` has no return:

- a. `x` is None
- b. `x` is 0
- c. Error always
- d. `x` is `f`

MCQ 4. return also:

- a. Exits the function immediately
- b. Loops forever
- c. Defines a class
- d. Opens a socket

MCQ 5. You can return:

- a. Any object (number, str, tuple, list, dict)
- b. Only ints
- c. Only True
- d. Only files

MCQ 6. `print(f())` prints the:

- a. Returned value (or None)
- b. Source code
- c. The name `f`
- d. A pie

Extra short questions

1. Why return rather than print inside?
2. Show both styles for doubling 5.
3. Returning two values.
4. What is None?

Extra long question

1. Write `mean(a,b,c)` that returns a float. Show a test print and a logic error version that prints but returns None. Explain the bug in a unit-test style.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. The caller can store, test, or reuse the value.
2. print inside vs return 10 and print in main.
3. return a,b → a tuple, unpack.
4. The default return if you fall off the end.

Long

1. Correct return (a+b+c)/3. Buggy print only → assert mean(1,2,3)==2 fails with None.
Teaching point of return.

3.2.3 Activity: Arithmetic Calculator Using Functions

LEARN IT

Each arithmetic operation is a **separate function**. That is modularity: addition is one small task, division is another (with a guard against dividing by zero). The main program only asks for two numbers and calls the functions.

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Cannot divide by zero"

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
print("Addition:", add(num1, num2))
print("Subtraction:", subtract(num1, num2))
print("Multiplication:", multiply(num1, num2))
print("Division:", divide(num1, num2))
```

WORKED EXAMPLE

Sample run with 12 and 4. `float` is used so decimals work; even whole numbers print with a decimal (12 becomes 12.0).

OUTPUT

```
Enter first number: 12
Enter second number: 4
Addition: 16.0
Subtraction: 8.0
Multiplication: 48.0
Division: 3.0
```

Check: $12 + 4 = 16$, $12 - 4 = 8$, $12 \times 4 = 48$, $12 \div 4 = 3$. In Python 3, `/` always produces a float, so 3.0 not 3.

Zero divisor:

```
def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Cannot divide by zero"

print(divide(10, 0))
print(divide(10, 2))
```

OUTPUT

```
Cannot divide by zero
5.0
```

Without the `if b != 0` test, `10 / 0` would raise `ZeroDivisionError` and stop the program.

REMEMBER IT

Four functions, four names: `add`, `subtract`, `multiply`, `divide`. The main program should look like a contents page — a sequence of calls — not a pile of formulae.

COMMON MISTAKE

`input()` returns a string. If you skip `float()` (or `int()`), `add` would concatenate: `"12" + "4"` is `"124"`. Convert first. Also, `divide` returns either a number or a string; do not later do arithmetic on the error message.

کیلکولیٹر میں ہر عمل الگ فنکشن ہے: جمع، تفریق، ضرب، تقسیم۔ یہ ماڈیولریٹی ہے۔ تقسیم سے پہلے صفر کی جانچ ضروری ہے ورنہ پروگرام رک جاتا ہے۔ `input()` سے ملنے والی تار کو `float()` سے عدد بنانا لازمی ہے ورنہ جمع جوڑنے کے بجائے الفاظ جوڑ دے گی۔

CHECK YOURSELF

1. What does `divide(10, 0)` return in this activity?
2. Why are the numbers converted with `float()` ?

Answers: 1. The string "Cannot divide by zero". 2. Because `input()` returns text; `float()` turns it into a number so `+` `-` `*` `/` do arithmetic, including decimals.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A calculator activity should use:

- a. A function per operation
- b. One 200-line print spaghetti only
- c. No functions
- d. Only globals named a,b,c,d,e

MCQ 2. Division must consider:

- a. Zero divisor
- b. NAND grouping
- c. OSI layer
- d. Figma frames

MCQ 3. Menu + functions is:

- a. Selection + modularity
- b. A stack hardware
- c. A pie
- d. A foreign key

MCQ 4. return lets you:

- a. Print nicely in one place
- b. Hide errors
- c. Skip tests
- d. Avoid operators

MCQ 5. float(input) may raise:

- a. ValueError
- b. A pie
- c. A K-map
- d. A sprint

MCQ 6. This activity prepares:

- a. Reusable functions + later try/except
- b. Only Access
- c. Only HCI colours
- d. Only ER

Extra short questions

1. Outline four functions for $+$ $-$ $\times$ $\div$.
2. How do you stop divide by zero?
3. Where does the menu live?
4. Why not copy the formula 12 times?

Extra long question

1. Write a mini calculator with four functions, a loop menu, and a safe divide. Sample session of 3 operations then quit.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. add, sub, mul, div with two params, return.

2. if `b==0`: return a message / raise / skip.
3. In main: while True: choice...
4. One fix in one function.

Long

1. Working structure, sample I/O, ZeroDivision handled. Mention return vs print.

3.2.4 Scope of Local and Global Variables

LEARN IT

Scope is the region of a program where a variable can be seen and used. Python has two scopes you must know for this chapter.

i) Local scope. A **local variable** is created inside a function. It exists only while that function is running. The main program cannot read it.

ii) Global scope. A **global variable** is created outside all functions. Any function may **read** it, and the main program can use it too.

WORKED EXAMPLE

Local — the name is invisible outside.

```
def calc():  
    secret = 50  
    print(secret)  
  
calc()  
print(secret)
```

OUTPUT

```
50  
Traceback (most recent call last):  
  File "local_scope.py", line 5, in <module>  
    print(secret)  
NameError: name 'secret' is not defined
```

First the function prints 50. Then the main program tries `print(secret)` and fails: `secret` was born inside `calc` and died when `calc` finished.

Global — readable inside and outside.

```
x = 10  
  
def show():  
    print("Inside function:", x)  
  
show()  
print("Outside function:", x)
```

OUTPUT

```
Inside function: 10  
Outside function: 10
```

Assignment inside a function creates a new local with the same name. It does not change the global:

```
x = 10  
  
def show():  
    x = 99  
    print("Inside:", x)  
  
show()  
print("Outside:", x)
```

OUTPUT

```
Inside: 99  
Outside: 10
```

Inside, `x = 99` is a local. After the call, the global is still 10.

REMEMBER IT

Local = inside the room; global = in the corridor. Everyone can see the corridor. Things left in a room are gone when you leave. Prefer locals. Too many globals make bugs hard to find because any function might be using them.

COMMON MISTAKE

If you only **read** `x` inside the function, Python uses the global. If you **assign** to `x` anywhere in that function, Python treats `x` as local for the whole function. Mixing the two without care causes `UnboundLocalError`. Keep function inputs as parameters and outputs as `return` values instead of editing globals.

اردو میں سمجھیں

دائرہ کار (Scope) بتاتا ہے کہ متغیر کہاں دکھائی دیتا ہے۔ لوکل متغیر فنکشن کے اندر بنتا ہے اور فنکشن ختم ہوتے ہی غائب ہو جاتا ہے؛ باہر سے پڑھنے پر `NameError` آتا ہے۔ گلوبل متغیر تمام فنکشنز سے باہر بنتا ہے اور اندر سے پڑھا جا سکتا ہے۔ فنکشن کے اندر اسی نام کو نئی قدر دینے سے نیا لوکل بنتا ہے، گلوبل نہیں بدلتا۔

CHECK YOURSELF

1. Where is a local variable defined?
2. In the last example, what does the main program print for `x` after `show()` ?

Answers: 1. Inside a function. 2. 10 (the global is unchanged).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A local variable lives:

- a. Inside the function
- b. In the whole file always
- c. On the disk
- d. In OSI

MCQ 2. A global variable lives:

- a. At module level (the file)
- b. Only in one for-loop
- c. Only in a dict key
- d. Only in RAM of another PC

MCQ 3. Reading a global from a function:

- a. Works
- b. Always error
- c. Deletes it
- d. Makes it local

MCQ 4. Assigning to a name in a function without global:

- a. Creates a local (does not change the outer)
- b. Always updates global
- c. Syntax error
- d. Opens a file

MCQ 5. global x is needed to:

- a. Rebind a global from inside a function
- b. Import math
- c. Draw a pie
- d. Sort a list

MCQ 6. Prefer:

- a. Parameters and return, not lots of globals
- b. All globals
- c. No names
- d. eval

Extra short questions

1. Define local vs global.
2. A classic bug.
3. How to share results instead of globals?
4. Can two functions have locals named total?

Extra long question

1. Show a 10-line program where a global count is incremented with global, then rewrite it with return so you do not need global. Explain which is clearer.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Local: created in function, dies when it ends. Global: file-level.
2. `x=1; def f(): x=x+1` without `global` → `UnboundLocalError`.
3. return values / pass them as arguments.
4. Yes — they are different variables.

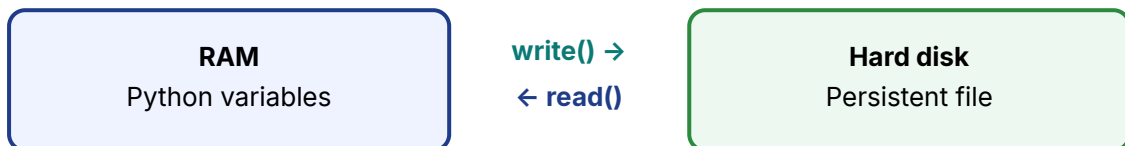
Long

1. Both outputs match. Prefer `return`. `UnboundLocalError` snippet as a warning.

3.3 & 3.4 File Handling and Operations (Modes r, w, a) ★ Golden

LEARN IT

File handling is a program's ability to create, read, write, update and delete files on disk. Variables live in **RAM** and vanish when the program ends. Files live on a **hard disk or SSD** and stay there — they are **persistent**.



Read copies data from disk into RAM. Write copies data from RAM onto disk.

Types of files

Type	What it stores	Examples
Text file	Human-readable characters. You can open it in a text editor.	<code>.txt</code> , <code>.csv</code> , <code>.py</code>
Binary file	Raw bytes, not meant to be read as text.	<code>.jpg</code> , <code>.mp3</code> , <code>.dat</code>

This chapter uses text files.

The `open()` function and modes

You must open a file before you use it: `open(filepath, mode)`.

Mode	Name	Description
<code>'r'</code>	Read	Open for reading. Fails if the file does not exist.
<code>'w'</code>	Write	Open for writing. Erases existing content. Creates the file if it is missing.
<code>'a'</code>	Append	Open for writing at the end . Keeps old content. Creates the file if it is missing.
<code>'rb'</code> / <code>'wb'</code>	Binary	Read or write bytes (images, audio).

File object methods

Method	What it does
<code>.read()</code>	Whole file as one string.
<code>.readlines()</code>	All lines as a list of strings (each usually ends with <code>\n</code>).
<code>.write("text")</code>	Writes a string. You must add <code>\n</code> yourself for a new line.
<code>.close()</code>	Saves to disk, closes the file, frees RAM. Essential if you did not use <code>with</code> .

WORKED EXAMPLE

Create a file in write mode.

```
new_file = open('myfile.txt', 'w')
new_file.write("The quick brown fox jumps over the lazy dog!!!\n")
new_file.close()
print("File has been created successfully.")
```

OUTPUT

```
File has been created successfully.
```

If `myfile.txt` did not exist, `'w'` created it. The sentence is now on disk. `.close()` flushes the buffer so the bytes actually reach the disk.

Danger of `'w'`: opening an existing file in `'w'` deletes everything first.

```
f = open('myfile.txt', 'w')
f.write("FIRST\n")
f.close()
f = open('myfile.txt', 'w')
f.write("SECOND\n")
f.close()
with open('myfile.txt', 'r') as f:
    print(f.read(), end="")
```

OUTPUT

```
SECOND
```

`FIRST` is gone. To keep old lines and add a new one, use `'a'`:

```
f = open('notes.txt', 'w')
f.write("Line 1\n")
f.close()
f = open('notes.txt', 'a')
f.write("Line 2\n")
f.close()
with open('notes.txt', 'r') as f:
    print(f.read(), end="")
```

OUTPUT

```
Line 1
Line 2
```

REMEMBER IT

r = Read existing, w = Wipe then write, a = Add at end. Remember: `write` does not add a newline unless you type `\n`.

COMMON MISTAKE

Forgetting `.close()` can leave data in a memory buffer so the file on disk looks empty or incomplete — or the file can stay locked. Section 3.6 shows `with`, which closes for you even after an error. Never use `'w'` when you only meant to add a line; that is the most common way students wipe their own data.

اردو میں سمجھیں

فائل ہینڈلنگ سے ڈیٹا ڈسک پر مستقل (Persistent) رہتا ہے، برخلاف RAM کے متغیرات کے جو پروگرام بند ہوتے ہی مٹ جاتے ہیں۔ `open(path, mode)` سے فائل کھلتی ہے: `'r'` پڑھنے کے لیے (فائل ہونی چاہیے)، `'w'` لکھنے کے لیے (پرانہ مواد مٹ جاتا ہے)، `'a'` آخر میں جوڑنے کے لیے۔ کام کے بعد `.close()` ضروری ہے اگر `with` استعمال نہ ہو۔

EXAM TIP

Define file handling, contrast RAM with disk, list modes `r / w / a` with one property each, and warn that `w` overwrites. Short programs that write one line and close the file are common. Mention `.read()`, `.readlines()`, `.write()` and `.close()` by name.

CHECK YOURSELF

1. Which mode overwrites an existing file?
2. Which mode fails if the file is missing?
3. Why must `.write` often include `\n`?

Answers: 1. `'w'`. 2. `'r'`. 3. Because `write()` does not start a new line by itself; without `\n` the next write continues on the same line.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. `open(path, "r")` is for:

- a. Reading
- b. Wiping the file
- c. Appending only
- d. Creating a DB

MCQ 2. Mode "w":

- a. Write (truncates existing)
- b. Read only
- c. Append
- d. Binary XOR

MCQ 3. Mode "a":

- a. Append at the end
- b. Overwrite from start
- c. Read
- d. Delete the file

MCQ 4. You should:

- a. `close()` (or use `with`)
- b. Leave files locked
- c. Never flush
- d. Always use `w` for logs

MCQ 5. `read()` returns:

- a. The whole content as `str` (text mode)
- b. A dict always
- c. A pandas DataFrame
- d. A stack

MCQ 6. This is ★ because practicals ask you to:

- a. Write a small file program
- b. Draw only ER
- c. Only HCI
- d. Only OSI

Extra short questions

1. The three text modes and a danger of `w`.
2. `write` vs `writelines`.
3. Why `close`?
4. A `FileNotFoundError` happens when:

Extra long question

1. Write a program that appends a line to `marks.txt` and then reads all lines. Show the file after two runs. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. r read, w write/overwrite, a append. w erases previous content.
2. write a string; writelines a sequence of strings.
3. Flush + unlock. Crash before close can lose data.
4. r on a path that does not exist.

Long

1. open a; write; close; open r; print. Second run has two lines. Warn that w would have kept only the last.

3.6 with Statement (Context Manager) ★ Golden

LEARN IT

It is easy to forget `.close()`. If the program crashes before `.close()` runs, the file may be incomplete or corrupted. Python's `with` statement is a **context manager**: it opens the file, runs the indented block, and **always closes** the file when the block ends — even if an error occurs inside.

with open(...) as f → use f → block ends → file closed automatically

```
with open('myfile.txt', 'w') as new_file:
    new_file.write("Context managers are safe!\n")
# no .close() here – with already closed the file
```

After the indented block, `new_file` is closed. You do not write `new_file.close()`.

3.7 Activity: reading file content

`.read()` pulls the whole file into RAM. For a huge file that can waste memory. The professional pattern is to **iterate line by line**.

```
with open('myfile.txt', 'w') as new_file:
    new_file.write("Context managers are safe!\n")

with open('myfile.txt', 'r') as read_file:
    for line in read_file:
        print(line, end="")
print("\n\nFile read successfully.")
```

OUTPUT

Context managers are safe!

File read successfully.

`end=""` stops `print` from adding a second newline: each line from the file already ends with `\n`.

WORKED EXAMPLE

Three ways to read the same small file.

```
with open('sample.txt', 'w') as f:
    f.write("Ali 80\nSara 90\nAyesha 75\n")

with open('sample.txt', 'r') as f:
    print("≡ .read() ≡")
    print(repr(f.read()))

with open('sample.txt', 'r') as f:
    print("≡ .readlines() ≡")
    print(f.readlines())

with open('sample.txt', 'r') as f:
    print("≡ line by line ≡")
    for line in f:
        print(line, end="")
```

OUTPUT

```
≡ .read() ≡
'Ali 80\nSara 90\nAyesha 75\n'
≡ .readlines() ≡
['Ali 80\n', 'Sara 90\n', 'Ayesha 75\n']
≡ line by line ≡
Ali 80
Sara 90
Ayesha 75
```

`repr` shows the hidden `\n` characters. `.readlines()` is handy when you need a list (the grade tracker uses this). Line-by-line looping is best for large files because only one line sits in memory at a time.

REMEMBER IT

with = “take, use, put back.” Prefer `with open(...) as f:` in every new program. You still choose the mode: `'r'`, `'w'` or `'a'`.

COMMON MISTAKE

You cannot read from `f` after the `with` block — the file is already closed. Do all reading and writing inside the block. Also, `print(line)` without `end=""` looks double-spaced because both the line and `print` add a newline.

with بیان ایک کانٹیکسٹ مینیجر (Context Manager) ہے جو فائل کو بلاک کے اختتام پر خود بند کر دیتا ہے، چاہے اندر غلطی ہو جائے۔ بڑی فائلیں ایک ایک سطر کے لوپ سے پڑھنا بہتر ہے بجائے پورے مواد کو .read() سے میموری میں بھرنے کے۔ print(line, end="") دگنی خالی سطر روکتا ہے۔

EXAM TIP

Define the `with` statement as a context manager that guarantees close. Contrast it with manual `open / close`. Be ready to write a four-line program that writes with `with` and a four-line program that reads line by line. Mention memory: iterating is safer for large files than `.read()`.

CHECK YOURSELF

1. What does a context manager guarantee for a file?
2. Why pass `end=""` when printing each line from a file?

Answers: 1. That the file is closed when the with block ends, even if an error occurs. 2. Because each line already ends with `\n`; print would otherwise add another newline and the output would look double-spaced.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. with open(...) as f:

- a. Closes the file automatically, even after errors
- b. Never closes
- c. Deletes the path
- d. Converts to pandas

MCQ 2. The with-block is a:

- a. Context manager
- b. Thread
- c. Socket always
- d. K-map

MCQ 3. If an exception happens inside with:

- a. The file still closes
- b. The file stays locked forever
- c. Python uninstalls
- d. The disk formats

MCQ 4. You still choose the mode:

- a. r/w/a as before
- b. with replaces modes
- c. Modes are only for SQL
- d. Modes are HCI

MCQ 5. This is ★ because it is the:

- a. Safe modern pattern for files
- b. Old optional style
- c. Only for pandas
- d. Only for Figma

MCQ 6. f is only valid:

- a. Inside the with block
- b. After the block always
- c. Before open
- d. In another file automatically

Extra short questions

1. Write the canonical read pattern.
2. Why it beats try/finally for beginners.
3. Can you nest with for two files?
4. What if the path is wrong in with-open r?

Extra long question

1. Rewrite a grade-tracker snippet from open/close to with, and show what happens if int() fails on a line. Why is with safer? ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. with open("a.txt") as f: data=f.read()
2. Less boilerplate; close is guaranteed.
3. Yes — with open(a) as f, open(b) as g:
4. Still FileNotFoundError — with does not create the file in r.

Long

1. Exception in parse still closes. Compare to forgotten close on the error path. Sample file 3 lines, one bad.

3.8 Errors and Exceptions (Try-Except)

LEARN IT

An **exception** is an error raised while the program is running — for example opening a file that is not there. Without handling, Python prints a traceback and **stops**. A `try / except` block catches the error and lets you show a clear message instead of crashing.

Error type	Meaning
<code>FileNotFoundError</code>	The path does not exist.
<code>PermissionError</code>	The operating system will not let you read or write that file.
<code>IOError</code>	A general input/output problem (disk full, disk disconnected).

Put risky code in `try`. Put the recovery in `except ErrorType`. A final `except Exception as e` can catch anything else, using `e` as the error object.

```
try:
    with open('missing_data.txt', 'r') as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

OUTPUT

```
Error: The file does not exist. Please check the name.
```

Without `try`, the same `open` raises:

OUTPUT

```
Traceback (most recent call last):
  File "missing.py", line 1, in <module>
    open('missing_data.txt', 'r')
FileNotFoundError: [Errno 2] No such file or directory: 'missing_data.txt'
```

WORKED EXAMPLE

When the file exists, the except blocks are skipped.

```
with open('ok.txt', 'w') as f:
    f.write("Present\n")

try:
    with open('ok.txt', 'r') as f:
        content = f.read()
        print(content, end="")
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
```

OUTPUT

Present

Professional software does not dump a red traceback on a user. If a file is missing or a disk fails, the user should see a calm sentence such as "Please check the file name" or "Please try again later."

REMEMBER IT

try = attempt; except = backup plan. Catch the **specific** error first (`FileNotFoundError`), then a general `Exception` if you need it. Order matters: a bare `except Exception` first would swallow the specific error.

COMMON MISTAKE

Do not wrap your entire program in one silent `except:` that ignores every mistake. You would hide real bugs. Catch the errors you expect, and keep the message honest. `'w'` and `'a'` create a missing file, so they do not raise `FileNotFoundError`; `'r'` does.

اردو میں سمجھیں

ایکسپشن (Exception) چلتے پروگرام کی غلطی ہے، جیسے غیر موجود فائل کھولنا۔ `try` میں خطرناک کوڈ لکھیں اور `except FileNotFoundError` میں صاف پیغام دکھائیں تاکہ پروگرام کریش نہ ہو۔ پیشہ ورانہ سافٹ ویئر صارف کو سرخ ٹریس بیک کی بجائے سمجھ آنے والا جملہ دکھاتا ہے۔

CHECK YOURSELF

1. Which exception is raised when `open('missing_data.txt', 'r')` cannot find the file?
2. Does the sample program crash, or print a message, when the file is missing?

Answers: 1. `FileNotFoundError`. 2. It prints "Error: The file does not exist. Please check the name." and continues.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. try/except is for:

- a. Handling runtime errors without crashing the whole program
- b. Fixing syntax errors (missing colons)
- c. Faster Big O
- d. Drawing pie charts

MCQ 2. FileNotFoundError is raised when:

- a. open r on a missing path
- b. 1/0
- c. int("a")
- d. A logic bug that prints 3 instead of 4

MCQ 3. except ValueError catches:

- a. int("hi")
- b. Missing files
- c. SyntaxError of a colon
- d. Logic silent bugs

MCQ 4. except Exception is:

- a. Broad — use with care
- b. The way to hide all bugs always
- c. A sort
- d. A key

MCQ 5. else on try runs when:

- a. No exception occurred
- b. Always
- c. Only on errors
- d. On syntax errors

MCQ 6. finally runs:

- a. Always (cleanup)
- b. Only on success
- c. Only on error
- d. Never with with

Extra short questions

1. Why exceptions in file programs?
2. Show a try around int(input).
3. Does try fix logic errors?
4. Order of except clauses.

Extra long question

1. Write a reader that tries to open marks.txt, handles missing file, skips bad lines with ValueError, and prints a count of good rows.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Missing files and bad lines are expected; crash is a poor UX.
2. try: `n=int(input())` except `ValueError`: `print("Enter a number")`.
3. No — wrong formula still wrong.
4. Specific first, broader later.

Long

1. Full code. Sample: file missing message; file with 80, x, 90 → 2 good. Mention not using bare except.

3.9 Mini File Handling Project: Grade Tracker

LEARN IT

This project uses 'w' to create a grades file, 'a' to add a student later, and 'r' to read every line and compute an average. It combines file handling, string methods and arithmetic.

Write names and marks ('w') → Append a new student ('a') → Read lines ('r') → Split, total, average

```
with open("grades.txt", "w") as f:
    f.write("Alice: 88\n")
    f.write("Bob: 75\n")
    f.write("Carol: 92\n")
print("Grades file created.")

with open("grades.txt", "a") as f:
    f.write("David: 81\n")
print("New student appended.")

with open("grades.txt", "r") as f:
    lines = f.readlines()
    total = 0
    for line in lines:
        name, grade = line.strip().split(":")
        print(f"{name} scored {grade}")
        total += int(grade)
    average = total / len(lines)
    print(f"\nClass average: {average:.1f}")
```

WORKED EXAMPLE

What happens in memory and on disk.

1. 'w' creates `grades.txt` (or wipes an old one) with three lines.
2. 'a' adds `David: 81\n` at the end. Alice, Bob and Carol stay.
3. `.readlines()` returns
`['Alice: 88\n', 'Bob: 75\n', 'Carol: 92\n', 'David: 81\n']`.
4. `.strip()` removes the trailing `\n`. `.split(":")` turns `"Alice: 88"` into `["Alice", " 88"]`.
5. `int(grade)` accepts the extra space: `int(" 88")` is 88.
6. `Total = 88 + 75 + 92 + 81 = 336`. `Average = 336 / 4 = 84.0`. The format `:.1f` prints one decimal place.

OUTPUT

```
Grades file created.
New student appended.
Alice scored  88
Bob scored   75
Carol scored  92
David scored  81

Class average: 84.0
```

The extra space before each mark in the output comes from the space after the colon in the file (`"Alice: 88"`). `int()` ignores it. To tidy the printout you could use `grade.strip()` as well; the average would not change.

REMEMBER IT

Same file, three modes, three `with` blocks. Never open the same file in `'w'` when you only meant to add David — that would delete Alice, Bob and Carol.

COMMON MISTAKE

If a line has no colon, `split(":")` will not produce two parts and the unpacking `name, grade = ...` will raise `ValueError`. Keep the `Name: mark` format. An empty file would make `len(lines)` zero and then `total / 0` would fail — a real system should check that, just as `divide` checked for zero.

اردو میں سمجھیں

گریڈ ٹریکر پہلے 'w' سے فائل بناتا ہے، پھر 'a' سے نیا طالب علم جوڑتا ہے، پھر 'r' سے تمام سطریں پڑھ کر اوسط نکالتا ہے۔ ہر سطر کو `strip` اور `split(":")` سے نام اور نمبر میں توڑا جاتا ہے۔ اوسط = کل نمبر ÷ طلبہ کی تعداد۔ 'w' دوبارہ استعمال کرنے سے پرانے ریکارڈ مٹ جائیں گے۔

CHECK YOURSELF

1. What is the class average in the sample, and how is it calculated?
2. Why is David written with mode "a" rather than "w" ?

Answers: 1. 84.0 because $(88+75+92+81)/4 = 336/4 = 84.0$. 2. Because "a" appends; "w" would erase the three existing students.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. `open(path, "r")` is for:

- a. Reading
- b. Wiping the file
- c. Appending only
- d. Creating a DB

MCQ 2. Mode "w":

- a. Write (truncates existing)
- b. Read only
- c. Append
- d. Binary XOR

MCQ 3. Mode "a":

- a. Append at the end
- b. Overwrite from start
- c. Read
- d. Delete the file

MCQ 4. You should:

- a. `close()` (or use `with`)
- b. Leave files locked
- c. Never flush
- d. Always use `w` for logs

MCQ 5. `read()` returns:

- a. The whole content as `str` (text mode)
- b. A dict always
- c. A pandas DataFrame
- d. A stack

MCQ 6. This is ★ because practicals ask you to:

- a. Write a small file program
- b. Draw only ER
- c. Only HCI
- d. Only OSI

Extra short questions

1. The three text modes and a danger of `w`.
2. `write` vs `writelines`.
3. Why `close`?
4. A `FileNotFoundError` happens when:

Extra long question

1. Write a program that appends a line to `marks.txt` and then reads all lines. Show the file after two runs. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. r read, w write/overwrite, a append. w erases previous content.
2. write a string; writelines a sequence of strings.
3. Flush + unlock. Crash before close can lose data.
4. r on a path that does not exist.

Long

1. open a; write; close; open r; print. Second run has two lines. Warn that w would have kept only the last.

Programming Fundamentals

17 single-page lessons · 6 Golden · read, close, say aloud, then sit the exam

Chapter in one look

- Python programs rest on primitive types (`int`, `float`, `bool`) and abstract structures: lists, tuples, sets and dictionaries.
- A **list** is ordered and mutable, written with `[]`. Index from 0; `[-1]` is last. Slice stop is exclusive. Methods include `append`, `insert`, `remove`, `sort`, `reverse`, `clear`.
- A **tuple** is ordered and immutable, written with `()`. Item assignment raises `TypeError`. Only `count` and `index` exist as methods.
- A **set** is unordered and unique, written with `{}`. Use `|` & `-` & `^` for union, intersection, difference and symmetric difference. Prefer `discard` over `remove`.
- A **dictionary** maps unique keys to values. `d[key]` can raise `KeyError`; `d.get(key, default)` is safe. Duplicate keys keep the last value.
- `min`, `max`, `len`, `sum` and `sorted` work across structures. On a dict, `min / max / sorted` use keys; total values with `sum(d.values())`. `sorted()` returns a new list; `.sort()` changes a list in place.
- Functions written with `def` give reusability, modularity, readability and maintainability. Built-in functions already exist; user-defined functions solve your tasks.
- `return` sends a value to the caller. Without `return`, the result is `None`. Local variables live inside a function; global variables live outside all functions.
- Files persist on disk. Modes: `r` read (must exist), `w` write (overwrites), `a` append. Methods: `read`, `readlines`, `write`, `close`.
- The `with` statement is a context manager that closes the file automatically. Loop line by line to read large files safely.
- `try / except FileNotFoundError` stops a missing file from crashing the program.
- The grade tracker writes with `w`, appends with `a`, reads with `r`, splits each `Name: mark` line, and computes the mean.

Key terms

Term	Meaning	اردو
Data structure	A way of organising values so a program can store and use them.	ڈیٹا سٹرکچر
Primitive type	A basic single value: <code>int</code> , <code>float</code> , <code>bool</code> .	بنیادی قسم
List	Ordered, mutable sequence written with <code>[]</code> .	فہرست
Index	The position of an item, starting at 0 (or -1 from the end).	انڈیکس
Slicing	Copying a portion of a sequence with <code>start:stop:step</code> .	سلائسنگ
Traversal	Visiting each element in turn, usually with a loop.	گشت / تراورسل

Mutable	Can be changed after creation.	قابل تبدیلی
Immutable	Cannot be changed after creation.	ناقابل تبدیلی
Tuple	Ordered, immutable sequence written with <code>()</code> .	ٹپل
Set	Unordered collection of unique items written with <code>{}</code> .	سیٹ
Union	All unique items from two sets <code>()</code> .	اجتماع
Intersection	Items common to both sets <code>(&)</code> .	اشتراک

CHAPTER 3 EXAM • CLOSED BOOK

Programming Fundamentals

75 marks • 1 hour 15 minutes • no notes • no phones

CHAPTER EXAM RULES

Instructions. Sit this exam after Lesson 17 and the recap. **Section A** — all 15 MCQs (1 mark). **Section B** — all 10 short questions (3 marks). **Section C** — attempt any **three** long questions (10 marks). At least one Section C answer must be a ★ Golden topic. Write in your notebook. Do not open the key until time is up.

Marks. 15 + 30 + 30 = 75. A 5-mark / 10-mark answer needs Define • Explain • Example • Diagram • Working.

سبق اور خلاصے کے بعد یہ امتحان بغیر نوٹس کے حل کریں۔ وقت پورا ہونے پر کلید کھولیں۔

Section A — Multiple choice (15 × 1 = 15)

- Which structure is ordered and mutable?
 - List
 - Tuple
 - Set
 - Frozen set
- What does `nums[2:6]` return if `nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`?
 - `[2, 3, 4, 5, 6]`
 - `[2, 3, 4, 5, 6, 7]`
 - `[2, 3, 4, 5]`
 - `[3, 4, 5, 6]`
- `t = (1, 2, 3)` then `t[0] = 99` raises
 - `KeyError`
 - `TypeError`
 - `ValueError`
 - `IndexError`

4. A set is best described as
- a. ordered with duplicates
 - b. indexed and mutable
 - c. key-value pairs
 - d. unordered with unique elements
5. If `A = {1, 2, 3, 4}` and `B = {3, 4, 5, 6}`, then `A & B` is
- a. `{3, 4}`
 - b. `{1, 2}`
 - c. `{1, 2, 3, 4, 5, 6}`
 - d. `{1, 2, 5, 6}`
6. `{"a": 1, "a": 2, "a": 3}` prints
- a. `{'a': 1}`
 - b. `{'a': 1, 'a': 2, 'a': 3}`
 - c. `{'a': 3}`
 - d. a `KeyError`
7. The safe way to read a possibly missing dictionary key is
- a. `d[key]`
 - b. `d.get(key)`
 - c. `d.keys(key)`
 - d. `d.index(key)`
8. For `s = {5, 1, 9, 3, 7}`, `sum(s)` is
- a. 5
 - b. 9
 - c. 7
 - d. 25
9. Which statement is true?
- a. `sorted(x)` returns a new list and leaves `x` unchanged
 - b. `sorted(x)` works only on lists
 - c. `.sort()` works on tuples
 - d. `.sort()` returns a new list
10. A function is created with the keyword
- a. `func`
 - b. `def`
 - c. `define`
 - d. `return`
11. A function that has no `return` statement returns
- a. `0`
 - b. `False`
 - c. `None`
 - d. an empty string
12. A variable defined inside a function is
- a. local
 - b. global
 - c. persistent
 - d. public
13. Opening a file with mode `'w'`
- a. fails if the file already exists
 - b. adds new text at the end
 - c. is read-only
 - d. erases existing content (and creates the file if needed)

4. The `with open(...) as f:` statement is used mainly because
- a. it makes the file binary
 - b. it closes the file automatically
 - c. it is faster than lists
 - d. it prevents `KeyError`
5. Reading a missing file in mode `'r'` without `try` raises
- a. `TypeError`
 - b. `KeyError`
 - c. `FileNotFoundError`
 - d. `ValueError`

Section B — Short questions (10 × 3 = 30)

1. Define a data structure. Distinguish primitive from abstract data structures with one example of each.
2. What is negative indexing? Give the expression that returns the last item of a list named `fruits`.
3. State four differences between a list and a tuple.
4. Differentiate `.remove()` and `.discard()` on a set.
5. Why can a dictionary not have two identical keys? What happens if you write the same key twice?
6. Contrast `sorted()` with list `.sort()`.
7. List four benefits of using functions.
8. Define local scope and global scope.
9. Write the meaning of file modes `'r'`, `'w'` and `'a'`.
10. What is a context manager? Why is `with` safer than calling `.close()` yourself?

Section C — Long questions (attempt any 3 × 10 = 30)

1. Compare list, tuple, set and dictionary in a table (syntax, order, mutability, duplicates, how you access an item, one typical use). Then write two lines of Python that create one example of each holding the numbers 1, 2 and 2, and state what each structure actually stores.
2. Write a complete Python program that takes a sentence, counts the frequency of each word using a dictionary and `.get()`, prints a two-column table, and prints the most frequent word. Trace your program on `"python is easy python is fun python is powerful"`.
3. Write an arithmetic calculator with four user-defined functions `add`, `subtract`, `multiply` and `divide`. `divide` must not crash when the second number is 0. Explain how `return` is used and why `input` values are converted with `float`.
4. Explain file handling. Describe modes `r`, `w` and `a`, the `with` statement, and `try/except` `FileNotFoundError`. Include a short program that writes one line with `with` and another that reads a file that might be missing.

5. The grade tracker writes three students with `'w'`, appends David with `'a'`, then computes the class average. Write the full program, show the exact output for marks 88, 75, 92 and 81, and explain `strip`, `split(":")` and `average:.1f`.

Answer key. Finish the Chapter Exam first, then mark. Do not peek while the clock is running.

پہلے امتحان مکمل کریں، پھر اس کلید سے نمبر لگائیں۔

Check yourself — answers

- 3.1 Data Structures in Python.** Answers: 1. Primitive: any two of int, float, bool. Abstract: any two of list, tuple, set, dictionary. 2. Because it is a higher-level container built from primitive values, holding many items and supporting operations such as append and slice.
- 3.1.1 Lists (Ordered, Mutable, Methods).** Answers: 1. `[2, 3, 4, 5]`. 2. `['banana', 'apple']` (only the first apple is removed). 3. To stop scanning once the value is found, so the rest of a long list is not checked.
- 3.1.2 Tuples (Ordered, Immutable).** Answers: 1. `count` and `index`. 2. `TypeError: 'tuple' object does not support item assignment`. 3. `([1, 2, 99], [3, 4])`.
- 3.1.3 Sets (Unordered, Unique, Math Operations).** Answers: 1. `{1, 2, 3, 4}` (duplicates removed; order may vary). 2. `discard` does not raise `KeyError` if the value is missing. 3. `{1, 2, 5, 6}`.
- 3.1.4 Dictionaries (Key-Value pairs).** Answers: 1. `{'a': 3}`. 2. `0`. 3. A set.
- 3.1.5 Common Built-in Functions (min, max, len, sum).** Answers: 1. 5, 1, 9, 25. 2. A list: `[1, 3, 5, 7, 9]`. 3. `sum(d.values()) → 240`.
- 3.1.6 Activity: Students Attendance Program.** Answers: 1. Present 0, Absent 3. 2. So that YES, Yes and yes all match the string "yes".
- 3.1.7 Activity: Word Frequency Counter.** Answers: 1. Three times. 2. Because a brand-new word is not yet a key; `.get` returns 0 instead of raising `KeyError`.
- 3.2 Functions in Python.** Answers: 1. `def`. 2. Twice, because `greet()` is called twice.
- 3.2.1 Types of Functions (Built-in vs User-defined).** Answers: 1. Built-in. 2. `def`.
- 3.2.2 Functions with Return Values.** Answers: 1. 15. 2. `None`.
- 3.2.3 Activity: Arithmetic Calculator Using Functions.** Answers: 1. The string "Cannot divide by zero". 2. Because `input()` returns text; `float()` turns it into a number so `+` `-` `*` `/` do arithmetic, including decimals.
- 3.2.4 Scope of Local and Global Variables.** Answers: 1. Inside a function. 2. 10 (the global is unchanged).
- 3.3 & 3.4 File Handling and Operations (Modes r, w, a).** Answers: 1. `'w'`. 2. `'r'`. 3. Because `write()` does not start a new line by itself; without `\n` the next write continues on the same line.
- 3.6 with Statement (Context Manager).** Answers: 1. That the file is closed when the `with` block ends, even if an error occurs. 2. Because each line already ends with `\n`; `print` would otherwise add another newline and the output would look double-spaced.
- 3.8 Errors and Exceptions (Try-Except).** Answers: 1. `FileNotFoundError`. 2. It prints "Error: The file does not exist. Please check the name." and continues.
- 3.9 Mini File Handling Project: Grade Tracker.** Answers: 1. 84.0 because $(88+75+92+81)/4 = 336/4 = 84.0$. 2. Because "a" appends; "w" would erase the three existing students.

Answer key

Multiple choice

- | | | | | |
|------|------|------|-------|-------|
| 1. a | 4. d | 7. b | 10. b | 13. d |
| 2. c | 5. a | 8. d | 11. c | 14. b |
| 3. b | 6. c | 9. a | 12. a | 15. c |

Short questions — model answers

1. A data structure organises values in a program. Primitive types hold one value (e.g. `int` 17). Abstract structures hold collections and richer operations (e.g. a list of marks).
2. Negative indexing counts from the right. `fruits[-1]` is the last item. `fruits[-2]` is the second last.
3. List: `[]`, mutable, many methods, slightly slower, use when data will change. Tuple: `()`, immutable, only `count` / `index`, slightly faster, use for protected records. Both are ordered and allow duplicates and indexing.
4. Both delete an element. `remove` raises `KeyError` if the value is absent. `discard` does nothing in that case, so it is the safer choice.
5. Each key must identify one value (like one roll number per student). A repeated key does not crash; the last assigned value overwrites the earlier one, e.g. `{"a": 3}`.
6. `sorted(x)` is a built-in function for any iterable; it returns a new list and does not change `x`. `.sort()` is a list method that sorts in place and returns `None`.
7. Reusability (write once, call many times), modularity (small named tasks), readability (shorter files), maintainability (fix one definition).
8. Local: defined inside a function; invisible outside; gone when the function ends. Global: defined outside all functions; readable throughout the program.
9. `'r'` read, error if missing. `'w'` write, create if missing, erase old content. `'a'` append at the end, create if missing, keep old content.
10. A context manager sets up a resource and cleans it up. `with open(...)` as `f` closes the file when the block ends, even after an exception, so you cannot forget `.close()` and you are safer if the program crashes mid-write.

Long questions — model answers

1. Use the comparison table from 3.1.4 (brackets, ordered/mutable/duplicates/access/empty/use). Examples:

```
print([1, 2, 2])
print((1, 2, 2))
print({1, 2, 2})
print({"a": 1, "b": 2, "b": 2})
```

OUTPUT

```
[1, 2, 2]
(1, 2, 2)
{1, 2}
{'a': 1, 'b': 2}
```

The list and tuple keep both 2s and an order. The set drops the extra 2. The dictionary keeps unique keys; the second "b" overwrites the first.

2. Use the program in 3.1.7. Trace: nine tokens; python and is end at 3; others at 1. Output matches the worked example, including Most Frequent: 'python' (3 times) because max returns the first key among ties. Heart of the loop: `word_freq[word] = word_freq.get(word, 0) + 1`.
3. Use the four functions from 3.2.3. `return` sends each result to `print` in the main program. `divide` returns the string "Cannot divide by zero" when `b == 0`. `float(input(...))` is required because `input` returns a string and `"12" + "4"` would concatenate. Sample 12 and 4: 16.0, 8.0, 48.0, 3.0.
4. File handling stores data on disk so it outlives the program. Modes: `r` read / must exist; `w` wipe-or-create; `a` add-or-create. `with` closes automatically. Try/except:

```
with open("note.txt", "w") as f:
    f.write("Saved\n")
try:
    with open("note.txt", "r") as f:
        print(f.read(), end="")
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
```

OUTPUT

```
Saved
```

If the name were wrong, the except message would print instead of a traceback.

5. Full program as in 3.9. Disk after append:

OUTPUT

```
Alice: 88
Bob: 75
Carol: 92
David: 81
```

Output of the program (exact):

OUTPUT

```
Grades file created.  
New student appended.  
Alice scored 88  
Bob scored 75  
Carol scored 92  
David scored 81
```

```
Class average: 84.0
```

`strip` removes `\n`; `split(":")` separates name and mark; `int(grade)` allows the extra space; `{average:.1f}` prints one decimal. $336 \div 4 = 84.0$.

Self-assessment: I can...

- ☐ Choose list, tuple, set or dictionary for a given problem and justify the choice.
- ☐ Explain tuple immutability and the inner-list exception, and use `count / index`.
- ☐ Update a dictionary, access values with `.get()`, and loop with `.items()`.
- ☐ Write a user-defined function with parameters and a `return` value, and contrast it with a built-in.
- ☐ Open a text file in `r`, `w` or `a` using `with`, and read it line by line.
- ☐ Index, slice, traverse and update a list, including `[-1]` and the main methods.
- ☐ Build a set, drop duplicates, and compute union, intersection, difference and symmetric difference.
- ☐ Apply `min`, `max`, `len`, `sum` and `sorted` correctly, including on dictionaries.
- ☐ State whether a name is local or global and predict a `NameError`.
- ☐ Catch `FileNotFoundError` and complete the grade-tracker project with the correct average.

Lecture 04

Data and Analysis

Chapter lecture — 8 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

BIEK Computer Science XII · Bilingual support: English + اردو

بی آئی ای کے کمپیوٹر سائنس — بارہویں جماعت

08 book lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XII

How to use this BIEK study-guide lecture

Each PDF follows one textbook lecture from the BIEK / Sindh Computer Science notes (New Sindh Curriculum). Read *Learn it*, study the SVG diagram, try the worked example on paper, then answer *Check yourself* before the answers. **Extra board practice** (MCQs, short and long questions) is added after the book content — it is not in the textbook. Career-only extras that were not in the book lecture list have been removed. Math is rendered with MathJax. ★ Golden topics are high-yield in board exams. The green box repeats the idea in Urdu.

ہر لیکچر کتاب کے درس کے مطابق ہے۔ کتاب کے بعد اضافی MCQs اور سوالات ہیں۔ جو اضافی کیریئر صفحات کتاب کی فہرست میں نہیں تھے وہ ہٹا دیے گئے ہیں۔

Lectures in this pack

Chapter 4: Data and Analysis. Study one lecture at a time, then use the chapter review and answer key at the end of this PDF.

L01. 4.1 Concept of Data Analysis

L02. 4.1.1 – 4.1.2 Data Sources and Database Connection

L03. 4.2 Creating a SQLite Database in Python

L04. 4.3 Basic Operations on Data using Pandas

L05. 4.3.3 Handling Missing Values (NaN)

L06. 4.4 Data Organization and Representation

L07. 4.5 Data Visualization & Types of Graphs

L08. 4.6 Descriptive Statistics

Data and Analysis

This chapter shows you how to pull data from a SQLite database with Python, clean it in Pandas, draw the right chart, and summarise a dataset with mean, median, mode and standard deviation. These are the same steps a data analyst uses every day.

اس باب میں آپ سیکھیں گے کہ پائتھن سے ایس کیو لائٹ ڈیٹا بیس کیسے جوڑا جاتا ہے، پینڈاز میں ڈیٹا کیسے صاف ہوتا ہے، درست چارٹ کیسے بنتا ہے، اور اوسط، میڈین، موڈ اور معیاری انحراف سے خلاصہ کیسے نکلتا ہے۔

What you will learn

- Define data analysis and say why Python is used for it.
- Create a SQLite database, table, insert, select, commit and close.
- Create calculated columns and handle NaN with isnull, fillna and dropna.
- Choose and code bar, line, pie, histogram, scatter and box plots.
- Name the five parts of a Python database connection in order.
- Build a Pandas DataFrame from a dictionary, CSV or Excel file.
- Organise data in tables and frequency distributions.
- Compute mean, median, mode, range, variance, standard deviation, min and max by hand, and know that pandas `.std()` uses `n-1`.

4.1 Concept of Data Analysis

LEARN IT

Data analysis is examining, organising and interpreting data so you can extract useful information. Schools study results, shops study sales, hospitals study patient records — all to make a decision, spot a trend or solve a problem.

Python is popular for this work because it is readable, talks to files and databases, does statistics, and draws graphs. Five lines of Pandas often replace an hour of spreadsheet clicking.

WORKED EXAMPLE

A Hyderabad college has 400 CS marks in a spreadsheet. Analysis answers: What is the average? How many failed? Is the second shift weaker than the first? The numbers are data; those answers are information.

اردو میں سمجھیں

ڈیٹا کا تجزیہ یعنی اعداد کو ترتیب دے کر ان سے فیصلے کے قابل معلومات نکالنا۔ پائٹھن اس لیے منتخب کیا جاتا ہے کہ وہ فائلیں، ڈیٹا بیس، حساب اور گراف سب سنبھال لیتا ہے۔

CHECK YOURSELF

1. Give one school use of data analysis.
2. Name two Python superpowers for analysis.

Answers: 1. Study results to improve teaching (or similar). 2. Any two: read CSV/Excel/databases, organise tables, statistics, charts.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Data analysis is:

- a. Examining data to extract useful information
- b. Only collecting rumours
- c. Only drawing logos
- d. Only cabling

MCQ 2. Python is used because:

- a. Files, databases, stats, plots in one language
- b. It is analog
- c. It cannot loop
- d. It cannot import

MCQ 3. Raw marks vs "class average 62":

- a. Data vs information
- b. Information vs data
- c. Both keys
- d. Both NAND

MCQ 4. A decision is the:

- a. Point of analysis
- b. First NAND
- c. Last OSI layer always
- d. Sprint name

MCQ 5. Without a question, analysis is:

- a. Aimless charting
- b. Always AI
- c. Always IoT
- d. Always HCI

MCQ 6. XII tools for analysis include:

- a. sqlite3, pandas, matplotlib, statistics
- b. Only LogiSim
- c. Only Figma
- d. Only Access macros

Extra short questions

1. Define data analysis with a college example.
2. Four Python superpowers listed in the notes.
3. What is not analysis?
4. Link to XI digital literacy.

Extra long question

1. A Hyderabad college has 400 CS marks. Write four analysis questions and which Python tool answers each.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Process marks to decide who needs extra class.
2. Read files, talk to DB, statistics, charts (and readable syntax).
3. Dumping 400 rows on a slide with no summary.
4. Same collect-present loop, now with code.

Long

1. Average: pandas mean. Fail count: filter. Shift compare: groupby. Trend: matplotlib line if by year. Mention cleaning NaNs first.

4.1.1 – 4.1.2 Data Sources and Database Connection ★ Golden

LEARN IT

Data sources are where information comes from: school records, websites, sensors, databases. **Connectivity** is the conversation between your Python program and that source so you can read and update data.

Five components of a database connection, in the order they actually work:

#	Component	Role
1	Database	Organised store (SQLite = one .db file; MySQL = a server for big apps)
2	Driver	Translator between Python and that DBMS (sqlite3 is built in)
3	Connection object	Opens, manages and closes the pipeline
4	Cursor object	Runs SQL and walks through result rows
5	SQL queries	CREATE, INSERT, SELECT, UPDATE, DELETE



EXAM TIP

Write the five names in order and one line each. A common trap is swapping cursor and connection: the connection is the pipe; the cursor is the messenger that travels through it.

اردو میں سمجھیں

خمسہ اجزاء: ڈیٹا بیس، ڈرائیور، کنکشن، کرسر، ایس کیو ایل۔ کنکشن پائپ ہے، کرسر اس پائپ میں چل کر حکم چلاتا ہے۔

CHECK YOURSELF

1. Which object executes `SELECT *`?
2. Is SQLite a driver, a database, or both?

Answers: 1. The cursor. 2. SQLite is the database; the `sqlite3` module is the driver.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. The five connection parts in order start with:

- a. Database then driver
- b. Cursor then database
- c. SQL then mouse
- d. Chart then file

MCQ 2. The cursor:

- a. Executes SQL and fetches rows
- b. Is the .db file
- c. Is matplotlib
- d. Is Figma

MCQ 3. The connection object is the:

- a. Pipeline
- b. SQL text
- c. Driver brand only
- d. Pie

MCQ 4. sqlite3 is the:

- a. Built-in driver (+ DB engine in-process)
- b. A web browser
- c. An OSI layer
- d. A sort

MCQ 5. SQL examples:

- a. SELECT, INSERT, CREATE
- b. def, for, if
- c. AND, OR, NOT gates
- d. HTTP GET only

MCQ 6. Swapping cursor and connection in an answer is a:

- a. Classic 0-mark mix-up
- b. Required trick
- c. Big O class
- d. UX heuristic

Extra short questions

1. Name the five components in order.
2. Why a driver?
3. SQLite vs MySQL in one line.
4. Draw the flow.

Extra long question

1. Label a 9-line sqlite snippet with the five components. Explain commit vs close. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Database, driver, connection, cursor, SQL.
2. Translates Python calls to the DBMS protocol/API.
3. SQLite: file, no server. MySQL: server for many users.
4. Python → driver → connection → cursor → SQL → DB.

Long

1. import=driver; connect=connection; cursor=cursor; execute=SQL;
school.db=database. commit saves; close unlocks. Missing commit → empty next run.

4.2 Creating a SQLite Database in Python

LEARN IT

SQLite is built into Python. The whole database is one `.db` file next to your script — no password, no port, no server. Nine steps you must be able to write from memory:

```
import sqlite3
conn = sqlite3.connect("school.db")    # 1-2  driver + connection
cur = conn.cursor()                   # 3    cursor
cur.execute("CREATE TABLE IF NOT EXISTS students (id INTEGER, name TEXT, marks
INTEGER)")
cur.execute("INSERT INTO students VALUES (1, 'Ali', 85)")
conn.commit()                         # 6    SAVE or the row vanishes
cur.execute("SELECT * FROM students")
rows = cur.fetchall()
for r in rows:
    print(r)
conn.close()                          # 9    unlock the file
```

OUTPUT

```
(1, 'Ali', 85)
```

Step	Why it matters
<code>connect</code>	Creates the file if it does not exist
<code>cursor</code>	Vehicle that carries SQL
<code>execute</code>	Runs one SQL command
<code>commit</code>	Writes INSERT/UPDATE/DELETE to disk
<code>fetchall</code>	Brings SELECT results into a Python list of tuples
<code>close</code>	Frees memory and unlocks the file

COMMON MISTAKE

Forgetting `commit()` is the classic bug: the program prints nothing next time because the insert was never saved. Forgetting `close()` can lock `school.db` so the next run fails.

REMEMBER IT

Strings inside SQL use single quotes: `'Ali'`. Python strings around the SQL use double quotes (or a triple-quoted string).

ایس کیو لائٹ پائٹھن کے ساتھ آتی ہے، پورا ڈیٹا بیس ایک فائل ہے۔ ترتیب: امپورٹ، کنکٹ، کرسر،
COMMIT .EXECUTE، COMMIT، SELECT، FETCALL، CLOSE نہ ہو تو اندراج ضائع ہو جاتا ہے۔

CHECK YOURSELF

1. What does * mean in SELECT *?
2. Which call must follow INSERT?

Answers: 1. All columns. 2. conn.commit() .

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. `sqlite3.connect("school.db")` will:

- a. Open or create the file
- b. Always error if missing
- c. Start MySQL server
- d. Draw a pie

MCQ 2. After INSERT you must:

- a. `commit()`
- b. only close
- c. drop the table
- d. `pip install sqlite`

MCQ 3. `fetchall()` returns:

- a. A list of tuples (rows)
- b. A DataFrame always
- c. A dict of dicts always
- d. A string of SQL

MCQ 4. `CREATE TABLE IF NOT EXISTS` avoids:

- a. Error on re-run
- b. The need for types
- c. The need for keys
- d. `commit`

MCQ 5. `close()` is needed to:

- a. Unlock the file
- b. Create pandas
- c. Sort rows
- d. Encrypt OSI

MCQ 6. `SELECT *` means:

- a. All columns
- b. All tables in the folder
- c. A pie of all apps
- d. Drop everything

Extra short questions

1. Nine-step sequence from the notes.
2. What does `*` mean?
3. Two classic bugs.
4. How do you see the data next run?

Extra long question

1. Write a complete program: table `students(id, name, marks)`, insert Ali 85 and Sara 90, print rows. Number the five connection components in comments.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. import, connect, cursor, create, insert, commit, select, fetch, close.
2. Every column.
3. No commit; forgotten close; SQL string quotes mixed up.
4. It is in school.db — connect again and SELECT.

Long

1. Full working code and output two tuples. Comments:
driver/connection/cursor/SQL/database.

4.3 Basic Operations on Data using Pandas ★ Golden

LEARN IT

Pandas is a third-party library for tables. Install once: `pip install pandas` in the terminal (not inside the `.py` file). Convention: `import pandas as pd`.

A **DataFrame** is a 2-D table of rows and columns, like Excel or an SQL table. You can filter, sort and calculate whole columns without writing a `for` loop.

```
import pandas as pd
data = {
    "Name": ["Ali", "Sara", "Ahmed"],
    "Math": [85, 90, 78],
    "Science": [80, 88, 70],
}
df = pd.DataFrame(data)
print(df)
```

OUTPUT

	Name	Math	Science
0	Ali	85	80
1	Sara	90	88
2	Ahmed	78	70

`read_csv('students.csv')` and `read_excel('students.xlsx', sheet_name='Sheet1')` load files. `df.head()` prints the first five rows — use it on large files so you do not flood the screen.

```
df["Total"] = df["Math"] + df["Science"]
df["Average"] = df["Total"] / 2
print(df)
```

OUTPUT

	Name	Math	Science	Total	Average
0	Ali	85	80	165	82.5
1	Sara	90	88	178	89.0
2	Ahmed	78	70	148	74.0

WORKED EXAMPLE

Check: Ali $85+80=165$, $165/2=82.5$. Sara $90+88=178$, 89.0 . Ahmed $78+70=148$, 74.0 .

اردو میں سمجھیں

پینڈا ڈیٹا فریم دیتا ہے۔ ڈکشنری، سی ایس وی یا ایکسل سے لوڈ کریں۔ نئے کالم بغیر لوپ کے پوری قطار پر حساب کر کے بنتے ہیں۔ `head()` صرف پہلی پانچ قطاریں دکھاتا ہے۔

CHECK YOURSELF

1. What is `pd`?
2. Why not `print(df)` on a million-row file?

Answers: 1. The usual alias for pandas. 2. It would print everything; use `df.head()`.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Common DS operations include:

- a. Insert, delete, search, traverse
- b. Only compile
- c. Only ping
- d. Only print the logo

MCQ 2. Traverse means:

- a. Visit elements in some order
- b. Delete all
- c. Encrypt all
- d. Sort the OS

MCQ 3. Search cost depends on:

- a. The structure (array vs sorted vs hash vs tree)
- b. The PDF theme
- c. The teacher name
- d. The sprint

MCQ 4. Insert in a full array may:

- a. Fail or require resizing
- b. Always $O(1)$ no matter what
- c. Create a graph edge automatically
- d. Pop a stack

MCQ 5. Delete from a queue happens at the:

- a. Front
- b. Rear only always
- c. Random index
- d. Root of a tree

MCQ 6. You choose a DS by the:

- a. Operations you will do most
- b. First letter of the course
- c. Colour
- d. NAND count

Extra short questions

1. List five operations.
2. Give costs qualitatively: array index vs list search.
3. What does traverse a tree mean?
4. Why "update" might be search+insert.

Extra long question

1. For array, stack, queue, tree: name the natural insert/delete ends and one costly operation. Table.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Create, insert, delete, search, traverse, update, sort (as applicable).
2. Array index $O(1)$; unsorted list search $O(n)$.
3. Visit every node (pre/in/post or level order).
4. You must find the item first.

Long

1. Array: insert middle costly. Stack: top only cheap. Queue: rear in front out. Tree: search/insert depend on balance. 8-row table.

4.3.3 Handling Missing Values (NaN) ★ Golden

LEARN IT

Missing cells are **NaN** ("Not a Number"). Statistics crash or lie if you ignore them. Three tools: `isnull()` (True where empty), `fillna(value)` (replace), `dropna()` (delete the row).

```
import pandas as pd
df = pd.DataFrame({
    "Name": ["Ali", "Sara", "Ahmed", "Ayesha"],
    "Marks": [85.0, None, 78.0, 92.0],
    "Grade": ["A", "A", None, "A"],
})
print(df.isnull().sum())
```

OUTPUT

```
Name      0
Marks      1
Grade      1
dtype: int64
```

WORKED EXAMPLE

Replace Marks with the mean of the known marks, and Grade with a label. Known marks 85, 78, 92 → mean = $255/3 = 85.0$.

```
avg = df["Marks"].mean()          # 85.0
df["Marks"] = df["Marks"].fillna(avg)
df["Grade"] = df["Grade"].fillna("Not Assigned")
print(df)
```

OUTPUT

	Name	Marks	Grade
0	Ali	85.0	A
1	Sara	85.0	A
2	Ahmed	78.0	Not Assigned
3	Ayesha	92.0	A

`df = df.dropna()` would instead delete Sara's row and Ahmed's row (any row that still had a NaN).

EXAM TIP

State: identify with `isnull / sum`, fill numeric gaps with the mean, fill categories with a label, or drop rows if filling would invent a lie. Never analyse first and clean second.

COMMON MISTAKE

Filling a salary gap with the mean of a company that includes the CEO can hide poverty (the same outlier problem as the mean). Know *why* the value is missing before you fill.

اردو میں سمجھیں

خالی خانے NaN ہیں۔ isnull تلاش، fillna بھرنا (اکثر اوسط سے)، dropna قطار ہٹانا۔ پہلے صاف کرو، پھر تجزیہ۔

CHECK YOURSELF

1. What does `df.isnull().sum()` return?
2. Does `dropna()` delete a column or a row by default?

Answers: 1. Count of NaNs in each column. 2. Any **row** that contains at least one NaN.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. NaN means:

- a. Not a Number — missing
- b. A huge number
- c. Zero always
- d. A pie slice

MCQ 2. `isnull().sum()` gives:

- a. Count of missing per column
- b. The mean
- c. The PK
- d. The chart type

MCQ 3. `fillna(mean)` :

- a. Replaces gaps with the average
- b. Deletes the column names
- c. Sorts
- d. Commits SQL

MCQ 4. `dropna()` by default:

- a. Drops rows that contain any NaN
- b. Drops the whole DataFrame always
- c. Fills with 0
- d. Draws a box plot

MCQ 5. You should clean NaNs:

- a. Before analysing
- b. After publishing
- c. Never
- d. Only in Figma

MCQ 6. Filling salary with the mean can:

- a. Hide poverty if a CEO is in the data
- b. Fix ethics
- c. Create keys
- d. Route packets

Extra short questions

1. Three tools: find, fill, drop.
2. Mean of 85, None, 78, 92 for filling.
3. When drop is better than fill.
4. `isnull()` returns what type of frame?

Extra long question

1. Given Marks [85, None, 78, 92] and Grade [A,A,None,A]: show `isnull` sum, fill marks with mean and grade with "Not Assigned", print the frame. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. `isnull/sum; fillna; dropna`.

2. Known $85+78+92=255/3=85.0$.
3. If inventing a value would lie (unknown category, huge outlier).
4. Booleans True where missing.

Long

1. Counts 1 and 1. Sara marks 85.0, Ahmed grade Not Assigned. Code + table. Warn about dropna deleting two rows if used first.

4.4 Data Organization and Representation

LEARN IT

Organised data is easier to read. Three everyday forms:

- **Table** — exact values side by side (Ali 85 Math, 80 Science).
- **Frequency distribution** — how many items fall in each range (bin).
- **Graph** — a picture of a trend or comparison (next topics).

Range of marks	Frequency (students)
50–59	5
60–69	7
70–79	9
80–89	4
90–100	2

Most students scored 70–79. That pattern is hard to see in a raw list of 27 marks, and obvious in this table. Matplotlib (`import matplotlib.pyplot as plt`) turns tables into graphs: `import` → `figure` → `plot` → `labels` → `title` → `show()`.

اردو میں سمجھیں

ٹیبل درست عدد دکھاتی ہے، تعدد کی جدول بتاتی ہے کتنے طلبہ کس رینج میں ہیں، گراف رجحان دکھاتا ہے۔
میٹپلاٹ لب گراف بناتی ہے۔

CHECK YOURSELF

1. How many students scored 80 or above in the table?
2. Name the library that draws graphs in Python.

Answers: 1. $4 + 2 = 6$. 2. Matplotlib (pyplot).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Tabular form is for:

- a. Exact values in rows/columns
- b. Only a pretty poster
- c. Only NAND
- d. Only sprints

MCQ 2. A frequency table shows:

- a. How many in each bin/range
- b. The ER diagram
- c. The OSI stack
- d. The Figma layers

MCQ 3. Most students in the notes' table scored:

- a. 70–79
- b. 90–100
- c. 0–10
- d. Exactly 100

MCQ 4. Graphical representation is for:

- a. Seeing patterns fast
- b. Storing PKs
- c. Compiling C
- d. Routing

MCQ 5. Matplotlib import line:

- a. `import matplotlib.pyplot as plt`
- b. `import pie`
- c. `import osi`
- d. `import nand`

MCQ 6. `plt.show()` :

- a. Renders the figure
- b. Commits SQL
- c. Closes sqlite
- d. Starts Figma

Extra short questions

1. Three organisation methods in the notes.
2. From the frequency table, how many scored 80+?
3. Steps of a matplotlib figure.
4. Why frequencies for large n?

Extra long question

1. Build a frequency table for 12 marks you invent, then say which graph you would draw and why. Include axis titles.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Table, frequency distribution, graph.
2. $4+2=6$.
3. import, figure, plot, labels, title, show.
4. A list of 2000 marks is unreadable; bins are not.

Long

1. Bins of 10. Most-filled bin noted. Histogram (numeric distribution) or column if you treat bands as categories — say which and why bars touch or not.

4.5 Data Visualization & Types of Graphs ★ Golden

LEARN IT

Pick the graph that matches the question. Then label axes and give a title — an unlabelled chart scores zero in an exam.

Chart	Shows	Watch out
Bar / column	Compare discrete categories (subjects, departments)	Column = vertical; bar often = horizontal
Line	Change over time (temperature, visitors)	Needs an ordered x-axis (months, years)
Pie	Parts of one whole as percentages	Bad with many or almost-equal slices
Histogram	Distribution of one numeric variable in bins	Bars touch (continuous range), unlike a bar chart
Scatter	Relationship of two numeric variables	Each dot is one pair (x, y)
Box plot	Min, Q1, median, Q3, max, and outliers	Best tool for spotting extreme values

4.5.1 Bar / column

```
import matplotlib.pyplot as plt
subjects = ["English", "Math", "Science", "History"]
marks = [64, 82, 71, 49]
plt.bar(subjects, marks, color="skyblue", edgecolor="black")
plt.title("Marks vs Subjects")
plt.xlabel("Subjects")
plt.ylabel("Marks out of 100")
```

4.5.2 Line

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
temps = [12, 15, 20, 25, 30, 35]
plt.plot(months, temps, marker="o")
plt.title("Average Monthly Temperature")
plt.grid(True)
```

4.5.3 Pie

```
subjects = ["Math", "Physics", "CS", "Biology"]
students = [25, 15, 30, 10] # total 80
plt.pie(students, labels=subjects, autopct="%1.1f%%", startangle=90)
```

Check: Math $25/80 = 31.25\% \rightarrow 31.2\%$. Physics 18.8% . CS 37.5% . Biology 12.5% .

4.5.4 Histogram

```
scores = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 90, 95]
plt.hist(scores, bins=[40, 50, 60, 70, 80, 90, 100], edgecolor="black")
```

4.5.5 Scatter

```
plt.scatter([10, 20, 30, 40, 50, 60, 70, 80],
            [2, 4, 5, 4, 7, 8, 9, 12], color="green", alpha=0.7)
plt.title("Rainfall vs Crop Yield") # positive correlation
```

4.5.6 Box plot

```
plt.boxplot([[55, 60, 65, 70, 75, 80, 85],
            [40, 65, 70, 72, 75, 80, 95]],
            tick_labels=["Class A", "Class B"])
```

The box is the middle 50% (Q1 to Q3). The line inside is the median. Whiskers reach min and max (or a fence, with outliers drawn as dots). Class B is more spread out than Class A.



Column compares; line shows a trend; pie shows share; the box summarises spread around the median.

EXAM TIP

"Which graph and why?" is a 3-mark gift. Write the name, what it shows, and one reason the others are worse.

اردو میں سمجھیں

بار/کالم اقسام کا موازنہ، لائن وقت کا رجحان، پائی حصے، ہسٹوگرام مسلسل تقسیم (بار آپس میں لگے)، اسکیٹر دو متغیرات کا تعلق، باکس پلاٹ پانچ اعداد اور آؤٹ لائر۔

CHECK YOURSELF

1. Why do histogram bars touch?
2. Rainfall vs yield — which chart?

Answers: 1. The x-axis is a continuous numeric range, not separate categories. 2. Scatter (two numeric variables, looking for correlation).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Bar/column charts compare:

- a. Discrete categories
- b. Parts of one whole only
- c. A single time point of one var as a distribution
- d. A 5-number summary

MCQ 2. Line graphs are for:

- a. Trends over time
- b. Unrelated categories with gaps
- c. Shares of a budget
- d. Outliers in five numbers

MCQ 3. Pie charts need:

- a. Parts of one whole; few slices
- b. 60 slices of 1%
- c. Time on x
- d. Two numeric axes of correlation

MCQ 4. Histogram bars:

- a. Touch (continuous bins)
- b. Must have large gaps
- c. Are always time
- d. Replace the mean

MCQ 5. Scatter plots show:

- a. Two numeric variables (correlation)
- b. One category vs colour
- c. FIFO order
- d. OSI layers

MCQ 6. A box plot shows:

- a. Min, Q1, median, Q3, max (+ outliers)
- b. Only the mean
- c. Only mode
- d. Only n

Extra short questions

1. One-line job of each of the six charts.
2. Why histogram $\neq$ bar.
3. When pie fails.
4. Rainfall vs yield?

Extra long question

1. Six school questions $\rightarrow$ pick the chart and a matplotlib function (bar, plot, pie, hist, scatter, boxplot). Justify in one sentence each. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Bar categories; line time; pie share; hist distribution; scatter relationship; box spread/outliers.
2. Numeric bins, touching bars vs separate categories.
3. Too many / nearly equal slices.
4. Scatter; look for positive correlation.

Long

1. Enrolment by faculty: bar. Hits by month: plot. Club shares: pie. Heights: hist. Hours vs marks: scatter. Two classes' spread: boxplot. Unlabelled charts score 0.

4.6 Descriptive Statistics ★ Golden

LEARN IT

Descriptive statistics squeeze a long list into a few honest numbers. **Central tendency** asks “what is typical?”. **Spread** asks “how mixed is the list?”.

4.6.1 Mean, median, mode

Measure	How	Use when
Mean	Sum ÷ n. Uses every value.	Values sit close together. Example: ordinary test scores.
Median	Middle after sorting. Odd n: centre. Even n: mean of the two centre values.	Outliers would wreck the mean (CEO salary).
Mode	Most frequent value. May be none, one, or several.	Most popular choice (shoe size).

WORKED EXAMPLE

Marks 70, 80, 90. Mean = $240/3 = 80$.

Odd list 35, 10, 15, 70, 95. Sort: 10, 15, 35, 70, 95. Median = **35**.

Bottles sold 20, 25, 20, 30, 20, 25, 20. Mode = **20** (appears four times).

Outlier trap: nine people earn 10, one earns 1000. Mean = $1090/10 = 109$ (looks rich).

Median = 10 (tells the truth).

4.6.2 Spread: range, variance, standard deviation, min, max

- **Min / max** — smallest and largest. Marks 70, 80, 90, 60, 75 → min 60, max 90.
- **Range** = max – min. Sales 120, 150, 180 → range 60.
- **Population variance** $\sigma^2 = \frac{1}{n} \sum (x_i - \mu)^2$.
- **Sample variance** s^2 uses $\div (n - 1)$. `pandas.Series.std()` uses the sample version.
- **Standard deviation** $\sigma = \sqrt{\sigma^2}$. Same units as the data (marks, not marks²). Low SD → tight, consistent. High SD → spread out, unpredictable.

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2, \quad \sigma = \sqrt{\sigma^2}, \quad s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

WORKED EXAMPLE

Full working on 70, 80, 90. Mean $\mu = 80$. Deviations -10, 0, +10. Squares 100, 0, 100. Sum = 200.

Population variance $\sigma^2 = 200/3 = \mathbf{66.67}$ (2 d.p.). Population SD $\sigma = \sqrt{200/3} = \mathbf{8.16}$ (2 d.p.).

Sample variance $s^2 = 200/2 = \mathbf{100}$. Sample SD $s = 10$. That is what `pd.Series([70,80,90]).std()` returns.

EXAM TIP

Always state whether you used n or $n-1$. Show the sorted list for the median. For SD, show: mean $\rightarrow$ deviations $\rightarrow$ squares $\rightarrow$ divide $\rightarrow$ square root. A box plot pictures min, Q1, median, Q3, max of the same data.

COMMON MISTAKE

Do not report variance as if it were in "marks". Variance is marks^2 . Quote the standard deviation when you want a sentence in the original units.

اردو میں سمجھیں

اوسط سب کا مجموعہ تقسیم تعداد، میڈین ترتیب کے بعد بیچ، موڈ سب سے زیادہ بار آنے والا۔ رینج بڑا منفی چھوٹا۔ ویرینس اوسط سے فاصلوں کے مربع کی اوسط ہے، معیاری انحراف اس کا جذر ہے اور اصل اکائی میں ہوتا ہے۔ پینڈاز std() نمونے والا ($n-1$) استعمال کرتا ہے۔

CHECK YOURSELF

1. Median of 4, 8, 6, 10, 2?
2. Range of 120, 150, 180?
3. Does pandas `.std()` divide by n or $n-1$?

Answers: 1. Sorted 2,4,6,8,10 $\rightarrow$ 6. 2. 60. 3. $n-1$ (sample).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Mean of 70,80,90 is:

- a. 80
- b. 70
- c. 90
- d. 240

MCQ 2. Median of 10,15,35,70,95 is:

- a. 35
- b. 10
- c. 70
- d. 45

MCQ 3. Mode of 20,25,20,30,20,25,20 is:

- a. 20
- b. 25
- c. 30
- d. 7

MCQ 4. Range of 120,150,180 is:

- a. 60
- b. 30
- c. 150
- d. 180

MCQ 5. pandas .std() uses:

- a. $n-1$ (sample)
- b. n always
- c. $2n$
- d. zero

MCQ 6. SD is the square root of:

- a. Variance
- b. Mean
- c. Median
- d. n

Extra short questions

1. When mean vs median vs mode.
2. Population variance formula idea.
3. Why square root for SD?
4. Outlier trap with 9×10 and one 1000.

Extra long question

1. For 4,8,6,10,2 compute mean, median, range, population variance and SD. Then say what pandas std reports. Show every step. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Mean typical no outliers; median with outliers; mode popular category/value.

2. Mean of squared deviations; $\div n$. Sample $\div (n-1)$.
3. Back to original units.
4. Mean 109 vs median 10.

Long

1. Sorted 2,4,6,8,10. Mean 6, median 6, range 8. Devs -4,-2,0,2,4 sq 16,4,0,4,16 sum 40.
 $\sigma^2=8$, $\sigma=\sqrt{8}\approx 2.828$. pandas $40/4=10$, $\sqrt{10}\approx 3.162$.

Data and Analysis

8 single-page lessons · 5 Golden · read, close, say aloud, then sit the exam

Chapter in one look

- Data analysis turns raw facts into decisions. Python reads files and databases, calculates and plots.
- Connection chain: database → driver → connection → cursor → SQL.
- SQLite: connect, cursor, execute, commit, fetchall, close. No commit means no save.
- A DataFrame is a 2-D table. Load with a dictionary, `read_csv` or `read_excel`. New columns are vectorised arithmetic.
- NaN: find (`isnull`), fill (`fillna` , often with the mean), or drop (`dropna`).
- Tables give exact values; frequency bins show how many in each range.
- Bar/column = categories; line = time; pie = parts of a whole; histogram = numeric distribution (touching bars); scatter = two variables; box = five-number summary + outliers.
- Mean uses every value and is sensitive to outliers; median is the middle; mode is the most common.
- Range = max–min. Variance = mean squared deviation ($\div n$ or $\div n-1$). SD = $\sqrt{\text{variance}}$. pandas `.std()` uses $n-1$.

Key terms

Term	Meaning	اردو
Data analysis	Examining data to extract useful information.	ڈیٹا کا تجزیہ
Connectivity	Link between a program and a data source.	رابطہ کاری
Driver	Translator between Python and a DBMS.	ڈرائیور
Connection	Open pipeline to the database.	کنکشن
Cursor	Object that executes SQL and fetches rows.	کرسر
SQLite	Built-in file-based database.	ایس کیو لائٹ
commit	Save INSERT/UPDATE/DELETE to disk.	کمیت / محفوظ کرنا
Pandas	Library for table-like data.	پینڈا
DataFrame	2-D labelled table of rows and columns.	ڈیٹا فریم
NaN	Missing value ("Not a Number").	خالی / غیر موجود قدر
Frequency	How many items fall in a class/bin.	تعدد

Histogram	Distribution chart whose bars touch.	ہسٹوگرام
-----------	--------------------------------------	----------

CHAPTER 4 EXAM · CLOSED BOOK

Data and Analysis

75 marks · 1 hour 15 minutes · no notes · no phones

CHAPTER EXAM RULES

Instructions. Sit this exam after Lesson 08 and the recap. **Section A** — all 15 MCQs (1 mark). **Section B** — all 10 short questions (3 marks). **Section C** — attempt any **three** long questions (10 marks). At least one Section C answer must be a ★ Golden topic. Write in your notebook. Do not open the key until time is up.

Marks. 15 + 30 + 30 = 75. A 5-mark / 10-mark answer needs Define · Explain · Example · Diagram · Working.

سبق اور خلاصے کے بعد یہ امتحان بغیر نوٹس کے حل کریں۔ وقت پورا ہونے پر کلید کھولیں۔

Section A — Multiple choice (15 × 1 = 15)

- The object that executes SQL is the:
 - driver
 - cursor
 - DataFrame
 - histogram
- After INSERT you must:
 - close then commit
 - commit
 - drop the table
 - call head()
- Pandas is installed with:
 - import pandas
 - pip install pandas
 - sqlite3.connect
 - plt.show()
- `df.head()` shows:
 - the last 5 rows
 - the first 5 rows
 - only NaNs
 - the standard deviation
- Missing Pandas values appear as:
 - NULL in every chart
 - NaN
 - 0 always
 - a pie slice
- To delete every row that contains a gap:
 - fillna
 - dropna
 - fetchall
 - commit

7. A line graph is best for:
- a. parts of a whole
 - b. a trend over months
 - c. two unrelated categories
 - d. a primary key
8. Histogram bars:
- a. must have gaps
 - b. touch, because bins are a continuous range
 - c. always show time
 - d. replace the mean
9. Rainfall vs crop yield is a:
- a. pie chart
 - b. scatter plot
 - c. primary key
 - d. mode
10. A box plot's inner line is the:
- a. mean
 - b. mode
 - c. median
 - d. range
11. Mean of 70, 80, 90 is:
- a. 70
 - b. 80
 - c. 90
 - d. 240
12. Median of 10, 15, 35, 70, 95 is:
- a. 10
 - b. 35
 - c. 70
 - d. 45
13. Mode of 20, 25, 20, 30, 20, 25, 20 is:
- a. 25
 - b. 30
 - c. 20
 - d. 7
14. Range of 120, 150, 180 is:
- a. 30
 - b. 60
 - c. 150
 - d. 180
15. pandas `.std()` divides by:
- a. n
 - b. n-1
 - c. 2n
 - d. zero

Section B — Short questions (10 × 3 = 30)

1. List the five components of a Python database connection in order.
2. Why is `conn.commit()` required after INSERT?
3. What is a DataFrame? How do you load a CSV into one?
4. Differentiate `fillna` and `dropna`.

5. When is a pie chart a poor choice?
6. Differentiate a bar chart and a histogram.
7. Why is the median better than the mean for company salaries?
8. Define range, min and max with the list 70, 80, 90, 60, 75.
9. Why do we take the square root of variance?
10. State two careers that use this chapter's skills.

Section C — Long questions (attempt any $3 \times 10 = 30$)

1. Write a complete Python program that creates `school.db`, a `students` table, inserts Ali 85 and Sara 90, commits, prints all rows, and closes. Number the five connection components beside the matching lines.
2. Starting from the dictionary of Ali/Sara/Ahmed marks, write Pandas code to build a DataFrame, add Total and Average columns, and print it. Show the exact output.
3. Given Marks [85, None, 78, 92] and Grade ["A", "A", None, "A"], show `isnull().sum()`, then fill Marks with the mean and Grade with "Not Assigned". Compute the mean by hand.
4. For each goal pick a chart and justify in one sentence: (a) CS vs IT vs Eng enrolment, (b) website hits from Jan to Jun, (c) share of four clubs in one college, (d) distribution of 200 heights, (e) hours studied vs marks, (f) comparing spread of two classes.
5. For 4, 8, 6, 10, 2 compute mean, median, range, population variance and population SD. Show every step. Then say what pandas `.std()` would report instead.
6. Explain, with a numeric example, how an outlier distorts the mean but not the median. Link this to when you would draw a box plot.

Answer key. Finish the Chapter Exam first, then mark. Do not peek while the clock is running.

پہلے امتحان مکمل کریں، پھر اس کلید سے نمبر لگائیں۔

Check yourself — answers

- 4.1 Concept of Data Analysis.** Answers: 1. Study results to improve teaching (or similar). 2. Any two: read CSV/Excel/databases, organise tables, statistics, charts.
- 4.1.1 – 4.1.2 Data Sources and Database Connection.** Answers: 1. The cursor. 2. SQLite is the database; the `sqlite3` module is the driver.
- 4.2 Creating a SQLite Database in Python.** Answers: 1. All columns. 2. `conn.commit()`.
- 4.3 Basic Operations on Data using Pandas.** Answers: 1. The usual alias for pandas. 2. It would print everything; use `df.head()`.
- 4.3.3 Handling Missing Values (NaN).** Answers: 1. Count of NaNs in each column. 2. Any **row** that contains at least one NaN.
- 4.4 Data Organization and Representation.** Answers: 1. $4 + 2 = 6$. 2. Matplotlib (pyplot).
- 4.5 Data Visualization & Types of Graphs.** Answers: 1. The x-axis is a continuous numeric range, not separate categories. 2. Scatter (two numeric variables, looking for correlation).
- 4.6 Descriptive Statistics.** Answers: 1. Sorted 2,4,6,8,10 → 6. 2. 60. 3. $n-1$ (sample).

Answer key

Multiple choice

- | | | | | |
|------|------|------|-------|-------|
| 1. b | 4. b | 7. b | 10. c | 13. c |
| 2. b | 5. b | 8. b | 11. b | 14. b |
| 3. b | 6. b | 9. b | 12. b | 15. b |

Short questions — model answers

- Database, driver, connection object, cursor object, SQL queries.
- Without commit the insert stays in memory and disappears when the program ends, so the next run cannot see the row.
- A 2-D table of labelled rows and columns. `df = pd.read_csv("file.csv")`.
- `fillna(v)` replaces NaN with v (the row stays). `dropna()` deletes any row that still contains a NaN.
- When there are many slices, or the values are almost equal, so the eye cannot compare them.
- Bar: separate categories, gaps between bars. Histogram: one numeric variable grouped into bins; bars touch.
- A huge CEO salary pulls the mean up; the median stays with the typical worker.
- Min 60, max 90, range 30.
- Variance is in squared units. The SD brings the spread back to the original units (marks, rupees, °C).

10. Any two: data analyst, DBA, data scientist, visualisation specialist, BI analyst, backend developer.

Long questions — model answers

1. Use the nine-step listing in 4.2 with two INSERTs. Label: sqlite3 = driver, connect = connection, cursor = cursor, execute strings = SQL, school.db = database.
2. Output is the three-row table with Total 165, 178, 148 and Average 82.5, 89.0, 74.0 (see the worked example).
3. isnull sum: Marks 1, Grade 1. Mean of 85, 78, 92 = $255/3 = 85.0$. After fill: Sara Marks 85.0, Ahmed Grade "Not Assigned".
4. (a) column/bar — compare categories. (b) line — time. (c) pie — parts of a whole. (d) histogram — numeric distribution. (e) scatter — two numeric variables. (f) box — spread and outliers.
5. Sorted 2,4,6,8,10. Sum 30, mean 6, median 6, range 8. Deviations -4,-2,0,2,4. Squares 16,4,0,4,16 sum 40. $\sigma^2 = 40/5 = 8$. $\sigma = \sqrt{8} = 2.828$ (3 d.p.). pandas `.std()` uses $40/4 = 10$, $\sqrt{10} \approx 3.162$.
6. Nine salaries of 10 and one of 1000: mean 109, median 10. Draw a box plot — the 1000 appears as an outlier dot above the whisker, so the distortion is visible.

Self-assessment: I can...

- | | |
|--|--|
| ☐ name the five connection components in order. | ☐ write SQLite connect / create / insert / commit / select / close. |
| ☐ build a DataFrame and add a calculated column. | ☐ find, fill and drop missing values. |
| ☐ choose bar, line, pie, histogram, scatter or box for a problem. | ☐ compute mean, median, mode, range, variance and SD by hand. |
| ☐ state that pandas std uses $n-1$. | |

Lecture 05

Application and Impacts of Computing

Chapter lecture — 15 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

BIEK Computer Science XII · Bilingual support: English + اردو

بی آئی ای کے کمپیوٹر سائنس — بارہویں جماعت

15 book lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XII

How to use this BIEK study-guide lecture

Each PDF follows one textbook lecture from the BIEK / Sindh Computer Science notes (New Sindh Curriculum). Read *Learn it*, study the SVG diagram, try the worked example on paper, then answer *Check yourself* before the answers. **Extra board practice** (MCQs, short and long questions) is added after the book content — it is not in the textbook. Career-only extras that were not in the book lecture list have been removed. Math is rendered with MathJax. ★ Golden topics are high-yield in board exams. The green box repeats the idea in Urdu.

ہر لیکچر کتاب کے درس کے مطابق ہے۔ کتاب کے بعد اضافی MCQs اور سوالات ہیں۔ جو اضافی کیریئر صفحات کتاب کی فہرست میں نہیں تھے وہ ہٹا دیے گئے ہیں۔

Lectures in this pack

Chapter 5: Application and Impacts of Computing. Study one lecture at a time, then use the chapter review and answer key at the end of this PDF.

L01. 5.1 Machine Learning, Neural Networks & Deep Learning

L02. 5.1.1 Machine Learning (ML)

L03. 5.1.2 Neural Networks (NN)

L04. 5.1.3 Deep Learning (DL)

L05. 5.1.4 Components of Neural Networks

L06. 5.1.5 Components of Deep Learning

L07. 5.2 Applications of Neural Networks and Deep Learning

L08. 5.3 Secure Collaboration (Authentication, Access Control)

L09. 5.3.2 Data Protection (Encryption, Passwords, Backup, Firewall)

L10. 5.4 Security Threats and Mitigation Techniques

L11. 5.4.2 Identify Security Threats

L12. 5.4.3 Apply Mitigation Techniques

L13. 5.5 Equity and Equal Access in Digital Collaboration

L14. 5.5.1 Functions of Equity and Equal Access

L15. 5.5.2 Collaboration Tools

Application and Impacts of Computing

This chapter shows you how computers learn from data with machine learning, neural networks and deep learning, how to collaborate online without leaking secrets, how to protect data and stop cyber threats, and how to keep digital teamwork fair for every student in Sindh.

اس باب میں آپ سیکھیں گے کہ مشین لرننگ، نیورل نیٹ ورکس اور ڈیپ لرننگ کمپیوٹر کو ڈیٹا سے کیسے سکھاتے ہیں، آن لائن کام کو محفوظ کیسے رکھا جائے، ڈیٹا کی حفاظت اور سائبر خطرات کا مقابلہ کیسے کیا جائے، اور ڈیجیٹل تعاون کو سب کے لیے منصفانہ کیسے بنایا جائے۔

What you will learn

- Define **machine learning (ML)** and describe the three steps: data input, learning, prediction.
- Define **deep learning (DL)** and draw the relationship $AI \supset ML \supset DL$.
- List the six components of deep learning and give applications of NN and DL in daily life and in Pakistan.
- Protect data using encryption (including a Caesar cipher), strong passwords, the 3-2-1 backup rule and a firewall.
- Distinguish **equal access** from **equity** and describe the functions of equity in digital collaboration.
- Explain a **neural network (NN)** as a brain-inspired model with an input layer, hidden layers and an output layer.
- Name the eight components of a neural network and compute a neuron's output from given numbers.
- Explain **secure collaboration**: authentication, multi-factor authentication (MFA) and access control.
- Identify common threats (malware, phishing, DoS) and apply mitigation techniques.
- Name common collaboration tools and use them fairly and safely.

5.1 Machine Learning, Neural Networks & Deep Learning

LEARN IT

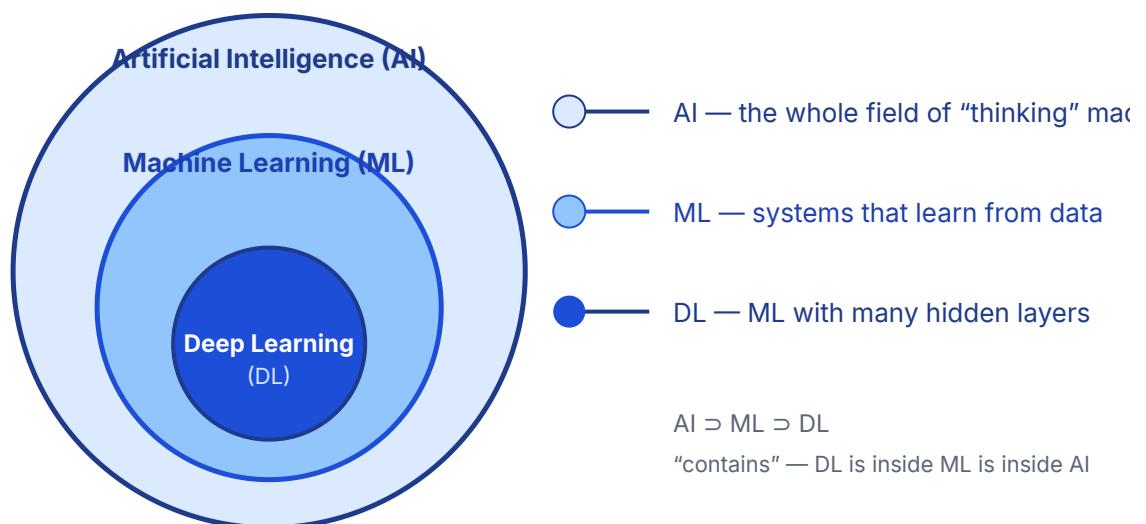
Machine learning (ML), **neural networks (NN)** and **deep learning (DL)** are important areas of **artificial intelligence (AI)**. **AI** is the broad field in which computers perform tasks that normally need human thinking, such as recognising a face, understanding speech or making a decision.

These technologies help computers **learn from data** and make useful decisions **without being explicitly programmed for every single step**. You do not write a separate rule for every possible face or every Urdu accent. You show the computer many examples, and it finds the patterns itself.

They are used in many modern systems:

- **Face unlock** on smartphones.
- **Speech recognition** (for example Siri, Alexa, or Urdu voice typing on a phone).
- **Photo search** categorisation (typing "car" in a gallery and seeing car photos).
- **Medical diagnosis** from X-rays and scans.
- **Language translation** (English to Urdu or Sindhi).

Remember the nesting: **AI is the biggest set**. Inside AI sits **ML**. Inside ML sits **DL**. Every deep-learning system is a kind of machine learning, and every machine-learning system is a kind of AI — but not every AI system is deep learning.



The set relationship $AI \supset ML \supset DL$. Deep learning is a method of machine learning; machine learning is a part of artificial intelligence.

WORKED EXAMPLE

Worked example: place each system in the correct set. Sara in Karachi uses three features on her phone.

1. **A chess app that follows fixed if-then rules** written by a programmer (“if the queen is attacked, move it”). This is **AI** (the machine decides), but it is **not ML**, because it does not learn from data.
2. **A JazzCash fraud filter** that was trained on thousands of past transactions and then flags a new unusual payment. This is **ML** (and therefore also AI). If it uses only a simple model with few layers, it may **not** be DL.
3. **Face unlock** that studied millions of face images through many hidden layers. This is **DL** — and therefore also ML and AI.

Exam sentence you can write: *All DL is ML, all ML is AI, but not all AI is ML, and not all ML is DL.*

REMEMBER IT

Russian-doll memory trick. Picture three boxes. The outer box is **AI**. Inside it is a smaller box **ML**. Inside that is the smallest box **DL**. You cannot take DL out of ML, and you cannot take ML out of AI.

COMMON MISTAKE

Mistake: “AI, ML and DL are three different, separate subjects.”

Fix: They nest. $DL \subset ML \subset AI$. A deep-learning face-unlock system is all three at once.

اردو میں سمجھیں

مصنوعی ذہانت (AI) سب سے بڑا میدان ہے۔ اس کے اندر مشین لرننگ (ML) ہے، اور مشین لرننگ کے اندر ڈیپ لرننگ (DL) ہے۔ یہ ٹیکنالوجیز کمپیوٹر کو ڈیٹا سے سکھاتی ہیں تاکہ ہر قدم الگ الگ پروگرام کیے بغیر فیصلے کیے جا سکیں، جیسے چہرے سے فون کھولنا یا آواز پہچاننا۔

CHECK YOURSELF

1. Which is the largest set: AI, ML or DL?
2. Is every neural-network face-unlock system an example of AI? Why?
3. Give one modern system from this section that uses these technologies.

Answers: 1. AI ($AI \supset ML \supset DL$). 2. Yes — DL is inside ML, and ML is inside AI. 3. Any of: face unlock, speech recognition, photo search, medical diagnosis, language translation.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. ML, NN and DL are parts of:

- a. AI
- b. OSI layer 1 only
- c. Access wizards only
- d. Waterfall only

MCQ 2. They all need:

- a. Data to learn from
- b. Only copper cable
- c. Only primary keys
- d. Only Figma

MCQ 3. The course order is:

- a. ML as the idea, NN as a model, DL as deep NNs
- b. DL then stone tools
- c. SQL then NAND
- d. HCI then K-maps

MCQ 4. Pakistan's AI policy is mentioned as:

- a. A future/local context
- b. A Python library
- c. A chart type
- d. A file mode

MCQ 5. A student mistake is:

- a. Treating the three terms as identical
- b. Giving one example
- c. Mentioning data
- d. Mentioning layers

MCQ 6. This lecture is the map; later ones:

- a. Define each term properly
- b. Delete the chapter
- c. Only careers
- d. Only MVPs

Extra short questions

1. Place ML, NN, DL in one nested sentence.
2. What do they share?
3. What should an XI/XII answer still include?
4. One Pakistani hope from the notes.

Extra long question

1. Draw a nested diagram $AI \supset ML \supset NN \supset DL$ and write one example at each ring.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. DL is a kind of NN which is a way to do ML which is a kind of AI.

2. Learn patterns from data instead of only hand-written rules.
3. A human-responsible, data-quality, bias warning.
4. Agriculture, health, Urdu NLP, public service — any from the book.

Long

1. AI: chess rules engine. ML: spam filter. NN: digit recognition. DL: speech-to-Urdu. One limit at the DL ring (needs lots of data/compute).

5.1.1 Machine Learning (ML)

LEARN IT

A **machine learning model** is the system that **learns from data and gives an output**. Think of it as the **trained brain** of a computer program.

In ordinary programming you write every rule yourself: *if marks ≥ 40 then "Pass"*. In machine learning you **do not list every rule**. You give the model many examples (data), it finds the patterns, and later it decides about new cases it has never seen.

How it works — three steps:

1. Data input → 2. Learning (find patterns) → 3. Prediction on new data

1. **Data input.** The model is given massive amounts of data — for example thousands of past BISE marksheets, or millions of crop photos from Sindh fields.
2. **Learning.** It studies the data and **learns patterns and rules automatically**. It might notice that students who scored low in Chapter 3 also failed the board exam, or that yellow spots on cotton leaves often mean a pest.
3. **Prediction.** After training, it can make **accurate predictions or decisions** when given **new, unseen data** — a new student's record, or a new leaf photograph.

Note from the source: there are many types of machine-learning models. A **neural network** is one important type. **Deep learning** is a highly advanced method of machine learning.

WORKED EXAMPLE

Worked example: a wheat-yield model for a farm near Sukkur.

Step 1 — Data input. For 8 years the farm recorded: rainfall (mm), temperature, fertiliser bags used, and the actual yield (maunds per acre).

Step 2 — Learning. The model studies those rows. It finds, for example, that yield rises with fertiliser up to a point, and falls when temperature is very high in April.

Step 3 — Prediction. This year the farmer types in this season's rainfall and temperature. The model has never seen this exact year before, but it predicts "about 32 maunds per acre." The farmer plans storage and a buyer in advance.

Nobody wrote a giant list of "if rain = 120 and temp = 38 then yield = ...". The model learned the pattern from data.

REMEMBER IT

D-L-P. Machine learning is **Data** in, **Learn** patterns, **Predict**. "Data Learns to Predict."

COMMON MISTAKE

Mistake: "The computer is intelligent on day one."

Fix: An ML model is useful only **after training** on enough good data. Garbage data → garbage predictions. Also, neural networks are *one* type of ML, not the only type; deep learning is a *method* that uses deep neural networks.

اردو میں سمجھیں

مشین لرننگ ماڈل وہ نظام ہے جو ڈیٹا سے سیکھ کر نتیجہ دیتا ہے — کمپیوٹر پروگرام کا تربیت یافتہ دماغ۔ پہلے بہت سا ڈیٹا دیا جاتا ہے، پھر ماڈل خود اصول اور پیٹرن سیکھتا ہے، اور تربیت کے بعد نئے ان دیکھے ڈیٹا پر پیش گوئی کرتا ہے۔ نیورل نیٹ ورک مشین لرننگ کی ایک اہم قسم ہے، اور ڈیپ لرننگ اس کا ایک اعلیٰ طریقہ ہے۔

CHECK YOURSELF

1. What are the three steps of how an ML model works?
2. What is a machine-learning model compared to in the source?
3. After training, what kind of data can the model handle?

Answers: 1. Data input, learning, prediction. 2. The trained brain of a computer program. 3. New, unseen data.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Machine learning lets a model:

- a. Improve from examples, not only hard-coded rules
- b. Run without data
- c. Replace all maths
- d. Ignore errors always

MCQ 2. Training data is:

- a. Examples used to fit the model
- b. The final user password
- c. OSI headers
- d. Figma frames

MCQ 3. A model that memorizes noise is:

- a. Overfitting
- b. Perfect science
- c. A stack
- d. A pie

MCQ 4. Supervised learning uses:

- a. Labels (the right answers)
- b. No data
- c. Only unsupervised always
- d. Only cables

MCQ 5. ML in this book is the “trained brain” metaphor for:

- a. Mapping inputs to outputs after seeing examples
- b. A DBMS trigger
- c. A foreign key
- d. A sprint

MCQ 6. Garbage data yields:

- a. Garbage predictions
- b. Perfect fairness
- c. $O(1)$ time always
- d. Free RAM

Extra short questions

1. Define ML in the book’s words/your own accurate line.
2. Give two applications.
3. What is a label?
4. One ethical issue.

Extra long question

1. Explain ML to a principal who wants to predict drop-outs: data, train, test, a risk, a human check.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. A model that learns from data to produce outputs without being explicitly programmed for every case.
2. Spam; recommendations; marks prediction; vision.
3. The known answer attached to a training example.
4. Bias in the training data is copied into decisions.

Long

1. Past attendance+marks → model → test on held-out year. Risk: punishing poverty correlated with absence. Human counsellor decides, model only flags.

5.1.2 Neural Networks (NN) ★ Golden

LEARN IT

A **neural network** is a computer model that **learns patterns from data**. It is **directly inspired by the human brain**, but it works as a **mathematical computer system** — not as living tissue.

Your brain has billions of nerve cells (**neurons**) that pass signals to each other. A neural network copies this idea with small processing units also called **neurons** or **nodes**. These nodes are **connected** and **pass information from one layer to another**, performing calculations at each step.

A typical neural network has three kinds of layer:

- **Input layer** — receives the raw data (pixels of a face, samples of a sound, marks of a student).
- **Hidden layer(s)** — the “thinking” in the middle. Nodes here combine the inputs and learn features (eye distance, nose shape, an edge in a photo).
- **Output layer** — gives the final answer (unlock / keep locked; “this is a cat”; “pass / fail”).

Input layer → Hidden layer(s) → Output layer

WORKED EXAMPLE

Practical example from the source: face unlock.

Step 1 — Input layer. You look at your phone. The camera captures your face as numbers (brightness of each pixel). Those numbers enter the input layer.

Step 2 — Hidden layers. The system studies face features such as **eye distance** and **nose shape**. Early hidden nodes may detect edges and shadows; later hidden nodes combine those into “this looks like Sara’s face.” It compares them with stored data from when you first set up the phone.

Step 3 — Output layer. If the features match, the phone **unlocks**. If they do not (your brother holds the phone), it stays locked.

The same three-layer story works for an NADRA-style biometric gate at a college: camera (input) → feature matching (hidden) → open the turnstile or not (output).

REMEMBER IT

I-H-O. Input receives, Hidden thinks, Output decides. “In Hidden Out.” A neural network is a **brain-inspired maths machine**, not a copy of a living brain.

COMMON MISTAKE

Mistake: "A neural network is an actual human brain inside the computer."

Fix: It is only **inspired by** the brain. It is mathematics: numbers, multiplications and additions running on ordinary hardware.

اردو میں سمجھیں

نیورل نیٹ ورک ایک کمپیوٹر ماڈل ہے جو ڈیٹا سے پیٹرن سیکھتا ہے۔ یہ انسانی دماغ سے متاثر ہے مگر ایک ریاضیاتی نظام کے طور پر کام کرتا ہے۔ چھوٹی اکائیاں نیورون یا نوڈ کہلاتی ہیں، جو تھوں میں جڑی ہوتی ہیں: ان پٹ تھ ڈیٹا لیتی ہے، پوشیدہ تھ سوچتی ہے، اور آؤٹ پٹ تھ فیصلہ دیتی ہے — جیسے چہرے سے فون کھولنا۔

EXAM TIP

Very common questions: "What is a neural network?" (2–3 marks), "Name its three layers and explain each with the face-unlock example" (5 marks), and "How is a neural network related to the human brain?"

Full-mark recipe: (1) definition — computer model that learns patterns from data, inspired by the brain, mathematical; (2) neurons/nodes connected across layers; (3) Input / Hidden / Output, each in one sentence; (4) walk the camera → features (eye distance, nose shape) → unlock path. Draw the three boxes if the paper asks for a diagram.

CHECK YOURSELF

1. What are the small processing units of a neural network called?
2. In face unlock, which layer studies eye distance and nose shape?
3. Is a neural network a living brain? Explain in one sentence.

Answers: 1. Neurons or nodes. 2. Hidden layers. 3. No — it is inspired by the brain but works as a mathematical computer system.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A neural network is inspired by:

- a. The brain's networks of neurons (loosely)
- b. OSI cables
- c. Access forms
- d. Bubble sort

MCQ 2. It learns:

- a. Weights that turn inputs into outputs
- b. SQL indexes
- c. TCP windows
- d. Figma auto-layout

MCQ 3. Simple NNs have:

- a. Input, hidden, output layers
- b. Only a pie
- c. Only one Excel cell
- d. No numbers

MCQ 4. This is ★: define, then:

- a. One diagram + two uses
- b. Only careers
- c. Only MVPs
- d. Only keys

MCQ 5. NN vs linear formula:

- a. NN can stack non-linear layers for complex patterns
- b. NN cannot use data
- c. NN is a queue
- d. NN is HCI only

MCQ 6. Digits on paper are a classic:

- a. NN / vision example
- b. Primary key example
- c. Waterfall example
- d. K-map example

Extra short questions

1. Define neural network.
2. Three layer types.
3. Two applications.
4. Need for data.

Extra long question

1. Draw a 3-layer net for pass/fail from 3 marks. Label input, hidden, output. Explain one training idea in words (weights change). ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. A computer model of connected units that learns patterns from data, loosely like neurons.
2. Input, hidden, output.
3. Image recognition; speech; translation.
4. Without enough varied examples the net does not generalise.

Long

1. 3 input nodes, few hidden, 1 output. Training: compare prediction to label, adjust weights. Limit: not a biological brain.

5.1.3 Deep Learning (DL)

LEARN IT

Deep learning is a machine-learning method in which computers use **multi-layer neural networks** to learn **complex patterns** from data.

The word **“deep”** is used because data passes through **several hidden layers** before the system produces the final output. Each layer learns a **different level of detail**:

- **Early layers** learn simple features — edges, shapes, colours.
- **Deeper layers** combine those simple features to understand complex patterns — a whole face, or a specific car model (a Cultus versus a Corolla).

A network with only one or two hidden layers is a **shallow** neural network. Deep learning is always “deep”: it needs **huge amounts of data** and **processing power**.

Neural network vs deep learning

	Neural network (the model)	Deep learning (the method)
What it is	A system/model that has layers, nodes and connections to help it learn.	A specific method that trains neural networks containing many hidden layers .
Depth	Can be simple (shallow) with just one or two hidden layers.	Always “deep,” requiring huge amounts of data and processing power to learn complex patterns.
Role	The architecture (the “machine”).	The way we train a deep version of that machine.

WORKED EXAMPLE

Real-world example from the source: photo gallery search.

A modern gallery app on a smartphone uses deep learning to recognise faces, objects or places in thousands of photos.

Step 1. During training (done by the company, using millions of images) early layers learned edges and colours; deeper layers learned "wheel", "petal", "minaret".

Step 2. The trained network has already scanned your phone's photos and tagged them.

Step 3. You type "**car**" or "**flower**" in the search bar. The app instantly shows related photos because its deep neural network has already learned visual patterns from millions of training images.

A student in Hyderabad can type "Ajrak" or "cricket" and jump to those pictures without scrolling for ten minutes.

REMEMBER IT

Deep = many hidden layers. Shallow NN: 1-2 hidden layers. Deep NN: many hidden layers, huge data, strong computers. Simple features first, complex meaning later.

COMMON MISTAKE

Mistake: "Deep learning and neural networks are two names for the same thing."

Fix: A neural network is the **model** (layers + nodes + connections). Deep learning is the **method** of training networks that have **many** hidden layers. A small NN with one hidden layer is not deep learning.

اردو میں سمجھیں

ڈیپ لرننگ مشین لرننگ کا وہ طریقہ ہے جس میں کئی پوشیدہ تہوں والے نیورل نیٹ ورک استعمال ہوتے ہیں۔ "ڈیپ" اس لیے کہ ڈیٹا کئی تہوں سے گزر کر پیچیدہ پیٹرن سیکھتا ہے: پہلی تہیں کنارے اور رنگ سیکھتی ہیں، گہری تہیں پورا چہرہ یا گاڑی کی قسم پہچانتی ہیں۔ یہ طریقہ بہت سے ڈیٹا اور طاقتور کمپیوٹنگ کا طالب ہے۔

CHECK YOURSELF

1. Why is the method called “deep” learning?
2. What do early layers learn, and what do deeper layers learn?
3. Can a neural network with one hidden layer be called deep learning?

Answers: 1. Because data passes through several hidden layers. 2. Early: simple features (edges, shapes, colours). Deeper: complex patterns (a whole face, a car model). 3. No — that is a shallow network; DL always uses many hidden layers.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Deep learning uses:

- a. Many-layer neural nets
- b. One if-statement
- c. Only Access macros
- d. Only OSI

MCQ 2. "Deep" refers to:

- a. Many hidden layers
- b. Deep sea cables only
- c. Deep stacks in Python only
- d. Deep queues

MCQ 3. DL shines at:

- a. Images, speech, complex patterns
- b. Two-row Excel averages
- c. Primary keys
- d. K-maps of 2 vars

MCQ 4. A cost of DL is:

- a. Lots of data and compute
- b. No data needed
- c. No energy use
- d. Perfect fairness always

MCQ 5. DL is a subset of:

- a. NN / ML / AI
- b. HCI only
- c. SQL only
- d. SDLC only

MCQ 6. Medical imaging in the book is a:

- a. DL application
- b. A foreign key
- c. A pie of NAND
- d. A sprint

Extra short questions

1. Define deep learning.
2. How is it different from a shallow net?
3. Two applications.
4. One Pakistani constraint.

Extra long question

1. Compare ML vs NN vs DL in a 6-row table (definition, depth, data need, example, limit). 10 marks.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. ML using multi-layer neural networks to learn complex patterns.

2. More layers can build higher-level features (edges → shapes → faces).
3. X-ray analysis; voice assistants; translation.
4. GPUs/data/labels are expensive; start with smaller models.

Long

1. Clear nesting. Example row: spam vs digit net vs speech net. Limits: data, bias, black box, energy.

5.1.4 Components of Neural Networks ★ Golden

LEARN IT

The components of a neural network can be understood through its **layered architecture**. Data **enters**, gets **processed**, and **exits**, with mathematical adjustments happening inside.

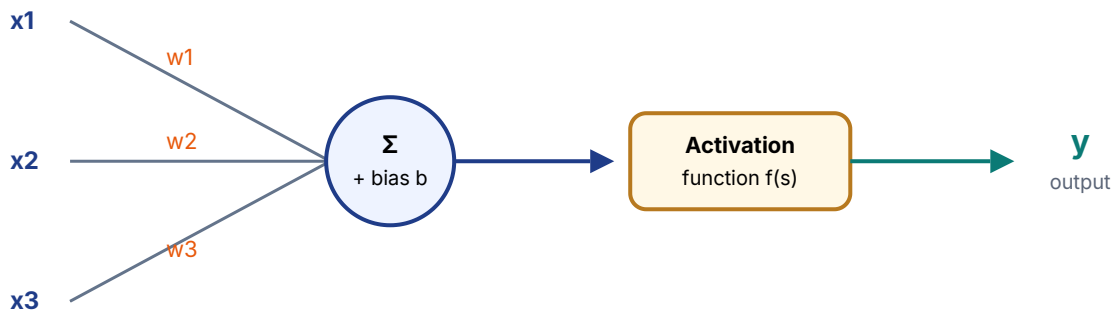
#	Component	Description
1	Input layer	Receives the initial raw data (for example image pixels, sound waves, text).
2	Neurons / nodes	Small processing units that receive, calculate, and pass information forward.
3	Hidden layer(s)	The layers between input and output. They process the data and learn patterns from it.
4	Weights	Numbers that show the importance or strength of connections between neurons.
5	Bias	An extra constant value added to help the model adjust its result and improve accuracy.
6	Activation function	A mathematical gate that decides whether the information is important enough to move to the next layer.
7	Output layer	Gives the final computed result, decision or prediction (for example "This is a cat").
8	Loss function	Checks the error (how much the model's answer is wrong compared with reality) so the network can correct itself.

What happens inside one neuron

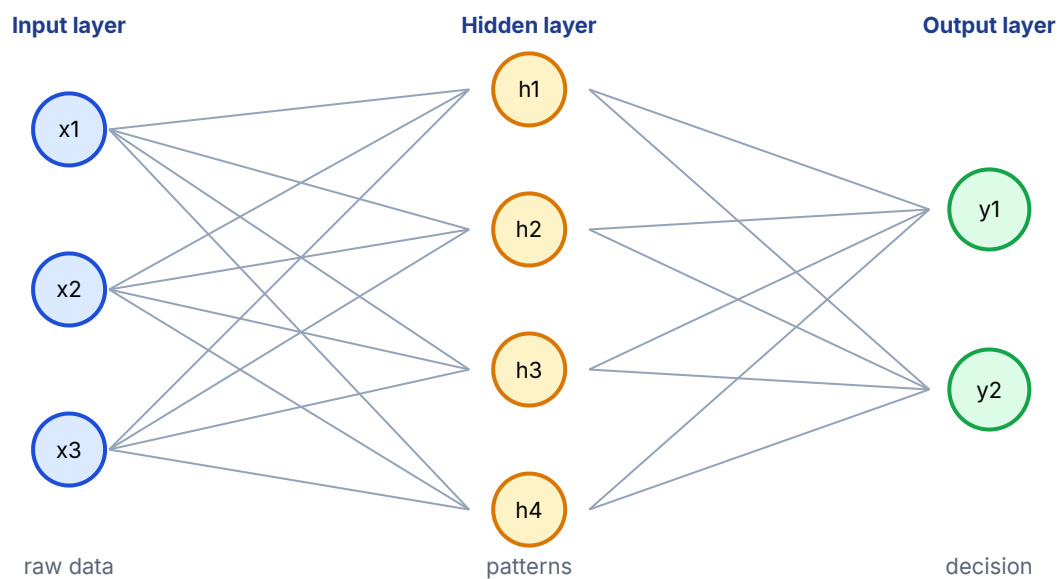
Each neuron does a short calculation:

1. Multiply each input x by its weight w .
2. Add those products together, then add the **bias** b . This total is the **weighted sum** s .
3. Pass s through an **activation function**. A simple exam-friendly gate is a **threshold (step) function**: if s is at least the threshold, the neuron **fires** (output = 1); otherwise output = 0.

In symbols: $s = (x_1 \times w_1) + (x_2 \times w_2) + \dots + b$, then $y = 1$ if $s \geq \text{threshold}$, else $y = 0$.



One neuron: inputs $\times$ weights are summed, bias is added, then an activation function decides the output y .



A small neural network: 3 input nodes, 4 hidden nodes, 2 output nodes. Each line is a weighted connection.

WORKED EXAMPLE

Worked example: neuron calculation with numbers. A college in Larkana uses a tiny network to guess whether a student is ready for a test. One hidden neuron receives two inputs:

Symbol	Meaning	Value
x1	Hours studied yesterday (scaled 0 to 1)	0.8
x2	Attendance this month (scaled 0 to 1)	0.5
w1	Weight of hours studied	0.4
w2	Weight of attendance	0.6
b	Bias	0.1
threshold	Activation gate	0.5

Step 1 — weighted products.

$$x1 \times w1 = 0.8 \times 0.4 = \mathbf{0.32}$$

$$x2 \times w2 = 0.5 \times 0.6 = \mathbf{0.30}$$

Step 2 — add bias to get the weighted sum s.

$$s = 0.32 + 0.30 + 0.1 = \mathbf{0.72}$$

Step 3 — activation (threshold 0.5). Is $0.72 \geq 0.5$? **Yes**, so the neuron fires: **y = 1** (ready for the test).

Step 4 — a second student with $x1 = 0.2$ (almost no study) and $x2 = 0.3$:

$$s = (0.2 \times 0.4) + (0.3 \times 0.6) + 0.1 = 0.08 + 0.18 + 0.1 = \mathbf{0.36}$$

0.36 is **less than** 0.5, so **y = 0** (not ready). The same neuron, different inputs, different decision.

You can check the first student in Python 3:

```
x1, x2 = 0.8, 0.5
w1, w2 = 0.4, 0.6
b = 0.1
s = x1 * w1 + x2 * w2 + b
y = 1 if s >= 0.5 else 0
print(round(s, 2))
print(y)
```

OUTPUT

0.72

1

Loss function, briefly. If the true answer for the first student was actually "not ready" (true $y = 0$) but the model said 1, the loss is high. Training then slightly changes weights and bias so the next guess is closer to reality.

REMEMBER IT

Weights = importance. Bias = extra nudge. Activation = yes/no gate. Loss = how wrong. A large weight means that input matters more. Bias shifts the sum up or down so the gate can open at the right place.

COMMON MISTAKE

Mistake: Forgetting to add the bias, or multiplying instead of adding after the products.

Fix: Always **multiply first** (each x by its w), **then add** the products, **then add b** , **then** apply the activation. Write the four lines in the exam as we did above.

اردو میں سمجھیں

نیورل نیٹ ورک کے اجزاء ہیں: ان پٹ تہ، نیورون، پوشیدہ تہ، وزن (Weights)، بائیس (Bias)، ایکٹیویشن فنکشن، آؤٹ پٹ تہ اور لاس فنکشن۔ ہر نیورون ان پٹ کو وزن سے ضرب دے کر جمع کرتا ہے، بائیس جوڑتا ہے، پھر ایک گیٹ فیصلہ کرتا ہے کہ سگنل آگے جائے یا نہیں۔ وزن تعلق کی طاقت بتاتا ہے؛ لاس فنکشن غلطی ناپتا ہے تاکہ ماڈل خود درست ہو سکے۔

EXAM TIP

Golden questions: "List the components of a neural network" (often 8 names + one line each), "What are weights and bias?", and a **numerical neuron**: given x , w , b and a threshold, find s and y .

Show every arithmetic step. Underline the final y . If asked for a diagram, draw Input → Hidden → Output with circles and connecting lines, and label weights on the lines.

Mention the loss function as the component that measures error so the network can correct itself.

CHECK YOURSELF

1. What do weights show?
2. Using $x_1 = 1$, $x_2 = 0$, $w_1 = 0.5$, $w_2 = 0.5$, $b = 0$, threshold = 0.4, what is y ?
3. What does the loss function check?

Answers: 1. The importance or strength of connections between neurons. 2. $s = 1 \times 0.5 + 0 \times 0.5 + 0 = 0.5$; $0.5 \geq 0.4$ so $y = 1$. 3. The error — how much the model's answer is wrong compared with reality.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Input layer:

- a. Receives the features
- b. Always outputs the class
- c. Stores SQL
- d. Draws Figma

MCQ 2. Hidden layers:

- a. Transform features (learned)
- b. Are optional pie charts
- c. Are OSI hubs
- d. Are primary keys

MCQ 3. Output layer:

- a. Produces the prediction
- b. Reads the keyboard
- c. Commits sqlite
- d. Opens VS Code

MCQ 4. Weights are:

- a. Strengths of connections, learned from data
- b. Font weights only
- c. TCP weights
- d. Agile story points

MCQ 5. Activation functions:

- a. Add non-linearity so layers can do more than a straight line
- b. Sort the array
- c. Encrypt the disk
- d. Draw ER

MCQ 6. This is ★: a 5-mark wants:

- a. Named parts + one sentence each + tiny diagram
- b. Only the word "brain"
- c. Only careers
- d. Only MVPs

Extra short questions

1. List components of a NN from the notes.
2. What does a neuron do conceptually?
3. Why more than one hidden layer can help.
4. What is not a component?

Extra long question

1. Draw and label input, 2 hidden, output. Annotate weights on two edges. Explain in 4 sentences how a pass/fail signal moves left to right. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Layers (in/hidden/out), neurons/nodes, weights, activations, (bias).
2. Weighted sum of inputs, then activation.
3. Features of features (deep).
4. A pie chart; a primary key; a sprint.

Long

1. Diagram. Forward pass description. Training mentioned as changing weights, not needed in full backprop maths. One misuse: calling it an exact brain.

5.1.5 Components of Deep Learning

LEARN IT

Deep learning relies on specific advanced components to achieve its high accuracy. These components **differentiate it from basic machine learning**.

#	Component	What it means
1	Large input data	Deep learning is “ data hungry .” It needs massive datasets (thousands or millions of records) for better learning and accuracy.
2	Many hidden layers	Uses dozens or hundreds of hidden layers to learn deeper, more intricate patterns.
3	Automatic feature learning	Unlike older models where humans had to manually highlight important features, deep learning extracts important features by itself .
4	Learning algorithm	Math routines (like backpropagation) that help the system reduce mistakes and improve learning continuously.
5	High computing power	Processing millions of data points across deep layers needs specialised, strong hardware, such as GPUs (Graphics Processing Units).
6	Training process	The model is trained again and again in loops called epochs to gradually improve its final performance.

A **GPU** was originally made for video games’ graphics, but it can do thousands of small multiplications in parallel — exactly what deep networks need. An **epoch** is one full pass through the training data. Ten epochs means the model has seen every training example ten times, adjusting weights a little each time.

Backpropagation means sending the error (from the loss function) **backwards** through the layers so each weight can be nudged in the direction that reduces the next error.

WORKED EXAMPLE

Worked example: cotton-leaf disease detector for a farm in Mirpurkhas. Match each DL component.

1. **Large input data.** Researchers collect 80,000 labelled photos: healthy leaves and leaves with whitefly damage.
2. **Many hidden layers.** The network has 20 hidden layers so it can go from "green edge" to "this exact pest pattern."
3. **Automatic feature learning.** No agronomist types in "look for yellow circles." The network finds the useful spots itself.
4. **Learning algorithm.** After a wrong guess ("healthy" when it was diseased), backpropagation lowers the weights that caused the mistake.
5. **High computing power.** Training runs on a GPU workstation at a university lab in Karachi; a school laptop would take weeks.
6. **Training process.** They train for 30 epochs. After epoch 1 accuracy is 62%; after epoch 30 it is 94% on photos it has not trained on.

REMEMBER IT

D-L-A-L-G-T. Data (lots), Layers (many), Automatic features, Learning algorithm, GPU power, Training epochs. "Deep Learners Always Love Good Training."

COMMON MISTAKE

Mistake: "Deep learning is just 'a neural network'."

Fix: Name the six extras: huge data, many hidden layers, automatic features, a learning algorithm (backpropagation), GPUs, and repeated epochs. A shallow NN with a small spreadsheet is ordinary ML, not DL.

اردو میں سمجھیں

ڈیپ لرننگ کو عام مشین لرننگ سے یہ چیزیں الگ کرتی ہیں: بہت بڑا ڈیٹا، بہت سی پوشیدہ تہیں، خود بخود خصوصیات نکالنا، سیکھنے کا الگورتھم جیسے بیک پروپیگیشن (Backpropagation)، طاقتور ہارڈویئر جیسا کہ جی پی یو (GPU)، اور بار بار تربیت جسے ایپوک (Epoch) کہتے ہیں۔

CHECK YOURSELF

1. Why is deep learning called “data hungry”?
2. What is an epoch?
3. Name the hardware often used for DL, and the learning algorithm named in the source.

Answers: 1. It needs massive datasets (thousands or millions of records) for accuracy. 2. One training loop — one full pass through the training data. 3. GPUs (Graphics Processing Units); backpropagation.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. DL components include:

- a. Deep stacks of layers, lots of data, lots of compute, specialised architectures
- b. A single if
- c. A single Access form
- d. A single NAND

MCQ 2. CNNs are often for:

- a. Images
- b. Undo stacks
- c. Printer queues
- d. Primary keys

MCQ 3. RNNs/transformers often for:

- a. Sequences (text/speech)
- b. Pie charts
- c. ER diamonds
- d. K-map groups

MCQ 4. GPUs help because:

- a. Many multiply-adds in parallel
- b. They store FKs
- c. They draw OSI
- d. They boil water

MCQ 5. Pretrained models let you:

- a. Reuse features (transfer) when you have less data
- b. Skip ethics
- c. Skip labels forever
- d. Skip HCI

MCQ 6. A practical school limit is:

- a. Not enough labelled images/GPU
- b. Too many GPUs always
- c. Too few ethics issues
- d. Too much labelled data always

Extra short questions

1. Name DL-specific components vs a tiny NN.
2. Why not train ImageNet in a college lab from scratch?
3. One architecture name at definition level.
4. Ethics still apply.

Extra long question

1. A team wants DL for Urdu speech marks entry. List DL components they need and three reasons to start smaller (ML or rules).

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Depth, specialised layers (conv), big datasets, accelerators, regularisation.
2. Compute and labelled data; use a pretrained net if anything.
3. CNN for images / transformer for language — no need for full maths.
4. Medical DL that is biased still harms patients.

Long

1. Need: audio data, labels, model, GPU, evaluation. Start smaller: n small, accents, privacy of children's voices, electricity/cost. Human checks the marks.

5.2 Applications of Neural Networks and Deep Learning

LEARN IT

Machine learning, neural networks and deep learning will play a key role in Pakistan's future. The **National AI Policy 2025** highlights how artificial intelligence is now part of national planning, focusing on **innovation, skill-building, ethical use and digital growth**.

Sector	Potential impact in Pakistan
Education	Smart learning systems, online tutoring, automatic assessments, and finding where students need specific help.
Healthcare	Disease detection, medical image analysis, and managing hospital data. Crucial for rural areas lacking specialist doctors.
Agriculture	Monitoring crops, predicting weather, managing irrigation, and estimating yields — highly important for Pakistan's economy.
Business and banking	Fraud detection, customer support (chatbots), market forecasting, and intelligent risk analysis.
Local languages	Speech recognition, translation and text-to-speech tools for Urdu, Sindhi and other regional languages.

To benefit from this future, Pakistan needs skilled individuals. You should focus on **programming (such as Python), mathematics, data analysis, problem-solving** and the **ethical use of technology**.

WORKED EXAMPLE

Worked example: pick a sector and write a three-line “impact paragraph” for 5 marks.

Agriculture: A cotton grower in Sanghar photographs leaves on a cheap phone. A deep-learning model trained on Pakistani crop images flags whitefly early. The farmer sprays only the affected block, saves pesticide cost, and protects yield — which matters because agriculture is a large part of Pakistan's economy.

REMEMBER IT

Five sectors: E-H-A-B-L. Education, Healthcare, Agriculture, Business/banking, Local languages. Policy words: innovation, skills, ethics, digital growth.

COMMON MISTAKE

Mistake: Writing only “AI will make Pakistan modern” with no sector or example.

Fix: Name the National AI Policy 2025, pick two sectors from the table, and give a local example (rural hospital without a specialist; Urdu/Sindhi speech tools; bank fraud).

اردو میں سمجھیں

قومی اے آئی پالیسی 2025 کے مطابق مصنوعی ذہانت پاکستان کی منصوبہ بندی کا حصہ ہے: جدت، مہارت، اخلاقی استعمال اور ڈیجیٹل ترقی۔ تعلیم، صحت، زراعت، بینکنگ اور اردو و سندھی جیسی مقامی زبانوں میں اس کے اثرات ہوں گے۔ طلبہ کو پائتھن، ریاضی، ڈیٹا کا تجزیہ، مسئلہ حل کرنا اور ٹیکنالوجی کا اخلاقی استعمال سیکھنا چاہیے۔

CHECK YOURSELF

1. Name the national policy mentioned for AI in Pakistan.
2. Why is AI in healthcare especially important for rural areas?
3. List three skills students should learn to prepare for this future.

Answers: 1. National AI Policy 2025. 2. Those areas often lack specialist doctors; AI can help with disease detection and medical image analysis. 3. Any three of: programming (Python), mathematics, data analysis, problem-solving, ethical use of technology.

LEARN IT

Neural networks and deep learning are widely used in modern life. Many intelligent systems use them in the background to **recognise information, make predictions and support users**.

#	Application	What it does	Example
1	Image recognition	Identifies objects, faces or patterns in images. Used in security cameras, photo search and biometric locks.	Unlocking a smartphone using face recognition; a camera at a Karachi plaza matching a face with a staff list.

2	Speech recognition	Converts spoken words into digital text or commands. Deep learning helps with accents, pronunciation and voice patterns.	A student speaks to Google Assistant or Alexa to set a morning alarm; Urdu voice search on a phone.
3	Natural Language Processing (NLP)	Helps computers understand, process and respond to human language grammatically and in context. Core of chatbots, translators and AI writers.	Using Google Translate to turn a difficult English paragraph into Urdu; a bank chatbot answering in Urdu.
4	Healthcare systems	Analyse complex medical images (X-rays, CT scans, MRIs) to detect diseases early and accurately.	A hospital system highlighting abnormal patterns in a lung X-ray that might indicate pneumonia or tumours — useful at a DHQ hospital that has few radiologists.
5	Recommendation systems	Analyse past behaviour and preferences to suggest content, products or services.	A student watches computer-science videos on YouTube; the platform recommends more programming tutorials.
6	Autonomous vehicles	Self-driving cars use neural networks to process real-time camera and radar data: roads, signals, pedestrians, obstacles.	A self-driving car detects a pedestrian crossing and automatically applies the brakes.
7	Fraud detection	Banks compare current activity with previous spending patterns to catch unusual transactions.	An HBL or Meezan app flags a sudden Rs 80,000 payment from a new city at 3 a.m.

8	Assistive technologies	AI supports people with disabilities.	Speech-to-text tools for users who have difficulty typing; screen readers that speak Urdu text aloud.
---	-------------------------------	---------------------------------------	---

WORKED EXAMPLE

Worked example: match the app to the application name — a day in Ahmed's life in Hyderabad.

1. 7:00 am — phone unlocks by face. → **Image recognition.**
2. 7:05 am — "Ok Google, set alarm for Maghrib." → **Speech recognition.**
3. 8:00 am — Google Translate helps with an English article. → **NLP.**
4. 4:00 pm — YouTube suggests a Python tutorial after he watched Chapter 5 notes. → **Recommendation system.**
5. 9:00 pm — JazzCash texts "We blocked a payment that does not match your pattern." → **Fraud detection.**

REMEMBER IT

Eight applications: I-S-N-H-R-A-F-A. Image, Speech, NLP, Healthcare, Recommendations, Autonomous vehicles, Fraud, Assistive tech. "I See New Hospitals Recommend Autonomous Fraud-Assist."

COMMON MISTAKE

Mistake: Mixing up speech recognition and NLP.

Fix: Speech recognition turns **sound into text** (or a command). NLP understands the **meaning of language** (grammar, context, translation, chat replies). You can have NLP on typed text with no microphone at all.

اردو میں سمجھیں

نیورل نیٹ ورکس اور ڈیپ لرننگ کی مشہور ایپلی کیشنز یہ ہیں: تصویر کی پہچان، آواز کی پہچان، قدرتی زبان کی پروسیسنگ (NLP)، صحت کے نظام، تجویز کرنے والے نظام، خود چلنے والی گاڑیاں، فراڈ کی نشاندہی اور معاون ٹیکنالوجیز۔ روزمرہ میں چہرے سے فون کھولنا، یوٹیوب کی تجاویز اور بینک کا مشکوک لین دین پکڑنا سب اسی خاندان سے ہیں۔

CHECK YOURSELF

1. Which application converts spoken words into text or commands?
2. Give the source example of a recommendation system.
3. How do self-driving cars use neural networks?

Answers: 1. Speech recognition. 2. YouTube recommending more programming tutorials after a student watches computer-science videos. 3. They process real-time camera and radar data to understand roads, traffic signals, pedestrians and obstacles (for example braking for a pedestrian).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Vision applications include:

- a. Face unlock, medical images, number-plate reads
- b. Primary keys
- c. Bubble sort
- d. OSI ping

MCQ 2. Language applications include:

- a. Translation, speech-to-text, chatbots
- b. NAND grouping
- c. Foreign keys
- d. Pie charts of cables

MCQ 3. Healthcare DL must still:

- a. Be checked by clinicians
- b. Replace doctors unsupervised always
- c. Ignore bias
- d. Train on one patient

MCQ 4. A local example could be:

- a. Urdu voice assistants / crop images
- b. K-maps of farms
- c. Access reports of OSI
- d. Figma of NAND

MCQ 5. Recommendation systems are:

- a. ML/NN apps (what to watch/buy)
- b. Queues in printers only
- c. Stacks of plates only
- d. ER only

MCQ 6. The exam wants:

- a. Named field + how NN/DL helps + one risk
- b. Only the word Pakistan
- c. Only careers list
- d. Only math of backprop

Extra short questions

1. Four application areas from the book.
2. One risk in health.
3. One risk in security/surveillance.
4. Why "application" questions still need a definition line.

Extra long question

1. Pick vision, health, language. For each: system, data it needs, a benefit, a harm. 10-mark table.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Vision, speech, health, transport, security, education — any four with a concrete system.
2. False negative on an X-ray; bias against a skin tone.
3. Misidentification; privacy.
4. 1 mark for the term, rest for use+limit.

Long

1. Face gate / images / fast entry / racism in scores. X-ray / labelled scans / earlier catch / missed cases. Urdu STT / audio / inclusion / leaking class recordings.

5.3 Secure Collaboration (Authentication, Access Control)

★ Golden

LEARN IT

Digital tools allow people to **share files, attend online classes, edit documents and work together from different locations**. Collaboration is useful only when **data is protected** and **communication remains safe**.

Secure collaboration means using digital tools to work together in a **safe, controlled and responsible** way. It depends on several key elements.

i) Authentication / identity verification

Authentication is **confirming the identity of a user before allowing access**. Common methods:

- **Username and password**
- **PIN** (a short number code)
- **OTP — One-Time Password**, a code that works only once, usually sent by SMS or an authenticator app
- **Biometrics** — fingerprint or face

Source example: a **banking app sending an OTP to your phone before login**.

Multi-factor authentication (MFA) and two-factor authentication (2FA)

A **factor** is a category of proof:

- **Something you know** — password or PIN.
- **Something you have** — your phone, an OTP, a bank SMS.
- **Something you are** — fingerprint, face (biometrics).

Multi-factor authentication (MFA) means the system asks for **two or more different factors** before it trusts you. **Two-factor authentication (2FA)** is the common case of MFA that uses **exactly two** factors — usually password + OTP.

If a thief in another city steals only your password, MFA still blocks them because they do not have your phone (or your fingerprint).

ii) Access control

Access control decides **what an authenticated user is allowed to do**: View, Edit, Comment, Download, Share.

Authentication answers **"Who are you?"** Access control answers **"What may you do?"**

Source example: a teacher shares a Google Doc with students as **"view only"**, but gives group leaders **"edit"** permissions.

iii) Accountability and tracking

User actions are linked to **verified accounts**. You can see who opened, edited, deleted or shared a file. Example: a school **LMS** (Learning Management System) tracks which teacher uploaded an assignment and which student submitted it.

iv) Data privacy

Data privacy means protecting personal and sensitive information and using it **only for the purpose it was collected**. It involves:

- **Protection** — keeping names, numbers, marks and passwords secure.
- **Controlled sharing** — sharing only with the right people.
- **Confidentiality** — data stays private during storage and transmission.
- **Compliance** — following legal and ethical rules for data handling.

v) Safe communication / transmission

Exchanging information through digital platforms **securely**, protecting messages from unauthorised interception or wiretapping. Example: a teacher shares class instructions through an **official, encrypted LMS** rather than an open public forum or an unknown WhatsApp broadcast.

WORKED EXAMPLE

Worked example: a Class XII Google Classroom in Sukkur.

Step 1 — Authentication. Each student signs in with a school email and password. The college turns on MFA: after the password, Google sends an OTP to the student's registered Jazz/Zong number.

Step 2 — Access control. All 40 students can **view** the assignment sheet. Only the four group leaders can **edit** the shared report. Nobody except the teacher can **delete** the class materials. A student who left the college is removed from the class (access revoked).

Step 3 — Accountability. Version history shows that Ali edited paragraph 2 at 8:14 pm and that Hina submitted the PDF at 9:02 pm.

Step 4 — Privacy and safe transmission. Marks stay inside Classroom; the teacher does not post CNIC numbers or phone numbers in a public Facebook group. Traffic to Google is encrypted, unlike an open public forum.

If Ali shares his password with a cousin, authentication is broken. If the teacher gives every student "edit" on the marksheet, access control is broken even though login still works.

REMEMBER IT

A-A-A-P-S. Authentication, Access control, Accountability, Privacy, Safe transmission.
MFA = at least two different factor types (know / have / are). 2FA is MFA with two factors.

COMMON MISTAKE

Mistake: Treating authentication and access control as the same thing.

Fix: Login only proves identity. After login, access control still limits View / Edit / Comment / Download / Share. A student who is authenticated must not automatically get teacher rights.

اردو میں سمجھیں

محفوظ تعاون کا مطلب ہے ڈیجیٹل ٹولز سے محفوظ، کنٹرول شدہ اور ذمہ دارانہ انداز میں ساتھ کام کرنا۔ تصدیق (Authentication) یہ جانچتی ہے کہ صارف کون ہے؛ رسائی کنٹرول (Access Control) یہ طے کرتا ہے کہ وہ کیا کر سکتا ہے (صرف دیکھنا، ترمیم، تبصرہ)۔ کثیر عنصری تصدیق (MFA) دو یا زیادہ مختلف ثبوت مانگتی ہے، جیسے پاس ورڈ اور او ٹی پی (OTP)۔ حساب دہی، ڈیٹا پرائیویسی اور محفوظ ترسیل بھی ضروری ہیں۔

EXAM TIP

Typical items: "Define secure collaboration", "Differentiate authentication and access control", "What is MFA/2FA?", "Explain five elements of secure collaboration with examples."

Write the five headings (authentication, access control, accountability, privacy, safe transmission). For MFA, name the three factor types and say that 2FA is password + OTP in a banking app. Use the Google Doc "view only vs edit" example — examiners look for it.

CHECK YOURSELF

1. What question does authentication answer, and what question does access control answer?
2. Password + OTP is an example of which idea?
3. Give the source example of access control in a Google Doc.

Answers: 1. Authentication: who are you? Access control: what may you do? 2. Two-factor / multi-factor authentication (2FA/MFA). 3. Teacher shares the document as view-only with students but gives group leaders edit permission.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Authentication is:

- a. Proving who you are
- b. What you are allowed to do
- c. Encrypting a disk
- d. A pie chart

MCQ 2. MFA means:

- a. More than one factor (password + OTP/phone)
- b. Many files always
- c. Many Figma frames
- d. Many OSI layers

MCQ 3. Access control is:

- a. What an authenticated user may do
- b. The login spelling
- c. A histogram
- d. A stack

MCQ 4. Accountability needs:

- a. Actions tied to accounts (logs)
- b. Shared "class" password
- c. No names
- d. Disabled logs

MCQ 5. A shared Google-doc with "anyone can edit" fails:

- a. Access control / accountability
- b. Big O
- c. NAND
- d. Median

MCQ 6. This is ★: define collaboration risk, then:

- a. Auth, MFA, access control, tracking
- b. Only careers
- c. Only MVPs
- d. Only charts

Extra short questions

1. Define authentication vs access control.
2. Two MFA factors.
3. Why logs?
4. A class example of bad practice.

Extra long question

1. Design login for a college LMS: password rules, MFA, roles (student/teacher/admin), and what an audit log stores. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Auth: who. Access: what they can do.
2. Something you know (password) + have (phone OTP) / are (fingerprint).
3. After a leak you can see who opened the file.
4. One Facebook password on the projector.

Long

1. Roles table. Students cannot edit marks. Teachers cannot drop the DB. MFA for admin.
Logs: user, time, action, IP. No shared accounts.

5.3.2 Data Protection (Encryption, Passwords, Backup, Firewall)

LEARN IT

Data protection means keeping digital information safe from **unauthorised access, misuse, loss, damage or accidental deletion**.

1. Data encryption

Encryption changes readable data (**plaintext**) into a coded form called **ciphertext**.

Unauthorised users cannot understand ciphertext without the key.

- **Symmetric encryption** — the **same key** is used to encrypt and to decrypt.
- **Asymmetric encryption** — **two keys**: a **public key** to encrypt, a **private key** to decrypt.

A simple exam cipher is the **Caesar cipher** (a symmetric substitution cipher). You shift every letter a fixed number of places along the alphabet. Shift 3 is the classic key: A → D, B → E, ..., Z → C (the alphabet wraps around).

2. Strong passwords

A strong password includes **uppercase, lowercase, numbers and symbols**. It resists **brute-force** guessing (trying every possible combination). Source example: `Zk@2026#Pass`.

Weak: `ahmed123`, `pakistan`, `sindh`, your date of birth. Never reuse the same password on the college LMS and on JazzCash.

3. Backup and recovery — the 3-2-1 rule

A **backup** is a copy of data in a safe location (cloud or external drive) so you can **restore** it after hardware damage, ransomware or accidental deletion.

The **3-2-1 backup rule** is a practical standard:

- **3** copies of the data (the original plus two backups).
- **2** different types of storage media (for example laptop disk + USB, or disk + cloud).
- **1** copy **off-site** (a different building or the cloud), so a fire or theft at home does not destroy every copy.

4. Firewall and antivirus software

A **firewall** is a security **barrier** that **monitors network traffic** and **blocks suspicious internet access**. Think of it as a checkpoint on the road between your device (or college lab) and the internet.

Antivirus software detects and removes harmful software (viruses, spyware and other malware) from the **local machine**.

Firewall ≈ guards the **door to the network**. Antivirus ≈ searches **inside the house** for intruders already present.

WORKED EXAMPLE

Worked example A — Caesar cipher, shift = 3. Encrypt the plaintext **SINDH**, then decrypt the result.

Write the alphabet and count three letters forward. Wrap after Z back to A.

Plain	S	I	N	D	H
+3	V	L	Q	G	K

Ciphertext = **VLQ GK**.

Decrypt: shift each letter **back** by 3: V → S, L → I, Q → N, G → D, K → H. You recover **SINDH**.

Spaces and punctuation are often left unchanged. The same key (3) is used both ways, so Caesar is **symmetric**.

Second check: **CAT** + 3 → C→F, A→D, T→W → **FDW**.

```
def caesar(text, shift):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = ord("A") if ch.isupper() else ord("a")
            result += chr((ord(ch) - base + shift) % 26 + base)
        else:
            result += ch
    return result

print(caesar("SINDH", 3))
print(caesar("VLQ GK", -3))
print(caesar("CAT", 3))
```

OUTPUT

```
VLQ GK
SINDH
FDW
```

Worked example B — 3-2-1 for Fatima's Class XII project in Hyderabad.

1. **Copy 1 (original):** project folder on her laptop hard disk.
2. **Copy 2 (different media):** the same folder on a USB drive she keeps in her bag.
3. **Copy 3 (off-site):** uploaded to Google Drive (cloud servers are not in her house).

That is 3 copies, 2 media types (internal disk + USB, with cloud as the off-site copy), 1 off-site. When her brother deletes the laptop folder, she restores from USB or Drive.

Password check: Zk@2026#Pass has uppercase Z, lowercase letters, digits 2026, and symbols @ and #. fatima has none of those extras — reject it.

3 copies



2 media types



1 copy off-site

REMEMBER IT

Cipher = lock the message. Password = lock the door. 3-2-1 = spare keys in three places. Firewall = gate. Antivirus = indoor search. Caesar: shift letters; same key both ways (symmetric).

COMMON MISTAKE

Mistake 1: Shifting the wrong way, or forgetting wrap-around (X, Y, Z + 3 → A, B, C).

Mistake 2: Two copies on the same laptop (Documents and Desktop) and calling it 3-2-1. That is still one disk in one house.

Fix: Count letters on a written alphabet. For backups, different media and one copy that would survive a house fire.

اردو میں سمجھیں

ڈیٹا پروٹیکشن کا مطلب ہے معلومات کو غیر مجاز رسائی، غلط استعمال، نقصان یا حادثاتی حذف سے بچانا۔ انکرپشن پڑھے جانے والے متن کو سائفر ٹیکسٹ بنا دیتی ہے؛ سیزر سائفر میں ہر حرف کو مقررہ جگہوں سے آگے سرکایا جاتا ہے۔ مضبوط پاس ورڈ میں بڑے، چھوٹے حروف، عدد اور علامات ہوں۔ 1-2-3 بیک اپ: تین کاپیاں، دو قسم کے میڈیا، ایک کاپی باہر (آف سائٹ)۔ فائر وال نیٹ ورک ٹریفک روکتی ہے؛ اینٹی وائرس مقامی نقصان دہ سافٹ ویئر ہٹاتا ہے۔

CHECK YOURSELF

1. Encrypt PAK with a Caesar shift of 3.
2. What do the 3, the 2 and the 1 stand for in 3-2-1 backup?
3. In asymmetric encryption, which key encrypts and which key decrypts?

Answers: 1. SDN (P→S, A→D, K→N). 2. 3 copies, 2 different media, 1 off-site copy. 3. Public key encrypts; private key decrypts.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Encryption makes data:

- a. Unreadable without a key
- b. Faster to sort
- c. A primary key
- d. A pie

MCQ 2. A strong password is:

- a. Long, unique, not a word+123
- b. Ali123 for every site
- c. Written on the monitor
- d. OTP posted in the group

MCQ 3. Backup is:

- a. A second copy you can restore
- b. A stronger GPU
- c. A prettier UI
- d. A faster sort

MCQ 4. A firewall:

- a. Filters traffic in/out
- b. Encrypts Excel cells one-by-one always
- c. Sorts arrays
- d. Draws ER

MCQ 5. Ransomware defence includes:

- a. Offline/offline-ish backups
- b. Paying and hoping always
- c. No updates
- d. Shared admin

MCQ 6. Data protection is about:

- a. Confidentiality, integrity, availability (CIA idea)
- b. Only colours
- c. Only NAND
- d. Only sprints

Extra short questions

1. Four protection techniques from the notes.
2. Why unique passwords?
3. 3-2-1 backup idea (simple).
4. Firewall vs antivirus in one line.

Extra long question

1. A college lab is hit by ransomware. Using encryption, passwords, backup, firewall: what should already have been in place, and what do you do now (without paying if possible)?

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Encryption, strong passwords, backup, firewall (and updates, least privilege).
2. One leaked site should not open email+JazzCash.
3. More than one copy, not only on the same laptop.
4. Firewall: traffic gate. AV: looks for malware on the host (both useful).

Long

1. Before: backups offline, least admin, updates, firewall. Now: isolate PCs, restore from backup, change passwords, report. Paying is not a syllabus requirement to endorse.

5.4 Security Threats and Mitigation Techniques ★ Golden

LEARN IT

A **security threat** is any action, event or weakness that can **harm a computer system or its data**. **Mitigation techniques** are protective measures used to prevent threats or reduce their impact. This section names the common threats; later sections show how to spot them and how to reduce the damage.

#	Threat	What it is	Example
1	Malware	Harmful software designed to damage a computer, steal data, or gain unauthorised access. Includes viruses, worms, spyware, ransomware and Trojans .	Installing cracked software that secretly corrupts files, or a "free game" USB that encrypts a college lab's coursework (ransomware).
2	Phishing	A social-engineering trick that deceives users into sharing personal information (passwords, bank details, OTPs) using fake emails or websites that look genuine.	An SMS that looks like JazzCash or HBL asking you to "verify" by typing your OTP on a fake page.
3	Unauthorised access	Entering a system, account or file without permission. Often caused by weak passwords, shared passwords, or leaving devices unlocked.	A classmate using an unlocked lab PC to open someone else's Google Classroom.

4	Data theft / identity theft	Data theft is stealing or copying data without permission. Identity theft is using another person's stolen details (name, CNIC, bank details) to pretend to be them online and commit fraud.	A leaked list of CNICs used to open a SIM or a wallet in someone else's name.
5	Denial of Service (DoS)	A website or system is flooded with thousands of fake requests, causing it to slow down or crash, so real users cannot access the service.	Hackers flooding an online exam portal so students cannot log in on BISE result day or during an e-test.
6	Human error	Mistakes by legitimate users: accidentally deleting files, sending confidential data to the wrong email, or falling for a simple scam.	A teacher emailing the whole marksheet to the wrong parent; a student forwarding an OTP "to help the bank."

Malware types in one line each: a **virus** attaches to files and spreads when you share them; a **worm** spreads by itself across a network; **spyware** secretly watches you; **ransomware** locks your files and demands money; a **Trojan** pretends to be useful software but hides harm.

Social engineering means tricking a **person**, not breaking a lock with maths. Phishing is the most common exam example.

Phishing
Fake bank email / SMS
steals password or OTP

DoS attack
Flood of fake requests
server overload / crash

Identity theft
Stolen CNIC / details
fraud in your name

Three high-yield threats: phishing (fake messages), DoS (server overload), identity theft (stolen CNIC or bank details).

WORKED EXAMPLE

Worked example: classify the incident.

1. Bilal downloads a "cracked" Photoshop. Next day his project files are unreadable and a note asks for Bitcoin. → **Malware** (ransomware), started by cracked software.
2. Ayesha gets an email "Your Meezan account will close in 2 hours. Click here and send OTP." The link is meezn-pk.tk, not the real bank. → **Phishing**.
3. On e-test morning the college portal spins forever; the admin graph shows 200,000 requests from one pattern of fake traffic. → **DoS**.
4. Someone opens a JazzCash wallet using Ahmed's CNIC copy from a leaked form. → **Identity theft** (after data theft).
5. The lab assistant emails the password list to the whole staff by choosing the wrong group. → **Human error**.

REMEMBER IT

Malware = bad program. Phishing = fake message. DoS = flood. Identity theft = stolen 'you'. Human error = honest mistake. Cracked software and unknown USBs are the usual door for malware in Pakistani colleges.

COMMON MISTAKE

Mistake: Calling every cyber problem a "virus", or mixing phishing with malware.

Fix: A virus is only one kind of malware. Phishing is a **trick against the user**; it may not install any file at all. DoS does not steal your password — it makes the service unusable for everyone.

اردو میں سمجھیں

سیکیورٹی خطرہ وہ عمل یا کمزوری ہے جو نظام یا ڈیٹا کو نقصان پہنچاے۔ میلویئر نقصان دہ سافٹ ویئر ہے (وائرس، ورم، سپائی ویئر، رینسم ویئر، ٹروجن)۔ فہنگ جعلی ای میل یا ویب سائٹ سے پاس ورڈ اور او ٹی پی چھینتی ہے۔ ڈو ایس (DoS) جعلی درخواستوں کی بارش سے سائٹ کریش کر دیتا ہے۔ دیگر خطرات: غیر مجاز رسائی، ڈیٹا/شناخت کی چوری، اور انسانی غلطی۔

EXAM TIP

Expect "Define a security threat", "Explain malware / phishing / DoS with examples", and "Differentiate data theft and identity theft." List all six threats if the question says "common security threats." For DoS, write: flood of fake requests → slowdown or crash → real users blocked, and quote the online exam portal. For phishing, write social engineering + fake genuine-looking email/site + passwords/OTPs.

CHECK YOURSELF

1. Name five types of malware listed in the source.
2. What does a DoS attack do to a website?
3. How is identity theft different from data theft?

Answers: 1. Viruses, worms, spyware, ransomware, Trojans. 2. Floods it with thousands of fake requests so it slows down or crashes and real users cannot access it. 3. Data theft is stealing/copying data; identity theft is using stolen personal details (name, CNIC, bank) to pretend to be that person and commit fraud.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Malware is:

- a. Malicious software (virus, worm, trojan, ransomware...)
- b. A friendly backup
- c. A primary key
- d. A caption

MCQ 2. Phishing is:

- a. Tricking you into giving secrets (fake mail/site)
- b. A fishing sport in the syllabus
- c. A sort
- d. A chart

MCQ 3. DoS aims to:

- a. Make a service unavailable by flooding it
- b. Encrypt with your key
- c. Fix UX
- d. Normalise tables

MCQ 4. A threat is:

- a. Anything that can harm system or data
- b. Only earthquakes
- c. Only NAND delay
- d. Only slow Wi-Fi in exams

MCQ 5. This is ★: name, one example, one mitigation.

- a. The 3-mark recipe
- b. Only the name
- c. Only Python
- d. Only Figma

MCQ 6. A fake BIEK result link is likely:

- a. Phishing
- b. A K-map
- c. A stack
- d. A median

Extra short questions

1. Define malware, phishing, DoS.
2. One extra threat from the notes (theft/identity).
3. A student-level mitigation for phishing.
4. Why USB etiquette?

Extra long question

1. Table of four threats: what it is, a Sindh-college example, a sign, a mitigation. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Malware: bad software. Phishing: social trick. DoS: flood to outage.
2. Data/identity theft — using someone's CNIC/account.
3. Check URL, don't tap unknown SMS, official portal only.
4. Worms travel on sticks.

Long

1. Malware, phishing, DoS, theft. Signs: slowness, fake from-addresses, site down, unknown logins. Mitigations: updates/AV, URL check, backups+capacity, MFA.

5.4.2 Identify Security Threats

LEARN IT

Identifying security threats means **noticing signs** that data or systems may be at risk. Early identification helps you act before serious damage occurs.

Sign / indicator	What it likely means
Slow performance, missing files, or random pop-ups.	Malware or background unauthorised activity.
Failed login attempts or strange login locations.	Attempted unauthorised access .
Emails urging quick action, asking for an OTP, or containing bad grammar / strange links.	Classic phishing .
Unexpected deletion, copying or movement of records.	Data theft or insider misuse.
Sudden massive spike in server requests or a very slow website.	An ongoing DoS attack.

The first line of defence is also simple behaviour: avoid **unsafe downloads, cracked software, unverified websites and infected USB drives**.

WORKED EXAMPLE

Worked example: read the signs in a college computer lab in Khairpur.

1. Three PCs show pop-ups and Word files have vanished after students used a "game" USB. → Treat as **malware**. Disconnect from the network; run antivirus; stop USB use.
2. Google Classroom emails: "Your account will be deleted in 30 minutes. Confirm password here." Link spelling is go0gle. → **Phishing**. Do not click; report to the teacher.
3. The e-portal that was fine at 8:00 am is unreachable at 8:05; the technician sees a sudden spike in requests. → Suspect **DoS**.
4. Nadia's Gmail security page shows a login from another country at 4 a.m. → **Unauthorised access** attempt. Change password; turn on MFA.

REMEMBER IT

Slow/pop-ups → malware. Odd logins → unauthorised access. Urgent OTP email → phishing. Files moving → theft. Traffic spike → DoS.

COMMON MISTAKE

Mistake: Ignoring early signs ("the PC is just old") until the ransomware note appears.

Fix: One unusual login or one OTP email is enough to investigate. Early action is cheaper than recovery.

اردو میں سمجھیں

خطرے کی جلد پہچان نقصان سے پہلے اقدام ممکن بناتی ہے۔ سست روی، غائب فائلیں یا پاپ اپ میلویئر کی علامت ہیں؛ عجیب لاگ ان غیر مجاز رسائی کی؛ فوری او ٹی پی والی ای میل فشنگ کی؛ ریکارڈ کا اچانک منتقل ہونا ڈیٹا چوری کی؛ سرور پر درخواستوں کا دھماکہ ڈو ایس کی۔ کریک سافٹ ویئر، نامعلوم یو ایس بی اور غیر تصدیق شدہ ویب سائٹوں سے بچیں۔

CHECK YOURSELF

1. Random pop-ups and missing files most likely indicate what?
2. An email with bad grammar that demands your OTP is a sign of what?
3. Name four unsafe habits that invite these threats.

Answers: 1. Malware (or background unauthorised activity). 2. Phishing. 3. Unsafe downloads, cracked software, unverified websites, infected USB drives.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An IDE is:

- a. One app to write, run and debug code
- b. A printer
- c. A sort
- d. A cable

MCQ 2. VS Code is:

- a. A popular editor/IDE
- b. A DBMS
- c. An OSI layer
- d. A gate

MCQ 3. The terminal/run button is for:

- a. Executing the program
- b. Drawing pie charts only
- c. Sending TCP SYN only
- d. K-maps

MCQ 4. Syntax highlighting helps you:

- a. See keywords and mistakes
- b. Sort faster
- c. Encrypt disks
- d. Route packets

MCQ 5. A file explorer in VS Code shows:

- a. Project files
- b. OSI layer 1
- c. NAND groups
- d. Beachhead users

MCQ 6. Debugging in an IDE uses:

- a. Breakpoints and variable watch
- b. Only bubble sort
- c. Only pie charts
- d. Only Agile stickers

Extra short questions

1. Define IDE.
2. Name four VS Code areas.
3. Why not use Notepad only?
4. What is an extension?

Extra long question

1. Describe how you would create, run and debug a first Python file in VS Code. Include two interface elements from the notes.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Integrated Development Environment: editor + run + debug + files in one place.

2. Activity bar/sidebar (files), editor, terminal, status bar (and extensions).
3. No easy run, no highlighting, no debugger.
4. An add-on (Python plugin) that teaches VS Code a language.

Long

1. New file .py, type print, Run Python, read terminal. Mention top menu, explorer, terminal panel. Breakpoint on a wrong variable.

5.4.3 Apply Mitigation Techniques

LEARN IT

Mitigation techniques are actions taken to **prevent** security threats or **reduce their harmful effects** once identified. Technology helps, but it cannot stop all threats if the user is careless.

#	Technique	How it helps
1	Antivirus and firewalls	Antivirus scans and removes harmful software (for example from USB drives). A firewall monitors network traffic and blocks suspicious access from the internet.
2	Strong passwords and 2FA	Complex passwords slow brute-force guessing. 2FA adds an extra layer: an OTP on your phone as well as the password.
3	Data encryption	Converts data into unreadable code during transmission. Protects passwords and sensitive data if they are intercepted.
4	Regular backups	Copies on cloud or external drives allow quick recovery after a crash, human error or ransomware.
5	User awareness (the human factor)	Training users to avoid unknown links, suspicious attachments and password sharing. The human is often the weakest link — and the strongest defence when trained.

WORKED EXAMPLE

Worked example: match each threat to a mitigation — BISE e-test week at a college in Larkana.

Threat seen	Mitigation to apply
USB games in the lab, pop-ups starting.	Scan with antivirus ; block unknown USBs; enable the lab firewall .
Phishing SMS about "update your portal password."	User awareness : do not click; verify on the official site typed by hand. Turn on 2FA .
Fear that intercepted Wi-Fi could show passwords.	Use sites with encryption (HTTPS / encrypted LMS); never send passwords in plain WhatsApp.
Ransomware risk to project files.	Regular 3-2-1 backups so files can be restored without paying.
Shared passwords on a sticky note on the monitor.	Strong unique passwords + 2FA; remove the note; awareness session.

REMEMBER IT

Five shields: A-F, P-2, E, B, U. Antivirus+Firewall, Passwords+2FA, Encryption, Backups, User awareness. Ransomware is why backups matter: if files are locked, a clean copy brings them back.

COMMON MISTAKE

Mistake: "If I installed antivirus I am 100% safe."

Fix: Antivirus does not stop you from typing an OTP into a fake page. Mitigation is a **set** of techniques plus human caution.

اردو میں سمجھیں

تحفظ کی تدابیر خطرات کو روکتی یا ان کا نقصان کم کرتی ہیں: اینٹی وائرس اور فائر وال، مضبوط پاس ورڈ اور دو عنصری تصدیق (2FA)، انکریپشن، باقاعدہ بیک اپ، اور صارف کی آگاہی۔ ٹیکنالوجی تب ناکام ہو جاتی ہے جب صارف نامعلوم لنک کھولے یا پاس ورڈ بانٹے۔

CHECK YOURSELF

1. How does 2FA add a layer on top of a password?
2. Why do regular backups help after ransomware?
3. What three careless behaviours should awareness training stop?

Answers: 1. It also requires an OTP sent to your phone (a second factor). 2. You can restore a clean copy instead of losing the files (or paying). 3. Unknown links, suspicious attachments, and password sharing.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Mitigation means:

- a. Prevent or reduce harm from threats
- b. Ignore them
- c. Rename the threat
- d. Draw them in Figma

MCQ 2. Updates/patches:

- a. Close known holes
- b. Always break Python
- c. Replace HCI
- d. Replace ER

MCQ 3. Least privilege means:

- a. Only the rights you need
- b. Everyone is admin
- c. No passwords
- d. No backups

MCQ 4. User education fights:

- a. Phishing / bad USB / weak passwords
- b. Cosmic rays only
- c. NAND delay
- d. Median bias only

MCQ 5. Incident plan includes:

- a. Isolate, preserve, restore, report
- b. Panic format twice
- c. Pay first always
- d. Post OTPs

MCQ 6. Technical + human controls together are:

- a. Stronger than either alone
- b. Illegal
- c. Useless
- d. Only for banks

Extra short questions

1. Five mitigation techniques from the book.
2. Map one technique to phishing, one to ransomware, one to DoS.
3. What not to do?
4. Why backups must be tested?

Extra long question

1. Write a one-page lab policy: 8 rules that mitigate the threats in 5.4.1. Tag each rule with the threat.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Updates, AV, firewall, backup, MFA, training, least privilege, encryption — any five.
2. Phishing: training+MFA. Ransomware: backup. DoS: filtering/capacity (high level).
3. Share the malware “to warn friends” as an attachment.
4. A backup you cannot restore is theatre.

Long

1. No shared admin (malware/theft); USB scan; official links (phishing); weekly backup (ransomware); firewall; MFA email; report SMS; no pirated cracks. Table.

5.5 Equity and Equal Access in Digital Collaboration

LEARN IT

Collaboration tools are vital for online learning and teamwork. To use these tools fairly, all users must participate **without barriers**. You must understand the difference between **equal access** and **equity**.

Equal access

Giving everyone the **exact same** opportunity, tools and resources.

Example: providing every student with the **same login ID** to access a Zoom class.

Equity

Giving **additional, specific support** to users who need it, so they can reach the **same outcome**.

Example: turning on live **subtitles/captions** in Zoom specifically for a student who has hearing difficulties.

Equal access is the same starting kit. Equity is extra help so a person who faces a barrier can still finish the race with everyone else. Fair collaboration needs **both**: the same Zoom link for all (equal access) **and** captions, recorded lectures or a low-data app where needed (equity).

WORKED EXAMPLE

Worked example: one class, two ideas. A teacher in Thatta starts online classes.

Equal access: every student receives the same Zoom ID and password. That is fair in the sense of "same tool," but a student with hearing loss still cannot follow the spoken lecture, and a student in a village with a weak 2G signal cannot join video.

Equity: the teacher turns on captions, posts a low-size audio recording, and accepts the assignment as a Google Doc in the browser (not only through a heavy app). Now those two students can reach the **same learning outcome**.

REMEMBER IT

Equal = same tools. Equity = extra support for the same outcome. Same shoes for all vs a ramp for the wheelchair user.

COMMON MISTAKE

Mistake: "We gave everyone the same login, so the class is fair."

Fix: Same login is equal access only. If some students still cannot hear, see, afford a smartphone, or get electricity, you still need equity.

مساوی رسائی (Equal Access) سب کو ایک جیسے اوزار دیتی ہے، جیسے ایک ہی زوم لاگ ان۔ انصاف (Equity) ضرورت مند کو اضافی مدد دیتی ہے تاکہ وہی نتیجہ نکل سکے، جیسے سماعت کی مشکل والے طالب علم کے لیے زندہ ذیلی عنوانات (Captions)۔ منصفانہ تعاون کے لیے دونوں چاہییں۔

CHECK YOURSELF

1. Define equal access in one sentence.
2. Define equity in one sentence.
3. Why are Zoom captions an example of equity, not merely equal access?

Answers: 1. Giving everyone the exact same opportunity, tools and resources. 2. Giving additional, specific support so users who need it can reach the same outcome. 3. Because they are extra support for a student with hearing difficulties, not the same identical tool given to all.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Equity in digital collaboration means:

- a. Fair access to tools and participation, not only "same link for all"
- b. Giving everyone the identical laptop always without support
- c. Banning assistive tech
- d. English-only 4pt PDFs

MCQ 2. Equal access includes:

- a. Devices, connectivity, language, disability support
- b. Only the top 10 students
- c. Only computer labs at midnight
- d. Only paid apps

MCQ 3. A student on 2G with no laptop needs:

- a. Low-data options / lab time
- b. A 4K video lecture only
- c. A VR headset
- d. A GPU DL assignment first

MCQ 4. Collaboration tools fail equity when:

- a. They require an expensive phone feature
- b. They have captions and Urdu
- c. They work on a lab PC
- d. They allow keyboard only

MCQ 5. This is ★ for 5.5.1: functions of equity are:

- a. Inclusion, fairness of voice, accessible tools, support
- b. Faster CPUs
- c. Shorter OSI
- d. Nicer NAND

MCQ 6. A fair group project:

- a. Roles + shared docs + offline option
- b. One member who owns the only laptop does all
- c. Voice chat only at 2am
- d. Uncaptioned 1-hour video mandatory

Extra short questions

1. Define equity vs "everyone got the WhatsApp".
2. Functions of equity (book).
3. Two practical college moves.
4. Link to XI assistive tech.

Extra long question

1. A group of 5: one blind, one with no smartphone, one in a village on weekends. Design the collaboration so all can submit. 10 marks ★.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Same message is not the same opportunity if some have no data bundle or cannot hear.
2. Access, participation, support, accessible design, respect.
3. Lab hours; printed alternative; captions; Urdu UI; extra time.
4. Screen readers + accessible docs are equity, not charity.

Long

1. Shared doc that works in lab PCs; keyboard/screen reader; tasks that don't need 2am Zoom; recorded captions; warden-lab timetable. Assessment not based on who has 4G.

5.5.1 Functions of Equity and Equal Access ★ Golden

LEARN IT

Equity and equal access are not slogans. They **do work** in a digital classroom. The source lists four functions.

Function	What it does	Pakistani example
1. Providing need-based support (equity)	Help tailored to individuals.	Audio descriptions for visually impaired students; a screen reader so a blind student in Karachi can hear a digital textbook.
2. Removing barriers	Help students in remote areas attend online classes.	Mobile-friendly, low-data apps so a student in Tharparkar or the katcha areas of Sindh can join without a fibre connection.
3. Enabling active participation	Students who miss live class can still take part.	Recorded lectures so load-shedding or an internet outage in Hyderabad does not make a student fall behind.
4. Supporting different learning needs	Offer more than one format.	Videos, text notes and audio of the same Chapter 5 topic so each student learns in the way that suits them best.

WORKED EXAMPLE

Worked example: design four functions for one class WhatsApp + Classroom group in Nawabshah.

1. **Need-based support:** Ali is visually impaired — share materials as selectable text (not only photos) so a screen reader can speak them; add a short audio description of each diagram.
2. **Remove barriers:** post a 5 MB audio summary instead of a 200 MB video for students on expensive mobile data.
3. **Active participation:** upload the Zoom recording within an hour; students whose lights went out at 7 pm watch it at 11 pm.
4. **Different learning needs:** the same lesson as (a) a 12-minute video, (b) a one-page PDF, (c) an Urdu audio note.

Discussion from the source: if a teacher **only** accepts assignments via an app that needs a **high-end smartphone**, is that equal access? **No.** It creates a **hardware barrier** for lower-income students. Web-based or light apps give better access.

REMEMBER IT

Four functions: Support, Remove, Participate, Formats. “Some Rural Pupils Fail” without them — Support (need-based), Remove barriers, Participate via recordings, Formats (video/text/audio).

COMMON MISTAKE

Mistake: Describing only captions and stopping.

Fix: Examiners want all four functions. Include remote-area low-data apps, recorded lectures for power cuts, and multiple formats — these are in the Sindh source on purpose.

اردو میں سمجھیں

انصاف اور مساوی رسائی کے چار کام ہیں: ضرورت کے مطابق مدد (جیسے نابینا طالب علم کے لیے آڈیو وضاحت)، رکاوٹیں ہٹانا (دور دراز علاقوں کے لیے کم ڈیٹا والی ایپس)، فعال شرکت ممکن بنانا (لوڈ شیڈنگ کے لیے ریکارڈ شدہ لیکچر)، اور مختلف اندازِ تعلیم کی حمایت (ویڈیو، متن، آڈیو)۔ مہنگے فون والی ایپ پر صرف اسی طرح اسائنمنٹ لینا مساوی رسائی نہیں۔

EXAM TIP

Golden question: “Explain the functions of equity and equal access in digital collaboration” (5–8 marks). Write the four headings in order, one example each, and add the hardware-barrier point (high-end app vs light/web app). Another favourite: “Differentiate equal access and equity with examples” — same login vs captions.

CHECK YOURSELF

1. List the four functions of equity and equal access.
2. How do recorded lectures enable active participation in Pakistan?
3. Why is a high-end-smartphone-only assignment unfair?

Answers: 1. Need-based support; removing barriers; enabling active participation; supporting different learning needs. 2. Students who face power or internet outages can watch later and not fall behind. 3. It creates a hardware barrier for lower-income students; web-based or light apps give better access.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Equity in digital collaboration means:

- a. Fair access to tools and participation, not only "same link for all"
- b. Giving everyone the identical laptop always without support
- c. Banning assistive tech
- d. English-only 4pt PDFs

MCQ 2. Equal access includes:

- a. Devices, connectivity, language, disability support
- b. Only the top 10 students
- c. Only computer labs at midnight
- d. Only paid apps

MCQ 3. A student on 2G with no laptop needs:

- a. Low-data options / lab time
- b. A 4K video lecture only
- c. A VR headset
- d. A GPU DL assignment first

MCQ 4. Collaboration tools fail equity when:

- a. They require an expensive phone feature
- b. They have captions and Urdu
- c. They work on a lab PC
- d. They allow keyboard only

MCQ 5. This is ★ for 5.5.1: functions of equity are:

- a. Inclusion, fairness of voice, accessible tools, support
- b. Faster CPUs
- c. Shorter OSI
- d. Nicer NAND

MCQ 6. A fair group project:

- a. Roles + shared docs + offline option
- b. One member who owns the only laptop does all
- c. Voice chat only at 2am
- d. Uncaptioned 1-hour video mandatory

Extra short questions

1. Define equity vs "everyone got the WhatsApp".
2. Functions of equity (book).
3. Two practical college moves.
4. Link to XI assistive tech.

Extra long question

1. A group of 5: one blind, one with no smartphone, one in a village on weekends. Design the collaboration so all can submit. 10 marks ★.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Same message is not the same opportunity if some have no data bundle or cannot hear.
2. Access, participation, support, accessible design, respect.
3. Lab hours; printed alternative; captions; Urdu UI; extra time.
4. Screen readers + accessible docs are equity, not charity.

Long

1. Shared doc that works in lab PCs; keyboard/screen reader; tasks that don't need 2am Zoom; recorded captions; warden-lab timetable. Assessment not based on who has 4G.

5.5.2 Collaboration Tools

LEARN IT

Digital collaboration tools are online platforms that support **communication, file sharing, learning and teamwork** across different locations.

Tool	Primary function and educational use
Google Docs	Write, edit, comment and share documents in real time. Excellent for group assignments.
Zoom	Online meetings, virtual classes, screen sharing and video-based communication.
Microsoft Teams	A unified workspace for online meetings, file sharing, class groups and structured teamwork.
Google Classroom / LMS	Learning Management Systems help teachers manage lectures, distribute assignments, conduct quizzes and grade student activities.
WhatsApp	Quick class announcements, urgent reminders and instant group communication. Must be used responsibly.
Email	Formal communication, official notices, submitting large attachments, and keeping a professional written record.

Responsibly on WhatsApp means: no leaking of classmates' marks or photos, no fake forwards, and no forcing a student who has no smartphone to miss official work that was never posted on the LMS.

WORKED EXAMPLE

Worked example: choose the right tool for each job in a Hyderabad college group project.

1. Write the report together after school, from different houses. → **Google Docs** (real-time edit and comments).
2. Sunday 7 pm teacher explanation with slides. → **Zoom** or **Teams** (video + screen share). Turn on captions (equity).
3. Collect homework, quiz and grades in one place. → **Google Classroom / LMS**.
4. "Tomorrow's test is postponed — tell everyone in the next ten minutes." → **WhatsApp** class group (then repeat on Classroom so nobody is left out).
5. Send the principal a formal complaint with a 8 MB scanned letter. → **Email** (written record + attachment).

REMEMBER IT

Docs = write together. Zoom/Teams = meet. Classroom = official class hub. WhatsApp = fast but informal. Email = formal record.

COMMON MISTAKE

Mistake: Running the whole course only on WhatsApp.

Fix: WhatsApp is for quick reminders. Assignments, marks and lectures belong on an LMS so there is access control, a record, and a path that does not depend on one person's phone.

اردو میں سمجھیں

ڈیجیٹل تعاون کے اوزار مختلف مقامات سے بات چیت، فائل شیئرنگ اور ٹیم ورک چلاتے ہیں: گوگل ڈاکس مشترکہ تحریر کے لیے، زوم اور مائیکروسافٹ ٹیمز میٹنگ کے لیے، گوگل کلاس روم یا ایل ایم ایس باقاعدہ پڑھائی کے لیے، واٹس ایپ فوری اطلاع کے لیے (ذمہ داری سے)، اور ای میل رسمی ریکارڈ کے لیے۔

CHECK YOURSELF

1. Which tool is excellent for real-time group writing?
2. What four jobs does an LMS / Google Classroom help a teacher do?
3. What is email especially good for?

Answers: 1. Google Docs. 2. Manage lectures, distribute assignments, conduct quizzes, grade activities. 3. Formal communication, official notices, large attachments, and a professional written record.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Collaboration tools include:

- a. Docs, chat, video, boards, LMS
- b. Only NAND trainers
- c. Only K-maps
- d. Only copper

MCQ 2. A good tool match: editing a report together →

- a. Shared document
- b. Only SMS
- c. Only a pie chart
- d. Only a stack

MCQ 3. Video class needs:

- a. Captions + low-bandwidth option
- b. 4K only
- c. No mute
- d. Forced camera in hostels always

MCQ 4. Permissions on a shared folder are:

- a. Access control (Ch 5.3)
- b. A histogram
- c. A median
- d. A sprint

MCQ 5. Chat is poor for:

- a. Final marks sheets (no audit, easy leak)
- b. Quick questions
- c. Memes the teacher banned
- d. Reminders

MCQ 6. Choose tools after:

- a. The task and the weakest connection in the group
- b. The trendiest brand
- c. The teacher's cousin's startup
- d. The GPU

Extra short questions

1. Four tool types and a school use.
2. One security setting per tool.
3. One equity setting.
4. When not to use public WhatsApp.

Extra long question

1. Plan a 1-week group CS project using three tools. For each: purpose, permission, equity tweak, security tweak.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Docs: assignment. Chat: quick. Video: guest lecture. LMS: submit. Board: brainstorm.
2. Docs: restricted link. Video: waiting room. Chat: no OTP sharing.
3. Captions; allow phone upload not only desktop app.
4. CNIC photos, marks, minors' data.

Long

1. LMS submit; Google Doc restricted; recorded Meet with captions. No personal numbers in the file. Lab slot for the student without data. Table.

Application and Impacts of Computing

15 single-page lessons · 5 Golden · read, close, say aloud, then sit the exam

Chapter in one look

- **ML** is a model (a trained brain) that learns from data in three steps: input, learning, prediction on new unseen data — without a hand-written rule for every case.
- **AI $\supset$ ML $\supset$ DL**: deep learning sits inside machine learning, which sits inside artificial intelligence.
- A **neural network** is a brain-inspired mathematical model of connected neurons in an input layer, hidden layer(s) and output layer (face unlock: camera $\rightarrow$ features $\rightarrow$ unlock).
- **Deep learning** trains neural networks with many hidden layers; early layers learn simple features, deeper layers learn complex patterns. It needs huge data and computing power.
- NN components: input layer, neurons, hidden layers, weights, bias, activation function, output layer, loss function. A neuron computes $s = \sum(x \times w) + b$, then an activation gate.
- DL components: large data, many hidden layers, automatic feature learning, learning algorithm (backpropagation), GPUs, training epochs. Pakistan's National AI Policy 2025 covers education, health, agriculture, banking and local languages.
- Applications: image and speech recognition, NLP, healthcare imaging, recommendations, autonomous vehicles, fraud detection, assistive tech.
- **Secure collaboration**: authentication (who), MFA/2FA (two or more factors), access control (what you may do), accountability, privacy, safe transmission.
- **Data protection**: encryption (plaintext $\rightarrow$ ciphertext; symmetric vs asymmetric; Caesar shift), strong passwords, 3-2-1 backup, firewall and antivirus.
- Threats: malware, phishing, unauthorised access, data/identity theft, DoS, human error. Spot them by pop-ups, odd logins, urgent OTP emails, missing files, traffic spikes.
- Mitigation: antivirus + firewall, strong passwords + 2FA, encryption, regular backups, user awareness.
- **Equal access** = same tools for all; **equity** = extra support for the same outcome. Functions: need-based support, removing barriers, recorded participation, multiple formats. Tools: Docs, Zoom, Teams, Classroom/LMS, WhatsApp, email.

Key terms

Term	Meaning	اردو
Artificial intelligence (AI)	The broad field in which machines perform tasks that normally need human thinking.	مصنوعی ذہانت
Machine learning (ML)	A model that learns patterns from data and then predicts on new data.	مشین لرننگ
Neural network (NN)	A brain-inspired mathematical model of connected neurons in layers.	نیورل نیٹ ورک
Deep learning (DL)	An ML method that trains neural networks with many hidden layers.	ڈیپ لرننگ
Weight	A number showing the strength or importance of a connection between neurons.	وزن (وزنِ تعلق)

Bias	An extra constant added to a neuron's sum to improve accuracy.	بائیس
Activation function	A gate that decides whether a signal is important enough to pass on.	ایکٹیویشن فنکشن
Loss function	A measure of how wrong the model's answer is compared with reality.	لاس فنکشن
Epoch	One full training pass through the dataset.	ایپوک (تربیتی چکر)
GPU	Graphics Processing Unit — specialised hardware used to train deep networks.	جی پی یو
NLP	Natural Language Processing — computers understanding and producing human language.	قدرتی زبان کی پروسپسنگ
Authentication	Confirming a user's identity before access (password, PIN, OTP, biometrics).	تصدیق شناخت

CHAPTER 5 EXAM • CLOSED BOOK

Application and Impacts of Computing

75 marks • 1 hour 15 minutes • no notes • no phones

CHAPTER EXAM RULES

Instructions. Sit this exam after Lesson 15 and the recap. **Section A** — all 15 MCQs (1 mark). **Section B** — all 10 short questions (3 marks). **Section C** — attempt any **three** long questions (10 marks). At least one Section C answer must be a ★ Golden topic. Write in your notebook. Do not open the key until time is up.

Marks. 15 + 30 + 30 = 75. A 5-mark / 10-mark answer needs Define • Explain • Example • Diagram • Working.

سبق اور خلاصے کے بعد یہ امتحان بغیر نوٹس کے حل کریں۔ وقت پورا ہونے پر کلید کھولیں۔

Section A — Multiple choice (15 × 1 = 15)

1. A machine-learning model works in which order?

- a. Prediction, then data input, then learning
- b. Data input, learning, then prediction on new data
- c. Only prediction, with no data
- d. Writing a separate rule for every case

2. A neural network is best described as:
 - a. a living human brain placed in a computer
 - b. a backup disk
 - c. a brain-inspired mathematical model of connected neurons
 - d. a type of firewall
3. The word "deep" in deep learning refers to:
 - a. several hidden layers through which data passes
 - b. very loud speakers
 - c. using only one neuron
 - d. storing data underground
4. In the relationship $AI \supset ML \supset DL$, the innermost set is:
 - a. AI
 - b. ML
 - c. equal access
 - d. DL
5. Weights in a neural network show the:
 - a. colour of the screen
 - b. importance or strength of connections between neurons
 - c. size of the hard disk
 - d. student's roll number
6. For $x_1 = 1$, $x_2 = 0$, $w_1 = 0.5$, $w_2 = 0.5$, $b = 0$ and threshold 0.4, the neuron output y is:
 - a. 1
 - b. 0
 - c. 0.4
 - d. 2
7. Which of the following is a component of deep learning (not of a tiny shallow network)?
 - a. Exactly one hidden layer and ten rows of data
 - b. Manual listing of every feature by a human, always
 - c. Many hidden layers, huge datasets and GPUs
 - d. A Caesar cipher
8. Unlocking a phone by matching facial features is mainly an example of:
 - a. a DoS attack
 - b. 3-2-1 backup
 - c. equal access
 - d. image recognition
9. Authentication means:
 - a. choosing who may edit a file
 - b. confirming a user's identity before allowing access
 - c. shifting letters in a cipher
 - d. flooding a server with requests
10. Password plus an OTP sent to a phone is an example of:
 - a. MFA / 2FA
 - b. a worm
 - c. a shallow neural network
 - d. equity captions

- .1. With a Caesar cipher of shift 3, the plaintext CAT becomes:
a. DBU
b. CAW
c. FDW
d. ZXQ
- .2. The 3-2-1 backup rule requires:
a. three passwords only
b. one copy on the same laptop twice
c. no off-site copy
d. three copies, two media types, one copy off-site
- .3. Phishing is:
a. a social-engineering trick using fake emails or sites to steal passwords or OTPs
b. a type of GPU
c. an activation function
d. a collaboration tool like Zoom
- .4. A Denial of Service (DoS) attack:
a. always steals a CNIC
b. floods a system with fake requests so it slows or crashes
c. is the same as a strong password
d. is a hidden layer
- .5. Turning on live captions in Zoom for a student with hearing difficulties is an example of:
a. malware
b. a DoS attack
c. equal access only, with no extra support
d. equity

Section B — Short questions (10 × 3 = 30)

1. Define a machine-learning model and state the three steps of how it works.
2. What is a neural network? Name its three kinds of layer and link them to face unlock.
3. Differentiate a neural network (the model) from deep learning (the method).
4. State the relationship $AI \supset ML \supset DL$ in words.
5. List eight components of a neural network. What do weights and bias do?
6. Name six components of deep learning.
7. Differentiate authentication, MFA and access control with one example each.
8. Encrypt SINDH using a Caesar cipher with shift 3. State whether this cipher is symmetric or asymmetric.
9. Define malware, phishing and DoS. Give one Pakistani-style example of each.
- .0. Differentiate equal access and equity. List four functions of equity and equal access.

Section C — Long questions (attempt any 3 × 10 = 30)

1. Explain machine learning, neural networks and deep learning. Draw and label the set diagram $AI \supset ML \supset DL$. Include the face-unlock walk-through and the photo-gallery

example.

2. A neuron has inputs $x_1 = 0.8$, $x_2 = 0.5$, weights $w_1 = 0.4$, $w_2 = 0.6$, bias $b = 0.1$ and threshold 0.5. (a) Compute the weighted sum s and the output y . (b) Repeat for $x_1 = 0.2$, $x_2 = 0.3$. (c) Name all eight NN components and sketch a 3–4–2 network. (d) List the six DL components and map them onto a cotton-leaf disease detector in Sindh.
3. Describe eight applications of neural networks and deep learning with examples. Then discuss the future of AI in Pakistan with reference to the National AI Policy 2025 and five sectors.
4. What is secure collaboration? Explain authentication, MFA/2FA, access control, accountability, data privacy and safe transmission, using a Google Classroom / banking-app scenario from Pakistan.
5. Define data protection. Explain encryption (symmetric vs asymmetric) with a full Caesar-cipher working of SINDH (shift 3). Describe strong passwords, the 3-2-1 backup rule, firewall and antivirus.
6. Define a security threat. Explain malware, phishing, unauthorised access, data/identity theft, DoS and human error. Give a table of identifying signs and a matching set of mitigation techniques. End with equity versus equal access and a fair choice of collaboration tools.

Answer key. Finish the Chapter Exam first, then mark. Do not peek while the clock is running.

پہلے امتحان مکمل کریں، پھر اس کلید سے نمبر لگائیں۔

Check yourself — answers

1. **5.1 Machine Learning, Neural Networks & Deep Learning.** Answers: 1. AI ($AI \supset ML \supset DL$). 2. Yes — DL is inside ML, and ML is inside AI. 3. Any of: face unlock, speech recognition, photo search, medical diagnosis, language translation.
2. **5.1.1 Machine Learning (ML).** Answers: 1. Data input, learning, prediction. 2. The trained brain of a computer program. 3. New, unseen data.
3. **5.1.2 Neural Networks (NN).** Answers: 1. Neurons or nodes. 2. Hidden layers. 3. No — it is inspired by the brain but works as a mathematical computer system.
4. **5.1.3 Deep Learning (DL).** Answers: 1. Because data passes through several hidden layers. 2. Early: simple features (edges, shapes, colours). Deeper: complex patterns (a whole face, a car model). 3. No — that is a shallow network; DL always uses many hidden layers.
5. **5.1.4 Components of Neural Networks.** Answers: 1. The importance or strength of connections between neurons. 2. $s = 1 \times 0.5 + 0 \times 0.5 + 0 = 0.5$; $0.5 \geq 0.4$ so $y = 1$. 3. The error — how much the model's answer is wrong compared with reality.
6. **5.1.5 Components of Deep Learning.** Answers: 1. It needs massive datasets (thousands or millions of records) for accuracy. 2. One training loop — one full pass through the training data. 3. GPUs (Graphics Processing Units); backpropagation.
7. **5.2 Applications of Neural Networks and Deep Learning.** Answers: 1. National AI Policy 2025. 2. Those areas often lack specialist doctors; AI can help with disease detection and medical image analysis. 3. Any three of: programming (Python), mathematics, data analysis, problem-solving, ethical use of technology.
8. **5.3 Secure Collaboration (Authentication, Access Control).** Answers: 1. Authentication: who are you? Access control: what may you do? 2. Two-factor / multi-factor authentication (2FA/MFA). 3. Teacher shares the document as view-only with students but gives group leaders edit permission.
9. **5.3.2 Data Protection (Encryption, Passwords, Backup, Firewall).** Answers: 1. SDN ($P \rightarrow S$, $A \rightarrow D$, $K \rightarrow N$). 2. 3 copies, 2 different media, 1 off-site copy. 3. Public key encrypts; private key decrypts.
10. **5.4 Security Threats and Mitigation Techniques.** Answers: 1. Viruses, worms, spyware, ransomware, Trojans. 2. Floods it with thousands of fake requests so it slows down or crashes and real users cannot access it. 3. Data theft is stealing/copying data; identity theft is using stolen personal details (name, CNIC, bank) to pretend to be that person and commit fraud.
11. **5.4.2 Identify Security Threats.** Answers: 1. Malware (or background unauthorised activity). 2. Phishing. 3. Unsafe downloads, cracked software, unverified websites, infected USB drives.
12. **5.4.3 Apply Mitigation Techniques.** Answers: 1. It also requires an OTP sent to your phone (a second factor). 2. You can restore a clean copy instead of losing the files (or paying). 3. Unknown links, suspicious attachments, and password sharing.
13. **5.5 Equity and Equal Access in Digital Collaboration.** Answers: 1. Giving everyone the exact same opportunity, tools and resources. 2. Giving additional, specific support so users who need it can reach the same outcome. 3. Because they are extra support for a student with hearing difficulties, not the same identical tool given to all.
14. **5.5.1 Functions of Equity and Equal Access.** Answers: 1. Need-based support; removing barriers; enabling active participation; supporting different learning needs. 2. Students who face power or internet outages can watch later and not fall behind. 3. It creates a hardware barrier for lower-income students; web-based or light apps give better access.

15. **5.5.2 Collaboration Tools.** Answers: 1. Google Docs. 2. Manage lectures, distribute assignments, conduct quizzes, grade activities. 3. Formal communication, official notices, large attachments, and a professional written record.

Answer key

Multiple choice

- | | | | | |
|------|------|------|-------|-------|
| 1. b | 4. d | 7. c | 10. a | 13. a |
| 2. c | 5. b | 8. d | 11. c | 14. b |
| 3. a | 6. a | 9. b | 12. d | 15. d |

Short questions — model answers

1. An ML model is the system that learns from data and gives an output — the trained brain of a program. Steps: (1) data input — massive data is given; (2) learning — it studies data and finds patterns/rules automatically; (3) prediction — after training it decides on new, unseen data.
2. A neural network is a computer model that learns patterns from data; it is inspired by the human brain but works as a mathematical system of connected neurons/nodes. Input layer receives the camera image; hidden layers study features such as eye distance and nose shape; output layer unlocks the phone if they match stored data.
3. A neural network is the model: layers, nodes and connections; it can be shallow (one or two hidden layers). Deep learning is the method of training networks that have many hidden layers, needing huge data and processing power for complex patterns.
4. Artificial intelligence is the largest set; machine learning is a part of AI; deep learning is a part of ML. All DL is ML and all ML is AI, but not all AI is ML and not all ML is DL.
5. Input layer; neurons/nodes; hidden layer(s); weights; bias; activation function; output layer; loss function. Weights show connection strength/importance. Bias is an extra constant that helps adjust the result and improve accuracy.
6. Large input data; many hidden layers; automatic feature learning; learning algorithm (e.g. backpropagation); high computing power (GPUs); training process (epochs).
7. Authentication confirms identity before access (password, PIN, OTP, biometrics — e.g. bank OTP). MFA/2FA requires two or more factor types, typically password + OTP. Access control decides what that user may do (view/edit/comment/download/share — e.g. Google Doc view-only for the class, edit for group leaders).
8. $S \rightarrow V$, $I \rightarrow L$, $N \rightarrow Q$, $D \rightarrow G$, $H \rightarrow K$ so ciphertext VLQ GK. Caesar uses the same shift key to encrypt and decrypt, so it is symmetric.
9. Malware: harmful software (viruses, worms, spyware, ransomware, Trojans) — e.g. cracked software that corrupts files. Phishing: fake genuine-looking email/SMS/site stealing passwords or OTPs — e.g. fake JazzCash OTP page. DoS: flood of fake requests that slows or crashes a service — e.g. online exam portal students cannot enter.
10. Equal access: same tools/opportunity for everyone (same Zoom login). Equity: extra specific support so people with barriers reach the same outcome (captions for hearing difficulties). Functions: need-based support; removing barriers (low-data apps for remote

areas); enabling participation (recorded lectures during load-shedding); supporting different learning needs (video, text, audio).

Long questions — model answers

1. **ML**: trained-brain model; data → learn patterns → predict on unseen data; used in face unlock, speech, photo search, diagnosis, translation. **NN**: brain-inspired maths model; neurons pass information through Input, Hidden, Output; face unlock: camera (input), eye distance/nose shape (hidden), unlock (output). **DL**: ML method with multi-layer NNs; “deep” = several hidden layers; early layers: edges/shapes/colours; deeper: whole face or car model; gallery search example. **Diagram**: three nested sets labelled AI, ML, DL with AI outermost. Sentence: $DL \subset ML \subset AI$.
2. **(a)** $s = 0.8 \times 0.4 + 0.5 \times 0.6 + 0.1 = 0.32 + 0.30 + 0.1 = 0.72$; $0.72 \geq 0.5$ so $y = 1$. **(b)** $s = 0.2 \times 0.4 + 0.3 \times 0.6 + 0.1 = 0.08 + 0.18 + 0.1 = 0.36$; $0.36 < 0.5$ so $y = 0$. **(c)** Eight components as in the table; sketch three input circles, four hidden, two output, labelled layers. **(d)** Huge labelled leaf photos; many hidden layers; features learned automatically; backpropagation; GPU training; many epochs until accuracy rises — cotton pest detector for a Sindh farm.
3. Cover all eight: image recognition (face lock, cameras); speech (assistant alarm, Urdu voice); NLP (Translate, chatbots); healthcare (X-ray/CT/MRI, pneumonia); recommendations (YouTube CS videos); autonomous vehicles (brake for pedestrian); fraud (unusual bank transaction); assistive (speech-to-text). Then Policy 2025: innovation, skills, ethics, digital growth. Sectors: education (tutoring, assessments); healthcare (rural diagnosis); agriculture (crops, irrigation, yield); business/banking (fraud, chatbots); local languages (Urdu, Sindhi). Students need Python, maths, data analysis, problem-solving, ethics.
4. Secure collaboration = working together with digital tools in a safe, controlled, responsible way. Authentication: prove identity (password/PIN/OTP/biometrics); bank OTP example. MFA: two or more of know/have/are; 2FA = password + phone OTP. Access control: view/edit/comment/download/share; teacher view-only vs group-leader edit. Accountability: LMS tracks who uploaded/submitted. Privacy: protect names, marks, passwords; share only with the right people; confidentiality; legal/ethical compliance. Safe transmission: encrypted official LMS, not an open public forum. Classroom scenario: school email + MFA, role permissions, version history, no CNICs on public WhatsApp.
5. Data protection = keeping digital information safe from unauthorised access, misuse, loss, damage or accidental deletion. Encryption: plaintext → ciphertext; symmetric = same key both ways; asymmetric = public encrypt, private decrypt. Caesar shift 3: SINDH → VLQ GK; decrypt by -3; wrap-around at Z. Strong password: upper, lower, number, symbol (e.g. Zk@2026#Pass) against brute force. 3-2-1: three copies, two media, one off-site (laptop + USB + Google Drive). Firewall: barrier monitoring/blocking suspicious network traffic. Antivirus: detects/removes malware on the local machine.
6. A security threat is any action, event or weakness that can harm a system or its data. Malware (cracked software); phishing (fake bank mail); unauthorised access (unlocked PC, weak/shared passwords); data theft vs identity theft (stolen CNIC used for fraud); DoS

(exam portal flood); human error (wrong email, deleting files, falling for a scam). Signs table: pop-ups/missing files → malware; odd logins → unauthorised access; urgent OTP/bad grammar → phishing; records moved → theft; traffic spike → DoS. Mitigation: antivirus+firewall; strong passwords+2FA; encryption in transit; regular backups; user awareness (no unknown links, attachments, or shared passwords). Equal access vs equity; do not require a high-end phone-only app. Tools: Docs, Zoom, Teams, Classroom/LMS, WhatsApp (responsible), email (formal record).

Self-assessment: I can...

- ☐ define ML and explain data input → learning → prediction.
- ☐ describe a neural network and the face-unlock I-H-O path.
- ☐ list eight NN components and six DL components.
- ☐ explain authentication, MFA/2FA and access control with examples.
- ☐ identify malware, phishing and DoS from signs and name mitigations.
- ☐ choose a fair collaboration tool (Docs, Zoom, Teams, LMS, WhatsApp, email).
- ☐ draw $AI \supset ML \supset DL$ and give a system that belongs in each layer.
- ☐ compute a neuron's weighted sum and threshold output from numbers.
- ☐ give eight applications of NN/DL and Pakistan's AI policy sectors.
- ☐ encrypt a short word with a Caesar cipher and apply the 3-2-1 backup rule.
- ☐ distinguish equal access from equity and state four functions of equity.

Lecture 06

Entrepreneurship in the Digital Age

Chapter lecture — 16 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

BIEK Computer Science XII · Bilingual support: English + اردو

بی آئی ای کے کمپیوٹر سائنس — بارہویں جماعت

16 book lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XII

How to use this BIEK study-guide lecture

Each PDF follows one textbook lecture from the BIEK / Sindh Computer Science notes (New Sindh Curriculum). Read *Learn it*, study the SVG diagram, try the worked example on paper, then answer *Check yourself* before the answers. **Extra board practice** (MCQs, short and long questions) is added after the book content — it is not in the textbook. Career-only extras that were not in the book lecture list have been removed. Math is rendered with MathJax. ★ Golden topics are high-yield in board exams. The green box repeats the idea in Urdu.

ہر لیکچر کتاب کے درس کے مطابق ہے۔ کتاب کے بعد اضافی MCQs اور سوالات ہیں۔ جو اضافی کیریئر صفحات کتاب کی فہرست میں نہیں تھے وہ ہٹا دیے گئے ہیں۔

Lectures in this pack

Chapter 6: Entrepreneurship in the Digital Age. Study one lecture at a time, then use the chapter review and answer key at the end of this PDF.

L01. 6.1 Entrepreneur and Entrepreneurship

L02. 6.2 Entrepreneurship in the Digital Age

L03. Table 6.1 Local examples of digital entrepreneurship

L04. 6.3 From Problem to Business Idea

L05. 6.4 Prototype

L06. 6.5 Types of Prototypes (Low, Mid, High-Fidelity)

L07. 6.6 Prototype Development Cycle (Design, Build, Test, Iterate)

L08. 6.7 Class Activity: Create a Prototype

L09. 6.8 Minimum Viable Product (MVP)

L10. 6.9 Difference between Prototype and MVP

L11. 6.10 Identifying the Riskiest Assumption

L12. 6.11 Developing an MVP

L13. 6.12 Case Study: MVP for a Canteen Pre-Order System

L14. 6.13 Testing an MVP

L15. 6.14 Beachhead Market

L16. 6.15 & 6.16 Complete Student Project & Ethical Use

Entrepreneurship in the Digital Age

This chapter shows you how a real problem in your home, college or bazaar can become a business idea, and how you can test that idea cheaply with paper sketches, prototypes and a Minimum Viable Product (MVP) before you spend a lot of money. You will also learn how to choose your first users, the beachhead market, and how to collect feedback honestly and safely.

اس باب میں آپ سیکھیں گے کہ روزمرہ زندگی کے کسی حقیقی مسئلے کو کاروباری خیال میں کیسے بدلا جائے، اور پروٹوٹائپ (Prototype) اور ایم وی پی (MVP) کے ذریعے کم خرچ میں اس خیال کو کیسے آزمایا جائے۔

What you will learn

- Define **entrepreneur** and **entrepreneurship** and describe the habits of an entrepreneur.
- Explain how digital tools (WhatsApp, Google Forms, Facebook pages, simple websites) make it cheap and quick to start and test a business.
- Turn an everyday problem into a business idea using the steps Observe, Ask Users, Select One Problem, Suggest Solution, Test Early.
- Define a **prototype**, list its uses, and compare low-, mid- and high-fidelity prototypes.
- Draw and explain the prototype development cycle: Design, Build, Test, Iterate.
- Define a **Minimum Viable Product (MVP)** and state at least five differences between a prototype and an MVP.
- Identify the **riskiest assumption** of a business idea and design a small test for it.
- Prepare an MVP plan using the MVP planning checklist, and test the MVP with number-based and comment-based feedback.
- Choose a focused **beachhead market** for a new idea.
- Complete a small group project and use digital tools ethically, safely and fairly.

6.1 Entrepreneur and Entrepreneurship

LEARN IT

An **entrepreneur** is a person who **identifies a problem, thinks of a solution, and tries to change that solution into a product or service.**

An entrepreneur does not always begin with a large company or a big office. Many begin with a small idea, limited resources, and a strong wish to solve a problem. Think of a young person in Karachi who starts by delivering home-cooked lunch boxes to a few offices, or a student in Sukkur who repairs mobile phones for neighbours. They started small.

Entrepreneurs usually show three habits:

- **They observe people and listen to their needs.** They notice what annoys people again and again.
- **They take calculated risks.** A *calculated risk* is a risk you take after thinking about what you could lose and how to keep that loss small. It is not gambling.
- **They improve their idea step by step.** They do not expect the first attempt to be perfect.

What is entrepreneurship?

Entrepreneurship is the **process** of **planning, starting, testing, and improving** a business idea. It involves **creativity, problem-solving, teamwork, communication, and calculated risk.**

Notice the difference: the *entrepreneur* is the **person**; *entrepreneurship* is the **process** that person goes through.

REMEMBER IT

Simple meaning. An entrepreneur asks: "*What problem can I solve?*" Entrepreneurship is the process of testing and improving that solution until it becomes useful for people.

Remember the three-step journey:

1. Problem → 2. Solution → 3. Product

WORKED EXAMPLE

Worked example: spotting the three steps. Ayesha lives in Larkana. Her mother and other women in the street make beautiful hand-embroidered *rillis*, but they can only sell them to visitors.

Step 1, Problem: skilled women have products but very few buyers.

Step 2, Solution: show the rillis online so buyers from other cities can see them.

Step 3, Product/Service: a Facebook page with photos, prices and a "message to order" button.

Ayesha is the **entrepreneur**. Planning the page, posting photos, seeing which designs sell, and changing prices is **entrepreneurship**.

COMMON MISTAKE

Mistake: "Only rich people with big offices can be entrepreneurs."

Fix: The source clearly says many entrepreneurs begin with a small idea and limited resources. What matters is a real problem, a useful solution, and the willingness to improve it.

اردو میں سمجھیں

کاروباری شخص (Entrepreneur) وہ ہوتا ہے جو کسی مسئلے کو پہچانتا ہے، اس کا حل سوچتا ہے اور اس حل کو کسی پراڈکٹ یا سروس میں بدلنے کی کوشش کرتا ہے۔ کاروبار شروع کرنے، آزمانے اور بہتر بنانے کے پورے عمل کو انٹرپرائیورشپ (Entrepreneurship) کہتے ہیں۔ ایسا شخص لوگوں کی بات سنتا ہے، سوچ سمجھ کر خطرہ مول لیتا ہے اور اپنے خیال کو قدم بہ قدم بہتر بناتا ہے۔

CHECK YOURSELF

1. Is "entrepreneurship" a person or a process?
2. Name the three habits of an entrepreneur given in the source.
3. What is a calculated risk?

Answers: 1. A process (planning, starting, testing and improving a business idea). 2. Observing people and listening to their needs; taking calculated risks; improving the idea step by step. 3. A risk taken after thinking carefully about the possible loss and keeping that loss small.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An entrepreneur:

- a. Turns a problem into a product/service and takes the risk
- b. Only works in a bank job
- c. Only draws NAND
- d. Only writes OSI

MCQ 2. Entrepreneurship is the:

- a. Process of doing that (test and improve until useful)
- b. Logo
- c. First tweet
- d. Compiler

MCQ 3. A student can be entrepreneurial by:

- a. A small local solution (notes, tutoring, canteen queue)
- b. Waiting for a billion-dollar app
- c. Avoiding all users
- d. Skipping tests

MCQ 4. Risk in the book's sense includes:

- a. It might not work / no one pays
- b. NAND delay
- c. Median of 1
- d. OSI layer 2

MCQ 5. Not every shopkeeper in the bazaar is automatically:

- a. A digital entrepreneur (needs the digital-age piece)
- b. A failure
- c. A programmer
- d. A UX designer

MCQ 6. Ethics still apply:

- a. Don't scam, don't leak user data
- b. Scams are clever entrepreneurship
- c. Spy on classmates
- d. Copy a bank app's OTP

Extra short questions

1. Define entrepreneur and entrepreneurship.
2. A local non-app example.
3. Why "risk" is in the definition.
4. One quality of an entrepreneur from class discussion.

Extra long question

1. Is a class fellow who sells printed notes an entrepreneur? Use the book definitions, then say what would make it "digital age".

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Person vs process: problem → solution → offer to users, with risk.
2. A photocopy stall that also sells past papers at the gate — still entrepreneurship; digital comes next lecture.
3. You may waste time/money; users may not come.
4. Observes problems, tests, iterates — not just dreams.

Long

1. Yes if they identified a problem and run a small offer. Digital: WhatsApp catalogue, JazzCash, PDF store. Ethics: copyright of the textbook.

6.2 Entrepreneurship in the Digital Age ★ Golden

LEARN IT

Entrepreneurship in the digital age means using **digital tools** to **start, test, improve, and grow** a business idea.

These tools may include **mobile phones, social media, websites, online forms, digital payments, maps, videos, and simple apps**. In Pakistan you already know many of them: WhatsApp, Facebook, Instagram, TikTok, YouTube, Google Forms, Google Maps, and digital payments such as JazzCash, Easypaisa or Raast.

The power of digital tools

A digital entrepreneur **does not always need to create a complex mobile app**. Sometimes a **WhatsApp group**, a **Google Form**, a **Facebook page**, or a **simple website** is enough to test an idea.

In the past

- Large investment needed.
- Offices and shops had to be rented first.
- Long planning before anyone saw the product.
- A failed idea meant a big loss.

Today (digital age)

- A small idea can be tested **quickly**.
- Free tools: WhatsApp, Google Forms, Facebook.
- Feedback arrives in hours or days.
- If users do not like the idea, the team can **change it before spending too much money**.

WORKED EXAMPLE

Real-world relevance (from the source).

Example 1: A student selling **handmade cards through Instagram**. She posts photos of her Eid cards, people message her, and she delivers. No website, no app.

Example 2: A shopkeeper creating a **WhatsApp catalogue for Ajrak**. WhatsApp Business lets him list each Ajrak with a photo and price. Customers in Hyderabad or Karachi browse and order through chat.

Both are digital entrepreneurship **without writing a single line of code**.

Walk-through: how the Ajrak seller tests a new design cheaply.

1. He photographs 3 new Ajrak designs and adds them to the catalogue (cost: almost zero).
2. He sends the catalogue link to his customer broadcast list.
3. After one week he counts the orders per design.
4. He prints more of the design that sold most and stops the one nobody ordered.

He learned what customers want **before** printing a large stock.

REMEMBER IT

Remember "S-T-I-G": digital tools help you **Start, Test, Improve and Grow**. Think: "Start The Idea Growing."

COMMON MISTAKE

Mistake: "Digital entrepreneurship means building a mobile app."

Fix: An app is only one option. A WhatsApp group, Google Form, Facebook page or simple website is often enough, and much cheaper, to test whether people want your idea.

اردو میں سمجھیں

ڈیجیٹل دور میں انٹرپرائزورشپ کا مطلب بے موبائل فون، سوشل میڈیا، ویب سائٹ، آن لائن فارم اور ڈیجیٹل ادائیگی جیسے ٹولز کے ذریعے کاروبار شروع کرنا، آزمانا اور بڑھانا۔ ہر بار پیچیدہ ایپ بنانا ضروری نہیں؛ وائس ایپ گروپ یا گوگل فارم بھی کافی ہو سکتا ہے۔ اگر لوگوں کو خیال پسند نہ آئے تو زیادہ پیسہ خرچ ہونے سے پہلے اسے بدلا جا سکتا ہے۔

EXAM TIP

Common questions: "Define entrepreneurship in the digital age", "How have digital tools changed entrepreneurship?" and "Give examples of digital entrepreneurship without coding." Write the definition (start, test, improve, grow using digital tools), list 4–5 tools, give the past-versus-today contrast (big investment versus quick, cheap testing), and quote the Instagram cards and the WhatsApp Ajrak catalogue examples.

CHECK YOURSELF

1. List four digital tools that the source mentions.
2. Why is it easier to change a business idea today than in the past?
3. Give one example of digital entrepreneurship that needs no coding.

Answers: 1. Any four of: mobile phones, social media, websites, online forms, digital payments, maps, videos, simple apps. 2. Because a small idea can be tested quickly and cheaply, so if users do not like it the team can change it before spending too much money. 3. Selling handmade cards through Instagram, or a WhatsApp catalogue for Ajrak.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Digital-age entrepreneurship uses:

- a. Digital tools to start, test, improve, grow
- b. Only a physical shop with no phone
- c. Only NAND labs
- d. Only paper ledgers forever

MCQ 2. A WhatsApp business catalogue is:

- a. A digital channel
- b. OSI layer 1 hardware
- c. A primary key
- d. A K-map

MCQ 3. Low cost of testing online means:

- a. You can try an MVP cheaply
- b. You skip users
- c. You skip ethics
- d. You skip problems

MCQ 4. This is ★: define + two local examples + one risk.

- a. The recipe
- b. Only Silicon Valley names
- c. Only Python lists
- d. Only ER

MCQ 5. Payment via JazzCash is:

- a. Digital infrastructure an entrepreneur can use
- b. A sort algorithm
- c. A pie of NAND
- d. A Figma auto-layout

MCQ 6. Copying an app without a real user problem is:

- a. Not the book's method
- b. Guaranteed success
- c. A stack
- d. A median

Extra short questions

1. Define entrepreneurship in the digital age.
2. Two tools a student actually has.
3. A risk unique to digital.
4. Why local problems beat fake "Uber for X".

Extra long question

1. Take a photocopier stall. List 5 digital upgrades that fit Chapter 6 (not a full app). For each, the problem it tests. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Using digital tools to start/test/improve/grow a business.
2. Phone camera, WhatsApp, Google Form, Canva, GitHub pages — any two.
3. Cyber fraud, copycats, platform bans, data leak.
4. You can talk to users this week (canteen, hostels, tuition).

Long

1. WhatsApp queue, prepay JazzCash, Google Form for notes list, simple status stories, PDF sample. Each tests demand. Ethics: no pirated books.

Table 6.1 Local examples of digital entrepreneurship

LEARN IT

Digital entrepreneurship can begin from **ordinary problems**. The examples below are close to the context of Sindh and Pakistan, where many students, parents, and small service providers already use mobile phones.

Problem	Digital Idea	Possible First Users	Urdu Context
Parents miss college notices.	WhatsApp notice and reminder service.	Parents of first-year students.	والدین تک کالج کے نوٹس پہنچانے کے لیے واٹس ایپ سروس۔
Small embroidery shop has few customers.	Simple catalogue with order form on Facebook or WhatsApp.	Customers in the same city.	کڑھائی کی دکان کے لیے واٹس ایپ کیٹلاگ۔
Students forget test dates.	Google Calendar guide or simple reminder app.	Grade XII students.	ٹیسٹ کی تاریخیں یاد دلانے والی ایپ یا کیلنڈر۔
Canteen has long queues during break.	Pre-order form for snacks.	Students and teachers.	کینٹین کے لیے پری آرڈر فارم۔

Read each row from left to right: a **problem** → a **simple digital idea** → a **small group of first users**. You will see these same examples again later in the chapter (the canteen pre-order system becomes a full case study).

WORKED EXAMPLE

Worked example: adding your own row. Follow the same pattern for a new local problem.

Problem	Digital Idea	Possible First Users
Students in a Mirpurkhas college lose water bottles, calculators and ID cards and never find them.	Lost-and-found Google Form plus a WhatsApp group where found items are posted with a photo.	Students of one college.

Check: Is it a **real** problem? Yes, it happens every week. Is the idea **simple**? Yes, a form and a group. Are the first users **reachable**? Yes, they are in the same college.

COMMON MISTAKE

Important: A digital business idea should **not** be selected only because it sounds modern. It should solve a **real problem for real users**. If the problem is weak, the business idea will also be weak.

For example, "an AI app that chooses your shirt colour" sounds modern, but if nobody actually struggles with that, nobody will use it.

REMEMBER IT

Think "Problem first, tool second." Start with the pain, not with the technology.

اردو میں سمجھیں

جدول 6.1 بتاتی ہے کہ ڈیجیٹل کاروبار عام مسائل سے شروع ہو سکتا ہے، جیسے والدین کا کالج نوٹس نہ دیکھ پانا، کڑھائی کی دکان پر کم گاہک، ٹیسٹ کی تاریخ بھول جانا یا کینٹین پر لمبی قطار۔ کوئی خیال صرف اس لیے نہ چنیں کہ وہ جدید لگتا ہے؛ اسے حقیقی لوگوں کا حقیقی مسئلہ حل کرنا چاہیے۔

CHECK YOURSELF

1. In Table 6.1, what digital idea is suggested for a small embroidery shop?
2. Who are the possible first users of the canteen pre-order form?
3. Why should an idea not be chosen only because it sounds modern?

Answers: 1. A simple catalogue with an order form on Facebook or WhatsApp. 2. Students and teachers. 3. Because it must solve a real problem for real users; a weak problem gives a weak business idea.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Table 6.1 is there to show:

- a. Ordinary local problems can start digital businesses
- b. Only US startups count
- c. Only AI labs count
- d. Only OSI consultants count

MCQ 2. A good exam use of the table is:

- a. One example, the problem, the digital twist
- b. Copying 20 rows unread
- c. Ignoring Pakistan
- d. Only listing apps

MCQ 3. "Local" means:

- a. Your college/bazaar/city, not only Silicon Valley
- b. Only Karachi DHA
- c. Only English users
- d. Only GPUs

MCQ 4. If you invent an example it must still:

- a. Be realistic and ethical
- b. Promise billions
- c. Need NASA
- d. Need a compiler in C

MCQ 5. The table sits after 6.2 because:

- a. It illustrates digital-age entrepreneurship
- b. It replaces prototypes
- c. It is HCI only
- d. It is SQL only

MCQ 6. A rickshaw app idea should still start from:

- a. A real pain (wait, fare fights)
- b. A random blockchain
- c. A random NFT
- d. A random pie

Extra short questions

1. From memory, give two local digital entrepreneurship examples (book or equivalent).
2. For one example: problem → digital twist.
3. Why the board likes local examples.
4. One unethical "business".

Extra long question

1. Add three rows in Table 6.1 style for your college: problem, idea, digital tool, first test.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Food delivery, online tuition, used-books WhatsApp, ride apps, handicraft Instagram — mapped to a problem.
2. Long canteen line → pre-order form.
3. Shows you understood 6.2, not just American brand names.
4. Selling leaked papers; fake job ads.

Long

1. Queue, lost keys, hostel laundry. Tools: form, group, JazzCash. First test: 10 users this week. Ethics column.

6.3 From Problem to Business Idea

LEARN IT

A good business idea usually starts from **observing daily life**. Look around your home, college, market, village, or city and ask these simple questions:

1. **What problem do people face?**
2. **Who faces this problem?**
3. **How often does it happen?**
4. **What do people currently do to solve it?**
5. **Can technology help in a simple way?**

A short way to remember them: **What? Who? How often? Current fix? Tech help?**

WORKED EXAMPLE

Example scenario (from the source): study material before exams.

Students cannot find study material before exams. The quick answer is "students are careless", but that is not the real issue.

Apply the five questions:

1. **What problem?** Students cannot find notes before exams.
2. **Who?** Students of a class or college.
3. **How often?** Before every test and exam.
4. **Current fix?** Notes are shared in many places: WhatsApp, email, photocopies, and class groups. Students search all of them.
5. **Tech help?** Yes: a simple **subject-wise notes organizer** where all files are placed in one location (for example, one shared Google Drive folder per subject).

The real problem is **scattered notes**, and the idea solves exactly that.

Scope of the problem

Avoid choosing **very large problems** at the start, such as "improving the whole education system". A better problem is **smaller and clear**, such as helping Grade XII students track **practical file submission dates**. Smaller problems are **easier to test and improve**.

Table 6.2: From problem to idea

The journey from observing a problem to creating an idea follows a structured path.

Step	What Students Do	Example
------	------------------	---------

Observe	Notice repeated problems around home, college, or market.	Students forget practical file dates.
Ask Users	Talk to people who face the problem.	Ask five classmates how they remember deadlines.
Select One Problem	Choose one clear problem only.	Practical file reminder.
Suggest Solution	Think of a simple digital or paper-based solution.	Reminder form or app screen.
Test Early	Show the idea to users before final development.	Ask classmates if they will use it.

After this, the idea moves along the full path that the rest of this chapter follows:



REMEMBER IT

Mnemonic for Table 6.2: "Only Ask Simple Solutions Today": Observe, Ask users, Select one problem, Suggest solution, Test early.

COMMON MISTAKE

Mistake 1: Choosing a huge problem ("fix traffic in Karachi"). **Fix:** Narrow it ("help students of one college know when their van will arrive").

Mistake 2: Skipping "Ask Users" and guessing what people need. **Fix:** Talk to at least five real people who face the problem.

اردو میں سمجھیں

اچھا کاروباری خیال روزمرہ زندگی کے مشاہدے سے شروع ہوتا ہے۔ پوچھیں: مسئلہ کیا ہے، کس کو ہے، کتنی بار ہوتا ہے، لوگ ابھی کیا کرتے ہیں، اور کیا ٹیکنالوجی آسان طریقے سے مدد کر سکتی ہے؟ شروع میں چھوٹا اور واضح مسئلہ چنیں، کیونکہ اسے آزمانا اور بہتر بنانا آسان ہوتا ہے۔

CHECK YOURSELF

1. Write the five steps of Table 6.2 in order.
2. Why is "improving the whole education system" a poor first problem?
3. In the study-material example, what is the real problem?

Answers: 1. Observe, Ask Users, Select One Problem, Suggest Solution, Test Early. 2. It is too large and unclear; small, clear problems are easier to test and improve. 3. Notes are scattered in many places (WhatsApp, email, photocopies, class groups), not that students are careless.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An IDE is:

- a. One app to write, run and debug code
- b. A printer
- c. A sort
- d. A cable

MCQ 2. VS Code is:

- a. A popular editor/IDE
- b. A DBMS
- c. An OSI layer
- d. A gate

MCQ 3. The terminal/run button is for:

- a. Executing the program
- b. Drawing pie charts only
- c. Sending TCP SYN only
- d. K-maps

MCQ 4. Syntax highlighting helps you:

- a. See keywords and mistakes
- b. Sort faster
- c. Encrypt disks
- d. Route packets

MCQ 5. A file explorer in VS Code shows:

- a. Project files
- b. OSI layer 1
- c. NAND groups
- d. Beachhead users

MCQ 6. Debugging in an IDE uses:

- a. Breakpoints and variable watch
- b. Only bubble sort
- c. Only pie charts
- d. Only Agile stickers

Extra short questions

1. Define IDE.
2. Name four VS Code areas.
3. Why not use Notepad only?
4. What is an extension?

Extra long question

1. Describe how you would create, run and debug a first Python file in VS Code. Include two interface elements from the notes.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Integrated Development Environment: editor + run + debug + files in one place.

2. Activity bar/sidebar (files), editor, terminal, status bar (and extensions).
3. No easy run, no highlighting, no debugger.
4. An add-on (Python plugin) that teaches VS Code a language.

Long

1. New file .py, type print, Run Python, read terminal. Mention top menu, explorer, terminal panel. Breakpoint on a wrong variable.

6.4 Prototype ★ Golden

LEARN IT

A **prototype** is an **early model** of a product, service, website, or app. It shows **how the idea may look or work**. It is **NOT the final product**. It is made to **test the idea and collect feedback**.

Think of the clay model an architect makes before building a house, or the sample *kameez* a tailor stitches before making 50 for a shop. The model lets people say "make this bigger" or "move that here" before real money is spent.

Why is a prototype useful?

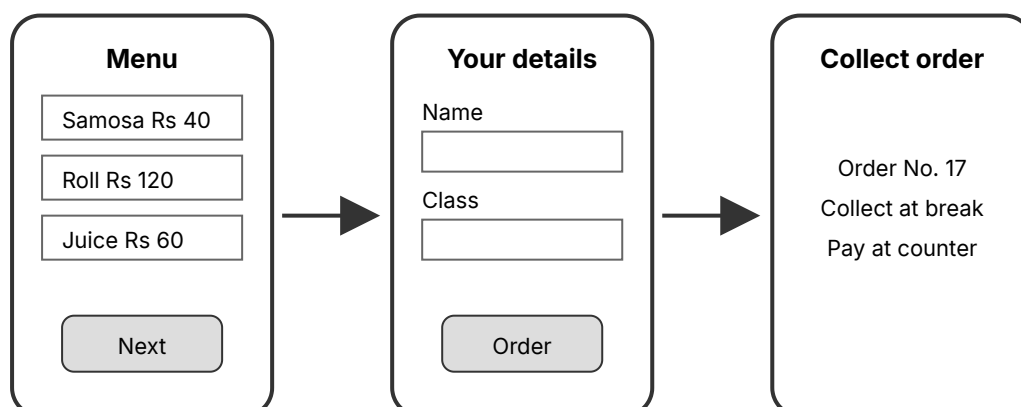
A prototype helps students and entrepreneurs **explain their idea clearly**, because users can **see something** instead of only listening to words.

- **It makes the idea visible.**
- **It helps the team find mistakes early.** Users can point out if a button is hard to find, if a screen has too much text, or if a feature is missing.
- **It saves time and money** before final coding begins. Fixing a pencil sketch takes a minute; fixing finished code can take days.

What form can a prototype take?

A prototype may be a **paper sketch**, a **cardboard model**, a **clickable PowerPoint**, a **Canva design**, a **Figma wireframe**, a **simple website layout**, or a **sample service process chart**. A *wireframe* is a simple outline drawing of a screen that shows where things go (boxes, buttons, text) without colours or pictures.

For example, for a **canteen pre-order system**, students can just **draw the screens on paper**.



A paper prototype of a canteen pre-order system: three hand-drawn screens joined by arrows (select item → enter name → collect order).

WORKED EXAMPLE

Worked example: how a paper prototype finds a mistake early. Bilal's team in a Nawabshah college draws the three screens above on A4 paper.

1. They hand screen 1 to a classmate and say, "Order a samosa."
2. The classmate looks for a quantity box and cannot find one. She asks, "What if I want two?"
3. The team notes: **feature missing: quantity.**
4. They add a small "Qty: []" box with a pencil in 30 seconds.

If they had already coded an app, adding quantity would have meant changing the database, the screen and the order list. The prototype **saved time and money.**

REMEMBER IT

Prototype = "practice model". It is like a rough draft of an essay: you show it, get corrections, and rewrite it before the final copy.

COMMON MISTAKE

Mistake: Thinking a prototype must work or must look beautiful.

Fix: A prototype only has to make the idea **visible** so users can react. A pencil sketch is a valid prototype.

اردو میں سمجھیں

پروٹوٹائپ (Prototype) کسی پراڈکٹ، سروس، ویب سائٹ یا ایپ کا ابتدائی نمونہ ہوتا ہے جو دکھاتا ہے کہ خیال کیسا نظر آئے گا یا کیسے کام کرے گا۔ یہ حتمی پراڈکٹ نہیں ہوتا بلکہ خیال کو آزمانے اور رائے (Feedback) لینے کے لیے بنایا جاتا ہے۔ یہ کاغذ کا خاکہ، گتے کا ماڈل، پاورپوائنٹ یا فگما (Figma) ڈیزائن بھی ہو سکتا ہے، اور اس سے غلطیاں جلد پکڑی جاتی ہیں اور وقت اور پیسہ بچتا ہے۔

EXAM TIP

Expect "Define prototype and write its benefits" (3–5 marks). Write: (1) definition with the words *early model, not the final product, test the idea and collect feedback*; (2) three benefits: makes the idea visible, finds mistakes early, saves time and money; (3) forms it can take (paper sketch, cardboard model, clickable PowerPoint, Canva, Figma wireframe); (4) the canteen paper-screens example.

CHECK YOURSELF

1. Is a prototype the final product?
2. Name four forms a prototype can take.
3. Give two kinds of problems users can point out by looking at a prototype.

Answers: 1. No. It is an early model made to test the idea and collect feedback. 2. Any four: paper sketch, cardboard model, clickable PowerPoint, Canva design, Figma wireframe, simple website layout, sample service process chart. 3. Any two: a button is hard to find, a screen has too much text, a feature is missing.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A prototype is:

- a. An early model of how it may look/work — not the final product
- b. The finished mass-produced item
- c. A database backup
- d. A pie of revenue

MCQ 2. You build it to:

- a. Learn, not to impress investors first
- b. Win a logo contest only
- c. Replace users
- d. Skip tests

MCQ 3. This is ★: define, purpose, what it is NOT.

- a. The 5-mark
- b. Only draw Figma pretty
- c. Only Python
- d. Only OSI

MCQ 4. A paper phone screen is still a:

- a. Prototype (low fidelity)
- b. MVP already charged
- c. Compiler
- d. Firewall

MCQ 5. Prototype before code when:

- a. The question is "do people understand the idea?"
- b. You already have 1 million users
- c. The law froze the spec like waterfall NASA
- d. You are only drawing NAND

MCQ 6. Throwing a prototype away can be:

- a. Success (you learned cheaply)
- b. Shame
- c. A SQL error
- d. A median

Extra short questions

1. Define prototype from the book.
2. Three purposes.
3. What it is not.
4. A non-app prototype.

Extra long question

1. Design a paper prototype of a "lost-and-found" college app: 3 screens, 3 things you would watch in a test. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Early model of a product/service/app that shows how it may look or work. Not the final thing.
2. Communicate idea, test with users, find problems early.
3. Not the finished product; not necessarily coded; not an MVP (next topics).
4. A cardboard token for a queue system.

Long

1. Home, report item, search. Watch: do they find report in 30s, do they understand photos, do they trust privacy. Don't code yet.

6.5 Types of Prototypes (Low, Mid, High-Fidelity)

LEARN IT

Prototypes are commonly divided into **three types** according to their level of detail, called **fidelity**. **Fidelity means how close the prototype is to the final product.** Low fidelity = far from final; high fidelity = very close to final.

6.5.1 Low-Fidelity Prototype

A **quick and rough** model made at the **very beginning**. It is **drawn on paper**. It does **not focus on colours or fonts**. It shows the **main idea and flow** (which screen comes after which). It is **very suitable for students** because it is **cheap, fast, and easy to change**.

6.5.2 Mid-Fidelity Prototype

Clearer than a paper sketch. It is created in **PowerPoint, Canva, or Figma**. It shows screens and **navigation** in a more organised way, but **without final graphics**. It is useful to **test screen-to-screen navigation**, for example by linking slides so that clicking "Next" opens the next screen.

6.5.3 High-Fidelity Prototype

Looks **very close to the final product**. It includes **realistic colours, buttons, sample data, and working features**. It **takes more time** and should **only be created after the basic idea is tested**.

Comparison of prototype types

For most school and college projects, you should **begin with a low-fidelity prototype**. A common mistake is to start building a full app before knowing whether users need it. A rough prototype protects you from this mistake.

Table 6.3: Types of prototypes

Type	Meaning	Best Use	Example
Low-Fidelity	A quick and rough model of the idea.	Very early testing.	Hand-drawn app screens for homework reminder.
Mid-Fidelity	A clearer model that shows layout and flow.	Testing navigation.	Clickable screens in PowerPoint or Figma.
High-Fidelity	A detailed model close to the final product.	Testing realistic experience.	Working demo of a small order form website.

Low-Fi (Paper) → Mid-Fi (PowerPoint) → High-Fi (Figma/HTML)

Extra comparison to help you revise:

Feature	Low-Fidelity	Mid-Fidelity	High-Fidelity
Tool	Paper and pencil	PowerPoint, Canva, Figma	Figma (detailed), HTML
Colours and fonts	No	Not final	Realistic
Clickable?	No (you move papers by hand)	Yes, basic links	Yes, with working features
Cost and time	Lowest, fastest	Medium	Highest, slowest
Easy to change?	Very easy	Fairly easy	Harder
When	Very beginning	After the flow is agreed	Only after the basic idea is tested

WORKED EXAMPLE

Worked example: one idea, three fidelities. Idea: a Homework Reminder for a college in Jacobabad.

1. **Low-fi:** Sana draws three screens on paper in 15 minutes: "Today", "Add homework", "Reminder". Classmates point at the drawings to "use" them.
2. **Mid-fi:** After feedback, she makes three PowerPoint slides with grey boxes and links them with hyperlinks, so tapping "Add" opens the Add slide. Now she tests whether people can move between screens easily.
3. **High-fi:** Once users clearly want it, she builds a small HTML page with real colours, a working form and sample homework data.

Each step costs more, so she only moves up when the previous step proves the idea is worth it.

REMEMBER IT

Fidelity = "faithfulness" to the final product. Remember **Paper → PowerPoint → Polished** (low → mid → high).

COMMON MISTAKE

Mistake: Starting with a high-fidelity prototype or a full app because it "looks professional".

Fix: Begin with low fidelity. If users do not need the idea, you have lost only paper and an hour, not weeks of work.

فِڈیلیٹی (Fidelity) کا مطلب ہے کہ پروٹوٹائپ حتمی پراڈکٹ سے کتنا ملتا جلتا ہے۔ لو فِڈیلیٹی کاغذ پر بنایا گیا سستا اور جلدی بننے والا خاکہ ہے؛ مڈ فِڈیلیٹی پاورپوائنٹ، کینوا یا فگما میں بنتا ہے اور اسکرینوں کے درمیان آنا جانا (Navigation) دکھاتا ہے؛ ہائی فِڈیلیٹی بالکل اصل پراڈکٹ جیسا لگتا ہے اور اسے بنیادی خیال آزمانے کے بعد ہی بنانا چاہیے۔

CHECK YOURSELF

1. What does "fidelity" mean?
2. Which type of prototype is best for testing navigation?
3. Why should students begin with low fidelity?

Answers: 1. How close the prototype is to the final product. 2. Mid-fidelity (e.g. clickable screens in PowerPoint or Figma). 3. It is cheap, fast and easy to change, and it protects them from building a full app before knowing whether users need it.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An IDE is:

- a. One app to write, run and debug code
- b. A printer
- c. A sort
- d. A cable

MCQ 2. VS Code is:

- a. A popular editor/IDE
- b. A DBMS
- c. An OSI layer
- d. A gate

MCQ 3. The terminal/run button is for:

- a. Executing the program
- b. Drawing pie charts only
- c. Sending TCP SYN only
- d. K-maps

MCQ 4. Syntax highlighting helps you:

- a. See keywords and mistakes
- b. Sort faster
- c. Encrypt disks
- d. Route packets

MCQ 5. A file explorer in VS Code shows:

- a. Project files
- b. OSI layer 1
- c. NAND groups
- d. Beachhead users

MCQ 6. Debugging in an IDE uses:

- a. Breakpoints and variable watch
- b. Only bubble sort
- c. Only pie charts
- d. Only Agile stickers

Extra short questions

1. Define IDE.
2. Name four VS Code areas.
3. Why not use Notepad only?
4. What is an extension?

Extra long question

1. Describe how you would create, run and debug a first Python file in VS Code. Include two interface elements from the notes.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Integrated Development Environment: editor + run + debug + files in one place.

2. Activity bar/sidebar (files), editor, terminal, status bar (and extensions).
3. No easy run, no highlighting, no debugger.
4. An add-on (Python plugin) that teaches VS Code a language.

Long

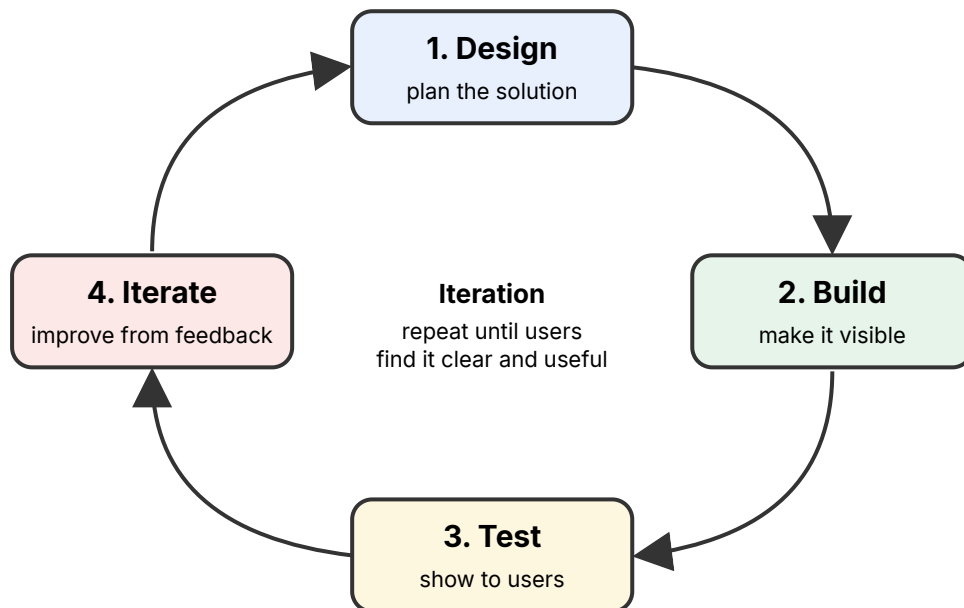
1. New file .py, type print, Run Python, read terminal. Mention top menu, explorer, terminal panel. Breakpoint on a wrong variable.

6.6 Prototype Development Cycle (Design, Build, Test, Iterate)

★ Golden

LEARN IT

Prototype development is **not a one-time activity**. A prototype is **created, tested, improved, and tested again**. This repeating process is called **iteration**. The main stages are **Design, Build, Test, and Iterate**.



The prototype development cycle: Design → Build → Test → Iterate, then back to Design.

Stage	What happens
1. Design	Understand the problem and plan the solution. Decide what the prototype should show (e.g. screen flow, features).
2. Build	Create the first version (physical or digital). The aim is not beauty; the aim is to make the idea visible.
3. Test	Show the prototype to users. Observe what they understand and what they find difficult.
4. Iterate	Improve the prototype based on user feedback. Remove unnecessary parts and add missing features.

WORKED EXAMPLE

Worked example: two rounds of the cycle. A team in Thatta wants a WhatsApp catalogue for a local Ajrak shop.

Round 1

- **Design:** The problem is that customers do not know which designs and prices are available. Plan: one page per design with photo, price and an "Order" message.
- **Build:** They make a paper booklet with 6 hand-drawn "catalogue pages".
- **Test:** They show it to 5 customers. Three ask, "Is this single or double-sided Ajrak? What size?"
- **Iterate:** They add size and type to every page.

Round 2

- **Design:** Add the size and type fields, and a delivery note.
- **Build:** They make the catalogue in WhatsApp Business with real photos.
- **Test:** 5 new customers browse it. Now nobody asks about size; one says the delivery charge is unclear.
- **Iterate:** They add "Delivery in Thatta: Rs 100" to each item.

Each round produces a clearer version, because each change comes from real feedback.

REMEMBER IT

Mnemonic: "Do Big Things Iteratively" = Design, Build, Test, Iterate. And remember the arrow at the end goes **back to Design**: it is a **cycle**, not a line.

COMMON MISTAKE

Mistake 1: Drawing the stages as a straight line that ends at "Iterate". **Fix:** Always draw a loop with an arrow from Iterate back to Design.

Mistake 2: Making the Build stage beautiful. **Fix:** The source says "The aim is not beauty; the aim is to make the idea visible."

اردو میں سمجھیں

پروٹوٹائپ ایک ہی بار میں نہیں بنتا؛ اسے بنایا، آزمایا، بہتر کیا اور دوبارہ آزمایا جاتا ہے۔ اس بار بار دہرانے کے عمل کو **ٹریشن (Iteration)** کہتے ہیں۔ چار مراحل ہیں: ڈیزائن (مسئلہ سمجھ کر حل کی منصوبہ بندی)، بلڈ (پہلا نمونہ بنانا)، ٹیسٹ (صارفین کو دکھانا) اور **ٹریٹ** (رائے کی بنیاد پر بہتری)۔

EXAM TIP

A favourite long question: "Explain the prototype development cycle with a diagram." Draw the four boxes in a **circle** with arrows, write one or two lines for each stage using the source's words, define **iteration**, and finish with a short example (e.g. the ClassBuddy activity in 6.7). Marks are often lost for drawing a straight line instead of a cycle.

CHECK YOURSELF

1. What is iteration?
2. At which stage do you observe what users find difficult?
3. What is the aim of the Build stage?

Answers: 1. The repeating process of creating, testing, improving and testing a prototype again. 2. The Test stage. 3. To make the idea visible (not to make it beautiful).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. The cycle is:

- a. Design → build → test → iterate (repeat)
- b. Build once, never look
- c. Test on no one
- d. Advertise first

MCQ 2. Iterate means:

- a. Change the prototype from what you learned
- b. Print more logos
- c. Hire 50 people
- d. Write OSI parsers

MCQ 3. This is ★ because:

- a. Entrepreneurship is not one-shot in the book
- b. Waterfall never iterates even in software
- c. Users are optional
- d. Ethics are optional

MCQ 4. If tests fail, you:

- a. Return to design
- b. Ship anyway always
- c. Delete the users
- d. Change the syllabus

MCQ 5. A cycle can be:

- a. Hours (paper) not months
- b. Only yearly
- c. Only after IPO
- d. Only in Silicon Valley

MCQ 6. Skipping test in the cycle turns it into:

- a. Just building in the dark
- b. HCI gold
- c. SQL gold
- d. Big O gold

Extra short questions

1. Name the four stages.
2. What do you carry from test to design?
3. How many cycles is "enough" for class?
4. Link to Agile from XI.

Extra long question

1. Document two cycles of a homework-reminder prototype. Each cycle: what you built, who tested, what changed. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Design, build, test, iterate.
2. Notes: where people stuck, what they asked for, what they ignored.
3. At least two documented rounds.
4. Same spirit: small loops with feedback. Don't over-claim they are identical.

Long

1. Cycle1 paper: people wanted SMS not email. Cycle2 Figma with SMS mock. Metric: 4/5 could complete. Table.

6.7 Class Activity: Create a Prototype

LEARN IT

Business idea scenario. Students often forget **homework deadlines, test dates, and required class items**. A group of students wants to design a simple "**Homework Helper**" or "**ClassBuddy**" reminder system.

The team will **not** immediately build a complete mobile app. They will first **understand the problem, draw a low-fidelity prototype, test it with classmates, and improve it**. This activity is the development cycle of 6.6 in action.

1. Understand → 2. Draw low-fi → 3. Test → 4. Analyse feedback → 5. Improve

Step 1: Understand the problem

The team **interviews ten students**. They find that **7 out of 10** students (that is 70%) forget at least one task in a week. Students want **reminders before leaving home**, and some prefer a **very simple design**.

Table 6.4: Understanding user needs

Finding	Meaning	Design Decision
7 out of 10 forget tasks.	The problem is common.	App should show a daily task list.
Reminders needed before leaving home.	Reminder time is important.	Add morning notification option.
Students want simple apps.	Design must be simple.	Use large buttons and clear labels.

Step 2: Draw a low-fidelity prototype

The team draws **three screens on paper**: (1) **Today's classes**, (2) **Add homework**, (3) **Show reminder**. **Arrows** show how a user moves from one screen to another.

Step 3: Test the prototype

The team gives the paper screens to **five classmates**. The **observer watches silently** to see if users can understand the design by themselves. Users are given one task: "**add one homework task**".

Step 4: Analyse feedback

The team looks for **repeated problems** in the feedback to identify what actually needs fixing.

Table 6.5: Changing feedback into improvement

User Feedback	Problem Identified	Improvement
"I cannot quickly find Add Homework."	Button is not visible.	Make Add Homework button larger and centre it.
"I want reminders by subject."	Reminder is too general.	Add subject name and due date fields.
"I want to see all tasks for the week."	Only daily view is not enough.	Add a 'Weekly View' button.

Step 5: Improve the prototype

The team **redraws the home screen** with a **larger Add Homework button**, a **subject list**, and a **weekly view**. This second version is clearer because it is based on real user feedback.

A prototype has not failed when users give negative feedback; it only fails if the team ignores useful feedback.

WORKED EXAMPLE

Worked example: link the activity to the cycle.

Cycle stage	ClassBuddy activity step
Design	Step 1: interview 10 students, make design decisions (Table 6.4).
Build	Step 2: draw three paper screens with arrows.
Test	Step 3: five classmates try "add one homework task" while the observer watches silently.
Iterate	Steps 4 and 5: analyse feedback (Table 6.5) and redraw the home screen.

Why watch silently? If the observer explains ("tap here"), the team never learns that the button is hard to find. Silence shows the real difficulty.

REMEMBER IT

Feedback → Problem → Improvement. Every comment should be turned into a problem statement and then one clear change, exactly like the three columns of Table 6.5.

COMMON MISTAKE

Mistake: Feeling that negative feedback means the prototype "failed", or changing the design for every single comment.

Fix: Negative feedback is useful information. Look for **repeated** problems and fix those first. The prototype only fails if you ignore useful feedback.

اردو میں سمجھیں

اس سرگرمی میں طلبہ "کلاس بڈی" (ClassBuddy) نامی یاد دہانی نظام بناتے ہیں۔ پہلے دس طلبہ سے بات کر کے مسئلہ سمجھتے ہیں، پھر کاغذ پر تین اسکرینیں بناتے ہیں، پانچ ساتھیوں سے خاموشی سے آزمواتے ہیں، رائے کا تجزیہ کرتے ہیں اور ڈیزائن بہتر کرتے ہیں۔ منفی رائے سے پروٹوٹائپ ناکام نہیں ہوتا؛ ناکامی تب ہے جب مفید رائے کو نظرانداز کیا جائے۔

CHECK YOURSELF

1. In Step 1, how many students forgot at least one task in a week, and what percentage is that?
2. Name the three paper screens drawn in Step 2.
3. What improvement was made for the feedback "I want to see all tasks for the week"?

Answers: 1. 7 out of 10, which is 70%. 2. Today's classes, Add homework, Show reminder. 3. A 'Weekly View' button was added.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. The class activity exists to:

- a. Make you build and test a prototype, not only define it
- b. Replace the exam
- c. Teach NAND
- d. Teach OSI

MCQ 2. Forgotten deadlines is a:

- a. Problem you can prototype
- b. GPU problem
- c. Primary key problem
- d. K-map problem

MCQ 3. Five classmates as testers are:

- a. Enough for a class test if you watch silently
- b. A statistically perfect census
- c. Useless
- d. A DoS attack

MCQ 4. The observer should:

- a. Watch, not rescue immediately
- b. Click for them
- c. Shout answers
- d. Grade their fashion

MCQ 5. You record:

- a. Where they hesitate and what they say
- b. Only a selfie
- c. Only the logo poll
- d. Only Big O

MCQ 6. After the activity you still:

- a. Iterate
- b. Ship to Play Store same hour
- c. Skip ethics
- d. Skip the write-up

Extra short questions

1. Summarise the book's activity scenario.
2. Three paper screens you would draw.
3. How to test without coding.
4. One ethical rule in class tests.

Extra long question

1. Write a lab report of the activity: idea, 3 screens, 5-user results, 3 changes. Use the cycle words.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Students forget homework/tests/items; team prototypes a reminder.
2. Today's tasks; add task; alarm confirm.
3. Ask a classmate to "use" paper as if it were a phone; you swap sheets.
4. Don't mock testers; don't collect their personal timetable into a public group.

Long

1. Results: 2/5 missed Add. Changes: bigger button, Urdu, confirm. Iterate. Marks-style headings.

6.8 Minimum Viable Product (MVP) ★ Golden

LEARN IT

A **Minimum Viable Product (MVP)** is the **simplest working version** of a product. It includes **only the most important features** needed to **test the idea with real users**.

- **Minimum:** it has only the **essential** features. No extras.
- **Viable:** it is **usable enough** to actually solve the main problem. ("Viable" means able to work successfully.)

An MVP is **not the final product**. It is also **not only a drawing**. It is a **small working version** used to test whether users actually need the solution.

WORKED EXAMPLE

Example from the source: canteen app.

Complete canteen app (final product)	Canteen MVP
Login, online payment, delivery tracking, discount offers...	A simple Google Form where students select snacks before break.

Step-by-step: how to cut a big idea down to an MVP.

1. List all features you dream of: login, payment, delivery tracking, discounts.
2. Ask of each one: "Without this, can a student still pre-order a snack?"
3. Login? Not needed; the student can type name and class. Payment? Not needed; pay at the counter. Delivery tracking? Not needed; students collect. Discounts? Not needed.
4. What is left is the core: **choose snack + name + class**. A Google Form does this for free.

That form is **minimum** (no extras) and **viable** (students really can pre-order with it).

REMEMBER IT

Simple meaning. An MVP answers: "**Will people actually use this solution?**" It tests **real use and behaviour**, not only the visual design.

Analogy: before opening a full restaurant, a cook in Hyderabad sells biryani from home to office workers for two weeks. If people keep ordering, the idea is worth growing. That home kitchen is the MVP.

COMMON MISTAKE

Mistake 1: Calling a paper sketch an MVP. **Fix:** A sketch is a prototype. An MVP must **work** and be used by **real users**.

Mistake 2: Thinking MVP means a low-quality or broken product. **Fix:** It is small but **viable**; it must genuinely solve the main problem.

اردو میں سمجھیں

ایم وی پی (Minimum Viable Product) کسی پراڈکٹ کا سب سے سادہ مگر کام کرنے والا ورژن ہے جس میں صرف ضروری خصوصیات ہوتی ہیں۔ "منیمم" کا مطلب ہے صرف ضروری چیزیں، اور "وائیل" کا مطلب ہے کہ وہ اصل مسئلہ حل کرنے کے قابل ہو۔ یہ حتمی پراڈکٹ نہیں بلکہ یہ جانچنے کا ذریعہ ہے کہ کیا لوگ واقعی اسے استعمال کریں گے، جیسے کینٹین کے لیے ایک سادہ گوگل فارم۔

EXAM TIP

Typical questions: "What is an MVP? Explain the words Minimum and Viable" and "Give an example of an MVP." Write the full form, the definition (simplest working version, only most important features, test with real users), explain both words, state that it is neither the final product nor only a drawing, and give the canteen Google Form example. Mention the key question: "Will people actually use this solution?"

CHECK YOURSELF

1. Write the full form of MVP.
2. What does "Viable" mean in MVP?
3. What question does an MVP answer?

Answers: 1. Minimum Viable Product. 2. Usable enough to actually solve the main problem. 3. "Will people actually use this solution?"

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An MVP is:

- a. The simplest working version that can test the riskiest assumption
- b. A paper sketch only
- c. The full app with every feature
- d. A logo

MCQ 2. Working means:

- a. Real users can get the core value (even if ugly/manual)
- b. It compiles Hello World
- c. It has 40 screens
- d. It has AI always

MCQ 3. This is ★: contrast with prototype in the next lecture, but here emphasise:

- a. Learning from a real offer
- b. Colours
- c. NAND
- d. OSI

MCQ 4. A WhatsApp group that actually takes canteen orders is:

- a. Closer to MVP than a Figma only
- b. Not digital
- c. A stack
- d. A pie

MCQ 5. Building extra features first is:

- a. The usual MVP failure
- b. Required
- c. Agile gold always
- d. HCI gold always

MCQ 6. The goal is to learn:

- a. Whether people use/pay, cheaply
- b. Whether the CEO likes purple
- c. Whether Big O is n^2
- d. Whether ER is 3NF

Extra short questions

1. Define MVP from the book.
2. What "viable" means.
3. What "minimum" means.
4. One example.

Extra long question

1. Design an MVP for "notes exchange" that you could run next week without an app store. List in/out features and the learning question. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Minimum Viable Product: simplest working version with only essential features, used to learn from real users.
2. Someone can complete the core job (order food, book notes).
3. Strip extras (reviews, AI, dark mode) until the core is tested.
4. Google Form + JazzCash + pickup window, before an app.

Long

1. In: list of notes, request, pickup. Out: chat, ratings, payments maybe later. Question: will 20 students actually exchange? How you measure.

6.9 Difference between Prototype and MVP

LEARN IT

Prototype and MVP are **related, but not the same**. A **prototype** is mainly used to **show and test the design** of an idea. An **MVP** is used to **test real use and market demand**. *Market demand* means whether enough people want the product and will use (or pay for) it.

Usually a prototype comes first and an MVP comes after it: Problem → Idea → Prototype → Feedback → MVP.

Table 6.6: Prototype and MVP comparison

Point	Prototype	MVP
Main purpose	To show how the idea may look or work.	To test whether users will actually use the solution.
Working status	May not work; can be a sketch or model.	Must work at least in a basic way.
Users	Small group gives feedback on design.	Early real users test the product in daily life.
Cost	Very low cost.	Low cost but may need some basic development.
Example	Paper screens for a canteen order system.	Google Form used for real canteen orders for three days.

REMEMBER IT

Key takeaway

Prototype = "Does it **look right** and make sense?"

MVP = "Will people **actually use it** and find value in it?"

Memory hook: **Prototype** → **P**icture of the idea; **MVP** → **M**arket test of the idea.

WORKED EXAMPLE

Worked example: classify each item. Decide whether each is a prototype or an MVP and give the reason.

1. Hand-drawn screens of a tuition-finder app shown to 6 friends. → **Prototype**: it does not work; it tests design.
2. A WhatsApp broadcast group that really sends van arrival times to parents of one route for a week. → **MVP**: it works and real users use it daily.
3. Clickable PowerPoint screens of a notes organizer. → **Prototype** (mid-fidelity): links work, but nobody uses it for real notes.
4. A shared Google Drive folder per subject that a class actually uses before the monthly test. → **MVP**: real use in daily life.

Rule used: **Does it work, and is it used by real users in real life?** If yes → MVP. If it only shows the design → prototype.

COMMON MISTAKE

Mistake: Thinking "high-fidelity prototype" and "MVP" are the same because both look good.

Fix: The difference is **purpose and real use**, not appearance. A beautiful Figma design that nobody uses for real orders is still a prototype; a plain Google Form taking real orders is an MVP.

اردو میں سمجھیں

پروٹوٹائپ اور ایم وی پی آپس میں جڑے ہیں مگر ایک جیسے نہیں۔ پروٹوٹائپ خیال کا ڈیزائن دکھاتا اور آزماتا ہے اور ضروری نہیں کہ کام کرے، جبکہ ایم وی پی کم از کم بنیادی طور پر کام کرتا ہے اور اصل صارفین روزمرہ زندگی میں اسے استعمال کرتے ہیں۔ پروٹوٹائپ پوچھتا ہے "کیا یہ ٹھیک لگتا ہے؟" اور ایم وی پی پوچھتا ہے "کیا لوگ واقعی اسے استعمال کریں گے؟"

CHECK YOURSELF

1. Which one must work at least in a basic way?
2. Give the source's example of a prototype and of an MVP for the canteen.
3. Which question does a prototype answer?

Answers: 1. The MVP. 2. Prototype: paper screens for a canteen order system. MVP: a Google Form used for real canteen orders for three days. 3. "Does it look right and make sense?"

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A prototype mainly:

- a. Shows/tests the design
- b. Takes real payments at scale always
- c. Replaces the company
- d. Is the beachhead

MCQ 2. An MVP mainly:

- a. Delivers the core working offer to real users to learn
- b. Is always paper
- c. Is always high-fi Figma only
- d. Has every feature

MCQ 3. You can have a prototype that is:

- a. Not working software
- b. Already the full product
- c. A DBMS
- d. An OSI stack

MCQ 4. You should not call a clickable mock the:

- a. MVP (if nobody can actually complete the job)
- b. Prototype
- c. Wireframe
- d. Test

MCQ 5. Order often is:

- a. Need → prototype → MVP → later product
- b. MVP → ignore users → prototype
- c. IPO → problem
- d. GPU → problem

MCQ 6. Exam 5-mark: table of:

- a. Purpose, working?, users, fidelity, example
- b. Only definitions swapped
- c. Only careers
- d. Only Python

Extra short questions

1. Four differences prototype vs MVP.
2. Can an MVP look ugly?
3. Can a prototype be digital?
4. One sentence you should memorise.

Extra long question

1. Table + one paragraph: for the canteen idea, what is the prototype and what is the MVP. Do not mix them.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Purpose design vs learning from a real offer; may not work vs must work; testers vs customers; cheap fake vs minimum real.
2. Yes — viability $\neq$ beauty.
3. Yes (Figma) — still not MVP if it doesn't deliver the job.
4. Prototype answers "do they understand it?"; MVP answers "do they use it?".

Long

1. Prototype: paper/Figma order flow. MVP: form that cooks actually receive and food is collected. Learning: % of orders picked up.

6.10 Identifying the Riskiest Assumption

LEARN IT

Every business idea is based on **assumptions**. An **assumption** is something we **believe to be true, but have not tested yet**.

The **riskiest assumption** is the belief that can make the **entire business fail** if it turns out to be wrong.

Entrepreneurs should **identify and test the riskiest assumption early**, because it **saves time and money**. If it is wrong, you find out before building everything else.

Table 6.7: Testing risky assumptions

Business Idea	Possible Risky Assumption	How to Test It (MVP)
Online tuition finder	Students will pay for online tutoring.	Ask 20 students and offer a trial class to see if they sign up.
Canteen pre-order form	Students will order snacks before break.	Test a Google Form for one week in real time.
Ajrak online catalogue	Customers will place orders through WhatsApp.	Share catalogue with 30 customers and record actual orders.
College van update system	Parents and students will check van updates daily.	Create a trial WhatsApp broadcast group for one route.

Notice that every test measures **what people actually do** (sign up, order, check), not just what they say.

WORKED EXAMPLE

Worked example: finding the riskiest assumption. Idea: a home-tutor booking service for Grade IX–X students in Sukkur.

Step 1, list assumptions:

1. Parents want home tutors for their children.
2. Good tutors in Sukkur are willing to join.
3. Parents will **pay a booking fee** through our service instead of finding tutors through relatives.
4. Parents will like our logo.

Step 2, ask of each: "If this is wrong, does the whole business fail?" Logo (4): no, easily changed. Parents wanting tutors (1): very likely true; many already hire tutors. Tutors joining (2): important, but easy to check by phoning a few tutors. Paying a booking fee (3): if parents refuse to pay, the business earns nothing, so **this is the riskiest assumption**.

Step 3, design a small MVP test: post the offer in 3 parent WhatsApp groups for one week, offer a first booking at a small fee, and **count how many actually pay**.

REMEMBER IT

Ask: "What would kill this idea?" The answer is usually the riskiest assumption. Test it **first**, cheaply, with real behaviour.

COMMON MISTAKE

Mistake: Testing easy, safe assumptions (colours, names) first and leaving the dangerous one for later.

Fix: Test the assumption that could make the whole business fail first. Also, do not trust only "Yes, I would use it". Measure actual sign-ups or orders.

اردو میں سمجھیں

ہر کاروباری خیال کچھ مفروضوں (Assumptions) پر قائم ہوتا ہے، یعنی وہ باتیں جنہیں ہم سچ سمجھتے ہیں مگر ابھی آزمایا نہیں۔ سب سے خطرناک مفروضہ (Riskiest Assumption) وہ ہے جو غلط نکلے تو پورا کاروبار ناکام ہو جائے۔ اسے شروع میں ہی چھوٹے ایم وی پی سے آزمانا چاہیے تاکہ وقت اور پیسہ بچ سکے۔

CHECK YOURSELF

1. Define assumption.
2. What is the risky assumption for the Ajrak online catalogue, and how is it tested?
3. Why test the riskiest assumption early?

Answers: 1. Something we believe to be true but have not tested yet. 2. Customers will place orders through WhatsApp; share the catalogue with 30 customers and record actual orders. 3. Because if it is wrong the whole business fails, so testing it early saves time and money.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An IDE is:

- a. One app to write, run and debug code
- b. A printer
- c. A sort
- d. A cable

MCQ 2. VS Code is:

- a. A popular editor/IDE
- b. A DBMS
- c. An OSI layer
- d. A gate

MCQ 3. The terminal/run button is for:

- a. Executing the program
- b. Drawing pie charts only
- c. Sending TCP SYN only
- d. K-maps

MCQ 4. Syntax highlighting helps you:

- a. See keywords and mistakes
- b. Sort faster
- c. Encrypt disks
- d. Route packets

MCQ 5. A file explorer in VS Code shows:

- a. Project files
- b. OSI layer 1
- c. NAND groups
- d. Beachhead users

MCQ 6. Debugging in an IDE uses:

- a. Breakpoints and variable watch
- b. Only bubble sort
- c. Only pie charts
- d. Only Agile stickers

Extra short questions

1. Define IDE.
2. Name four VS Code areas.
3. Why not use Notepad only?
4. What is an extension?

Extra long question

1. Describe how you would create, run and debug a first Python file in VS Code. Include two interface elements from the notes.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Integrated Development Environment: editor + run + debug + files in one place.

2. Activity bar/sidebar (files), editor, terminal, status bar (and extensions).
3. No easy run, no highlighting, no debugger.
4. An add-on (Python plugin) that teaches VS Code a language.

Long

1. New file .py, type print, Run Python, read terminal. Mention top menu, explorer, terminal panel. Breakpoint on a wrong variable.

6.11 Developing an MVP

LEARN IT

To develop an MVP, the team must **select only the essential features**. Extra features should be avoided at the beginning. A common mistake is **trying to build everything at once**. This makes the product **difficult, costly, and slow**.

An **MVP plan** should clearly explain: the **problem, target users, main feature, basic design, estimated cost, marketing method, feedback method, and improvement plan**.

Table 6.8: MVP planning checklist

MVP Element	Question to Ask	Example Answer
Problem	What problem are we solving?	Students wait too long at canteen.
Target users	Who will use it first?	Grade XII students in one college.
Main feature	What is the ONE feature that solves it?	Pre-order snacks before break.
Basic design	What steps are needed?	Menu, name, class, order time.
Cost	What free/low-cost tools to use?	Google Form or printed slips.
Marketing	How will early users hear about it?	WhatsApp message to class group.
Feedback	How will users respond?	Short feedback form after 3 days.

The eighth element, the **improvement plan**, answers "What will we change after feedback?" (you will see it in the case study in 6.12).

WORKED EXAMPLE

Worked example: filling the checklist for a new idea. Idea: a college notice reminder for parents in Hyderabad (from Table 6.1).

MVP Element	Answer
Problem	Parents miss college notices (fee dates, PTMs, holidays).
Target users	Parents of first-year (Class XI) students of one college.
Main feature	Send each notice to parents' phones on the same day.
Basic design	Photo of notice + 2-line summary in Urdu and Sindhi + date.
Cost	Free: WhatsApp broadcast list from the college office phone.
Marketing	Students give a printed slip to parents with the number to save.
Feedback	After 2 weeks, a 3-question voice-note or phone survey of 20 parents.
Improvement plan	If parents say notices come too late, send a reminder 1 day before each date.

Notice: only **one** main feature, a **free** tool, and a clear way to collect feedback.

REMEMBER IT

Mnemonic for the 8 elements: "People Try My Bakery; Cheap Muffins Find Improvement": Problem, Target users, **M**ain feature, **B**asic design, **C**ost, **M**arketing, Feedback, Improvement plan.

COMMON MISTAKE

Mistake: Writing a list of ten "main features".

Fix: The checklist asks for the **ONE** feature that solves the problem. Everything else waits until the MVP proves people want it.

ایم وی پی بناتے وقت صرف ضروری خصوصیات چنیں؛ سب کچھ ایک ساتھ بنانے سے کام مشکل، مہنگا اور سست ہو جاتا ہے۔ ایم وی پی منصوبے میں مسئلہ، پہلے صارفین، ایک بنیادی خصوصیت، سادہ ڈیزائن، کم خرچ ٹولز، تشہیر کا طریقہ، رائے لینے کا طریقہ اور بہتری کا منصوبہ واضح ہونا چاہیے۔

CHECK YOURSELF

1. Why is building everything at once a mistake?
2. List the eight elements of an MVP plan.
3. In Table 6.8, what is the marketing method?

Answers: 1. It makes the product difficult, costly and slow. 2. Problem, target users, main feature, basic design, estimated cost, marketing method, feedback method, improvement plan. 3. A WhatsApp message to the class group.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Developing an MVP means:

- a. Choosing only essential features and a way to deliver them
- b. Hiring 40 engineers first
- c. Designing 12 themes
- d. Writing the TOS of a unicorn

MCQ 2. A feature is essential if:

- a. The core job fails without it
- b. A competitor has it
- c. It is AI
- d. It is animated

MCQ 3. Manual-behind-the-scenes is allowed if:

- a. The user still gets the job done (concierge MVP)
- b. You lie that it is fully automatic forever
- c. You skip users
- d. You skip ethics

MCQ 4. Time-box the build to:

- a. Days/weeks, not years
- b. A decade
- c. Until perfect
- d. Until the logo shines

MCQ 5. You still need:

- a. A way to measure the learning
- b. No metric
- c. Only likes
- d. Only press

MCQ 6. Extra features before the core works are:

- a. Waste
- b. Professionalism
- c. Required by BIEK
- d. HCI law

Extra short questions

1. Steps to develop an MVP from the notes.
2. Concierge MVP meaning.
3. Tools a student can use.
4. One ethics line.

Extra long question

1. Turn the notes-exchange idea into an MVP plan: features in/out, tools, 7-day schedule, metric, kill-criteria.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Core job, essential features, drop extras, pick tools, build, measure.
2. You perform the service by hand (WhatsApp) while pretending the “product” exists — to test demand.
3. Forms, sheets, WhatsApp, JazzCash, a landing page.
4. If you take money, deliver or refund; don't ghost testers.

Long

1. In: list+request. Out: chat/AI. Tools: sheet+group. Metric: 10 completed exchanges. Kill if <2. Schedule table.

6.12 Case Study: MVP for a Canteen Pre-Order System

LEARN IT

Problem: In a college, students wait in long lines at the canteen. Sometimes snacks finish before students reach the counter. A group wants to test a canteen pre-order idea.

Goal: Their aim is **NOT** to create a complete app immediately. Their aim is to test **whether students will place orders before break** and **whether the canteen can manage these orders**.

The approach

1. The team creates a **paper prototype** showing three steps: **select item, enter name, collect order**.
2. **Feedback** from the prototype suggests the **menu should be short** and **payment should be at collection time**.
3. For the actual MVP, the team uses a **Google Form**. Students select **one of five items**.
4. The canteen prepares these pre-orders. This MVP is tested for **three days only** to learn quickly.

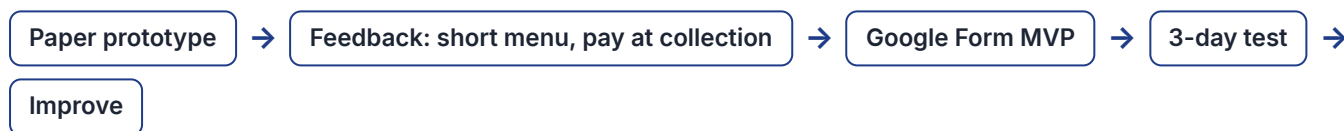


Table 6.9: MVP plan for canteen pre-order system

MVP Element	Decision
Features	Menu list, student name, class, item, and quantity.
Cost & Tools	Free Google Form or printed order slips.
Marketing	Class announcement and WhatsApp group message.
Testing period	Three days.
Feedback method	Ask students if they saved time and would use it again.
Improvement	Keep only 5 items; close orders 20 minutes before break.

Result: Students liked saving time, but **canteen staff found too many items difficult**. The team improved the MVP by **keeping only five items** and creating a **separate collection line**. This improvement is based on real feedback.

WORKED EXAMPLE

Worked example: trace the case study through the chapter's ideas.

Chapter idea	Where it appears in the case study
Real problem (6.3)	Long queues; snacks finish before students reach the counter.
Low-fidelity prototype (6.5)	Paper screens: select item, enter name, collect order.
Iteration (6.6)	Feedback led to a short menu and payment at collection.
MVP (6.8)	Google Form with one of five items, used for real orders.
Riskiest assumption (6.10)	Students will order before break, and the canteen can manage the orders.
Beachhead market (6.14)	One college canteen during break.

Why "close orders 20 minutes before break"? Staff need time to prepare. If orders arrive at the last minute, the pre-order line becomes as slow as the normal line, and the main benefit disappears.

REMEMBER IT

Remember the case study as **"5 items, 3 days, 20 minutes"**: five menu items, a three-day test, orders closing twenty minutes before break.

COMMON MISTAKE

Mistake: Thinking only the students' opinion matters.

Fix: The case study tested **two** things: whether students will order **and** whether the canteen can manage the orders. The staff's feedback (too many items) caused the key improvement.

اردو میں سمجھیں

ایک کالج میں طلبہ کینٹین کی لمبی قطاروں میں انتظار کرتے تھے۔ ٹیم نے فوراً ایپ بنانے کے بجائے پہلے کاغذی پروٹوٹائپ بنایا، پھر تین دن کے لیے گوگل فارم کو ایم وی پی کے طور پر آزمایا جس میں پانچ اشیاء تھیں۔ طلبہ کو وقت کی بچت پسند آئی مگر عملے کو زیادہ اشیاء مشکل لگیں، اس لیے صرف پانچ اشیاء رکھی گئیں، الگ قطار بنائی گئی اور بریک سے بیس منٹ پہلے آرڈر بند کیے گئے۔

CHECK YOURSELF

1. Which tool was used for the canteen MVP, and for how long was it tested?
2. What two things did the team want to test?
3. What two improvements were made after the result?

Answers: 1. A Google Form, tested for three days. 2. Whether students will place orders before break, and whether the canteen can manage these orders. 3. Keeping only five items and creating a separate collection line (orders also close 20 minutes before break).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. The canteen case is there to:

- a. Show a full MVP story on a local problem
- b. Teach NAND
- c. Teach OSI
- d. Teach ER only

MCQ 2. The problem is:

- a. Queues / food running out
- b. GPUs
- c. Primary keys
- d. Figma auto-layout

MCQ 3. A sensible MVP is:

- a. Pre-order a few items for pickup, not a full restaurant OS
- b. Drone delivery in week 1
- c. AI chefs
- d. Blockchain loyalty

MCQ 4. The kitchen is a:

- a. User/constraint you must test
- b. A pie
- c. A stack
- d. An OSI hub

MCQ 5. If nobody pre-orders, the idea:

- a. Failed the risky assumption — good you learned cheap
- b. Needs more gradients
- c. Needs a CEO
- d. Needs C++

MCQ 6. Write the case as:

- a. Problem, users, assumption, MVP, test, result, next
- b. Only the logo
- c. Only careers
- d. Only Python lists

Extra short questions

1. Retell the case in 5 bullets.
2. Two user groups with different needs.
3. A metric.
4. A next step if it works.

Extra long question

1. Write the canteen case as a 10-mark structured answer with a tiny flow diagram (order → kitchen → pickup).

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Queue; students+staff+cooks; assume pre-order helps; form/WhatsApp MVP; measure wait/orders; iterate.
2. Students (speed); kitchen (simple tickets, not 40 modifiers).
3. Orders completed; average wait; food wasted.
4. Add payment; then maybe an app.

Long

1. Headings as above. Diagram three boxes. One risk: kitchen overwhelmed at 10:55. Mitigation: cap orders. Ethics: allergies.

6.13 Testing an MVP

LEARN IT

MVP testing means giving the basic working version to **real users** and **studying their response**. The team should collect **both numbers and comments**.

- **Numbers** (quantitative data) show how many users tried the MVP and how many may use it again.
- **Comments** (qualitative data) explain what users liked, disliked, or found confusing.

Testing should be **planned before the MVP is launched**. Decide:

1. **How many users** will test it.
2. **How long** the test will continue.
3. **What information** will be collected.

Table 6.10: Feedback during MVP testing

Type of Feedback	Question	Example Result
Number-based	How many users tried it?	40 students used the form.
Number-based	How many would use it again?	30 out of 40 said yes.
Comment-based	What was confusing?	Did not know order closing time.
Observation	Where did users face difficulty?	Forgot to enter class name.

Observation means the team watches users and notes difficulties themselves, even ones users do not mention.

WORKED EXAMPLE

Worked example: studying the results in Table 6.10.

1. **Repeat-use rate** = (users who would use it again ÷ users who tried it) × 100 = (30 ÷ 40) × 100 = **75%**.
2. **Did not want to use again** = 40 - 30 = **10 students** = (10 ÷ 40) × 100 = **25%**.
3. **Comment:** "Did not know order closing time" → **Fix:** write "Orders close 20 minutes before break" at the top of the form and in the WhatsApp message.
4. **Observation:** "Forgot to enter class name" → **Fix:** make the Class field **required** in the Google Form so it cannot be submitted empty.
5. **Decide what to fix first:** the closing-time problem stops people from ordering at all, so fix it first; the class-name problem only makes collection slower.

Three out of four users would come back, which is a promising sign; fixing the two problems could raise that number further.

REMEMBER IT

Data skill link. Feedback is useful only when it is studied: **Count** the responses → **Look** for repeated problems → **Decide** which improvement comes first. Remember "**C-L-D**: Count, Look, Decide."

COMMON MISTAKE

Mistake: Launching the MVP first and deciding later what to measure.

Fix: Plan the number of users, the test length and the questions **before** launch. Otherwise you may forget to record important data (for example, how many people opened the form but did not submit).

اردو میں سمجھیں

ایم وی پی کی جانچ کا مطلب ہے کہ بنیادی کام کرنے والا ورژن اصل صارفین کو دیا جائے اور ان کے ردعمل کا مطالعہ کیا جائے۔ اعداد بتاتے ہیں کتنے لوگوں نے استعمال کیا اور کتنے دوبارہ کریں گے، جبکہ تبصرے بتاتے ہیں کیا پسند آیا اور کیا الجھن تھی۔ جانچ کی منصوبہ بندی پہلے سے کریں، جوابات گنیں، بار بار آنے والے مسائل تلاش کریں اور پہلے سب سے اہم بہتری کریں۔

CHECK YOURSELF

1. Name the three types of feedback in Table 6.10.
2. If 50 people tried an MVP and 35 would use it again, what is the repeat-use rate?
3. Name the three things to decide before launching an MVP test.

Answers: 1. Number-based, comment-based, observation. 2. $(35 \div 50) \times 100 = 70\%$. 3. How many users will test it, how long the test will continue, and what information will be collected.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Testing an MVP means:

- a. Real users, real core task, collect numbers and comments
- b. Only your friends saying "cool logo"
- c. Only the teacher's mark
- d. Only a unit test of Python

MCQ 2. Vanity metrics are:

- a. Likes with no usage
- b. Completed jobs
- c. Repeat orders
- d. Time to complete

MCQ 3. You want both:

- a. Numbers and quotes
- b. Only numbers
- c. Only vibes
- d. Only press

MCQ 4. A failed test is useful when:

- a. You write what you will change or stop
- b. You hide it
- c. You add features at random
- d. You buy ads

MCQ 5. Sample of 5–20 users is:

- a. OK for class MVPs if honest
- b. A national census
- c. Useless always
- d. A DoS

MCQ 6. Don't help the user in the test if you are measuring:

- a. Whether the offer is understandable/usable
- b. Kindness in general life
- c. SQL skill
- d. NAND skill

Extra short questions

1. What to measure (book + good practice).
2. How to collect.
3. Bias: testing only your friends.
4. Then what?

Extra long question

1. Write a test plan for the canteen MVP: n, tasks, 3 metrics, 3 questions, pass/fail thresholds, ethics (food safety).

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Did they finish? Time? Would they repeat? Pay? Quotes of confusion.
2. Tally sheet + 3 interview questions after.
3. They are too nice — include one stranger in the line.
4. Iterate the MVP (or the assumption).

Long

1. n=15 lunch buyers. Task: order and collect. Metrics: % complete, wait, errors.
Questions: would you do this tomorrow? Thresholds: 70% complete. Don't poison anyone; allergens.

6.14 Beachhead Market ★ Golden

LEARN IT

A **beachhead market** is a **small and focused group of first users**. These users have a **clear need** and are **likely to try the product early**.

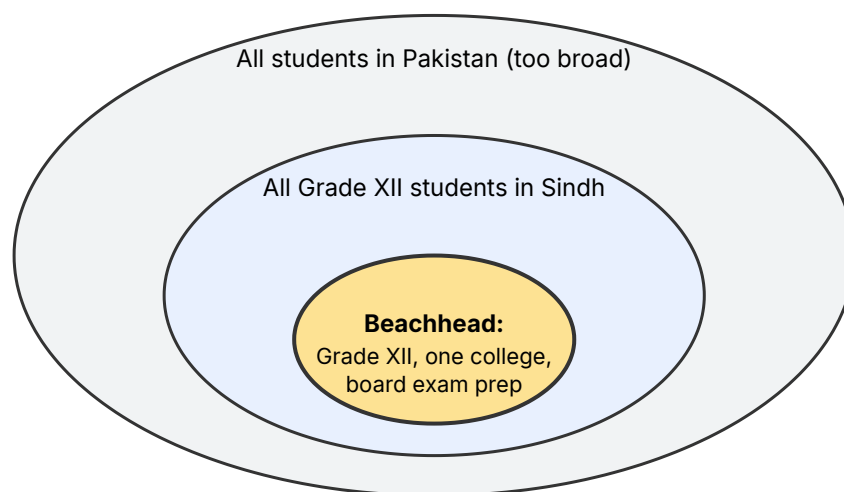
Instead of trying to serve everyone, entrepreneurs **first focus on one small group**. This helps them **test the MVP properly and improve it before expanding**.

The word comes from the army: a *beachhead* is the small piece of beach soldiers capture first when they land; from there they move inland. In business, your beachhead is the first small group you "win".

Characteristics of a good beachhead market

- It should be **specific and narrowly defined**.
- It should be **reachable** by the startup team.
- The users should have a **strong, immediate need** for the solution.

For example, targeting "**all students in Pakistan**" is too broad for a new study app. "**Grade XII students preparing for board exams in one college**" is much more focused and easier to reach.



Start with the small inner circle (beachhead), learn and improve, then expand outward.

Table 6.11: Choosing a focused beachhead market

Business Idea	Weak First Market	Better Beachhead Market
Study app	All students in Pakistan.	Grade XII students preparing for exams in one college.
Canteen pre-order	All canteens in the city.	One college canteen during lunch break.

Ajrak catalogue	All online shoppers.	Existing customers of one local shop.
Van update system	All transport users.	Students and parents of ONE specific college van route.

Choosing a small first market **does not mean the idea will remain small forever**. After learning from this group, the idea can be improved and expanded, even globally.

WORKED EXAMPLE

Worked example: test a proposed beachhead against the three characteristics. Idea: an app that reminds farmers about irrigation water turns (*warabandi*).

Proposed market	Specific?	Reachable?	Strong, immediate need?	Verdict
All farmers in Sindh	No	No (millions of people, many districts)	Varies	Weak
Farmers on one watercourse near Badin whose sons/daughters study in the team's college	Yes	Yes (through classmates)	Yes (missing a turn damages crops)	Good beachhead

Expansion path: one watercourse → nearby watercourses → the whole district → other districts.

REMEMBER IT

Remember S-R-U: Specific, Reachable, Urgent need. If you cannot meet your first users within a week, your market is too big.

COMMON MISTAKE

Mistake: Thinking a bigger first market means more success ("everyone can use my app").

Fix: A broad market is hard to reach and hard to learn from. Win a small group first, learn, improve, then expand.

بیج ہیڈ مارکیٹ (Beachhead Market) پہلے صارفین کا ایک چھوٹا اور مخصوص گروہ ہے جسے حل کی واضح اور فوری ضرورت ہو اور جو پراڈکٹ جلد آزمانے پر تیار ہو۔ سب کو ایک ساتھ خدمت دینے کے بجائے پہلے ایک چھوٹے گروہ پر توجہ دیں، جیسے "پاکستان کے تمام طلبہ" کے بجائے "ایک کالج کے بارہویں جماعت کے طلبہ"۔ وہاں سے سیکھ کر خیال کو بعد میں پھیلا یا جا سکتا ہے۔

EXAM TIP

Asked as "Define beachhead market and give its characteristics" or "Suggest a beachhead market for ...". Write the definition (small, focused group of first users with a clear need, likely to try early), the three characteristics (specific, reachable, strong immediate need), one weak-versus-better pair from Table 6.11, and the line that a small first market does not mean the idea stays small.

CHECK YOURSELF

1. Give the three characteristics of a good beachhead market.
2. What is the better beachhead market for the van update system?
3. Does choosing a small first market mean the business will stay small?

Answers: 1. Specific and narrowly defined; reachable by the team; users have a strong, immediate need. 2. Students and parents of ONE specific college van route. 3. No. After learning from the first group, the idea can be improved and expanded.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A beachhead market is:

- a. A small focused first group of users
- b. The whole country on day 1
- c. Random Twitter
- d. Every age/city/income at once

MCQ 2. You pick them because:

- a. They feel the pain strongly and you can reach them
- b. They are richest always
- c. They are abroad always
- d. They hate the product

MCQ 3. This is ★: define, why small, one example.

- a. The recipe
- b. Only the D-Day story without CS
- c. Only Python
- d. Only ER

MCQ 4. XI CS students in one college as first users of a notes app is a:

- a. Beachhead
- b. Global TAM
- c. OSI layer
- d. NAND net

MCQ 5. After the beachhead works you:

- a. Expand to the next similar group
- b. Immediately advertise on TV nationwide
- c. Quit
- d. Rewrite in assembly

MCQ 6. A beachhead that is too broad is:

- a. Hard to test and to talk to
- b. Ideal
- c. Required
- d. A median

Extra short questions

1. Define beachhead market.
2. Why not "all Pakistan" first?
3. Example from college.
4. How it links to MVP.

Extra long question

1. For a canteen pre-order, choose a beachhead (who, where, when), why they are best, and who you refuse at first. ★ Golden.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Small, focused first users with a clear need, likely to try.
2. You cannot learn; message becomes vague; cost explodes.
3. Hostel Block B, not all Karachi youth.
4. The MVP is aimed at that beachhead, not every imaginary user.

Long

1. XI 10:00 break, one stall. They queue worst. Refuse staff catering and evening shift until the kitchen process works. Success: 20 repeating students.

6.15 & 6.16 Complete Student Project & Ethical Use

LEARN IT

You will work in **groups** and complete one small entrepreneurship project. The project should be **simple, local, and possible in the classroom**.

The purpose is to practise the **complete journey**: problem → prototype → feedback → MVP plan → risky assumption → beachhead market.

Suggested project ideas: homework and test reminder system, college notice organizer, canteen pre-order system, small shop WhatsApp catalogue, college van update system, lost and found item reporting system, study notes organizer.

Table 6.12: Student project sheet (template)

Item	Your group's answer
Problem statement	
Target users	
Business idea	
Low-fidelity prototype description	
Feedback from users	
Improvement made after feedback	
MVP plan	
Riskiest assumption	
How to test the riskiest assumption	
Beachhead market	
Marketing method for first users	

WORKED EXAMPLE

Worked example: a completed project sheet (Lost and Found system).

Item	Answer
Problem statement	Students of our college in Mirpurkhas lose items (bottles, calculators, ID cards) and rarely get them back because there is no central place to report them.
Target users	Students and staff of our college.
Business idea	A Lost and Found WhatsApp group plus a Google Form to report lost or found items.
Low-fidelity prototype	Two paper screens: "Report lost item" (item, colour, place, phone of class rep) and "Found items list" with sketches.
Feedback from users	6 of 8 testers asked for photos; 3 did not want their own phone number shown.
Improvement made	Added a photo upload; replaced personal numbers with "collect from the library desk".
MVP plan	Google Form + one WhatsApp group run by two volunteers; free; tested for two weeks.
Riskiest assumption	Students who find items will take the trouble to report them.
How to test it	Count how many "found" reports arrive in two weeks compared with "lost" reports.
Beachhead market	Grade XII students of our college.
Marketing method	Announcement in morning assembly, a poster at the library and a message in each class group.

Table 6.13: Rubric for project assessment

A **rubric** is a marking guide that describes what excellent, satisfactory and weak work looks like for each part of a project.

Criteria	Excellent	Satisfactory	Needs Improvement
Problem ID	Clear, local, and important.	Understandable but needs detail.	Vague or too broad.

Prototype	Shows clear flow and features.	Shows the basic idea.	Incomplete or unclear.
Feedback & Fixes	Collected properly; changes clearly based on feedback.	Collected but analysis is weak; some changes made.	No clear feedback collected; changes not explained.
MVP Plan	Essential features and clear testing plan.	Basic features but needs testing clarity.	Confused with the final product.

REMEMBER IT

Use the rubric **before** you submit: read your project sheet row by row and ask, "Is this Excellent yet?" For "Feedback & Fixes", always write **which** feedback caused **which** change.

COMMON MISTAKE

Mistake: Writing an MVP plan that lists every feature of the dream app. The rubric marks this as "Confused with the final product".

Fix: Keep the MVP to the essential feature(s) and add a clear testing plan (how many users, how long, what you will measure).

اردو میں سمجھیں

طلبہ گروپوں میں ایک چھوٹا، مقامی اور آسان منصوبہ مکمل کرتے ہیں جس میں مسئلے سے پروٹوٹائپ، رائے، ایم وی پی منصوبہ، سب سے خطرناک مفروضہ اور بیچ ہیڈ مارکیٹ تک پورا سفر شامل ہوتا ہے۔ منصوبے کو جانچنے کے لیے روبرک (Rubric) استعمال ہوتا ہے جو مسئلے کی پہچان، پروٹوٹائپ، رائے و بہتری اور ایم وی پی منصوبے کو بہترین، تسلی بخش یا بہتری کی ضرورت کے درجوں میں پرکھتا ہے۔

CHECK YOURSELF

1. Name four suggested project ideas.
2. What does "Excellent" look like for Problem ID in the rubric?
3. Which rubric level says "Confused with the final product"?

Answers: 1. Any four: homework and test reminder system, college notice organizer, canteen pre-order system, small shop WhatsApp catalogue, college van update system, lost and found reporting system, study notes organizer. 2. Clear, local, and important. 3. Needs Improvement (for the MVP Plan).

LEARN IT

Digital entrepreneurship should be **responsible**. When collecting user feedback, you must respect:

- **Privacy**: people's personal information belongs to them.
- **Truthfulness**: report results honestly; do not make up numbers or hide bad feedback.
- **Fairness**: treat all users equally.

You should **not collect unnecessary personal information** or **share phone numbers or photos without permission**.

Equity point

A good digital solution should consider users who have **weak internet**, **limited devices**, or **language difficulty**. A design with **large buttons**, **simple Urdu/Sindhi instructions**, and **low data usage** helps ensure **equal access**. *Equity* means making sure everyone, including people with fewer resources, can use the solution.

WORKED EXAMPLE

Worked example: fix an unethical feedback form. A team's canteen feedback form asks:

1. Full name, father's name, CNIC/B-form number, home address. → **Problem**: unnecessary personal data. **Fix**: ask only class (or nothing personal at all).
2. "Upload your selfie." → **Problem**: photos are not needed and could be misused. **Fix**: remove.
3. The team plans to post all respondents' phone numbers in the class group. → **Problem**: sharing numbers without permission. **Fix**: never share; keep responses private to the team.
4. The form is in English only and has a 5 MB video at the top. → **Problem**: excludes users with language difficulty or weak internet. **Fix**: add Urdu/Sindhi instructions and remove the heavy video.

When presenting, the team reports "30 out of 40 would use it again" exactly, without rounding it up to "almost everyone". That is truthfulness.

REMEMBER IT

Ask yourself "P-T-F-E": Is it **Private**, **Truthful**, **Fair**, and **Equal-access**?

COMMON MISTAKE

Mistake: "More data is always better, so let's collect everything."

Fix: Collect only what you need to answer your test questions. Extra personal data creates risk for users and gives you no benefit.

ڈیجیٹل کاروبار ذمہ داری سے کرنا چاہیے۔ رائے جمع کرتے وقت لوگوں کی رازداری (Privacy)، سچائی اور انصاف کا خیال رکھیں؛ غیر ضروری ذاتی معلومات نہ لیں اور اجازت کے بغیر فون نمبر یا تصاویر شیئر نہ کریں۔ اچھا حل کمزور انٹرنیٹ، محدود موبائل اور زبان کی مشکل والے لوگوں کا بھی خیال رکھتا ہے، جیسے بڑے بٹن، آسان اردو یا سندھی ہدایات اور کم ڈیٹا کا استعمال۔

CHECK YOURSELF

1. Name the three values students must respect when collecting feedback.
2. Give two things students must not do with users' information.
3. List three design features that support equal access.

Answers: 1. Privacy, truthfulness, fairness. 2. Collect unnecessary personal information; share phone numbers or photos without permission. 3. Large buttons, simple Urdu/Sindhi instructions, low data usage.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. The student project should be:

- a. Small, local, possible in class time
- b. A full bank app
- c. A new OS
- d. A satellite

MCQ 2. It should include:

- a. Problem, idea, prototype/MVP, test, ethics
- b. Only a logo
- c. Only careers
- d. Only a Python hello

MCQ 3. Ethical use of digital tools means:

- a. Consent, no harassment, no leaked data, honest claims
- b. Scrape classmates' photos
- c. Fake testimonials
- d. Spam the college group

MCQ 4. The rubric rewards:

- a. A real test, not a fantasy pitch deck
- b. Unicorn valuation
- c. English poetry only
- d. NAND counts

MCQ 5. Safe tools:

- a. Forms/docs you have a right to use; no piracy
- b. Cracked Adobe
- c. Stolen question papers
- d. Phishing kits

MCQ 6. 6.15 and 6.16 together remind you:

- a. Build something small and don't harm people
- b. Skip users
- c. Skip tests
- d. Skip the write-up

Extra short questions

1. List the project pieces the book wants.
2. Two ethical rules for digital tools.
3. What "possible" means.
4. A fail on ethics even if the idea is clever.

Extra long question

1. Write a one-page project proposal that would score well: 8 headings including ethics and a 7-day test. Invent a local idea that is not the canteen copy-paste.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Team, local problem, idea, prototype, MVP or test, beachhead, results, ethics, reflection.
2. Permission for photos; no copyright theft; no unsafe personal data in public sheets.
3. You can test this week with people you can walk to.
4. Secretly recording students in the canteen.

Long

1. e.g. lab-equipment booking. Headings match rubric. Ethics: no shame list of late students on a public wall. Metric: 10 bookings honoured.

Entrepreneurship in the Digital Age

16 single-page lessons · 5 Golden · read, close, say aloud, then sit the exam

Chapter in one look

- **Entrepreneur:** a person who identifies a problem, thinks of a solution, and turns it into a product or service; observes people, takes calculated risks, improves step by step.
- **Entrepreneurship:** the process of planning, starting, testing and improving a business idea until it becomes useful (Problem → Solution → Product).
- **Digital age:** entrepreneurs use accessible digital tools (social media, forms, websites, WhatsApp, digital payments) to start and test ideas quickly with low investment; often no coding is needed.
- A digital idea must solve a **real problem for real users**, not just sound modern (Table 6.1 examples: notices, embroidery shop, test dates, canteen queues).
- **Business ideas** come from observing daily life and talking directly to users: Observe → Ask Users → Select One Problem → Suggest Solution → Test Early. Start with a small, clear problem.
- **Prototype:** an early model used to show and test the design of an idea; not the final product; makes the idea visible, finds mistakes early, saves time and money.
- **Fidelity** is how close a prototype is to the final product: Low (paper), Mid (PowerPoint/Canva/Figma, tests navigation), High (realistic, Figma/HTML, only after the idea is tested).
- **Development cycle:** Design → Build → Test → Iterate, repeated (iteration). Feedback is crucial; a prototype fails only if useful feedback is ignored.
- **MVP:** the simplest working version with only essential features (Minimum) that still solves the main problem (Viable); it tests real use and market demand.
- **Prototype vs MVP:** prototype asks "Does it look right?"; MVP asks "Will people actually use it?" A prototype may not work; an MVP must.
- **Riskiest assumption:** the untested belief that, if wrong, will make the business fail; test it early with real behaviour.
- **MVP plan:** problem, target users, ONE main feature, basic design, cost, marketing, feedback, improvement plan. Do not build everything at once.
- **Canteen case study:** paper prototype → Google Form MVP with 5 items for 3 days → staff found too many items hard → kept 5 items, separate line, orders close 20 minutes before break.
- **MVP testing:** plan users, duration and data in advance; collect numbers (40 tried, 30 would return = 75%) and comments/observations; count, look for repeated problems, fix the most important first.
- **Beachhead market:** a small, focused, reachable group of first users with a strong, immediate need; win it first, then expand.
- **Ethics and equity:** respect privacy, truthfulness and fairness; collect no unnecessary data; share nothing without permission; design for weak internet, limited devices and Urdu/Sindhi speakers.

Key terms

Term	Meaning	اردو
Entrepreneur	A person who identifies a problem, thinks of a solution, and turns it into a product or service.	کاروباری شخص
Entrepreneurship	The process of planning, starting, testing and improving a business idea.	کاروبار کاری

Calculated risk	A risk taken after carefully thinking about the possible loss and keeping it small.	سوچا سمجھا خطرہ
Digital entrepreneurship	Using digital tools to start, test, improve and grow a business idea.	ڈیجیٹل کاروبار کاری
Business idea	A proposed product or service that solves a real problem for real users.	کاروباری خیال
Target users	The people who will use the product first.	ہدف صارفین
Prototype	An early model of a product, service, website or app made to test the idea and collect feedback.	ابتدائی نمونہ
Fidelity	How close a prototype is to the final product.	اصل سے مطابقت کا درجہ
Low-fidelity prototype	A quick, rough paper model showing the main idea and flow.	کم تفصیل والا نمونہ
Mid-fidelity prototype	A clearer model in PowerPoint, Canva or Figma showing layout and navigation without final graphics.	درمیانی تفصیل والا نمونہ
High-fidelity prototype	A detailed model close to the final product with realistic colours, buttons, data and working features.	زیادہ تفصیل والا نمونہ
Wireframe	A simple outline drawing of a screen showing where items go, without colours or pictures.	اسکرین کا خاکہ

CHAPTER 6 EXAM • CLOSED BOOK

Entrepreneurship in the Digital Age

75 marks • 1 hour 15 minutes • no notes • no phones

CHAPTER EXAM RULES

Instructions. Sit this exam after Lesson 16 and the recap. **Section A** — all 15 MCQs (1 mark). **Section B** — all 10 short questions (3 marks). **Section C** — attempt any **three** long questions (10 marks). At least one Section C answer must be a ★ Golden topic. Write in your notebook. Do not open the key until time is up.

Marks. 15 + 30 + 30 = 75. A 5-mark / 10-mark answer needs Define • Explain • Example • Diagram • Working.

سبق اور خلاصے کے بعد یہ امتحان بغیر نوٹس کے حل کریں۔ وقت پورا ہونے پر کلید کھولیں۔

Section A — Multiple choice (15 × 1 = 15)

1. An entrepreneur is best described as a person who:
 - a. only invests money in big companies
 - b. identifies a problem and tries to turn a solution into a product or service
 - c. always works from a large office
 - d. only writes computer programs
2. In prototyping, "fidelity" means:
 - a. the cost of the prototype
 - b. the number of users
 - c. how close the prototype is to the final product
 - d. the speed of the internet
3. Which of these is a low-fidelity prototype?
 - a. Hand-drawn app screens on paper
 - b. A working demo website of an order form
 - c. Clickable screens in Figma
 - d. An app published on the Play Store
4. A mid-fidelity prototype is mainly useful for testing:
 - a. online payments
 - b. screen-to-screen navigation
 - c. market demand over three days
 - d. investor interest
5. The correct order of the prototype development cycle is:
 - a. Build, Design, Iterate, Test
 - b. Test, Build, Design, Iterate
 - c. Design, Test, Build, Iterate
 - d. Design, Build, Test, Iterate
6. Creating, testing, improving and testing a prototype again is called:
 - a. iteration
 - b. incubation
 - c. marketing
 - d. assumption
7. In MVP, the letter "V" stands for:
 - a. Visual
 - b. Valuable
 - c. Viable
 - d. Virtual
8. Which is the best MVP for a canteen pre-order idea?
 - a. A full app with login, payment, delivery tracking and discounts
 - b. A Google Form used for real orders for three days
 - c. Paper sketches shown to five friends
 - d. A colourful poster of the menu
9. A prototype mainly answers which question?
 - a. Does it look right and make sense?
 - b. Will people pay for it every month?
 - c. How big is the national market?
 - d. How much profit will we make?

- .0. The riskiest assumption is:
- a. the cheapest feature to build
 - b. the colour scheme of the app
 - c. the name of the company
 - d. the belief that, if wrong, can make the whole business fail
- .1. Which is the best beachhead market for a new study app?
- a. All students in Pakistan
 - b. All online shoppers
 - c. Grade XII students preparing for board exams in one college
 - d. All transport users
- .2. Which is NOT a characteristic of a good beachhead market?
- a. Specific and narrowly defined
 - b. Reachable by the startup team
 - c. Users have a strong, immediate need
 - d. As large as possible from the first day
- .3. "30 out of 40 said they would use it again" is which type of feedback?
- a. Number-based
 - b. Comment-based
 - c. Observation
 - d. Rubric-based
- .4. Which is an ethical practice when collecting user feedback?
- a. Collect CNIC numbers of all users
 - b. Share users' phone numbers in the class group
 - c. Collect only necessary information and share nothing without permission
 - d. Change results so the idea looks better
- .5. To support users with weak internet and language difficulty, a design should use:
- a. heavy videos on every screen
 - b. English-only instructions
 - c. features that need the latest phone
 - d. large buttons, simple Urdu/Sindhi instructions and low data usage

Section B — Short questions (10 × 3 = 30)

1. Define entrepreneur and entrepreneurship. How are they different?
2. What is meant by entrepreneurship in the digital age? Give two local examples that need no coding.
3. Write the five questions you should ask to find a problem in daily life.
4. Why should a student choose a small problem at the start? Give one example of a too-large and one of a suitable problem.
5. Define prototype and give three reasons why it is useful.
6. Differentiate between low-, mid- and high-fidelity prototypes, with one example of each.
7. Define MVP and explain the words "Minimum" and "Viable".

8. What is the riskiest assumption? Give the risky assumption and its test for the college van update system.
9. An MVP was tried by 40 students and 30 said they would use it again. Calculate the repeat-use rate and name two other kinds of feedback to collect.
10. State the three characteristics of a good beachhead market and give one weak and one better first market for an Ajrak catalogue.

Section C — Long questions (attempt any 3 × 10 = 30)

1. Explain the prototype development cycle with a labelled diagram. Use the ClassBuddy class activity to show what happens at each stage, and explain why negative feedback does not mean the prototype has failed.
2. Compare a prototype and an MVP under five headings (purpose, working status, users, cost, example). Use the canteen pre-order system as your example and state the key question each one answers.
3. **Plan an MVP for a college van update system** for a college in Hyderabad, where students and parents often do not know when the van will arrive. Fill in all eight elements of the MVP planning checklist, and also state the riskiest assumption, how to test it, and the beachhead market.
4. **Plan an MVP for a small Sindhi topi and Ajrak shop in Hala** that wants online customers. Describe a low-fidelity prototype, the MVP, its riskiest assumption and test, the beachhead market, the marketing method, and two ethical or equity points the shop must follow.
5. A team tested a Lost and Found MVP (Google Form + WhatsApp group) in a college of 60 Grade XII students for two weeks. 45 students tried it; 36 of them said they would use it again; 12 of the users said they did not know where to collect found items; the team observed that many reports had no photo. (a) Calculate the percentage of the class who tried it, the repeat-use rate among users, and the percentage of users confused about collection. (b) Classify each piece of feedback by type. (c) Decide which improvement to make first and justify it.

Answer key. Finish the Chapter Exam first, then mark. Do not peek while the clock is running.

پہلے امتحان مکمل کریں، پھر اس کلید سے نمبر لگائیں۔

Check yourself — answers

1. **6.1 Entrepreneur and Entrepreneurship.** Answers: 1. A process (planning, starting, testing and improving a business idea). 2. Observing people and listening to their needs; taking calculated risks; improving the idea step by step. 3. A risk taken after thinking carefully about the possible loss and keeping that loss small.
2. **6.2 Entrepreneurship in the Digital Age.** Answers: 1. Any four of: mobile phones, social media, websites, online forms, digital payments, maps, videos, simple apps. 2. Because a small idea can be tested quickly and cheaply, so if users do not like it the team can change it before spending too much money. 3. Selling handmade cards through Instagram, or a WhatsApp catalogue for Ajrak.
3. **Table 6.1 Local examples of digital entrepreneurship.** Answers: 1. A simple catalogue with an order form on Facebook or WhatsApp. 2. Students and teachers. 3. Because it must solve a real problem for real users; a weak problem gives a weak business idea.
4. **6.3 From Problem to Business Idea.** Answers: 1. Observe, Ask Users, Select One Problem, Suggest Solution, Test Early. 2. It is too large and unclear; small, clear problems are easier to test and improve. 3. Notes are scattered in many places (WhatsApp, email, photocopies, class groups), not that students are careless.
5. **6.4 Prototype.** Answers: 1. No. It is an early model made to test the idea and collect feedback. 2. Any four: paper sketch, cardboard model, clickable PowerPoint, Canva design, Figma wireframe, simple website layout, sample service process chart. 3. Any two: a button is hard to find, a screen has too much text, a feature is missing.
6. **6.5 Types of Prototypes (Low, Mid, High-Fidelity).** Answers: 1. How close the prototype is to the final product. 2. Mid-fidelity (e.g. clickable screens in PowerPoint or Figma). 3. It is cheap, fast and easy to change, and it protects them from building a full app before knowing whether users need it.
7. **6.6 Prototype Development Cycle (Design, Build, Test, Iterate).** Answers: 1. The repeating process of creating, testing, improving and testing a prototype again. 2. The Test stage. 3. To make the idea visible (not to make it beautiful).
8. **6.7 Class Activity: Create a Prototype.** Answers: 1. 7 out of 10, which is 70%. 2. Today's classes, Add homework, Show reminder. 3. A 'Weekly View' button was added.
9. **6.8 Minimum Viable Product (MVP).** Answers: 1. Minimum Viable Product. 2. Usable enough to actually solve the main problem. 3. "Will people actually use this solution?"
10. **6.9 Difference between Prototype and MVP.** Answers: 1. The MVP. 2. Prototype: paper screens for a canteen order system. MVP: a Google Form used for real canteen orders for three days. 3. "Does it look right and make sense?"
11. **6.10 Identifying the Riskiest Assumption.** Answers: 1. Something we believe to be true but have not tested yet. 2. Customers will place orders through WhatsApp; share the catalogue with 30 customers and record actual orders. 3. Because if it is wrong the whole business fails, so testing it early saves time and money.
12. **6.11 Developing an MVP.** Answers: 1. It makes the product difficult, costly and slow. 2. Problem, target users, main feature, basic design, estimated cost, marketing method, feedback method, improvement plan. 3. A WhatsApp message to the class group.
13. **6.12 Case Study: MVP for a Canteen Pre-Order System.** Answers: 1. A Google Form, tested for three days. 2. Whether students will place orders before break, and whether the canteen can manage these orders. 3. Keeping only five items and creating a separate collection line (orders also close 20 minutes before break).

14. **6.13 Testing an MVP.** Answers: 1. Number-based, comment-based, observation. 2. $(35 \div 50) \times 100 = 70\%$. 3. How many users will test it, how long the test will continue, and what information will be collected.
15. **6.14 Beachhead Market.** Answers: 1. Specific and narrowly defined; reachable by the team; users have a strong, immediate need. 2. Students and parents of ONE specific college van route. 3. No. After learning from the first group, the idea can be improved and expanded.
16. **6.15 & 6.16 Complete Student Project & Ethical Use.** Answers: 1. Any four: homework and test reminder system, college notice organizer, canteen pre-order system, small shop WhatsApp catalogue, college van update system, lost and found reporting system, study notes organizer. 2. Clear, local, and important. 3. Needs Improvement (for the MVP Plan).

Answer key

Multiple choice

- | | | | | |
|------|------|------|-------|-------|
| 1. b | 4. b | 7. c | 10. d | 13. a |
| 2. c | 5. d | 8. b | 11. c | 14. c |
| 3. a | 6. a | 9. a | 12. d | 15. d |

Short questions — model answers

1. An **entrepreneur** is a person who identifies a problem, thinks of a solution and tries to change that solution into a product or service. **Entrepreneurship** is the process of planning, starting, testing and improving a business idea, involving creativity, problem-solving, teamwork, communication and calculated risk. The entrepreneur is the person; entrepreneurship is the process.
2. It means using digital tools such as mobile phones, social media, websites, online forms, digital payments, maps, videos and simple apps to start, test, improve and grow a business idea. Examples: a student selling handmade cards through Instagram; a shopkeeper making a WhatsApp catalogue for Ajrak.
3. What problem do people face? Who faces this problem? How often does it happen? What do people currently do to solve it? Can technology help in a simple way?
4. Small, clear problems are easier to test and improve. Too large: "improving the whole education system". Suitable: "helping Grade XII students track practical file submission dates".
5. A prototype is an early model of a product, service, website or app that shows how the idea may look or work; it is not the final product and is made to test the idea and collect feedback. Useful because it makes the idea visible, helps find mistakes early (hard-to-find buttons, too much text, missing features), and saves time and money before final coding.
6. Low-fidelity: quick, rough paper model showing main idea and flow, e.g. hand-drawn homework-reminder screens. Mid-fidelity: clearer model in PowerPoint/Canva/Figma showing layout and navigation without final graphics, e.g. clickable PowerPoint screens. High-fidelity: detailed model close to the final product with realistic colours, buttons, sample data and working features, e.g. a working demo of a small order-form website.
7. An MVP (Minimum Viable Product) is the simplest working version of a product with only the most important features, used to test the idea with real users. Minimum: only the

essential features, no extras. Viable: usable enough to actually solve the main problem.

8. It is the belief that can make the entire business fail if it turns out to be wrong. For the van update system: "Parents and students will check van updates daily." Test: create a trial WhatsApp broadcast group for one route.
9. Repeat-use rate = $(30 \div 40) \times 100 = 75\%$. Other feedback: comment-based (e.g. "What was confusing?") and observation (e.g. where users faced difficulty, such as forgetting to enter class name).
10. Specific and narrowly defined; reachable by the team; users have a strong, immediate need. Ajrak catalogue: weak = all online shoppers; better = existing customers of one local shop.

Long questions — model answers

1. **Prototype development cycle.** Draw four boxes in a loop: Design → Build → Test → Iterate → back to Design. Prototype development is not one-time; repeating the cycle is called iteration.
 - **Design:** understand the problem and plan what the prototype shows. ClassBuddy: interview 10 students; 7 of 10 forget tasks, so show a daily task list; reminders are needed before leaving home, so add a morning notification; students want simplicity, so use large buttons and clear labels.
 - **Build:** create the first version to make the idea visible (not beautiful). ClassBuddy: three paper screens (Today's classes, Add homework, Show reminder) joined by arrows.
 - **Test:** show it to users and observe. ClassBuddy: five classmates try "add one homework task" while the observer watches silently.
 - **Iterate:** improve from feedback. ClassBuddy: larger, centred Add Homework button; subject name and due date fields; a Weekly View button.

Negative feedback is information that shows what to fix; the prototype fails only if the team ignores useful feedback.

2. **Prototype vs MVP.** Purpose: prototype shows how the idea may look or work; MVP tests whether users will actually use it. Working status: prototype may not work (sketch or model); MVP must work at least in a basic way. Users: prototype, a small group giving design feedback; MVP, early real users in daily life. Cost: prototype very low; MVP low but may need basic development. Example: prototype = paper screens for a canteen order system; MVP = Google Form used for real canteen orders for three days. Key questions: prototype, "Does it look right and make sense?"; MVP, "Will people actually use it and find value in it?"

3. **MVP plan: college van update system (Hyderabad).**

- Problem: students and parents do not know when the van will arrive, so students wait outside for a long time or miss the van.
- Target users: students and parents on one van route.
- Main feature: the driver or a route monitor sends a short "Van left / 5 minutes away" message.

- Basic design: one WhatsApp broadcast message with route stop name, time and status; voice note option for parents who cannot read.
- Cost: free, using a WhatsApp broadcast list on the monitor's phone.
- Marketing: a message in the class WhatsApp group and a printed slip for parents with the number to save.
- Feedback: after one week, a 3-question Google Form or phone call: Did you check the updates daily? Did you wait less? Would you continue?
- Improvement plan: if messages arrive too late, send them earlier; if many parents do not read, switch to voice notes; then expand to a second route.
- Riskiest assumption: parents and students will check van updates daily. Test: trial broadcast group for one route for one week, counting how many people read or reply to messages each day.
- Beachhead market: students and parents of ONE specific college van route.

4. MVP plan: topi and Ajrak shop in Hala.

- Low-fidelity prototype: paper catalogue pages, one per product, with a sketch, price, size/type and "Message to order"; shown to 5 customers to see if information is clear.
- MVP: a WhatsApp Business catalogue with real photos and prices for 10–15 best-selling items; orders by chat; payment by cash on delivery or JazzCash/Easypaisa.
- Riskiest assumption: customers will actually place orders through WhatsApp. Test: share the catalogue with 30 existing customers and record actual orders for two weeks.
- Beachhead market: existing customers of this one shop (people who have bought before and whose numbers the shop already has with permission).
- Marketing: broadcast message to existing customers, a QR code at the shop counter, posts in local Facebook groups.
- Ethics: share no customer numbers or photos without permission; give truthful prices, sizes and photos. Equity: simple Urdu/Sindhi descriptions and compressed photos for low data use.
- Improvement plan: drop designs nobody orders, add more of the popular ones, then expand to customers in Hyderabad and Karachi.

5. Lost and Found MVP analysis.

- (a) Tried: $(45 \div 60) \times 100 = 75\%$ of the class. Repeat-use rate: $(36 \div 45) \times 100 = 80\%$. Confused about collection: $(12 \div 45) \times 100 = 26.7\%$ (about 27%).
- (b) "45 tried" and "36 would use again": number-based. "Did not know where to collect": comment-based (12 users said it). "Many reports had no photo": observation.
- (c) Fix the collection confusion first: it is a repeated problem (more than a quarter of users) and it stops the system's main purpose, which is returning items to owners. Add "Collect found items from the library desk" at the top of the form and pin it in the WhatsApp group. Next, make the photo field strongly encouraged (or required for found items), since photos help owners recognise items. Re-test for another week and compare the numbers.

Self-assessment: I can...

- ☐ define entrepreneur and entrepreneurship and describe an entrepreneur's habits.
- ☐ use the five questions and the steps of Table 6.2 to turn a problem into an idea.
- ☐ compare low-, mid- and high-fidelity prototypes.
- ☐ turn user feedback into problems and improvements, as in Table 6.5.
- ☐ identify the riskiest assumption of an idea and design a test for it.
- ☐ choose a specific, reachable beachhead market.
- ☐ explain how digital tools let me start and test an idea cheaply, with local examples.
- ☐ define a prototype and list its benefits and possible forms.
- ☐ draw and explain the Design–Build–Test–Iterate cycle.
- ☐ define an MVP and explain how it differs from a prototype.
- ☐ fill in an MVP plan and analyse MVP test results with percentages.
- ☐ collect feedback ethically and design for equal access.

Lecture 07

Final Revision and Mock Paper

Final revision lecture — 1 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

BIEK Computer Science XII · Bilingual support: English + اردو

بی آئی ای کے کمپیوٹر سائنس — بارہویں جماعت

01 book lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XII

How to use this BLEK study-guide lecture

Each PDF follows one textbook lecture from the BLEK / Sindh Computer Science notes (New Sindh Curriculum). Read *Learn it*, study the SVG diagram, try the worked example on paper, then answer *Check yourself* before the answers. **Extra board practice** (MCQs, short and long questions) is added after the book content — it is not in the textbook. Career-only extras that were not in the book lecture list have been removed. Math is rendered with MathJax. ★ Golden topics are high-yield in board exams. The green box repeats the idea in Urdu.

ہر لیکچر کتاب کے درس کے مطابق ہے۔ کتاب کے بعد اضافی MCQs اور سوالات ہیں۔ جو اضافی کیریئر صفحات کتاب کی فہرست میں نہیں تھے وہ ہٹا دیے گئے ہیں۔

One-page recaps and a mock paper

Use these pages in the last week before the exam. Read one recap aloud from memory, then sit the mock paper in 2 hours 30 minutes with no notes.

امتحان سے ایک ہفتہ پہلے خلاصے زبانی دہرائیں، پھر ماک پیپر بغیر نوٹس کے حل کریں۔

Chapter 1 recap — Human Computer Interaction

- HCI = study of how people and computers work together so systems are usable, useful and satisfying.
- Traditional interaction = keyboard, mouse, menus. Natural interaction = touch, voice, gesture, gaze.
- Components: user, computer, interaction (input + output + feedback) inside an environment.
- UI = how it looks and the controls. UX = how the whole experience feels. Wireframe → prototype (Figma) → test.
- Evaluate with usability tests, heuristic evaluation, A/B tests and automated checks. Golden: accessibility, HCI problems, UI vs UX, testing methods.

Chapter 2 recap — Algorithms and Data Structures

- Correctness: trace table (variable values after every step) or stepwise reasoning.
- Clarity: modularity (small named parts) and readability (names, comments, indentation).
- Efficiency: count steps, conditions, repetitions. Big O: $O(1)$ constant, $O(\log n)$ binary search, $O(n)$ linear, $O(n \log n)$ good sorts, $O(n^2)$ nested loops.
- Linear DS: array, linked list, stack (LIFO, push/pop), queue (FIFO, enqueue/dequeue). Non-linear: tree (one root, parent-child), graph (nodes + edges).
- Pick the structure from the problem: undo → stack; printer jobs → queue; hierarchy → tree; maps → graph; fast index → array.

Chapter 3 recap — Python Data Structures, Functions and Files

- list [] ordered, mutable. tuple () ordered, immutable. set {} unordered, unique. dict {k:v} key-value, mutable.
- Functions: def, parameters, return. Local vs global scope. Built-in vs user-defined.
- Files: "r" read, "w" overwrite, "a" append. Always prefer `with open(...)` as `f` so the file closes itself.

- Exceptions: `try / except / else / finally`. Catch specific errors (`ValueError`, `FileNotFoundError`).

Chapter 4 recap — Data and Analysis

- SQLite in Python: `connect` → `cursor` → `execute` → `commit` → `close`. Always commit writes.
- Pandas `DataFrame` = table. load CSV/Excel, `isnull`, `dropna`, `fillna`, `sort`, `filter`, `groupby`.
- Charts: bar (categories), line (trend over time), pie (parts of a whole, few slices), histogram (distribution), scatter (relationship of two variables), box (spread and outliers).
- Mean = average. Median = middle after sorting. Mode = most frequent. Range = max–min. Variance / std-dev measure spread. pandas `.std()` uses sample $n-1$.

Chapter 5 recap — AI, Security and Collaboration

- AI ⊃ Machine Learning ⊃ Deep Learning. A neuron: weighted sum + bias, then an activation function. Deep = many hidden layers.
- Secure collaboration: authentication (passwords, MFA), access control (who may do what).
- Protect data: encryption, strong passwords, 3-2-1 backup, firewall. Threats: malware, phishing, DoS/DDoS, social engineering.
- Equity: everyone can take part (devices, language, disability, bandwidth). Tools: shared docs, video, LMS, versioning.

Chapter 6 recap — Entrepreneurship in the Digital Age

- Entrepreneur = person who turns a problem into a venture, taking risk. Digital tools make testing cheap (WhatsApp, Forms, a Facebook page).
- Prototype = experimental model. Low / mid / high fidelity. Cycle: Design → Build → Test → Iterate.
- MVP = smallest product that tests the riskiest assumption with real users. Prototype ≠ MVP (MVP is used in the real market).
- Beachhead market = the first, narrow group you can win. Measure retention, not only sign-ups. Use data ethically.

Mock paper — Computer Science XII

Time: 2 hours 30 minutes | Total: 75 marks | Answer all of A and B; attempt three from C.

Section A — Multiple choice (15 × 1 = 15)

1. HCI stands for:
 - a. Human Computer Interaction
 - b. High Code Interface
 - c. Hardware Control Input
 - d. Hybrid Computing Internet
2. A wireframe is:
 - a. the final coloured UI
 - b. a simple layout sketch of screens
 - c. a compiled program
 - d. an encryption key
3. Which is $O(\log n)$?
 - a. Linear search
 - b. Nested for-loops
 - c. Binary search
 - d. Reading one array cell
4. A stack is:
 - a. FIFO
 - b. LIFO
 - c. random access only
 - d. unordered unique items
5. Which Python type is immutable?
 - a. list
 - b. dict
 - c. set
 - d. tuple
6. The file mode that overwrites an existing file is:
 - a. r
 - b. a
 - c. w
 - d. x+
7. `with open("t.txt") as f` is useful because:
 - a. it encrypts the file
 - b. it closes the file automatically
 - c. it creates a DataFrame
 - d. it skips exceptions
8. After INSERT in sqlite3 you must:
 - a. rollback always
 - b. commit
 - c. drop the table
 - d. print the cursor
9. A pie chart is a poor choice when:
 - a. you have 3 parts of a total
 - b. you have 20 tiny slices
 - c. you show percentages of a whole
 - d. you label each slice

.0. pandas `df['x'].std()` uses:

- a. population n
- b. sample n-1
- c. median
- d. mode

.1. Deep Learning is a subset of:

- a. only statistics
- b. Machine Learning
- c. only HCI
- d. only networking

.2. MFA means:

- a. Many File Archives
- b. Multi-Factor Authentication
- c. Main Firewall Access
- d. Minimum Function API

.3. A Caesar cipher is a form of:

- a. phishing
- b. encryption
- c. a firewall
- d. a prototype

.4. An MVP is:

- a. the most polished version of a product
- b. the smallest product that tests the riskiest assumption
- c. a paper sketch only
- d. a full company registration

.5. A printer job list is best modelled as a:

- a. stack
- b. queue
- c. binary tree
- d. set

Section B — Short questions (10 × 3 = 30)

1. Differentiate UI and UX with one example.
2. What is a trace table? Why is it better than "it looks right"?
3. Draw a 3-node stack after push 10, push 20, pop. Label TOS.
4. Write one difference each: list vs tuple vs set vs dictionary.
5. Write a Python function `area(l, w)` that returns $l \times w$, and a call that prints it.
6. Name the five steps of a database connection in Python.
7. When would you draw a scatter plot rather than a line graph?
8. Compute mean, median and range of 4, 8, 6, 10, 2.
9. List three signs of a phishing message.
- .0. Differentiate a prototype and an MVP.

Section C — Long questions (3 × 10 = 30). Attempt any three.

1. A college wants a new student-portal. Describe need analysis, a low-fidelity wireframe of the login + timetable screens, three accessibility principles you would apply, and one usability test you would run.

- Trace this algorithm with a table for $n = 5$: $\text{total} = 0$; for i in $1..n$: $\text{total} = \text{total} + i$; return total . State its Big O and rewrite it as a formula that is $O(1)$.
- Write a Python program that reads a text file of lines "name,marks", stores results in a dictionary, and writes "name: grade" to an output file using `with`. Grade A if marks ≥ 80 else B. Include exception handling if the file is missing.
- Using the dataset 12, 18, 12, 20, 15, compute mean, median, mode, range, population variance and population standard deviation, showing every step. Then state which chart you would use to show the distribution and why.
- A Karachi canteen wants a pre-order app. Identify the problem, the riskiest assumption, an MVP, a beachhead market, two test metrics, and one ethical rule for student data.

Mock paper — answers

Section A

- | | | | | |
|------|------|------|-------|-------|
| 1. a | 4. b | 7. b | 10. b | 13. b |
| 2. b | 5. d | 8. b | 11. b | 14. b |
| 3. c | 6. c | 9. b | 12. b | 15. b |

Section B

- UI is the screens and controls (buttons, colours). UX is the whole journey (time to complete a task, frustration, trust). Example: a JazzCash send-money screen (UI) vs how safe and quick the payment felt (UX).
- A trace table lists each variable after every instruction. It proves correctness on a chosen input; "it looks right" misses off-by-one and uninitialised variables.
- After push 10, TOS=10. After push 20, TOS=20 (10 below). After pop, 20 removed, TOS=10.
- list: ordered mutable. tuple: ordered immutable. set: unordered unique. dict: keyed pairs.
- `def area(l, w): return l * w` then `print(area(3, 4))` $\rightarrow 12$.
- connect, cursor, execute, commit, close.
- Scatter: two measured variables (height vs marks) with no single time axis. Line: one quantity changing over an ordered time or sequence.
- Sorted: 2,4,6,8,10. Mean = $30/5 = 6$. Median = 6. Range = $10 - 2 = 8$.
- Unknown sender, urgent threat ("account will close today"), a link that does not match the real bank, request for OTP/password, poor Urdu/English, unexpected attachment.
- A prototype is a model used to learn (often internal). An MVP is a real, usable product with just enough features to test the riskiest assumption with actual customers.

Section C — outline

- Need analysis: who (students, teachers), tasks (view timetable, submit leave), constraints (mobile, slow internet). Wireframe: simple boxes for logo, fields, button, a second screen with a 5-day table. Accessibility: large tap targets, colour not the only signal, Urdu/English

labels, screen-reader names. Test: five students think-aloud while logging in; record errors and time.

2. $i=1$ total=1; $i=2$ total=3; $i=3$ total=6; $i=4$ total=10; $i=5$ total=15. Loop body runs n times $\rightarrow O(n)$. Formula $n \times (n+1) / 2$ is $O(1)$.

3. try: with open(path) as f: for line in f: name, m = line.strip().split(","); d[name]=int(m)
except FileNotFoundError: print("missing"). Then with open(out,"w") as g: write grades.
Show one sample dictionary.

4. Sorted 12,12,15,18,20. Mean $77/5=15.4$. Median 15. Mode 12. Range 8. Population variance: deviations $-3.4, -3.4, -0.4, 2.6, 4.6$; squares 11.56,11.56,0.16,6.76,21.16; sum 51.2; $/5 = 10.24$. std = 3.2. Histogram to show distribution (or bar of frequencies).

5. Problem: long canteen queues between periods. Riskiest assumption: students will pre-order 10 minutes before break. MVP: WhatsApp number + Google Form, pickup tokens, no app. Beachhead: one shift of first-year students. Metrics: % of orders picked up, time-to-collect, repeat use. Ethical: do not publish phone numbers; delete form responses after the term.